Principios y práctica del aprendizaje profundo de representaciones

o Una teoría matemática de la memoria

Sam Buchanan*, UC Berkeley & TTIC

Druv Pai*, UC Berkeley

Peng Wang, Universidad de Macau & Universidad de Michigan

Yi Ma, Universidad de Hong Kong & UC Berkeley

Este manuscrito se ha preparado para un curso sobre los principios del aprendizaje profundo y la inteligencia en general, a entregarse por los autores en sus respectivas instituciones. Esta versión previa a su publicación es de libre acceso para ver y descargar solo para uso personal y no se permiten su redistribución, su reventa ni su uso en obras derivadas.
Copyright © 2026 Sam Buchanan, Druv Pai, Peng Wang, Yi Ma

1 de marzo de 2026

* Estos autores contribuyeron por igual.

Prefacio

«*Todos los caminos conducen a Roma.*»

Objetivo. Este libro revela y estudia un problema común y fundamental que subyace a casi todas las prácticas modernas de inteligencia artificial. Se trata de *¿cómo aprender de manera eficaz y eficiente una distribución de datos de baja dimensión en un espacio de alta dimensión y luego transformar esta distribución en una representación compacta y estructurada?* Para cualquier sistema inteligente, natural o artificial, una representación de este tipo puede considerarse en general como *memoria* o *conocimiento* (empírico) aprendido a partir de datos percibidos del mundo externo. En los últimos años, a menudo se le denomina informalmente un «modelo del mundo.»

Público objetivo. Este libro de texto tiene como objetivo proporcionar una introducción sistemática a los principios matemáticos y computacionales del aprendizaje de representaciones (profundas) de tales distribuciones de datos, como forma computable de memoria, para *estudiantes de último año de licenciatura y de posgrado principiantes.* Los principales prerequisitos para este libro son el álgebra lineal a nivel universitario, probabilidad y estadística, así como optimización. Cierta familiaridad con los conceptos básicos del procesamiento de señales (representaciones esparsas y sensores comprimidos en particular), de teoría de la información, así como de control por retroalimentación mejoraría la comprensión del lector.

Motivación. La principal motivación para escribir este libro es que ha habido avances extraordinarios en estos últimos años, tanto por los autores como por numerosos colegas, que buscan establecer un enfoque riguroso y basado en principios para entender las redes neuronales profundas y, más en general, la inteligencia como tal. La metodología deductiva defendida por este nuevo enfoque, que es altamente complementaria, está en contraste directo con la metodología dominante que subyace a las prácticas contemporáneas en inteligencia artificial, que son inductivas y basadas en ensayo y error. La falta de comprensión de esos poderosos modelos y sistemas de IA ha generado

un creciente entusiasmo y temor en el seno de la sociedad. Creemos que es necesario, más que nunca, hacer un esfuerzo serio por establecer un enfoque basado en principios para comprender lo que es la inteligencia. Un objetivo general de este libro es proporcionar pruebas teóricas y experimentales sólidas que demuestren que es ahora posible estudiar la inteligencia como una disciplina científica y matemática. Argumentaremos que la inteligencia es la capacidad fundamental de desarrollar nuevas memorias (o conocimientos) o de corregir las existentes. Por lo tanto, podemos considerar este libro como un primer intento de desarrollar *una teoría matemática de la inteligencia*, al nivel de aprender conocimientos empíricos en forma de memoria, tal como sugiere el subtítulo del libro.

A nivel técnico, el marco teórico presentado en este libro ayuda a reconciliar una brecha antigua entre el enfoque clásico del modelado de estructuras de datos principalmente basado en modelos analíticos geométricos, algebráicos y probabilísticos (p.ej. subespacios, distribuciones gaussianas y ecuaciones) y el enfoque «moderno» basado en modelos no paramétricos diseñados empíricamente a partir de datos (p.ej. redes profundas). Como resultado, la unificación de estas dos metodologías aparentemente separadas es posible e incluso natural si se entiende que todas tratan de aprender y representar estructuras *de baja dimensión* en la distribución de datos de interés. Son simplemente maneras diferentes de descubrir, representar y explotar estas estructuras de baja dimensión. Desde esta perspectiva, incluso muchas técnicas computacionales aparentemente no relacionadas, desarrolladas independientemente en campos separados y en distintos momentos, pueden ser comprendidas mejor ahora en un marco teórico y computacional común y es probable que puedan estudiarse en su conjunto a partir de ahora. Como veremos en este libro, estas técnicas, entre otras, incluyen:

1. La codificación y decodificación compresiva con pérdidas desarrollada en la teoría de la información clásica y la teoría de la codificación frente al aprendizaje de representaciones moderno;

2. La eliminación de ruido y la reducción de dimensionalidad en el procesamiento clásico de señales frente a los modelos de difusión y eliminación de ruido de los métodos generativos modernos;

3. La inferencia bayesiana por vía del máximo a posteriori o del muestreo condicionado frente a la optimización con restricciones por la vía de técnicas de continuación [1].

Contenido principal. Creemos que el marco conceptual y computacional unificado que se presenta en este libro será de gran utilidad para aquellos lectores que verdaderamente quieran aclarar los misterios y malentendidos de las redes neuronales profundas y la inteligencia (artificial). Más allá de lo anterior, este marco pretende equipar a los lectores con principios rectores para

[1] como el método del lagrangiano aumentado

desarrollar sistemas significativamente mejores y verdaderamente inteligentes en el futuro. Más específicamente, aparte de una introducción (capítulo) informal, el contenido técnico principal del libro se organizará en forma de seis temas (capítulos) íntimamente relacionados:

1. Comenzaremos con los modelos clásicos y más básicos del análisis de componentes principales (PCA), del análisis de componentes independientes (ICA) y del aprendizaje de diccionarios (DL), las que suponen que las distribuciones de baja dimensión de interés poseen estructuras lineales e independientes. A partir de estos modelos simples idealizados que han sido ampliamente estudiados y comprendidos en procesamiento de señales y detección por compresión, introduciremos las ideas más fundamentales[2] sobre cómo aprender y cómo representar distribuciones de baja dimensión.

2. Para generalizar estos modelos clásicos y extender sus soluciones a distribuciones de baja dimensión que no puedan ser representadas fácilmente por un modelo analítico sencillo, introducimos un principio computacional universal para el aprendizaje de tales distribuciones: *la compresión*. Como veremos, la compresión de datos proporciona una visión unificadora de todos los enfoques clásicos y modernos aparentemente diferentes para el aprendizaje de distribuciones o representaciones, incluyendo la minimización de entropía, la eliminación de ruido mediante score-matching, compresión con pérdidas y distorsión de tasa, así como el aprendizaje discriminativo de representaciones mediante maximización de la ganancia de información.

3. En este marco unificador, las redes neuronales profundas (DNNs) modernas, como ResNet, CNN y Transformer, pueden todas interpretarse matemáticamente (desenrollarse) como algoritmos de optimización que iterativamente logran mejor compresión y mejor representación al reducir la longitud o la tasa de codificación o al ganar información. Este marco no solo ayuda a explicar las arquitecturas de redes profundas diseñadas empíricamente hasta ahora, sino que también conduce a nuevos diseños arquitectónicos que pueden ser significativamente más simples y eficientes.

4. Más allá de lo anterior, para que la representación aprendida para una distribución de datos sea correcta y consistente, se hacen necesarias las arquitecturas de *autocodificadores* que incluyen tanto codificación como decodificación. Para que un sistema de aprendizaje sea plenamente automático y de operación contínua, introduciremos un poderoso *esquema de transcripción en circuito cerrado* que permite que un sistema autocodificador se autocorrija y, por tanto, se autoperfeccione, por medio de un juego minimax entre codificador y decodificador.

5. Para conectar la teoría con la práctica, entonces estudiaremos cómo la distribución de datos y su representación aprendidas pueden utilizarse como un potente a priori para llevar a cabo inferencias bayesianas u

[2] tales como a eliminación de ruidos y la reducción de dimensionalidad

optimizaciones con restricciones que se manifiestan en casi todos los tipos de tareas y configuraciones más ampliamente utilizadas en la práctica de la inteligencia artificial moderna, incluyendo la estimación, el completamiento y la generación condicionados de datos reales de alta dimensión, tales como imágenes y textos.

6. Por último, pero no menos importante, mostraremos paso a paso cómo aprender de manera efectiva y eficiente las representaciones profundas de distribuciones de baja dimensión a partir de conjuntos de datos del mundo real a gran escala, incluyendo datos visuales, movimientos corporales y datos textuales. Estos se usarán en numerosas aplicaciones prácticas, tales como clasificación, completado, segmentación, generación de imágenes, estimación de movimiento y tareas similares para datos de texto.

Resumen. Para resumir, el contenido técnico presentado en este libro establece fuertes conexiones conceptuales y técnicas entre el enfoque analítico clásico y el enfoque moderno impulsado por datos, entre modelos paramétricos simples y los modelos no paramétricos profundos, entre las diversas prácticas inductivas y un marco deductivo unificado basado en principios fundamentales. Revelaremos que numerosos enfoques aparentemente no relacionados o incluso contradictorios entre sí, aunque se hayan desarrollado en campos distintos, con terminologías diversas y en momentos históricos diferentes, sin embargo todos aspiran a lograr un objetivo común:

perseguir y explotar las estructuras intrínsecas de baja dimensión de distribuciones de datos incrustadas en espacios de alta dimensión.

A tal fin, el libro nos guiará a través de un recorrido completo desde la formulación teórica a la deducción matemática, seguida de la implementación computacional y las aplicaciones prácticas.

Prefacio a la versión 2.0

Como libro de código abierto, la primera versión, titulada «*Aprendizaje de representaciones profundas de distribuciones de datos*», lanzada el 18 de agosto de 2025, se ha actualizado de manera frecuente y constante en los últimos seis meses. Al mismo tiempo, basados en nuestra experiencia usando el libro en un nuevo curso impartido en la Universidad de Hong Kong en el semestre de otoño de 2025, así como en los comentarios que hemos recibido de colegas, tutores y estudiantes, hemos identificado numerosos puntos a lo largo del libro que merecen una mayor revisión y ampliación.

Por ello hemos decidido introducir algunos cambios y actualizaciones sustanciales en el contenido y la organización del libro para la nueva *Versión 2.0* y hemos cambiado su título a:

Principios y práctica del aprendizaje profundo de representaciones
o Una teoría matemática de la memoria

Esta nueva versión también nos permite poner de relieve las fuertes conexiones conceptuales y técnicas entre los contenidos de los diferentes capítulos y secciones del libro, de manera que creemos que el valor pedagógico global del libro ha mejorado significativamente en comparación con la primera versión.

En nuestra opinión, la nueva versión logra una presentación mucho más unificada, completa y fluida del tema. La nueva versión también incorpora nuevos aportes teóricos recientemente desarrollados, así como un número creciente de aplicaciones prácticas para datos del mundo real que no se han documentado de manera apropiada ni explicado en forma sistemática en otros lugares. Esta Versión 2.0 servirá como solución oportuna para esta situación.

Principales cambios en la Version 2.0

- Hemos dividido en dos el Capítulo 3 de la primera versión, que ahora son los Capítulos 3 y 4. El nuevo Capítulo 3 se centra en el proceso de eliminación de ruido para el aprendizaje de distribuciones de baja dimensión. Hemos agregado una nueva sección en la que se caracterizan las condiciones bajo las que el proceso conduce a una generalización o memorización. El nuevo Capítulo 4 se centra en el aprendizaje de representaciones basado en un enfoque de codificación con pérdidas y en el principio de maximización de la ganancia de información. Como la

versión anterior solo ilustraba cómo aplicar el principio en el aprendizaje de representaciones de imágenes en una configuración supervisada, en la nueva versión se agrega un estudio de casos correspondientes a una configuración no supervisada. También proporciona una justificación teórica de algunos métodos populares de aprendizaje no supervisado tales como el aprendizaje contrastivo y DINO en particular.

- En el Capítulo 5 actualizado sobre el diseño de representaciones por vía de optimización desenrollada, hemos agregado una derivación basada en principios para una versión causal de la arquitectura de transformador transparente (white-box) CRATE, que resulta importante para el procesamiento de datos secuenciales tales como textos, como vemos frecuentemente en aplicaciones.

- En el Capítulo **??** actualizado sobre el aprendizaje de representaciones consistentes, hemos agregado una nueva sección que aumenta la claridad sobre las relaciones entre el aprendizaje de distribuciones y el aprendizaje de representaciones. Esto da una clarificación y justificación teóricas para los métodos populares y prácticos basados en autocodificación, como la autocodificación variacional o los autocodificadores de representaciones.

- En el Capítulo **??** actualizado hemos añadido una sección que ofrece una explicación basada en principios del aprendizaje de representaciones con datos pareados y generación condicionada a través de la perspectiva de la información mutua. Proporciona una justificación teórica para métodos populares y prácticos tales como CLIP y formas de atención cruzada.

- El Capítulo **??** ampliado sobre aplicaciones ahora incluye numerosas implementaciones detalladas del aprendizaje de distribuciones y de representaciones de datos reales, incluyendo imágenes bidimensionales, objetos naturales tridimensionales, movimientos corporales, así como lenguajes naturales, en casi todos las configuraciones populares y prácticas, supervisadas, débilmente supervisadas, no supervisadas y condicionadas.

- El Capítulo **??** final, sobre líneas de investigación abiertas, también se ha reescrito significativamente. Buscamos ofrecer una taxonomía más clara de los diferentes niveles de la inteligencia, de modo de poder clarificar qué hemos hecho y qué entendemos acerca de la inteligencia (con este libro) y qué no hemos hecho. Creemos que los problemas abiertos asociados con las formas superiores de inteligencia pueden formularse adecuadamente de manera de que puedan estudiarse cualitativa y cuantitativamente con medios científicos y matemáticos, en lugar de permanecer como un asunto misterioso o meramente filosófico.

Contribuidores a la Versión 2.0

Además de los autores, numerosos estudiantes y colegas han colaborado a este proyecto y han aportado contenido valioso a diferentes partes del libro durante la

preparación de la *Versión 2.0*. Se presenta a continuación una lista incompleta de las personas y sus contribuciones específicas, en orden alfabético:

- Tianzhe Chu: Experimentos en la Sección **??**, así como herramientas de IA para el libro.
- Prof. Shenghua Gao: Secciones **??** y **??**.
- Bingbing Huang: Secciones **??** y **??**.
- Kerui Min: Traducción al chino.
- Prof. Qing Qu: Sección 3.3.
- Shengbang Peter Tong: Secciones **??** y **??**.
- Chengyu Wang: Secciones **??** y **??**.
- Ziyang Robin Wu: Secciones **??** y desarrollo del sitio web.
- Chun-Hsiao Daniel Yeh: Secciones **??** y **??**.
- Brent Yi: Sección **??**.
- Jingfeng Yang: Sección **??**.
- Dr. Yaodong Yu: El capítulo 5 se basa en su tesis doctoral.
- Dr. Zibo Zhao: Sección **??**.

También queremos agradecer al Dr. Kevin Murphy y al Dr. Bill Mark por sus extensos comentarios técnicos sobre el manuscrito; a Jan Cavel por haber contribuido una traducción no oficial al rumano, así como a Stephen Butterfill y Jeroen Van Goey por haber contribuido correcciones y ajustes al manuscrito.

Declaratoria de acceso abierto

> «*A menudo comparo el código abierto con la ciencia. La ciencia ha adoptado toda esta noción del desarrollo abierto de ideas y de mejoras en base a las ideas de otros. Esto hizo de la ciencia lo que es hoy en día y permitió los increíbles avances que hemos logrado. Y eso lo comparo con la brujería y la alquimia, donde la abertura era algo que no se practicaba.*»
>
> – Linus Torvalds

Este libro está destinado a ser un proyecto de «código abierto» que se pondrá a disposición del público en el sitio web

https://ma-lab-berkeley.github.io/deep-representation-learning-book/

para todos los interesados en los principios científicos, matemáticos y computacionales de la inteligencia. El libro se actualizará de manera continua y periódica por todos los que quieran contribuir a este proyecto, incluyendo sus versiones traducidas, los materiales educativos, y diversas herramientas de IA para apoyar el estudio, la enseñanza y la investigación de este tema.

A través de este libro esperamos iniciar un «programa de conocimiento abierto» que promueva que todo el conocimiento científico sea de acceso abierto, para su organización, pedagogía, difusión y avance.

Agradecimientos

Apoyo financiero. Este libro se basa principalmente en resultados de la investigación desarrollada a partir de 2018. Gracias a los generosos fondos de arranque de la Universidad de California, Berkeley (2018) y la Universidad de Hong Kong (2023), Yi Ma pudo iniciar y centrarse en esta nueva y apasionante línea de investigación durante los últimos ocho años. Durante estos años, para la investigación en esta dirección, Yi Ma y su equipo en Berkeley han sido apoyados por las las siguientes subvenciones de investigación:

- El proyecto de multi-universidad *THEORINET* (Transferable, Hierarchical, Expressive, Optimal, Robust, and Interpretable NETworks) para las Fundaciones del Aprendizaje Profundo, conjuntamente financiado por la Fundación Simons y la National Science Foundation (DMS grant #2031899);
- El proyecto *Closed-Loop Data Transcription via Minimaxing Rate Reduction* financiado por Office of Naval Research (grant N00014-22-1-2102);
- El proyecto *Principled Approaches to Deep Learning for Low-dimensional Structures* financiado por la National Science Foundation (CISE grant #2402951).

Este libro no habría sido posible sin el apoyo financiero para estos proyectos de investigación. Los autores han obtenido una inspiración tremenda de los resultados de investigación desarrollados por colegas y estudiantes que han participado en estos proyectos.

Notación

En este libro utilizamos la notación de una variedad de disciplinas matemáticas, resumida a continuación. Se recomienda al lector que consulte esta lista como referencia a medida de que vaya leyendo los diversos capítulos del libro. Hemos intentado mantener su uso lo más coherente posible entre capítulos.

Como un primer ejemplo, para un número natural n el conjunto $\{1, 2, \ldots, n\}$ se denota por $[n]$.

Notación de álgebra lineal

- Los escalares (ya sean deterministas o aleatorios) no están en negrilla, por ejemplo, x, X.

- Los vectores (ya sean deterministas o aleatorios) están en negrilla minúscula, por ejemplo, $\boldsymbol{x}$, $\boldsymbol{\pi}$. Los componentes de vectores se denotan x_i o π_i, alternativamente $(\boldsymbol{x})_i$. Los términos genéricos (que pueden ser escalares, vectores, matrices o tensores de orden superior) también están en negrilla.

- Las matrices (ya sean deterministas o aleatorias) están en negrilla mayúscula, por ejemplo, $\boldsymbol{X}, \boldsymbol{\Pi}$. Los elementos de matrices se denotan X_{ij} o Π_{ij}, alternativamente $(\boldsymbol{X})_{ij}$. Las columnas de matrices son vectores en negrilla minúscula, por ejemplo, $\boldsymbol{x}_i$ es la i-ésima columna de $\boldsymbol{X}$, a no ser que se defina de otra manera.

- La transpuesta de una matriz se denota con $\top$, por ejemplo, $\boldsymbol{A}^\top$. La adjunta con *, p.ej., $\boldsymbol{A}^*$. La pseudo-inversa con $\dagger$, p.ej., $\boldsymbol{A}^\dagger$.

- El rango de una matriz es $\operatorname{rank}(\boldsymbol{A})$, la traza es $\operatorname{tr}(\boldsymbol{A})$, el determinante es $\det(\boldsymbol{A})$ y el determinante logarítmico es $\operatorname{logdet}(\boldsymbol{A})$.

- La multiplicación elemento por elemento de matrices es $\boldsymbol{A} \odot \boldsymbol{B}$. La cuadratura elemento por elemento es $\boldsymbol{A}^{\odot 2}$, etc. De manera similar, el producto Kronecker de matrices es $\boldsymbol{A} \otimes \boldsymbol{B}$. El producto Kronecker iterado es $\boldsymbol{A}^{\otimes 2}$, etc.

- Para una función $f\colon \mathbb{R} \to \mathbb{R}$, su aplicación elemento por elemento a una matriz $\boldsymbol{X}$ es $f[\boldsymbol{X}]$, esto es, con corchetes. Para una función $f\colon \mathbb{R}^d \to \mathbb{R}^k$ y una matriz $\boldsymbol{X} \in \mathbb{R}^{d\times n}$, podemos difundir f para aplicarla a cada columna sin utilizar una notación especial, esto es, $f(\boldsymbol{X}) = [f(\boldsymbol{x}_1), \dots, f(\boldsymbol{x}_n)] \in \mathbb{R}^{k\times n}$.
- La esfera unidad (euclidiana) es $\mathbb{S}^{n-1} \subseteq \mathbb{R}^n$. la bola unidad (euclidiana) es $\mathbb{B}^n \subseteq \mathbb{R}^n$.
- El conjunto de matrices ortogonales es $\mathsf{O}(m,n) \subseteq \mathbb{R}^{m\times n}$ o $\mathsf{O}(n) = \mathsf{O}(n,n)$.
- El conjunto de las matrices simétricas es $\mathsf{Sym}(n)$, el de las matrices PSD simétricas es $\mathsf{PSD}(n)$, el de las PD simétricas es $\mathsf{PD}(n)$.
- Los valores propios de una matriz simétrica $\boldsymbol{S} \in \mathsf{Sym}(n)$ son todos reales por el teorema espectral; se denotan $\lambda_i(\boldsymbol{S})$ y se ordenan de modo que $\lambda_1(\boldsymbol{S}) \geq \cdots \geq \lambda_n(\boldsymbol{S})$.
- Los valores singulares de $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ con $\operatorname{rank}(\boldsymbol{A}) = r$ se denotan $\sigma_i(\boldsymbol{A})$ y se ordenan de modo que $\sigma_1(\boldsymbol{A}) \geq \cdots \geq \sigma_r(\boldsymbol{A}) > 0$. Por convención, se define $\sigma_{r+1}(\boldsymbol{A}) = \sigma_{r+2}(\boldsymbol{A}) = \cdots = 0$.
- La proyección sobre un conjunto $\mathcal{K}$ se denota $\operatorname{proj}_{\mathcal{K}}$.
- Las normas ℓ^p de vectores se denotan como $\|\boldsymbol{x}\|_p$, $p \in [0,\infty]$.
- La norma operador euclidiana en matrices es $\|\boldsymbol{A}\| = \sigma_1(\boldsymbol{A})$.
- La norma Frobenius de matrices es $\|\boldsymbol{A}\|_F = \operatorname{tr}(\boldsymbol{A}^\top \boldsymbol{A})$.
- El producto escalar euclidiano de vectores es $\langle \boldsymbol{x}, \boldsymbol{y}\rangle = \boldsymbol{y}^\top \boldsymbol{x}$.
- El producto escalar Frobenius de matrices es $\langle \boldsymbol{X}, \boldsymbol{Y}\rangle = \operatorname{tr}(\boldsymbol{Y}^\top \boldsymbol{X})$.
- Un vector o matriz con todos los elementos iguales a uno se denota $\mathbf{1}$ (la dimensión debería ser obvia por contexto). De manera similar, un vector o matriz con todos los elementos iguales a cero se denota $\mathbf{0}$. La matriz identidad se denota como $\boldsymbol{I}$.

Notación de probabilidades

- La medida de probabilidad base es $\mathbb{P}$, es decir, la probabilidad de que ocurra un evento A es $\mathbb{P}[A]$. Podemos especificar la distribución de una variable aleatoria mediante subíndices, es decir, $\mathbb{P}_{\boldsymbol{x}\sim\mu}[\boldsymbol{x} \in S]$, describe las probabilidades asociadas con una variable aleatoria $\boldsymbol{x}$ con distribución (medida) μ. Si dos variables aleatorias $\boldsymbol{x}$ y $\boldsymbol{x}'$ tienen la misma distribución, escribimos $\boldsymbol{x} \overset{\mathrm{d}}{=} \boldsymbol{x}'$.
- El operador esperanza es $\mathbb{E}$, con la misma advertencia sobre el uso de subíndices.

- El operador covarianza es Cov, con la misma advertencia sobre el uso de subíndices.
- El operador correlación es Corr, con la misma advertencia sobre el uso de subíndices.
- En ciertos contextos de modelado probabilístico donde se supone una densidad (especialmente en Capítulo 3 y **??**), también se usa la notación $p_{\boldsymbol{x}}$ para la densidad de una variable aleatoria $\boldsymbol{x}$. Esto se extiende a las densidades condicionales, p.ej. $p_{\boldsymbol{x}|\boldsymbol{y}}$ para la densidad de $\boldsymbol{x}$ condicionada por $\boldsymbol{y}$.
- En general se usan letras griegas para denotar las realizaciones de variables aleatorias (en contraste con las variables aleatorias propiamente tales). Por ejemplo, $p_{\boldsymbol{x}}(\boldsymbol{\xi})$, $p_{\boldsymbol{x}|\boldsymbol{y}}(\boldsymbol{\xi} \mid \boldsymbol{\nu})$, $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y} = \boldsymbol{\nu}]$, etc.
- El conjunto de distribuciones de probabilidades sobre un conjunto finito $\mathcal{X}$ se denota $\Delta(\mathcal{X})$. (Para $\mathcal{X} = [n]$, esto es el simplex probabilístico que se puede incrustar en $\mathbb{R}^n$).
- La distribución gaussiana con media $\boldsymbol{\mu}$ y covarianza $\boldsymbol{\Sigma}$ se denota $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.
- La distribución uniforme sobre un conjunto compacto $\mathcal{X}$ se denota $\mathcal{U}(\mathcal{X})$.

Notación de aprendizaje automático

- Los codificadores y reductores de ruido normalmente se denotan por f y suelen tener parámetros $\theta \in \Theta$. Normalmente escribiremos las características como $\boldsymbol{z} = f_\theta(\boldsymbol{x})$.
- Los decodificadores normalmente se denotan por g y suelen tener parámetros η. Normalmente escribiremos $\hat{\boldsymbol{x}} = g_\eta(\boldsymbol{z})$ para denotar la autocodificación correspondiente a $\boldsymbol{x}$.
- Cuando f y g se implementan como redes neurales, tendrán el mismo número de capas sin pérdida de generalidad; escribiremos $f = f^L \circ f^{L-1} \circ \cdots \circ f^2 \circ f^1 \circ f^{\text{pre}}$ y $g = g^{\text{post}} \circ g^L \circ g^{L-1} \circ \cdots \circ g^2 \circ g^1$.
- Escribiremos las características a la entrada a la capa ℓ como $\boldsymbol{z}^\ell$ de modo que $\boldsymbol{z}^{\ell+1} = f^\ell(\boldsymbol{z}^\ell)$ con $\boldsymbol{z}^1 = f^{\text{pre}}(\boldsymbol{x})$ y $\boldsymbol{z} = \boldsymbol{z}^{L+1}$. Del mismo modo, las características de autocodificación serán $\hat{\boldsymbol{x}}^\ell$ con $\hat{\boldsymbol{x}}^{\ell+1} = g^\ell(\hat{\boldsymbol{x}}^\ell)$ con $\hat{\boldsymbol{x}}^1 = \boldsymbol{z}$ y $\hat{\boldsymbol{x}} = g^{\text{post}}(\hat{\boldsymbol{x}}^{L+1})$.
- Para la reducción de ruido, la optimización y para procesos con un índice de tiempo continuo, el tiempo casi siempre es un subíndice, p.ej., $\boldsymbol{x}_t$. En el caso de las secuencias discretas el índice puede ser superíndice o subíndice.

Notación para modelación y optimización

- Tal como en las secciones anteriores de notación de probabilidades y de aprendizaje automático, cuando tenemos un modelo, digamos, para las realizaciones de una distribución de datos definida en $\mathbb{R}^D$, siempre lo denotamos con variables «ordinarias», sin marca diacrítica, p.ej. $\boldsymbol{x}$.

- En los casos en que suponemos que los datos $\boldsymbol{x}$ son generados por un modelo de una clase restringida de *modelos paramétricos*, digamos, con un parámetro $\boldsymbol{\mu}$, también denotamos el *parámetro verdadero* sin marca diacrítica.[3]

- Las *variables de decisión* (o parámetros aprendibles) del ajuste de un modelo a datos observados se denotan con una marca diacrítica de «tilde», p.ej. $\tilde{\boldsymbol{x}}$, en casos donde existe un modelo subyacente $\boldsymbol{x}$, o con tipografía simple en casos donde no haya confusión (p.ej., si un modelo empíricamente ajustado para datos *no paramétricos* $\boldsymbol{x}$ involucra parámetros «medios» $\boldsymbol{\mu}$).

- Las *soluciones óptimas* de problemas de optimización se denotan con una marca diacrítica de «estrella», ya sea como superíndice o subíndice (p.ej. $\boldsymbol{x}_\star$ o $\boldsymbol{x}^\star$).

- Los *estimadores* para datos que corresponden a un modelo estadístico específico, especialmente para la media de ese modelo estadístico, se denotan con una marca diacrítica de «barra», p.ej. $\bar{\boldsymbol{x}}$ (especialmente para la reducción de ruido por error cuadrático medio en Capítulo 3).

- Las *aproximaciones* asociadas a procedimientos computacionales del modelado de datos se denotan con una marca diacrítica de «sombrero», p.ej. $\hat{\boldsymbol{x}}$ para la salida de un autocodificador o un modelo generativo que aproxime datos $\boldsymbol{x}$.[4]

[3] Esto podría surgir ya sea por haber hecho una suposición sobre de los datos, o bien debido a que los datos hayan sido generados sintéticamente. Esta convención nos permite adoptar una notación concisa tanto en contextos 'probabilísticos' como 'analíticos' de modelación. Aquellos lectores más familiares con estos últimos tipos de esquemas puede que estén acostumbrados a usar notaciones tales como $\boldsymbol{\mu}_\sharp$ o $\boldsymbol{\mu}_o$ para este caso; en contraste con nuestra notación con variables de decisión acentuadas.

[4] En general distinguimos estos casos de las soluciones óptimas $\boldsymbol{x}_\star$, aunque en algunos casos especiales sean lo mismo (p.ej., en varios ejemplos de Capítulo 2). Piénsese en la distinción entre el parámetro $\boldsymbol{\mu}$ de un modelo, que puede haberse ajustado con optimización, y los resultados del muestreo con ese parámetro aprendido.

Índice

Capítulo 1

Una introducción informal a la inteligencia

> "*Así como el aumento constante de la entropía es la ley fundamental del universo, así también lo es, para la vida, el ser cada vez más altamente estructurada y luchar contra la entropía.*"
>
> – Václav Havel

1.1 Inteligencia, cibernética e inteligencia artificial

El mundo que habitamos no es plenamente aleatorio ni completamente impredecible.[1] En cambio sigue cierto orden, ciertos patrones y leyes que lo hacen predecible en gran medida.[2] El propio surgimiento de la vida y su persistencia dependen de esta predictibilidad. Solo aprendiendo y memorizando lo que es predecible en el entorno, la vida puede sobrevivir y prosperar, ya que las decisiones y acciones acertadas dependen de predicciones confiables. Dado que el mundo ofrece una variedad aparentemente ilimitada de fenómenos predecibles, los seres inteligentes—animales y humanos—han desarrollado sentidos cada vez más agudos: la vista, el oído, el tacto, el gusto y el olfato. Estos sentidos recogen datos sensoriales a gran velocidad para percibir las regularidades del entorno. Por tanto, una tarea fundamental de todos los seres inteligentes es

aprender y memorizar la información predecible a partir de volúmenes masivos de datos sensoriales.

[1] Si el mundo fuese plenamente aleatorio, un ser inteligente no tendría necesidad de aprender o memorizar nada.

[2] Algunos deterministas, otros probabilísticos.

Antes de poder entender cómo esto se logra es necesario abordar tres interrogantes:

- ¿Cómo puede modelarse y representarse matemáticamente la información predecible?
- ¿Cómo se puede aprender dicha información computacionalmente, de manera efectiva y eficiente, a partir de datos?
- ¿Cómo se debería organizar y estructurar de la mejor manera esta información como memoria o conocimiento (empírico), ya sea en nuestra mente o en una máquina, para apoyar las futuras predicciones e inferencias?

Este libro tiene como objetivo proporcionar algunas respuestas a estas preguntas. Estas respuestas nos ayudarán a mejorar nuestra comprensión de la inteligencia, especialmente los principios y mecanismos computacionales que la habilitan. La evidencia sugiere que todas las formas de inteligencia—desde la inteligencia de bajo nivel observable en las formas de vida tempranas y primitivas hasta las formas más altas de inteligencia, la práctica de la ciencia moderna—comparten un conjunto común de principios y mecanismos. A continuación lo explicamos con más detalle.

Surgimiento y la evolución de la inteligencia. Una condición necesaria para el surgimiento de la vida en la Tierra hace aproximadamente cuatro mil millones de años es que el ambiente sea en gran medida predecible. La vida ha desarrollado mecanismos que le permiten aprender lo que sea predecible sobre el medio ambiente, codificar esa información y usarla para la supervivencia. Hablando en general, a esta habilidad de adquirir conocimientos sobre el mundo la llamamos *inteligencia*. En gran medida, la evolución de la vida es obra de los mecanismos de la inteligencia [**Bennett-2023**], al igual que la evolución del universo es obra de las leyes de la física. En las etapas tempranas de la vida, la inteligencia se desarrolla principalmente a través de dos tipos de mecanismos de aprendizaje: los *filogenéticos* y los *ontogenéticos* [**Wiener-Cybernetics-1961**].

La *inteligencia filogenética* se refiere al aprendizaje a través de la evolución de las especies. Las especies heredan y sobreviven principalmente basándose en la memoria y en el conocimiento codificado en el ADN o los genes de sus progenitores. En gran medida, podríamos decir que el ADN puede considerarse como los grandes modelos preentrenados de la naturaleza, porque estos desempeñan un papel muy parecido. La principal característica de la inteligencia filogenética es que los individuos tienen una capacidad de aprendizaje limitada. El aprendizaje se lleva a cabo mediante un mecanismo de «prueba-y-error» basado en mutaciones aleatorias de los genes y las especies evolucionan por medio de la selección natural —la supervivencia del más apto— tal como se muestra en la Figura 1.1. Esto puede verse como la implementación, por parte de la naturaleza, de lo que ahora se conoce como «aprendizaje por refuerzo», [**Sutton-Barto**] o potencialmente de una «búsqueda de arquitecturas

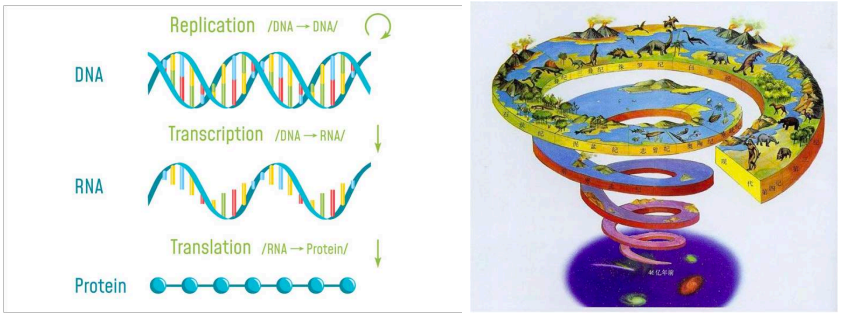


Figura 1.1: La evolución de la inteligencia filogenética: La memoria o el conocimiento del mundo exterior se codifica y transmite por vía del ADN (a la izquierda) y se decodifica de ADN a ARN y a proteínas. En etapas tempranas de la evolución de la vida (a la derecha), la inteligencia desarrolla memoria o conocimiento al nivel de la especie por vía de mutaciones (aleatorias) de genes y la selección natural —«que sobreviva el más apto»— lo que puede verse como una forma primitiva de aprendizaje por refuerzo.

neuronales»[**NAS-1**]. Sin embargo, tal proceso de «prueba-y-error» puede ser extremadamente lento, exigente en recursos e impredecible. Desde el surgimiento de las primeras formas de vida, aproximadamente hace 4400 a 3800 millones de años[**bell2015potentially**], la vida se ha basado en esta forma de evolución.[3]

La *inteligencia ontogenética* se refiere a aquellos mecanismos de aprendizaje que permiten que un individuo aprenda a través de sus propios sentidos, memorias y predicciones dentro de su entorno específico, mejorando y adaptando

[3]Los lectores más perspicaces quizá noten una sorprendente similitud entre la forma en que se ha desarrollado la vida primitiva y la forma en que evolucionan hoy en día los grandes modelos de lenguaje (LLMs).

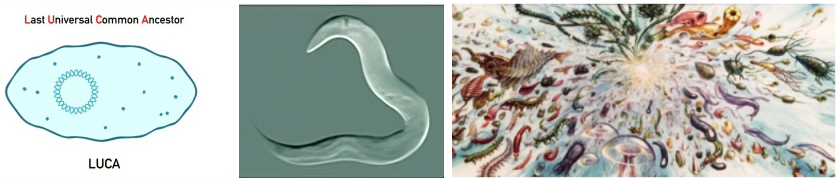


Figura 1.2: La evolución de la vida, desde el antepasado de toda la vida actual (LUCA—last universal common ancestor), un organismo de tipo unicelular que vivió hace 3500–4300 millones de años [**moody2024nature**], hasta el surgimiento del primer sistema nervioso en especies similares a gusanos (al medio), hace unos 550 millones de años [**wang2025preservation**] y hasta la explosión de las formas de vida en el Período Cámbrico (a la derecha), hace unos 530 millones de años.

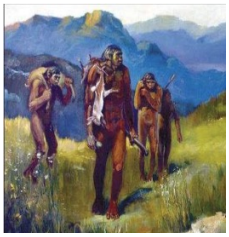
sus conductas. El aprendizaje ontogenético empezó a ser posible después del surgimiento del sistema nervioso hace alrededor de 550–600 millones de años (en organismos similares a gusanos) [**wang2025preservation**], como se muestra en la Figura 1.2 en el centro. Con su sistema sensorial y nervioso, un individuo puede formar y perfeccionar de manera continua su propia memoria del mundo, adicionalmente a lo que haya heredado mediante ADN o genes. Esta capacidad mejoró considerablemente las posibilidades de supervivencia del individuo contribuyendo a la explosión de las formas de vida en el Cámbrico, hace alrededor de 530 millones de años [**Parker-2004**]. En comparación con el aprendizaje filogenético, el aprendizaje ontogenético es más eficiente y predecible, pudiendo llevarse a cabo dentro de los límites de los recursos de un individuo.

Ambos tipos de aprendizaje se basan en la retroalimentación del entorno externo —penalizaciones (la muerte) o recompensas (alimento)— que se aplican a las acciones de una especie o de un individuo.[4] Este hallazgo inspiró a Norbert Wiener a concluir en su programa de cibernética [**Wiener-Cybernetics-1948**] que todos los seres inteligentes, ya sean especies o individuos, se basan en mecanismos de retroalimentación en circuito cerrado para aprender y mejorar su memoria y conocimiento del mundo. Además, desde las plantas pasando por peces, aves y mamíferos, las especies más avanzadas se basan progresivamente en el aprendizaje ontogenético: permanecen con sus padres por períodos más largos después de su nacimiento y aprenden de ellos, porque los individuos de una misma especie deben sobrevivir en ambientes muy diversos. En otras palabras, a medida que las especies se vuelven cada vez más «inteligentes», dependen cada vez menos de su ADN heredado («modelos base preentrenados») y cada vez más de su capacidad para aprender por sí mismas después de nacer.

La evolución de la inteligencia humana. Desde la aparición del *Homo sapiens* hace alrededor de 2.5 millones de años [**Harari-2015**], ha surgido una nueva forma superior de inteligencia, que evoluciona de manera más eficiente y económica. Las sociedades humanas desarrollaron lenguajes —primero orales, después escritos— como se muestra en la Figura 1.3. El lenguaje hace posible que los individuos se comuniquen y compartan información útil, permitiendo que la comunidad humana se comporte como un único organismo inteligente que aprende más rápido y retiene más conocimientos que cualquier individuo. Los textos escritos desempeñan, por tanto, un papel análogo al del ADN y los genes, ya que permiten que las sociedades acumulen y transmitan conocimientos entre territorios y generaciones. Podemos referirnos a este tipo de inteligencia como *inteligencia colectiva de la sociedad*, o *inteligencia societal*, diferenciándola de la inteligencia filogenética de las especies y de la inteligencia ontogenética de los individuos. Esta acumulación de conocimientos sostiene a todas las civilizaciones (antiguas).

Hace unos dos o tres mil años, la inteligencia humana dio otro gran salto cualitativo, permitiendo a filósofos y matemáticos desarrollar un conocimiento

[4] Las mutaciones génicas en el caso de las especies o las acciones ejecutadas por el individuo.



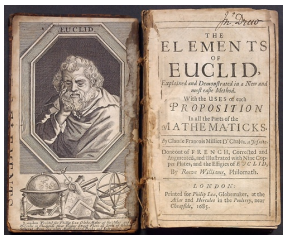


Figura 1.3: El desarrollo de la comunicación verbal y de los lenguajes orales (hace 70,000–30,000 años), de los lenguajes escritos (alrededor de 3000 AC) [**schmandt2014evolution**] y de las matemáticas abstractas (alrededor de 500–300 AC) [**heath1956thirteen**] constituyen tres hitos clave en la evolución de la inteligencia humana.

que va mucho más allá de la organización de observaciones empíricas. El desarrollo de conceptos y símbolos abstractos, como números, tiempo, espacio, lógica y geometría, dio origen a un lenguaje totalmente novedoso y riguroso de la *matemática*. Además, el desarrollo de la capacidad de formular hipótesis y comprobar su veracidad mediante la deducción lógica o la experimentación sentó las bases de la *ciencia* moderna. Por primera vez, los humanos pudieron descubrir y desarrollar nuevos conocimientos de manera proactiva y sistemática, con una precisión y un rigor sin precedentes. Esta forma de inteligencia ha mejorado significativamente la eficiencia en la adquisición de conocimientos sobre lo desconocido. A esta forma avanzada de inteligencia la llamaremos «inteligencia científica», debido a que es indispensable para el descubrimiento deductivo y científico.

Por lo tanto, según lo que podemos aprender de la naturaleza, cada vez que usamos la palabra «inteligencia», debemos especificar a qué nivel o forma nos referimos:

$$\textbf{inteligencia filogenética} \implies \textbf{ontogenética} \implies \textbf{social} \implies \textbf{científica}. \tag{1.1.1}$$

Se requieren caracterizaciones y distinciones claras porque queremos estudiar la inteligencia como disciplina de carácter científico-matemático. Aunque todas las formas puedan compartir el objetivo común de aprender y mejorar la memoria y el conocimiento del mundo, los mecanismos computacionales y las implementaciones físicas pueden diferir entre sí. Creemos que el lector entenderá y apreciará mejor sus rasgos comunes y sus diferencias después de estudiar este libro. Es por eso que dejamos las discusiones científicas o filosóficas más profundas sobre la relación entre estas diferentes formas o niveles de la inteligencia para el final del libro, en el **??**. De momento, basta con señalar que, en cuanto al tipo de conocimiento adquirido, las tres primeras etapas de la inteligencia comparten más rasgos en común: todas aprenden conocimiento (o memoria) de forma mayoritariamente empírica y luego lo utilizan de manera predominantemente inductiva. Este libro se centrará en estudiar y poner de

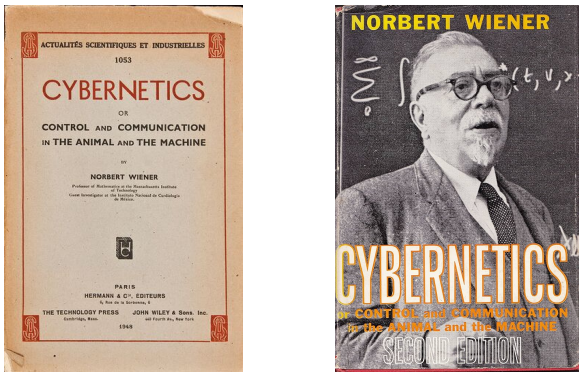


Figura 1.4: El libro «Cybernetics» (1948) de Norbert Wiener [**Wiener-Cybernetics-1948**] (a la izquierda) y su segunda edición (1961) [**Wiener-Cybernetics-1961**] (a la derecha).

manifiesto cómo este tipo de conocimiento puede aprenderse a partir de datos captados del mundo exterior. En este punto, los mecanismos que subyacen a la creación de conocimiento matemático y científico avanzado siguen siendo en gran parte un enigma, aunque creemos que deben construirse sobre la capacidad de desarrollar conocimiento empírico. Dejamos esta cuestión como problema abierto para su discusión en el **??**.

El origen de la inteligencia artificial — la cibernética. En la década de 1940, impulsados por el esfuerzo bélico, los científicos, inspirados por la inteligencia natural, buscaron emular la inteligencia animal mediante máquinas, dando origen al movimiento de la «Cibernética», encabezado por Norbert Wiener [**Kline-2011**]. Wiener estudió zoología en Harvard, durante sus estudios de licenciatura, antes de convertirse en matemático y especialista en teoría de control. Consagró su vida a la comprensión y construcción de sistemas autónomos que pudieran reproducir una inteligencia similar a la de los animales. En la actualidad, el programa de la Cibernética muchas veces se interpreta estrechamente como algo centrado sobre todo en los sistemas de control por retroalimentación, área en la que Wiener hizo sus contribuciones técnicas más significativas. Sin embargo, el programa era mucho más amplio y profundo: buscaba comprender la inteligencia en su totalidad —al menos en el nivel animal— e influyó en la obra de toda una generación de científicos de renombre, entre los que se cuentan Warren McCulloch, Walter Pitts, Claude Shannon, John von Neumann y Alan Turing.

Se considera que Wiener fue el primero en estudiar la inteligencia *como sistema*, en lugar de centrarse en sus componentes o aspectos aislados. Su visión integral se publicó en el célebre libro de 1948 *Cibernética: o el control y la comunicación en los animales y las máquinas* [**Wiener-Cybernetics-1948**]. En ese libro y en su segunda edición publicada

en 1961 [**Wiener-Cybernetics-1961**] (véase la Figura 1.4), intentó identificar diversas características y mecanismos necesarios de los sistemas inteligentes, incluyendo (pero sin limitarse a):

- Cómo *medir y almacenar* información (en el cerebro) y cómo comunicarse con los demás.[5] Esta idea llevó a Claude Shannon a formular la *teoría de la información* en 1948 [**Shannon-1948**].

- Cómo *corregir errores* en predicciones y estimaciones basándose en la información existente. El propio Wiener ayudó a formalizar la teoría del control por retroalimentación (en circuito cerrado) en la década de 1940.

- Cómo aprender a *tomar mejores decisiones* al interactuar con un entorno poco cooperativo o incluso hostil. John von Neumann formalizó este tema como *teoría de juegos* en 1944 [**von-Neumann-1944**].

En 1943, inspirados por el programa de Cibernética de Wiener, el científico de la cognición Warren McCulloch y el lógico Walter Pitts formalizaron conjuntamente el primer modelo computacional de una neurona [**McCulloch-Pitts**], llamado *neurona artificial* y más tarde lo ilustró en la Figura 1.15. Sobre la base de este modelo, Frank Rosenblatt construyó el Perceptrón Mark I en la década de 1950—una máquina física que contenía cientos de neuronas artificiales de ese tipo [**Rosenblatt:1957**]. El Perceptrón fue la primera red neuronal artificial materializada físicamente; véase la Figura 1.17. Cabe destacar que la arquitectura universal de computadoras de John von Neumann, propuesta en 1945, también fue diseñada para facilitar el objetivo de construir *máquinas computacionales* que pudieran materializar físicamente los mecanismos sugeridos por el programa de cibernética [**von-Neumann-1958**].

Los lectores más perspicaces se darán cuenta de que los años cuarenta fueron realmente mágicos: se inventaron numerosas ideas fundamentales y se formalizaron diversas teorías influyentes, entre las que se cuentan el modelo matemático de las neuronas, las redes neuronales artificiales, las teorías de la información, de control, de juegos y las computadoras. En la Figura 1.5 se muestra a algunos de los pioneros. Cada una de estas contribuciones ha evolucionado, transformándose en el fundamento de un campo científico o de ingeniería que sigue ejerciendo un tremendo impacto en nuestras vidas. Todos ellos se inspiraron en el objetivo de desarrollar máquinas que emulen la inteligencia en la naturaleza. Los registros históricos muestran que el movimiento de la cibernética encabezado por Wiener ejerció influencia sobre casi todos estos pioneros y su obra. En gran medida, el programa de Wiener puede considerarse el verdadero precursor del actual programa de «inteligencia encarnada»; de hecho, el propio Wiener articuló una visión de dicho programa con notable claridad y concreción [**Wiener-Cybernetics-1961**].

[5] Wiener fue el primero en señalar que la «información» no es ni materia ni energía, sino una cantidad independiente digna de estudiarse.



Figura 1.5: Pioneros de las fundaciones teóricas y computacionales de la inteligencia: Norbert Wiener (cibernética y teoría de control), Claude Shannon (teoría de la información), John von Neumann (teoría de juegos) y Alan Turing (teoría de la computación).

Aunque Wiener identificó numerosas características y mecanismos esenciales de la inteligencia (encarnada), no ofreció una receta clara para la integración de estas en un sistema autónomo inteligente completo. Desde la perspectiva actual, algunos de sus puntos de vista fueron incompletos o inexactos. En particular, en el último capítulo de la segunda edición de *Cybernetics* [**Wiener-Cybernetics-1961**], insistió en la necesidad de *abordar la no linealidad* si se quiere que las máquinas emulen los mecanismos de aprendizaje típicos de la naturaleza, pero no ofreció ninguna solución concreta ni eficaz para este difícil problema. Para ser justos, incluso la teoría de los sistemas lineales se aún estaba en su etapa embrionaria en aquella época,[6] y los sistemas no lineales parecían mucho menos accesibles.

No obstante, no podemos evitar de maravillarnos ante la clarividencia de Wiener respecto a la importancia de la no linealidad. Como se mostrará en este libro, la respuesta llegó sólo recientemente: la no linealidad puede ser tratada de manera efectiva a través de la linearización progresiva y la transformación que llevan a cabo las redes neuronales y representaciones profundas (véase el Capítulo 5). Más aún, demostraremos en este libro cómo todos los mecanismos listados con anterioridad pueden integrarse de manera natural en un sistema completo que exhibe ciertas características de un sistema autónomo inteligente (véase el **??**).

El origen de la inteligencia artificial. El subtítulo del libro *Cybernetics* de Wiener —*Control y comunicación en animales y máquinas*— revela que en la década de 1940 la investigación estaba primordialmente orientada a emular la inteligencia a nivel animal. Como se ha señalado anteriormente, las agendas de aquella época estaban dominadas por el movimiento cibernético de Wiener.

Alan Turing estuvo entre los primeros en reconocer esta limitación. En su célebre artículo de 1950 titulado «*Computing Machinery and Intelligence*»

[6] El famoso logro de Wiener, el filtro de Wiener [**Wiener-1942**; **Wiener-1949**], que presentaremos más adelante, solo se aplicaba a sistemas lineales.

[**Turing-1950**], Turing planteó formalmente la cuestión de «¿las máquinas pueden pensar?» En otras palabras, si las máquinas podrían imitar inteligencia de nivel humano hasta el punto en que la inteligencia de las máquinas fuera indistinguible de la inteligencia humana—lo que hoy se conoce como la *prueba de Turing*.

Alrededor de 1955, un grupo de científicos jóvenes y ambiciosos intentó desmarcarse del programa de cibernética, que por entonces era dominante, y forjar su propio legado. Aceptaron el desafío de Turing de imitar la inteligencia humana y propusieron organizar un taller en el Dartmouth College para el verano de 1956. En su propuesta declaraban [**McCarthy-1955**]:

> «*El estudio se basará en la conjetura de que todos los aspectos del aprendizaje o cualquier otra característica de la inteligencia pueden, en principio, describirse con tal precisión que se pueda construir una máquina capaz de simularlos. Se intentará descubrir cómo hacer que las máquinas utilicen el lenguaje, formar abstracciones y conceptos, resuelvan tipos de problemas que ahora están reservados a los humanos y se perfeccionen a sí mismas.*»

Su objetivo era formalizar y estudiar la inteligencia superior que distingue a los seres humanos de los animales. Su programa incluía la abstracción, los métodos simbólicos, el lenguaje natural y el razonamiento deductivo (inferencia causal, deducción lógica, etc.), es decir, la inteligencia científica. El organizador del taller, John McCarthy—entonces un joven profesor adjunto de matemáticas en el Dartmouth College—acuñó el ahora famoso término «inteligencia artificial» (IA) refiriéndose a las máquinas que muestran inteligencia científica, al describir formalmente los problemas y objetivos del taller.

¿Renacimiento de la «inteligencia artificial» o «cibernética»? A lo largo de la última década, la inteligencia de las máquinas ha experimentado un desarrollo explosivo, impulsado en su mayor parte por las redes neuronales artificiales profundas, a raíz del trabajo de Geoffrey Hinton y sus estudiantes, en 2012 [**krizhevsky2012imagenet**]. Este período es aclamado como el «Renacimiento de la IA». Sin embargo, en lo que respecta a las tareas que realmente se abordan (reconocimiento, generación, predicción) y a las técnicas desarrolladas (aprendizaje por refuerzo, aprendizaje por imitación, codificación, decodificación, eliminación de ruido y compresión), estamos en gran medida emulando mecanismos comunes de la inteligencia de las primeras formas de vida y de los animales. Incluso en este aspecto, como aclararemos en este libro, los modelos y sistemas actuales de «IA» no han implementado de manera completa ni correcta todos los mecanismos necesarios para la inteligencia a nivel animal que ya se conocían en el movimiento de la Cibernética de la década de 1940.

En sentido estricto, los recientes avances en inteligencia maquinal de la última década se ajustan mal al programa de IA de Dartmouth en 1956. En cambio, lo que se ha logrado es más cercano a los objetivos del clásico programa de cibernética de Wiener de la década de 1940. Probablemente

sea más apropiado denominar a la era actual el «Renacimiento de la Cibernética». El reciente auge de la denominada «IA encarnada» para robots y agentes autónomos se alinea aún más con los objetivos del programa de cibernética. Solo cuando comprendamos plenamente, desde una perspectiva científico-matemática, lo que realmente hemos logrado, podremos determinar qué problemas siguen sin resolverse y qué caminos conducen a la verdadera naturaleza de la inteligencia.[7] Ese es uno de los principales objetivos de este libro.

1.2 ¿Qué aprender?

1.2.1 Predictibilidad

Los datos que contienen información útil se manifiestan de muchas formas. En su forma más natural, se pueden modelar como secuencias que son predecibles y computables. El concepto de una secuencia predecible y computable fue fundamental para la teoría de la información [**Shannon-1948**] y la informática, lo que condujo en gran medida a la invención de las computadoras [**Turing-1936**]. El papel de las secuencias predecibles en la inferencia (inductiva) fue estudiado por Ray Solomonoff, Andrey Kolmogorov y otros en la década de 1960 [**Kolmogorov1998OnTO**] como generalización de la clásica teoría de la información de Claude Shannon [**Shannon-1948**]. Para ayudar a los lectores a comprender el concepto de secuencias predecibles, comenzaremos con algunos ejemplos concretos.

El caso escalar. Podría decirse que la secuencia discreta predecible más simple es la secuencia de los números naturales:

$$S = 1, 2, 3, 4, 5, 6, \ldots, n, n+1, \ldots \tag{1.2.1}$$

en la que el siguiente número x_{n+1} se define como el número anterior x_n más 1:

$$x_{n+1} = x_n + 1. \tag{1.2.2}$$

Se puede generalizar el concepto de predictibilidad a cualquier secuencia $\{x_n\}_{n=1}^{\infty}$ con $x_n \in \mathbb{R}$ si el número siguiente x_{n+1} puede computarse siempre a partir de su predecesor x_n:

$$x_{n+1} = f(x_n), \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.3}$$

donde $f(\cdot)$ es una función *computable* (escalar).[8] El trabajo pionero de Alan Turing de 1936 [**Turing-1936**] proporciona una definición rigurosa de computabilidad. En la práctica, a menudo suponemos además que f

[7] Por lo tanto, retomaremos este tema al final de este libro, en el último capítulo, el **??**.

[8] Aquí destacamos que la función $f(\cdot)$ misma es computable, lo que significa que puede implementarse como un programa en una computadora.

es computable de manera eficiente y tiene propiedades deseables, como la continuidad y la diferenciabilidad. La necesidad de estas propiedades quedará clara más adelante, una vez que comprendamos conceptos más sofisticados de computabilidad y su papel en el aprendizaje automático y la inteligencia artificial.

El caso multivariable. El valor siguiente también puede depender de dos predecesores. Por ejemplo, la famosa *secuencia de Fibonacci*

$$S = 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots \tag{1.2.4}$$

satisface

$$x_{n+2} = x_{n+1} + x_n, \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.5}$$

De manera más general, podemos escribir

$$x_{n+2} = f(x_{n+1}, x_n), \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.6}$$

para cualquier función computable f con dos parámetros de entrada. Extendiendo aún más, el siguiente valor puede depender de los d valores precedentes:

$$x_{n+d} = f(x_{n+d-1}, \ldots, x_n), \quad x_n \in \mathbb{R},\ n = 1, 2, 3, \ldots \tag{1.2.7}$$

El número entero d se denomina el *grado* de la recursión. La expresión anterior (1.2.7) se denomina *autoregresión* y la secuencia resultante es *autoregresiva*. Cuando f es lineal, decimos que es una *autoregresión lineal*.

El caso vectorial. Para simplificar la notación, definimos un vector $\boldsymbol{x} \in \mathbb{R}^d$ que recoge d valores consecutivos de la secuencia:

$$\boldsymbol{x}_n \doteq [x_{n+d-1}, \ldots, x_n]^\top, \quad \boldsymbol{x}_n \in \mathbb{R}^d,\ n = 1, 2, 3, \ldots \tag{1.2.8}$$

Con esta notación, la relación recursiva (1.2.7) se convierte en

$$\boldsymbol{x}_{n+1} = g(\boldsymbol{x}_n) \in \mathbb{R}^d, \quad n = 1, 2, 3, \ldots \tag{1.2.9}$$

donde $g(\cdot)$ está unívocamente determinada por la función f en (1.2.7) y mapea un vector d-dimensional a un vector de dimensión d. En diferentes contextos, un vector de este tipo a veces es llamado «estado» o «token». Nótese que (1.2.7) define un mapeo $\mathbb{R}^d \to \mathbb{R}$, mientras que aquí tenemos $g\colon \mathbb{R}^d \to \mathbb{R}^d$.

Predicción controlada. También podemos definir una secuencia predecible que depende de otra secuencia predecible como entrada:

$$\boldsymbol{x}_{n+1} = f(\boldsymbol{x}_n, \boldsymbol{u}_n) \in \mathbb{R}^d, \quad n = 1, 2, 3, \ldots, \tag{1.2.10}$$

donde $\{\boldsymbol{u}_n\}$ con $\boldsymbol{u}_n \in \mathbb{R}^k$ es una secuencia predecible (computable). En otras palabras, el vector siguiente $\boldsymbol{x}_{n+1} \in \mathbb{R}^d$ depende tanto de $\boldsymbol{x}_n \in \mathbb{R}^d$ como de

$\boldsymbol{u}_n \in \mathbb{R}^k$. En teoría de control, la secuencia $\{\boldsymbol{u}_n\}$ generalmente se denomina «entrada de control» y la secuencia $\boldsymbol{x}_n$ el «estado» o la «salida» del sistema (1.2.10). Un ejemplo clásico es un sistema dinámico lineal:

$$\boldsymbol{x}_{n+1} = \boldsymbol{A}\boldsymbol{x}_n + \boldsymbol{B}\boldsymbol{u}_n, \quad \boldsymbol{A} \in \mathbb{R}^{d\times d}, \boldsymbol{B} \in \mathbb{R}^{d\times k}, \tag{1.2.11}$$

que se estudia ampliamente en teoría de control [**Cal:Des**].

A menudo la entrada de control viene dada por una función computable del propio estado $\boldsymbol{x}_n$:

$$\boldsymbol{u}_n = h(\boldsymbol{x}_n), \quad n = 1, 2, 3, \ldots \tag{1.2.12}$$

Como resultado, la secuencia $\{\boldsymbol{x}_n\}$ se obtiene por la composición de dos funciones computables f y h:

$$\boldsymbol{x}_{n+1} = f(\boldsymbol{x}_n, h(\boldsymbol{x}_n)), \quad n = 1, 2, 3, \ldots \tag{1.2.13}$$

De esta manera, la secuencia $\{\boldsymbol{x}_n\}$ nuevamente se vuelve una secuencia autoregresiva predecible. Cuando la entrada $\boldsymbol{u}_n$ depende de la salida $\boldsymbol{x}_n$, decimos que la secuencia resultante es generada por un sistema «en bucle cerrado» (1.2.13). Dado que el sistema en bucle cerrado ya no depende de ninguna entrada externa, decimos que tal sistema se ha hecho *autónomo*. Esto puede verse como un caso especial de autoregresión. Por ejemplo, si elegimos $\boldsymbol{u}_n = \boldsymbol{F}\boldsymbol{x}_n$ en el sistema lineal anterior (1.2.11), el sistema en bucle cerrado se transforma en

$$\boldsymbol{x}_{n+1} = \boldsymbol{A}\boldsymbol{x}_n + \boldsymbol{B}\boldsymbol{u}_n = \boldsymbol{A}\boldsymbol{x}_n + \boldsymbol{B}\boldsymbol{F}\boldsymbol{x}_n = (\boldsymbol{A} + \boldsymbol{B}\boldsymbol{F})\boldsymbol{x}_n, \tag{1.2.14}$$

lo que es una autoregresión lineal.

Procesos continuos. Las secuencias predecibles tienen contrapartes continuas naturales, que denominamos procesos predecibles. El proceso más simple de este tipo es el propio tiempo, $x(t) = t$.

Más generalmente, un proceso $\boldsymbol{x}(t)$ es *predecible* si, en cada instante t, su valor en el instante $t + \delta t$ se determina por su valor en t, donde δt es un incremento infinitesimal. Por lo general, $\boldsymbol{x}(t)$ es continuo y suave, por lo que la variación $\delta\boldsymbol{x}(t) = \boldsymbol{x}(t+\delta t) - \boldsymbol{x}(t)$ es infinitesimalmente pequeña. Estos procesos típicamente se describen mediante ecuaciones diferenciales (multivariables)

$$\dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t)), \quad \boldsymbol{x} \in \mathbb{R}^d. \tag{1.2.15}$$

En teoría de sistemas [**Cal:Des**; **Sastry-Nonlinear**], la ecuación (1.2.15) se conoce como modelo de espacio de estados. Un proceso controlado se modela por

$$\dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t), \boldsymbol{u}(t)), \quad \boldsymbol{x} \in \mathbb{R}^d, \boldsymbol{u} \in \mathbb{R}^k, \tag{1.2.16}$$

donde $\boldsymbol{u}(t)$ es una señal de entrada computable.

Ejemplo 1.1. La segunda ley de Newton determina la trayectoria $\boldsymbol{x}(t) \in \mathbb{R}^3$ de un cuerpo en movimiento sometido a una fuerza $\boldsymbol{F}(t) \in \mathbb{R}^3$:

$$m\ddot{\boldsymbol{x}}(t) = \boldsymbol{F}(t). \tag{1.2.17}$$

Cuando no hay fuerza, es decir, $\boldsymbol{F}(t) \equiv \boldsymbol{0}$, esto se reduce a la primera ley de Newton: el objeto se mueve a velocidad constante $\boldsymbol{v} \in \mathbb{R}^3$:

$$\ddot{\boldsymbol{x}}(t) = \boldsymbol{0} \iff \text{existe } \boldsymbol{v} \in \mathbb{R}^3 \text{ tal que } \dot{\boldsymbol{x}}(t) = \boldsymbol{v}. \tag{1.2.18}$$

■

1.2.2 Baja dimensión

Aprender a predecir. Ahora supongamos que se han observado o proporcionado muchos segmentos de la secuencia:

$$\{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_i, \ldots, \boldsymbol{x}_N\} \tag{1.2.19}$$

extraídos de una secuencia predecible $\{x_n\}_{n=1}^{\infty}$. Sin pérdida de generalidad, supongamos que cada segmento tiene una longitud $D \gg d$, de manera que

$$\boldsymbol{x}_i = [x_{j(i)}, x_{j(i)+1}, \ldots, x_{j(i)+D-1}]^\top \in \mathbb{R}^D \tag{1.2.20}$$

para algún $j \in \mathbb{N}$. Luego se nos da un nuevo segmento $\boldsymbol{x}_t$ y se nos pide predecir sus valores futuros.

La dificultad es que la función generadora f y su orden d no se conocen:

$$x_{n+d} = f(x_{n+d-1}, \ldots, x_n). \tag{1.2.21}$$

Por lo tanto, el objetivo es aprender f y d a partir de los segmentos de muestra $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N$. La tarea principal del aprendizaje de la predicción es:

Dados numerosos segmentos de muestra de una secuencia predecible, ¿cómo podemos identificar de manera eficaz y eficiente la función f?

Predecibilidad y baja dimensión. Para identificar la función predictiva f, podemos observar una característica común de los segmentos de cualquier secuencia predecible dada por (1.2.21). Si tomamos un segmento largo, digamos de longitud $D \gg d$, y lo consideramos como un vector

$$\boldsymbol{x}_i = [x_i, x_{i+1}, \ldots, x_{i+D-1}]^\top \in \mathbb{R}^D, \tag{1.2.22}$$

entonces el conjunto de todos esos vectores $\{\boldsymbol{x}_i\}$ dista mucho de ser aleatorio y no puede ocupar todo el espacio $\mathbb{R}^D$. En cambio, tiene como máximo d grados de libertad: una vez que se conocen las primeras d entradas de cualquier $\boldsymbol{x}_i$, las entradas restantes quedan determinadas de manera unívoca. En otras palabras, todos los vectores $\{\boldsymbol{x}_i\}$ se encuentran en una superficie de dimensión d. En

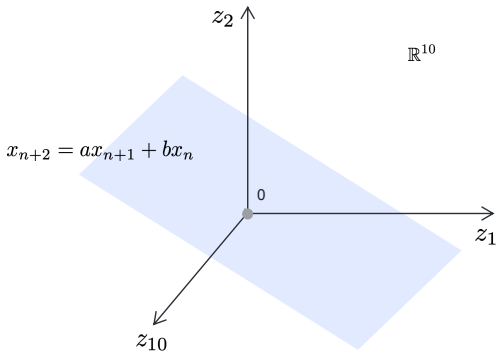


Figura 1.6: Un subespacio de dos dimensiones en un espacio ambiente de dimensión diez.

matemáticas, a una superficie de este tipo se le denomina subvariedad y se denota como $\mathcal{M} \subset \mathbb{R}^D$.

En la práctica, si elegimos una longitud de segmento D suficientemente grande, todos los segmentos muestreados a partir de la misma función predictiva se encuentran en una superficie con una dimensión intrínseca d, significativamente menor que la del espacio ambiente D. Por ejemplo, si la secuencia viene dada por la autoregresión lineal

$$x_{n+2} = ax_{n+1} + bx_n, \tag{1.2.23}$$

para constantes $a, b \in \mathbb{R}$ y si muestreamos segmentos de longitud $D = 10$, entonces todas las muestras se ubican en un plano de dimensión dos en $\mathbb{R}^{10}$, tal como se ilustra en la Figura 1.6. Al identificar este subespacio bidimensional se determinan completamente las constantes a y b en (1.2.23).

En términos más generales, cuando la función predictiva f es lineal, como en los sistemas que se muestran en (1.2.11) y (1.2.14), los segmentos largos siempre se encuentran en un subespacio lineal de baja dimensión. La identificación de la función predictiva equivale, en gran medida, a la identificación de este subespacio, un problema conocido como análisis de componentes principales. Discutiremos tales modelos y métodos clásicos en el Capítulo 2.

Esta observación se extiende a secuencias predecibles en general: si podemos identificar la superficie de baja dimensión en la que se encuentran las muestras del segmento, podemos identificar la función predictiva f.[9] No podemos dejar de subrayar la importancia de esta propiedad: *Todas las muestras de segmentos largos de una secuencia predecible se encuentran en una subvariedad de baja dimensión.* En gran medida, la ciencia y las matemáticas modernas tratan precisamente de identificar y describir esa estructura de baja dimensión de los

[9] Bajo condiciones moderadas, existe una correspondencia biunívoca entre la superficie de baja dimensión y la función f. Este hecho se ha aprovechado en problemas como la identificación de sistemas, que analizaremos más adelante.

17 Equations That Changed the World
by Ian Stewart

1.	**Pythagoras's Theorem**	$a^2 + b^2 = c^2$	Pythagoras,530 BC
2.	**Logarithms**	$\log xy = \log x + \log y$	John Napier, 1610
3.	**Calculus**	$\frac{\mathrm{d}f}{\mathrm{d}t} = \lim_{h\to 0} \frac{f(t+h) - f(t)}{h}$	Newton, 1668
4.	**Law of Gravity**	$F = G\frac{m_1 m_2}{r^2}$	Newton, 1687
5.	**The Square Root of Minus One**	$i^2 = -1$	Euler, 1750
6.	**Euler's Formula for Polyhedra**	$V - E + F = 2$	Euler, 1751
7.	**Normal Distribution**	$\Phi(x) = \frac{1}{\sqrt{2\pi\rho}} e^{\frac{(x-\mu)^2}{2\rho^2}}$	C.F. Gauss, 1810
8.	**Wave Equation**	$\frac{\partial^2 u}{\partial t^2} = c^2 \frac{\partial^2 u}{\partial x^2}$	J. d'Almbert, 1746
9.	**Fourier Transform**	$f(\omega) = \int_{\infty}^{\infty} f(x) e^{-2\pi i x \omega} \mathrm{d}x$	J. Fourier, 1822
10.	**Navier-Stokes Equation**	$\rho\left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v}\cdot\nabla\mathbf{v}\right) = -\nabla p + \nabla\cdot\mathbf{T} + \mathbf{f}$	C. Navier, G. Stokes, 1845
11.	**Maxwell's Equations**	$\nabla\cdot\mathbf{E} = \frac{\rho}{\varepsilon_0}$ $\quad \nabla\cdot\mathbf{H} = 0$ $\nabla\times\mathbf{E} = -\frac{1}{c}\frac{\partial \mathbf{H}}{\partial t}$ $\quad \nabla\times\mathbf{H} = \frac{1}{c}\frac{\partial E}{\partial t}$	J.C. Maxwell, 1865
12.	**Second Law of Thermodynamics**	$\mathrm{d}S \geq 0$	L. Boltzmann, 1874
13.	**Relativity**	$E = mc^2$	Einstein, 1905
14.	**Schrodinger's Equation**	$ih\frac{\partial}{\partial t}\Psi = H\Psi$	E. Schrodinger, 1927
15.	**Information Theory**	$H = -\sum p(x)\log p(x)$	C. Shannon, 1949
16.	**Chaos Theory**	$x_{t+1} = kx_t(1 - x_t)$	Robert May, 1975
17.	**Black-Scholes Equation**	$\frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS\frac{\partial V}{\partial S} + \frac{\partial V}{\partial t} - rV = 0$	F. Black, M. Scholes, 1990

Figura 1.7: 17 ecuaciones famosas que cambiaron el mundo. La mayoría de estas ecuaciones describe leyes físicas del universo. En cuanto a la inteligencia, la ecuación 15 de entropía jugará un papel muy especial y explicaremos por qué en este libro, especialmente en el capítulo 3.

fenómenos observados. Esto explica por qué los descubrimientos científicos más importantes y famosos acerca de nuestro mundo físico se describen a menudo mediante ecuaciones, tal y como ilustra la figura 1.7, ya que las ecuaciones restringen las cantidades observadas —que de otro modo serían independientes— a variedades de menor dimensión.

Según la teoría de cuerdas, que unifica casi todas las leyes físicas que conocemos sobre el mundo, todo el universo evoluciona en un espacio de 10 dimensiones, desde el Big Bang; véase la Figura 1.8. Como veremos en este libro, todos los métodos de aprendizaje modernos también explotan la propiedad de baja dimensión de los datos, ya sea de manera implícita o explícita.

Nota 1.1 (La entropía en física versus en inteligencia). En la figura 1.7, se observa que la única ley física que *no* se describe en forma de ecuación es la segunda ley de la termodinámica, la cual establece que la entropía de cualquier sistema físico cerrado, como el universo entero, siempre *aumenta*. En este libro se verá que

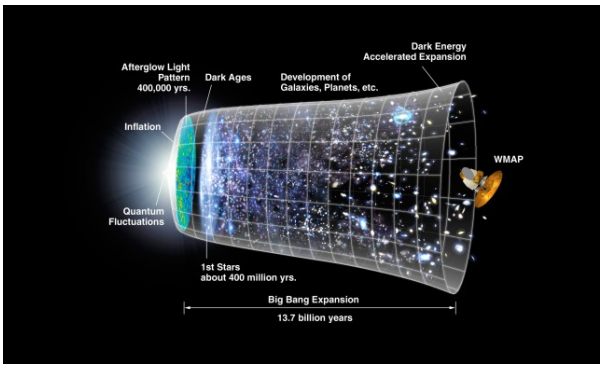


Figura 1.8: La evolución del Universo, según la mejor teoría física conocida hasta la fecha, se cree que tiene lugar en un espacio de 10 dimensiones.

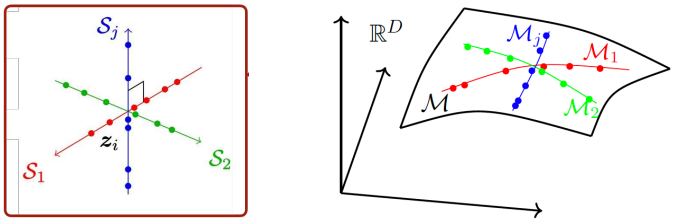


Figura 1.9: Datos distribuidos sobre una mezcla de subespacios (ortogonales) $\{\mathcal{S}_j\}$ (a la izquierda) o subvariedades $\{\mathcal{M}_j\}$ (a la derecha).

el objetivo de la inteligencia (y la ciencia) consiste en identificar estructuras en los fenómenos observados que puedan *reducir* la entropía en nuestra predicción. Esto se estudiará con bastante detalle en el capítulo 3 y en el **??**.

En situaciones reales, los datos observados suelen proceder de múltiples secuencias predecibles. Por ejemplo, una secuencia de video puede contener varios objetos en movimiento. En tales casos, los datos yacen sobre una mezcla de subespacios lineales de baja dimensión o subvariedades no lineales, tal como se ilustra en la Figura 1.9.

Propiedades de baja dimensión. La correlación temporal en secuencias predecibles no es la única razón de que los datos sean de baja dimensión. Por ejemplo, el espacio de todas las imágenes es enorme, pero en su mayor parte se compone de imágenes aleatorias sin estructura, como se muestra en la Figura 1.10 (a la izquierda). Las imágenes y los vídeos naturales, sin embargo, presentan un alto grado de redundancia debido a las fuertes correlaciones espaciales y temporales entre los valores de los píxeles. Esta redundancia nos permite identificar fácilmente si una imagen tiene ruido o está limpia, como se muestra en la Figura 1.10 (en la parte central y a derecha). En consecuencia, la



Figura 1.10: Una imagen de ruido aleatorio (a la izquierda) frente a una imagen con ruido (centro) y la imagen original sin ruido (a la derecha).

distribución de las imágenes naturales tiene una dimensión intrínseca muy baja en relación con el número total de píxeles.

Dado que el aprendizaje de estructuras de baja dimensión es tan importante como ubicuo, el libro *Análisis de datos de alta dimensión con modelos de baja dimensión: principios, cálculo y aplicaciones* [**Wright-Ma-2022**] comienza con la siguiente afirmación: «*El problema de identificar la estructura de baja dimensión de las señales o los datos en espacios de alta dimensión es uno de los problemas más fundamentales que, a lo largo de su larga historia, ha entrelazado numerosos campos de la ingeniería y las matemáticas, tales como la teoría de sistemas, el procesamiento de señales, el reconocimiento de patrones, el aprendizaje automático y la estadística.*»

Al restringir el punto de datos observado $\boldsymbol{x}$ a una superficie o subvariedad de baja dimensión, hacemos que sus componentes sean muy dependientes entre sí o estén correlacionadas y, en cierto sentido, sean «predecibles» a partir de los valores de las demás componentes. Por ejemplo, si sabemos que los datos están restringidos a una superficie de dimensión d en $\mathbb{R}^D$, podemos realizar varias tareas útiles más allá de la predicción:

- **autocompletado**: si se dan más de d entradas de una muestra típica $\boldsymbol{x}$, las entradas restantes suelen determinarse en forma unívoca;[10]
- **eliminación de ruido**: si los valores de una muestra $\boldsymbol{x}$ son perturbados por un ruido leve, este ruido puede eliminarse eficazmente proyectando $\boldsymbol{x}$ de vuelta sobre la superficie;
- **corrección de errores**: si un pequeño número de entradas desconocidas de $\boldsymbol{x}$ se corrompen en forma arbitraria, pueden corregirse de manera eficiente.

La Figura 1.11 ilustra estas propiedades usando una estructura lineal de baja dimensión—una línea de dimensión uno en un plano bidimensional.

En condiciones suaves, estas propiedades se generalizan a muchas otras estructuras de baja dimensión en espacios de alta dimensión

[10] La predicción se convierte en un caso especial de esta propiedad.

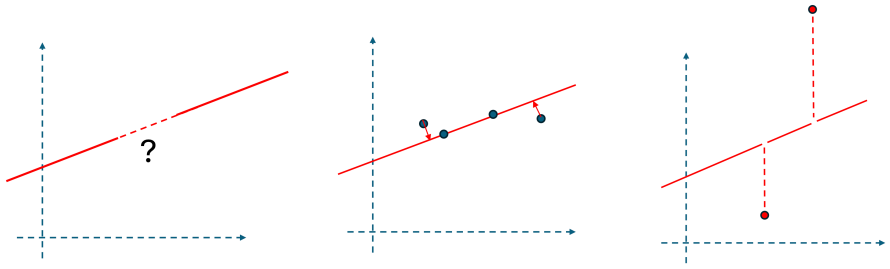


Figura 1.11: Ilustración de las propiedades de una estructura (lineal) de baja dimensión: permite el autocompletado (a la izquierda), la eliminación de ruido (al centro) y la corrección de errores (a la derecha).

[**Wright-Ma-2022**]. Veremos que estas propiedades útiles— por ejemplo, el autocompletado y la eliminación de ruido— inspiran el desarrollo de métodos eficaces en el aprendizaje de tales estructuras.

Para simplificar, hasta ahora hemos utilizado el caso determinista para introducir los conceptos de predictibilidad y baja dimensión, en el cual los datos se sitúan exactamente sobre estructuras geométricas como subespacios o superficies. Sin embargo, en la práctica, los datos siempre contienen cierta incertidumbre o aleatoriedad. En este caso, podemos suponer que los datos siguen una distribución probabilística con densidad $p(\boldsymbol{x})$. Una distribución se considera «de baja dimensión» si su densidad se concentra en torno a una estructura geométrica de baja dimensión—un subespacio, una superficie o una combinación de ambos— tal y como se muestra en la Figura 1.9. Esta estructura también se conoce como el «soporte» de la distribución. Una vez aprendida, esta densidad $p(\boldsymbol{x})$ sirve como una potente distribución a priori para estimar $\boldsymbol{x}$ a partir de observaciones parciales, ruidosas o corruptas:

$$\boldsymbol{y} = f(\boldsymbol{x}) + \boldsymbol{n}, \tag{1.2.24}$$

calculando la estimación condicional $\hat{\boldsymbol{x}}(\boldsymbol{y}) = \mathbb{E}(\boldsymbol{x} \mid \boldsymbol{y})$ o mediante el muestreo de la distribución condicional $\hat{\boldsymbol{x}}(\boldsymbol{y}) \sim p(\boldsymbol{x} \mid \boldsymbol{y})$.[11]

En numerosos trabajos anteriores se han estudiado las propiedades de conjuntos de datos cuyas distribuciones se apoyan en una superficie o subvariedad de baja dimensión, por ejemplo, [**ALLARD2012435**; **LITTLE2017504**]. Nos gustaría destacar aquí su importancia fundamental y la planteamos como el único supuesto en el que este libro basa su enfoque deductivo para comprender las redes profundas y la inteligencia en general:

> **El supuesto principal:** *Cualquier sistema inteligente o método de aprendizaje debería (y puede) basarse en la predictibilidad del*

[11] Las tecnologías modernas de IA generativa, como la generación condicionada de imágenes, se basan en gran medida en este principio, tal y como veremos en el **??**

mundo; por lo tanto, la distribución de las muestras de datos de alta dimensión observadas tiene un soporte de baja dimensión.

La pregunta que queda por resolver es cómo aprender estas estructuras de baja dimensión de manera correcta y computacionalmente eficiente a partir de datos de alta dimensión. Como veremos, los modelos paramétricos estudiados en los enfoques analíticos clásicos y los modelos no paramétricos, como las redes profundas, tan populares en la práctica actual, no son más que diferentes medios para alcanzar un mismo fin.

1.3 ¿Cómo aprender?

1.3.1 Enfoques analíticos basados en modelos

Tengamos en cuenta que, aunque el cálculo de una función predictiva sea factible, eso no implica que sea factible o escalable aprender dicha función a partir de un número finito de segmentos muestreados. Un enfoque clásico para garantizar la factibilidad consiste en formular supuestos explícitos sobre la familia de estructuras de baja dimensión con la que estamos trabajando. Históricamente, debido a las limitaciones en materia de cálculo y datos, los primeros modelos analíticos que se estudiaron fueron modelos simples e idealizados, ya que a menudo ofrecen soluciones eficientes en forma cerrada o numéricas. Además, ofrecen nuevas luces sobre problemas más generales y ya aportan soluciones útiles para casos importantes, aunque limitados. En el pasado, cuando los recursos computacionales eran escasos, los modelos analíticos que permitían soluciones eficientes en forma cerrada o soluciones numéricas eran los únicos que se podían implementar. Las *estructuras lineales* fueron la primera clase de modelos que se estudió a fondo.

Por ejemplo, podría decirse que el caso más sencillo es suponer que los datos se distribuyen en torno a un único subespacio de baja dimensión en un espacio de alta dimensión. De manera más o menos equivalente, se puede suponer que los datos se distribuyen según una distribución gaussiana de baja dimensión casi degenerada. Identificar dicho subespacio o gaussiana a partir de un número finito de muestras (con ruido) constituye el problema clásico del análisis de componentes principales (PCA) y se han desarrollado algoritmos eficaces para esta clase de modelos [**JolliffeI2002**]. La familia de modelos puede hacerse cada vez más compleja y expresiva. Asimismo se puede suponer que los datos se distribuyen en torno a una mezcla de componentes de baja dimensión (subespacios o gaussianas de baja dimensión), como en el análisis de componentes independientes (ICA) [**Ans-1985**], el aprendizaje de diccionarios (DL), el análisis de componentes principales generalizado (GPCA) [**Vidal-GPCA**] o la clase más general de modelos esparsos de baja dimensión que se han estudiado extensivamente en los últimos años en campos como la detección compresiva [**Wright-Ma-2022**].

En todas estas familias de modelos analíticos, el problema central consiste en identificar el modelo más compacto dentro de cada familia que mejor se ajuste

a los datos proporcionados. A continuación, ofrecemos una breve descripción de estos modelos analíticos clásicos, pero dejamos un estudio más sistemático para el Capítulo 2. En teoría, estos modelos analíticos han ofrecido profundas perspectivas sobre las propiedades geométricas y estadísticas de las estructuras de baja dimensión. A menudo proporcionan soluciones en forma cerrada o algoritmos eficientes y escalables, que resultan muy útiles para datos cuya distribución puede aproximarse bien mediante dichos modelos. Y lo que es más importante, en el caso de problemas más generales, dan una idea de lo fácil o difícil que puede resultar identificar estructuras de baja dimensión y cuáles son las ideas básicas para abordar ese tipo de problemas.

Sistemas dinámicos lineales

El filtro de Wiener. Como se explicó en la Sección 1.2.1, una tarea central de la inteligencia consiste en aprender qué es lo predecible en las secuencias de observaciones. La clase más simple de secuencias predecibles —o señales— es la generada por un proceso *lineal e invariante en el tiempo* (LTI):

$$x[n] = h[n] * z[n] + \epsilon[n], \tag{1.3.1}$$

donde $*$ es la operación de convolución, z es la entrada y h es la función de respuesta al impulso.[12] Aquí, $\epsilon[n]$ denota el ruido aditivo de la observación. Dado el proceso de entrada $\{z[n]\}$ y las observaciones del proceso de salida $\{x[n]\}$, el objetivo es encontrar el $h[n]$ óptimo tal que $\hat{x}[n] = h[n] * z[n]$ prediga $x[n]$ de manera óptima. La calidad de la predicción (es decir, su bondad) se mide mediante el error cuadrático medio (MSE):

$$\min_h \mathbb{E}[\epsilon[n]^2] = \mathbb{E}[\|x[n] - h[n] * z[n]\|_2^2]. \tag{1.3.2}$$

La solución óptima $h[n]$ se denomina filtro (de eliminación de ruido). Norbert Wiener —quien también fue quien inició el movimiento de Cibernética— estudió este problema en la década de 1940 y derivó una elegante solución en forma cerrada conocida como el *filtro de Wiener* [**Wiener-1942**; **Wiener-1949**]. Este resultado, también conocido como estimación y filtrado por mínima varianza, se convirtió en uno de los pilares del procesamiento de señales. Los lectores interesados pueden consultar [**MaY2003**] para obtener una derivación detallada de este tipo de estimador.

El filtro de Kalman. La idea de atenuar el ruido en un sistema dinámico o filttrarlo fue posteriormente ampliada por Rudolph Kalman en la década de 1960 al caso de un sistema lineal invariante en el tiempo descrito por un modelo de espacio de estados de dimensión finita:

$$\boldsymbol{z}[n] = \boldsymbol{A}\boldsymbol{z}[n-1] + \boldsymbol{B}\boldsymbol{u}[n] + \boldsymbol{\epsilon}[n]. \tag{1.3.3}$$

[12] Por lo general, se supone que h tiene ciertas propiedades deseables, como una longitud finita o un espectro de banda limitada.

El problema consiste en estimar el estado del sistema $\boldsymbol{z}[n]$ a partir de observaciones con ruido de la forma

$$\boldsymbol{x}[n] = \boldsymbol{C}\boldsymbol{z}[n] + \boldsymbol{w}[n], \tag{1.3.4}$$

donde $\boldsymbol{w}$ es ruido (blanco). El estimador de estado causal[13] óptimo que minimiza el error medio cuadrático mínimo (EMCM) de predicción

$$\min \mathbb{E}[\|\boldsymbol{x}[n] - \boldsymbol{C}\boldsymbol{z}[n]\|_2^2] \tag{1.3.5}$$

se expresa en forma cerrada mediante el denominado *filtro de Kalman* [**kalman1960new**]. Este resultado constituye una piedra angular de la teoría moderna de control, ya que permite estimar el estado de un sistema dinámico a partir de observaciones con ruido. Con ello, se puede introducir una retroalimentación lineal de estado, por ejemplo, $\boldsymbol{u}[n] = \boldsymbol{F}\hat{\boldsymbol{z}}[n]$, y hacer que el sistema en bucle cerrado sea totalmente autónomo, tal y como se muestra en la Ecuación (1.2.13). Los lectores interesados pueden consultar [**MaY2003**] para ver una derivación detallada del filtro de Kalman.

La identificación de sistemas dinámicos lineales. Para derivar el filtro de Kalman, se supone que los parámetros del sistema $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$ son conocidos. Si no se proporcionan de antemano, el problema se vuelve más complejo y se conoce como *identificación de sistemas*: cómo *aprender* los parámetros $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$ a partir de numerosas muestras de la secuencia de entrada $\{\boldsymbol{u}[n]\}$ y de la secuencia de observación $\{\boldsymbol{x}[n]\}$. Este es un problema clásico de la teoría de sistemas. Si el sistema es lineal, las secuencias de entrada y salida $\{\boldsymbol{u}[n], \boldsymbol{x}[n]\}$ se encuentran conjuntamente en un determinado subespacio de baja dimensión[14]. Por lo tanto, el problema de identificación equivale esencialmente a identificar este subespacio de baja dimensión [**OverscheeP1996**; **Liu-2009-CDC**; **Liu-2010-SIAM**].

Cabe señalar que los problemas anteriores tienen dos cosas en común: en primer lugar, se supone que las secuencias o señales sin ruido son generadas por una familia explícita de modelos paramétricos; en segundo lugar, estos modelos son esencialmente lineales. Conceptualmente, sea $\boldsymbol{x}_o$ una variable aleatoria cuya distribución «verdadera» está soportada sobre un subespacio lineal de baja dimensión, digamos S. En gran medida, tanto el filtro de Wiener como el filtro de Kalman tratan de estimar dicho $\boldsymbol{x}_o$ a partir de sus observaciones ruidosas:

$$\boldsymbol{x} = \boldsymbol{x}_o + \boldsymbol{\epsilon}, \quad \boldsymbol{x}_o \sim S, \tag{1.3.6}$$

donde $\boldsymbol{\epsilon}$ suele ser un ruido (o proceso) aleatorio gaussiano. Por lo tanto, sus soluciones se basan en identificar un subespacio lineal de baja dimensión que se ajuste mejor a los datos ruidosos observados. Al proyectar los datos sobre este subespacio, se obtienen las operaciones óptimas de eliminación de ruido, todas en forma cerrada.

[13] Esto significa que el estimador solo puede utilizar observaciones hasta el paso temporal actual n. El filtro de Kalman es siempre causal, mientras que el filtro de Wiener no tiene por qué serlo.

[14] que tiene la misma dimensión que el orden del modelo de espacio de estados (1.3.3).

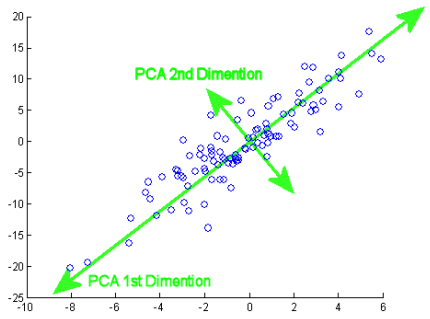


Figura 1.12: Una distribución con dos componentes principales.

Modelos lineales y lineales mixtos

Análisis de componentes principales. A partir de los problemas expuestos en el procesamiento clásico de señales y la identificación de sistemas, vemos que la tarea principal que subyace a todos estos problemas consiste en aprender un *único* subespacio lineal de baja dimensión a partir de observaciones con ruido. Matemáticamente, podemos modelar dicha estructura como

$$\boldsymbol{x} = \boldsymbol{u}_1 z_1 + \boldsymbol{u}_2 z_2 + \cdots + \boldsymbol{u}_d z_d + \boldsymbol{\epsilon} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\epsilon}, \quad \boldsymbol{U} \in \mathbb{R}^{D\times d}, \tag{1.3.7}$$

donde $\boldsymbol{\epsilon} \in \mathbb{R}^D$ es un ruido aleatorio de pequeña magnitud. La Figura 1.12 ilustra una distribución de este tipo con dos componentes principales. El problema consiste en encontrar la base del subespacio $\boldsymbol{U}$ a partir de numerosas muestras de $\boldsymbol{x}$. Un enfoque habitual consiste en minimizar el error de la proyección sobre el subespacio:

$$\min_{\boldsymbol{U}} \mathbb{E}[\|\boldsymbol{x} - \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{z}\|_2^2]. \tag{1.3.8}$$

En esencia, se trata de una tarea de eliminación de ruido: una vez que se ha encontrado correctamente la base $\boldsymbol{U}$, podemos eliminar el ruido de la muestra ruidosa $\boldsymbol{x}$ proyectándola sobre el subespacio de baja dimensión generado por $\boldsymbol{U}$:

$$\boldsymbol{x} \to \hat{\boldsymbol{x}} = \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{x}. \tag{1.3.9}$$

Si el ruido es pequeño y el subespacio de baja dimensión $\boldsymbol{U}$ se aprende correctamente, esperamos que $\boldsymbol{x} \approx \hat{\boldsymbol{x}}$. Por lo tanto, PCA es un caso especial de lo que se conoce como esquema de «autocodificador», que codifica los datos proyectándolos sobre un espacio de menor dimensión, y decodifica un código de menor dimensión para volver al espacio original:

$$\boldsymbol{x} \xrightarrow{\boldsymbol{U}^\top} \boldsymbol{z} \xrightarrow{\boldsymbol{U}} \hat{\boldsymbol{x}}. \tag{1.3.10}$$

Debido a la sencillez de la estructura de datos, tanto el codificador $\mathcal{E}$ como el decodificador $\mathcal{D}$ se convierten en operadores lineales simples (de proyección y elevación).

Este problema clásico de la estadística se conoce como análisis de componentes principales (PCA). Fue estudiado por primera vez por Pearson en 1901 [**Pearson1901**] y, posteriormente, de manera independiente por Hotelling en 1933 [**Hotelling1933**]. El tema se resume en forma sistemática en el libro clásico [**Jolliffe1986**; **JolliffeI2002**]. También se puede suponer explícitamente que los datos $\boldsymbol{x}$ se distribuyen según una única gaussiana de baja dimensión:

$$\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{U}\boldsymbol{U}^\top + \sigma \boldsymbol{I}), \quad \boldsymbol{U} \in \mathbb{R}^{D \times d}, \tag{1.3.11}$$

lo cual equivale a suponer que $\boldsymbol{z}$ en el modelo PCA (1.3.7) sigue una distribución normal estándar. Este problema se conoce como PCA probabilístico [**TippingM1999**] y tiene la misma solución computacional que el PCA.

En este libro reexaminamos el PCA en el Capítulo 2 desde la perspectiva del aprendizaje de una distribución de baja dimensión. Nuestro objetivo es utilizar este modelo sencillo e idealizado para transmitir algunas de las ideas más fundamentales para el aprendizaje de una representación compacta de una distribución de baja dimensión, incluyendo conceptos importantes como la compresión mediante la eliminación de ruido y la autocodificación para obtener una representación coherente.

Análisis de componentes independientes. El análisis de componentes independientes (ICA) fue propuesto originalmente por [**Ans-1985**] como un modelo clásico para *aprender una buena representación*. De hecho, en un principio se propuso como un simple modelo matemático de nuestra memoria. El modelo ICA adopta una forma aparentemente similar a la del modelo PCA mencionado anteriormente (1.3.7), al suponer que la variable aleatoria observada $\boldsymbol{x}$ es una superposición lineal de múltiples componentes independientes z_i:[15]

$$\boldsymbol{x} = \boldsymbol{u}_1 z_1 + \boldsymbol{u}_2 z_2 + \cdots + \boldsymbol{u}_d z_d + \boldsymbol{\epsilon} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\epsilon}. \tag{1.3.12}$$

Sin embargo, en este caso se supone que las componentes z_i son variables independientes *no gaussianas*. Por ejemplo, una opción muy utilizada es

$$z_i = \sigma_i \cdot w_i, \quad \sigma_i \sim \text{Ber}(p), \tag{1.3.13}$$

donde σ_i es una variable aleatoria de Bernoulli y w_i podría ser un valor constante u otra variable aleatoria, por ejemplo, una gaussiana.[16] El problema ICA tiene como objetivo identificar tanto $\boldsymbol{U}$ como $\boldsymbol{z}$ a partir de muestras observadas de $\boldsymbol{x}$. La Figura 1.13 ilustra la diferencia entre el ICA y el PCA.

Aunque el mapeo de $\boldsymbol{z}$ a $\boldsymbol{x}$ (la decodificación) parece lineal y sencilla una vez que se han aprendido $\boldsymbol{U}$ y $\boldsymbol{z}$, el mapeo de $\boldsymbol{x}$ a $\boldsymbol{z}$ (la codificación) puede ser complicado y es posible que no se pueda representar mediante un simple mapeo lineal. Por lo tanto, el ICA generalmente proporciona una autocodificación de la forma:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}} \boldsymbol{z} \xrightarrow{\boldsymbol{U}} \hat{\boldsymbol{x}}. \tag{1.3.14}$$

[15] En el PCA, las «componentes» son los vectores aprendidos, mientras que en el ICA son escalares. Se trata simplemente de una diferencia de nomenclatura y de convención.

[16] ¡Aunque w sea gaussiana, σw ya no es una variable gaussiana!

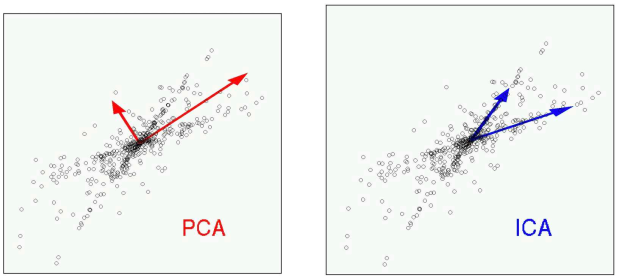


Figura 1.13: PCA (a la izquierda) versus ICA (a la derecha). Obsérvese que el PCA encuentra los vectores *ortogonales* que abarcan aproximadamente los datos, mientras que el ICA encuentra vectores (no necesariamente ortogonales) que se combinan *de manera independiente* para formar aproximadamente los datos, así como los coeficientes de esta combinación.

Por lo tanto, a diferencia del PCA, el ICA es algo más difícil de analizar y resolver. En la década de 1990, investigadores como Erkki Oja y Aapo Hyvärinen [**hyvarinen-1997**; **Hyvrinen-2000**] realizaron importantes contribuciones teóricas y algorítmicas al ICA. En el Capítulo 2, estudiaremos y presentaremos una solución para el ICA a partir de la cual quedará claro el mapeo de codificación $\mathcal{E}$. De hecho, Aapo Hyvärinen propuso el método general de *score matching* en 2005 [**hyvarinen05a**], en parte para aprender distribuciones tales como las mezclas gaussianas, como en el ICA. Abordaremos este tema con mayor detalle más adelante y de nuevo en el 3.

Estructuras esparsas y detección compresiva. Si p en (1.3.13) es muy pequeño, la probabilidad de que cualquier componente sea distinto de cero es baja. En este caso, decimos que $\boldsymbol{x}$ es un vector generado de manera esparsa y que se concentra en una unión de subespacios lineales cuya dimensión es $k = p \cdot d$. Por lo tanto, podemos ampliar el modelo ICA a una familia más general de estructuras de baja dimensión conocidas como modelos esparsos.

Un modelo k-esparso es el conjunto de todos los *vectores k-esparsos*:

$$\mathcal{Z} = \{\boldsymbol{z} \in \mathbb{R}^n \mid \|\boldsymbol{z}\|_0 \leq k\}, \tag{1.3.15}$$

donde $\|\cdot\|_0$ cuenta el número de entradas distintas de cero. Por lo tanto, $\mathcal{Z}$ es la unión de todos los subespacios de dimensión k alineados con los ejes de coordenadas, tal como se ilustra en la Figura 1.9 (a la izquierda). Un problema fundamental en el procesamiento de señales y la estadística consiste en recuperar un vector esparso $\boldsymbol{z}$ a partir de sus observaciones lineales

$$\boldsymbol{x} = \boldsymbol{A}\boldsymbol{z} + \boldsymbol{\epsilon}, \quad \boldsymbol{A} \in \mathbb{R}^{m \times n}, \tag{1.3.16}$$

donde $\boldsymbol{A}$ es dado, normalmente $m < n$, y $\boldsymbol{\epsilon}$ es un ruido pequeño. Este problema, que a primera vista parece sencillo, es NP-difícil de resolver e incluso difícil de aproximar (véase [**Wright-Ma-2022**] para más detalles).

A pesar de su rica historia, que se remonta al siglo XVIII [**Boscovichca1750**], no existía ningún algoritmo de eficacia probada para esta clase de problemas, aunque entre los años de 1960 y 1990 se propusieron muchos algoritmos heurísticos. Algunos resultaron eficaces en la práctica, pero carecían de una justificación rigurosa. Se produjo un gran avance a principios de la década de 2000, cuando matemáticos como David Donoho, Emmanuel Candés y Terence Tao [**donoho2005neighborly**; **Candes2005**; **CandesE2005-IT**] establecieron un marco teórico riguroso que caracteriza las condiciones precisas bajo las cuales el problema de la recuperación esparsa puede resolverse de manera efectiva y eficiente mediante la minimización convexa de ℓ^1:

$$\min \|\boldsymbol{z}\|_1 \quad \text{subject to} \quad \|\boldsymbol{x} - \boldsymbol{A}\boldsymbol{z}\|_2 \leq \epsilon, \tag{1.3.17}$$

donde $\|\cdot\|_1$ es la norma ℓ^1 de promoción de esparcidad de un vector y ϵ es una constante pequeña. Cualquier solución da como resultado una (auto) codificación esparsa

$$\boldsymbol{x} \xrightarrow{\mathcal{E}} \boldsymbol{z} \xrightarrow{\boldsymbol{A}} \hat{\boldsymbol{x}}. \tag{1.3.18}$$

Describiremos dicho algoritmo (y, por lo tanto, dicha correspondencia) en el Capítulo 2, revelando conexiones fundamentales entre la codificación esparsa y el aprendizaje profundo.[17]

Las condiciones para que la minimización de ℓ^1 tenga éxito son sorprendentemente generales: el número mínimo de mediciones m necesarias para una recuperación exitosa solo es proporcional a la dimensión intrínseca k. Este es el fenómeno de la *detección compresiva* [**CandesE2006-ICM**]. Se extiende a amplias familias de estructuras de baja dimensión, incluidas las matrices de bajo rango. Estos resultados cambiaron radicalmente nuestra comprensión de la recuperación de estructuras de baja dimensión en espacios de alta dimensión. David Donoho, entre otros, celebró este cambio de tendencia como la «bendición de la dimensionalidad» [**DonohoD2000**], en contraste con la habitual creencia pesimista en la «maldición de la dimensionalidad». La teoría completa y el conjunto de resultados se presentan en [**Wright-Ma-2022**].

No se puede subestimar la importancia computacional de este marco. Transformó problemas que antes se consideraban intratables en otros que no solo son abordables, sino que pueden resolverse de manera escalable mediante algoritmos extremadamente eficientes:

$$\text{intratable} \implies \text{tratable} \implies \text{escalable}. \tag{1.3.19}$$

Los algoritmos cuentan con rigurosas garantías teóricas de corrección, siempre que se cumplan unos requisitos precisos en cuanto a datos y cálculo. La naturaleza deductiva de este enfoque contrasta marcadamente con la práctica empírica e inductiva de las redes neuronales profundas. Sin embargo, ahora sabemos que ambos enfoques comparten un objetivo común: la búsqueda de estructuras de baja dimensión en espacios de alta dimensión.

[17] Ya en 2010 se señalaron similitudes entre los algoritmos de codificación esparsa y las redes profundas [**gregor2010learning**].

Aprendizaje de diccionarios. Desde el punto de vista conceptual, surge un problema aún más complejo que el de la codificación esparsa (1.3.16) cuando la matriz de observación $\boldsymbol{A}$ es desconocida y debe aprenderse a partir de un conjunto de observaciones (posiblemente ruidosas) $\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n]$:

$$\boldsymbol{X} = \boldsymbol{A}\boldsymbol{Z} + \boldsymbol{E}, \quad \boldsymbol{A} \in \mathbb{R}^{m \times n}. \tag{1.3.20}$$

Aquí solo se nos da $\boldsymbol{X}$, no el correspondiente $\boldsymbol{Z} = [\boldsymbol{z}_1, \boldsymbol{z}_2, \ldots, \boldsymbol{z}_n]$ ni el término de ruido $\boldsymbol{E} = [\boldsymbol{\epsilon}_1, \boldsymbol{\epsilon}_2, \ldots, \boldsymbol{\epsilon}_n]$, salvo que se sabe que los $\boldsymbol{z}_i$ son esparsos. Este es el *problema del aprendizaje de diccionarios*, que generaliza el problema del ICA (1.3.12) comentado anteriormente. En otras palabras, dado que la distribución de los datos $\boldsymbol{X}$ es la imagen de una distribución esparsa estándar $\boldsymbol{Z}$ bajo una transformación lineal $\boldsymbol{A}$, queremos aprender tanto $\boldsymbol{A}$ como su mapeo «inverso» $\mathcal{E}$ para obtener un autocodificador:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z} \xrightarrow{\boldsymbol{A}} \hat{\boldsymbol{X}}. \tag{1.3.21}$$

Tanto el PCA como el ICA y el aprendizaje de diccionarios parten del supuesto de que la distribución de los datos se apoya en estructuras lineales o lineales por tramos de baja dimensión, o se aproxima a ellas. Cada método requiere aprender una base (global o local) para estas estructuras a partir de muestras con ruido. En el Capítulo 2 estudiamos cómo identificar este tipo de estructuras de baja dimensión mediante estos modelos clásicos. En particular, veremos que todos estos modelos de baja dimensión, lineales o lineales por tramos, pueden aprenderse de manera eficiente mediante la misma familia de algoritmos rápidos conocida como *iteración de potencias* [**Zhai-2020**]. Aunque estos modelos lineales o lineales por tramos son algo idealizados para la mayoría de los datos del mundo real, comprenderlos es un primer paso esencial para comprender distribuciones de baja dimensión más generales.

Distribuciones generales

Las distribuciones de los datos del mundo real, como imágenes, vídeos y audio, son demasiado complejas como para ser modeladas mediante los modelos lineales o los procesos gaussianos citados anteriormente, que están, en cierta medida, idealizados. Normalmente no sabemos *a priori* de qué familia de modelos paramétricos se derivan. A lo largo de la historia, se han realizado numerosos intentos por desarrollar modelos analíticos para estos datos. En concreto, el ganador de la Medalla Fields, David Mumford, en la década de 1990 dedicó un esfuerzo considerable a intentar comprender y modelar las estadísticas de las imágenes naturales [**Mumford1996TheSD**]. Él y sus alumnos, entre ellos Song-Chun Zhu, se inspiraron en la física estadística y adoptaron sus técnicas, proponiendo numerosos modelos estadísticos y estocásticos para la distribución de imágenes naturales, tales como campos aleatorios o procesos estocásticos [**Zhu-Entropy-1997**; **Zhu1997LearningGP**; **Zhu1997Prior**; **Huang-Mumford**; **Mumford-1999**; **Lee-Mumford**]. Sin embargo, estos modelos analíticos solo lograron un éxito limitado a la hora de generar muestras

que se asemejaran mucho a las imágenes naturales. Es evidente que, para datos del mundo real como las imágenes, necesitamos desarrollar métodos más potentes y unificadores que permitan identificar sus estructuras de baja dimensión más generales y obtener representaciones eficientes, por ejemplo, con fines de codificación y decodificación [**ALLARD2012435**; **LITTLE2017504**].

En la práctica, en el caso de una distribución general de datos del mundo real, por lo general solo disponemos de muchas muestras de dicha distribución: la llamada *distribución empírica*. En tales casos, normalmente no podemos esperar contar con una forma analítica clara para sus estructuras de baja dimensión, ni para los operadores de eliminación de ruido resultantes.[18] Por lo tanto, necesitamos desarrollar una solución más general para estas distribuciones empíricas, no necesariamente en forma cerrada, pero al menos que se pueda calcular de manera eficiente. Si lo hacemos correctamente, las soluciones de los modelos lineales antes mencionados deberían convertirse en sus casos particulares.

Eliminación de ruido. En la década de 1950, los estadísticos comenzaron a interesarse por el problema de reducir el ruido en los datos muestreados de una distribución arbitraria. Sea $\boldsymbol{x}_o$ una variable aleatoria con función de densidad de probabilidad $p_o(\cdot)$. Supongamos que observamos una versión ruidosa de vx_o:

$$\boldsymbol{x} = \boldsymbol{x}_o + \sigma \boldsymbol{g}, \tag{1.3.22}$$

donde $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ es un ruido gaussiano estándar y σ es el nivel de ruido de la observación. Sea $p(\cdot)$ la función de densidad de probabilidad de $\boldsymbol{x}$.[19] Cabe destacar que la esperanza a posteriori de $\boldsymbol{x}_o$ dada $\boldsymbol{x}$ se puede calcular mediante una elegante fórmula, conocida como fórmula de Tweedie [**Robbins1956AnEB**]:[20]

$$\hat{\boldsymbol{x}}_o = \mathbb{E}[\boldsymbol{x}_o \mid \boldsymbol{x}] = \boldsymbol{x} + \sigma^2 \nabla \log p(\boldsymbol{x}). \tag{1.3.23}$$

Como se puede observar en la fórmula, la función $\nabla \log p(\boldsymbol{x})$ desempeña un papel muy especial en la reducción del ruido de la observación $\boldsymbol{x}$ en este caso. El ruido $\boldsymbol{g}$ se puede estimar explícitamente como

$$\hat{\boldsymbol{g}} = \frac{\boldsymbol{x} - \hat{\boldsymbol{x}}_o}{\sigma} = -\sigma \nabla \log p(\boldsymbol{x}), \tag{1.3.24}$$

para lo cual solo necesitamos conocer la distribución $p(\cdot)$ de $\boldsymbol{x}$, y no la distribución real $p_o(\cdot)$ para $\boldsymbol{x}_o$. Una consecuencia importante de este resultado es que, si añadimos ruido gaussiano a cualquier distribución, el proceso de eliminación de ruido puede llevarse a cabo fácilmente si logramos obtener de alguna manera la función $\nabla \log p(\boldsymbol{x})$.

[18] Esto difiere de los casos del PCA, el filtro de Wiener y el filtro de Kalman.

[19] Es decir, $p(\boldsymbol{x}) = \int_{-\infty}^{\infty} \varphi_\sigma(\boldsymbol{x} - \boldsymbol{z}) p_o(\boldsymbol{z}) \mathrm{d}\boldsymbol{z}$, donde φ_σ es la función de densidad de la distribución gaussiana $\mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$.

[20] Herbert Robbins atribuyó el mérito de esta fórmula a Maurice Kenneth Tweedie, según se desprende de su correspondencia personal.

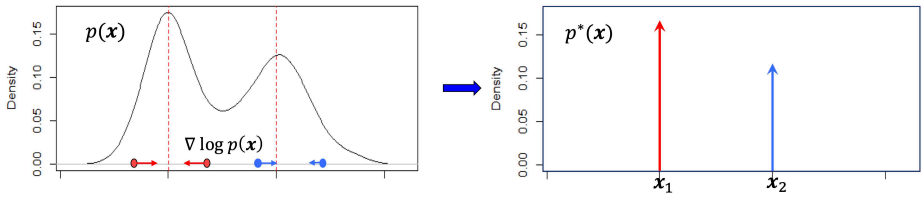


Figura 1.14: Interpretación geométrica de una función de *score* $\nabla \log p(\boldsymbol{x})$ para una distribución con densidad $p(\boldsymbol{x})$ a la izquierda. La operación generada por la función de *score* empuja la distribución hacia las zonas de mayor densidad. El objetivo es que, según una determinada medida de compacidad (por ejemplo, la entropía o la longitud de codificación), la distribución resultante esté más «comprimida». En el límite, la distribución converge hacia una que tiene un soporte de menor dimensión, como la $p^*(\boldsymbol{x})$ mostrada a la derecha.

Dado que se trata de un resultado tan importante y útil, ha sido redescubierto y utilizado en muchos contextos y ámbitos diferentes. Por ejemplo, tras la fórmula de Tweedie [**Robbins1956AnEB**], fue redescubierta unos años más tarde por [**Miyasawa61**], donde se denominó «eliminación de ruido bayesiana empírica». A principios de la década de 2000, la función $\nabla \log p(\boldsymbol{x})$ fue redescubierta en el contexto del aprendizaje de una distribución general y Aapo Hyvärinen [**hyvarinen05a**] la denominó «función de *score*». Su relación con la eliminación de ruido (bayesiana empírica) fue reconocida poco después por [**Vincent2011**]. El grupo de Eero Simoncelli ha realizado generalizaciones a otras distribuciones de medición (más allá del ruido gaussiano) [**Raphan10**]. Hoy en día, la aplicación más directa de la fórmula de Tweedie y la eliminación de ruido es la generación de imágenes mediante la eliminación iterativa de ruido [**Kadkhodaie21a**; **ho2020denoising**].

Minimización de entropía. El score tiene una interpretación intuitiva desde el punto de vista de la teoría de la información y la geometría. En teoría de la información, $-\log p(\boldsymbol{x})$ corresponde al número de bits que se necesitan para codificar $\boldsymbol{x}$[21]. El gradiente $\nabla \log p(\boldsymbol{x})$ apunta hacia regiones de mayor densidad de probabilidad, como se muestra en la Figura 1.14 (a la izquierda). Moverse en esta dirección reduce el número de bits necesarios para codificar $\boldsymbol{x}$. Por lo tanto, el operador $\nabla \log p(\boldsymbol{x})$ empuja a la distribución a «encogerse» hacia regiones de alta densidad. Formalmente, se puede demostrar que la entropía (diferencial)

$$H(\boldsymbol{x}) = -\int p(\boldsymbol{w}) \log p(\boldsymbol{w}) \mathrm{d}\boldsymbol{w} \tag{1.3.25}$$

disminuye bajo esta operación (véanse el Capítulo 3 y el **??**). Con un libro de códigos óptimo, la distribución resultante alcanza una tasa de codificación más baja y, por lo tanto, está más comprimida. Si se repite este proceso de

[21] al menos para variables discretas, como se detalla en el Capítulo 3.

eliminación de ruido indefinidamente, se obtiene una distribución cuya masa de probabilidad se concentra en un soporte de menor dimensión. Por ejemplo, la distribución $p(\boldsymbol{x})$ en la Figura 1.14 (a la izquierda) converge a $p^*(\boldsymbol{x})$ (a la derecha):[22][23]

$$H(\boldsymbol{x}) = -\int p(\boldsymbol{w}) \log p(\boldsymbol{w}) \mathrm{d}\boldsymbol{w} \quad \xrightarrow{\text{disminuyendo}} \quad H^*(\boldsymbol{x}) = -\int p^*(\boldsymbol{w}) \log p^*(\boldsymbol{w}) \mathrm{d}\boldsymbol{w}. \tag{1.3.26}$$

A medida que la distribución converge hacia $p^*(\boldsymbol{x})$, su entropía diferencial se aproxima al infinito negativo debido a una diferencia técnica entre las definiciones de entropía continua y discreta. El Capítulo 3 resuelve esto utilizando una medida unificada de *tasa-distorsión*.

Más adelante en este capítulo y en los Capítulos 3 y 4, exploramos cómo este sencillo marco de eliminación de ruido y compresión unifica potentes métodos para el aprendizaje de distribuciones de baja dimensión en espacios de alta dimensión, incluidas las distribuciones de imágenes naturales.

1.3.2 Enfoques empíricos basados en datos

En la práctica, resulta difícil modelar datos importantes del mundo real —como imágenes, sonidos y texto— con los modelos lineales idealizados, lineales por tramos u otros modelos analíticos que se han comentado en la sección anterior. Por ello, a lo largo de la historia se han propuesto muchos modelos y métodos no paramétricos para modelar o capturar la distribución empírica de los datos y, posteriormente, utilizar dichos «modelos» para la inferencia. Estos modelos, especialmente las redes neuronales profundas, a menudo se han inspirado en el sistema nervioso biológico, ya que el cerebro de un animal o de un ser humano procesa esos datos con una eficiencia y eficacia notables. En comparación con los modelos analíticos presentados en la sección anterior, estos modelos basados en datos son los menos comprendidos, a pesar de que han dominado en gran medida la práctica de la tecnología de IA durante aproximadamente la última década. Por lo tanto, uno de los principales objetivos de este libro es ofrecer un marco conceptual sólido y unificado (en gran parte a través de los capítulos 5 a **??**) para comprender tanto estos modelos empíricos «modernos» como los modelos analíticos clásicos y poner de manifiesto las conexiones fundamentales entre ellos.

Redes neuronales artificiales clásicas

[22] En sentido estricto, $p^*(\boldsymbol{x})$ es una función generalizada: $p^*(\boldsymbol{x}) = p^*(\boldsymbol{x}_1)\delta(\boldsymbol{x} - \boldsymbol{x}_1) + p^*(\boldsymbol{x}_2)\delta(\boldsymbol{x} - \boldsymbol{x}_2)$ con $p^*(\boldsymbol{x}_1) + p^*(\boldsymbol{x}_2) = 1$.

[23] Obsérvese que en esta sección analizamos el proceso de reducir el ruido y comprimir en forma iterativa una distribución con ruido y de alta entropía hasta que converja a la distribución de datos de baja entropía. Como veremos en el Capítulo 3, este problema es dual al problema de aprender una codificación óptima para la distribución de datos, lo cual también es una aplicación útil [**relic2024lossy**].

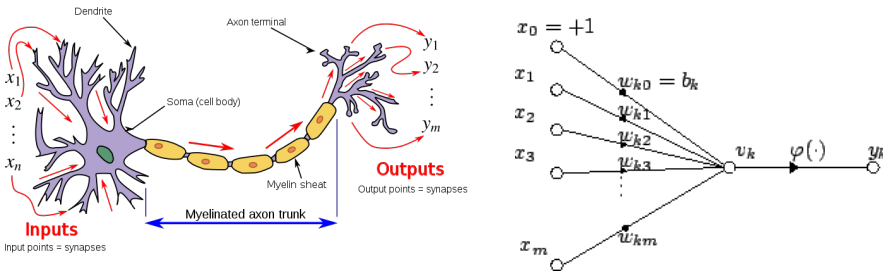


Figura 1.15: El primer modelo matemático de una neurona artificial (a la derecha) que emula la manera en que una neurona (a la izquierda) procesa señales.

La neurona artificial. Inspirándose en el sistema nervioso del cerebro, el primer modelo matemático de una neurona artificial[24] fue propuesto por Warren McCulloch[25] y Walter Pitts en 1943 [**McCulloch-Pitts**]. Describe la relación entre la entrada x_i y la salida o_j de la siguiente manera:

$$o_j = \varphi\Big(\sum_i w_{ji} x_i\Big), \tag{1.3.27}$$

donde $\varphi(\cdot)$ es alguna función de activación no lineal, que suele representarse mediante una función umbral. Este modelo se ilustra en la Figura 1.15. Como podemos ver, esta estructura ya comparte las principales características de una unidad básica en las redes neuronales profundas modernas. El modelo se basa en observaciones sobre el funcionamiento de una neurona individual en nuestro sistema nervioso. Sin embargo, los investigadores no sabían exactamente qué funciones podría realizar y llevar a cabo un conjunto de neuronas de este tipo. En un plano más técnico, tampoco tenían claro qué función de activación no lineal $\varphi(\cdot)$ debían utilizar. Por lo tanto, a lo largo de la historia se han propuesto muchas variantes.[26]

La red neuronal artificial. En la década de 1950, Frank Rosenblatt fue el primero en construir una máquina con una *red* de este tipo de neuronas artificiales, tal y como se muestra en la Figura 1.17. La máquina, denominada Mark I Perceptron, consta de una capa de entrada, una capa de salida y una única capa oculta de 512 neuronas artificiales, tal y como se muestra en la Figura 1.17 (a la izquierda), que es similar a lo que se ilustra en Figura 1.16 (a la izquierda). Se diseñó para clasificar imágenes ópticas de letras. Sin embargo, la capacidad de una red de una sola capa es limitada y solo puede aprender patrones linealmente separables. En el libro de 1969 *Perceptrons:*

[24] conocida como «unidad de umbral lineal» o «perceptrón»
[25] un profesor de psiquiatría de la Universidad de Chicago en aquella época
[26] Función escalonada, umbral duro o blando, unidad lineal rectificada (ReLU), sigmoide, etc. [**dubey2022activation**].

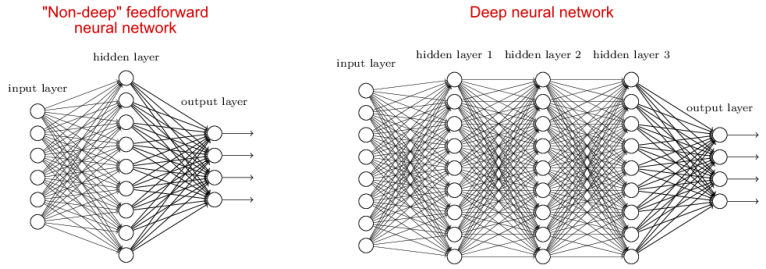


Figura 1.16: Una red con una sola capa oculta (a la izquierda) versus una red profunda (a la derecha).

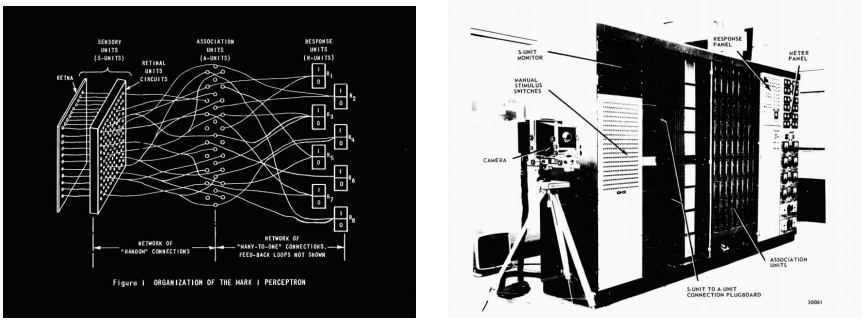


Figura 1.17: La máquina Perceptrón Mark I desarrollada por Frank Rosenblatt a finales de la década de 1950.

An Introduction to Computational Geometry de Marvin Minsky y Seymour Papert [**Minsky-1969**], se demostró que la arquitectura de una sola capa del Perceptrón Mark I no puede aprender una función XOR. Este resultado frenó considerablemente el interés por las redes neuronales artificiales, aunque más tarde se demostró que una red multicapa puede aprender una función XOR [**Rumelhart1986**]. De hecho, una red multicapa lo suficientemente grande, como se muestra en la Figura 1.16 (a la derecha), compuesta por neuronas simples de este tipo, puede simular cualquier máquina de estados finitos, incluso la máquina universal de Turing.[27] Sin embargo, el estudio de las redes neuronales artificiales entró posteriormente en su primer invierno en la década de 1970.

Redes neuronales convolucionales. Los primeros experimentos con redes neuronales artificiales, como el Perceptrón Mark I, durante las décadas de 1950 y 1960 resultaron algo decepcionantes. Sugirieron que el simple acto de conectar

[27] No hay que confundir lo que las redes neuronales son capaces de hacer en teoría con el hecho de que sea factible o fácil entrenar una red neuronal que realice determinadas funciones deseadas.

neuronas de manera general, como en los perceptrones multicapa (MLP), podría no ser suficiente. Para construir redes eficaces y eficientes, resulta de gran ayuda comprender el propósito o la función colectiva que deben cumplir las neuronas de la red, de modo que se puedan organizar y aprender de manera especializada. Así pues, en aquel momento decisivo, el estudio de la inteligencia artificial volvió a buscar inspiración en el sistema nervioso animal.

Se sabe que la mayor parte de nuestro cerebro se dedica a procesar información visual [**PalmerS1999**]. En las décadas de 1950 y 1960, David Hubel y Torsten Wiesel estudiaron sistemáticamente las cortezas visuales de los gatos. Descubrieron que la corteza visual contiene diferentes tipos de células —células simples y células complejas— que resultan sensibles a estímulos visuales de diferentes orientaciones y ubicaciones [**Hubel-Wiesel-1959**]. Hubel y Wiesel ganaron el Premio Nobel de Fisiología o Medicina de 1981 por este descubrimiento revolucionario.

En el ámbito de las redes neuronales artificiales, el trabajo de Hubel y Wiesel inspiró a Kunihiko Fukushima a diseñar la red «neocognitron» en 1980, la cual consiste en neuronas artificiales que emulan a las neuronas biológicas de la corteza visual [**Fukushima1980NeocognitronAS**]. Esta se conoce como la primera *red neuronal convolucional* (CNN), y su arquitectura se ilustra en la Figura 1.18. A diferencia del perceptrón, el neocognitrón tenía más de una capa oculta y podía considerarse una red profunda, tal y como se muestra en la Figura 1.16 (a la derecha).

Inspirándose también en el funcionamiento de las neuronas de la corteza visual del gato, Fukushima fue el primero en introducir la *unidad lineal rectificada* (ReLU):

$$\varphi(x) = \max\{0, x\} = \begin{cases} x, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0, \end{cases} \tag{1.3.28}$$

como función de activación $\varphi(\cdot)$ en 1969 [**Fukushima-1969**]. Sin embargo, no fue hasta los últimos años que ReLU comenzó a utilizarse ampliamente en las redes neuronales profundas (convolucionales) modernas. En este libro se explicará por qué ReLU es una buena opción una vez que analizemos las principales operaciones que llevan a cabo las redes profundas: la compresión.

Las redes de tipo CNN siguieron evolucionando en la década de 1980 y se introdujeron y estudiaron muchas variantes diferentes. Sin embargo, a pesar de las notables capacidades de las redes profundas y de las arquitecturas mejoradas inspiradas en la neurociencia, seguía siendo extremadamente difícil entrenar esas redes profundas para tareas reales, como la clasificación de imágenes. Para que una red funcionara, era necesario recurrir a muchas heurísticas y trucos inexplicables, lo que limitaba el atractivo y la aplicabilidad de las redes neuronales. Un avance significativo se produjo alrededor de 1989, cuando Yann LeCun utilizó con éxito la *retropropagación* (BP) para entrenar una red neuronal convolucional profunda destinada al reconocimiento de dígitos escritos a mano [**LeCun-1989**], conocida posteriormente como LeNet (véase la Figura 1.19). Tras varios años de desarrollo constante, su perseverancia dio

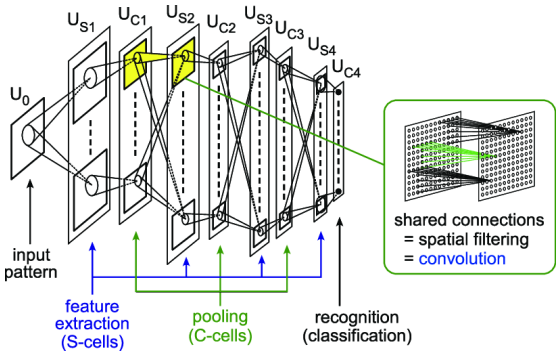


Figura 1.18: El origen de las redes neuronales convolucionales: el Neocognitron, creado por Kunihiko Fukushima en 1980. Cabe destacar que la alternancia entre capas de convoluciones y de agrupación emula las funciones de las células simples y las células complejas descubiertas en la corteza visual de los gatos.

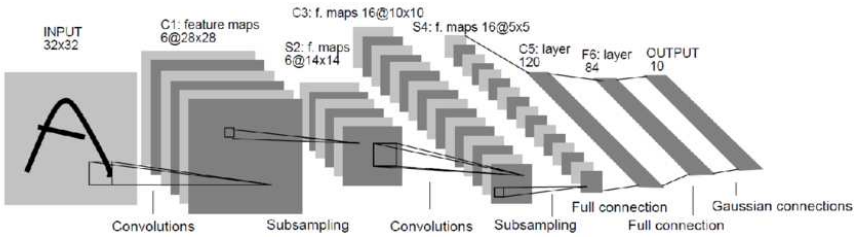


Figura 1.19: La red neuronal convolucional LeNet-5, diseñada por Yann LeCun en 1989.

sus frutos: el rendimiento de LeNet acabó siendo lo suficientemente bueno como para su uso práctico a finales de la década de 1990 [**LeCun-1998**]. El Servicio Postal de los Estados Unidos lo utilizaba para reconocer dígitos escritos a mano (en los códigos postales). LeNet se consideraba la red «prototipo» de todas las redes neuronales profundas modernas, tales como AlexNet [**krizhevsky2012imagenet**] y ResNet [**he2016deep**], de las que hablaremos más adelante. Por este trabajo, Yann LeCun recibió el Premio Turing de 2018.[28]

Retropropagación. A lo largo de la historia, el futuro de las redes neuronales profundas ha estado ligado a la facilidad y la eficiencia con que pueden entrenarse. La retropropagación (BP) se introdujo con este fin. Un perceptrón multicapa puede expresarse como una composición de transformaciones lineales y activaciones no lineales:

$$h(\boldsymbol{W}_1, \ldots, \boldsymbol{W}_L) = f^L(\boldsymbol{W}_L f^{L-1}(\boldsymbol{W}_{L-1} \cdots f^2(\boldsymbol{W}_2 f^1(\boldsymbol{W}_1 \boldsymbol{x})))). \tag{1.3.29}$$

[28] Junto con otros dos pioneros de las redes profundas, Yoshua Bengio y Geoffrey Hinton.

Para entrenar los pesos de la red $\{\boldsymbol{W}_l\}_{l=1}^{L}$ mediante el descenso por gradiente, debemos evaluar el gradiente $\partial h/\partial \boldsymbol{W}_l$. La regla de la cadena en cálculo muestra que los gradientes pueden calcularse de manera eficiente para este tipo de funciones, una técnica que más tarde se denominó retropropagación o BP; véase el **??** para más detalles. La BP ya era conocida en el ámbito del control óptimo y la programación dinámica durante las décadas de 1960 y 1970 y apareció en la tesis doctoral de Paul Werbos de 1974 [**Werbos-1974**; **Werbos1994TheRO**]. En 1986, David Rumelhart y otros fueron los primeros en aplicarla para entrenar un perceptrón multicapa (MLP) [**Rumelhart1986**]. Desde entonces, la BP se ha convertido en la técnica predominante para entrenar redes profundas, ya que es escalable y puede implementarse de manera eficiente en plataformas informáticas paralelas y distribuidas. Sin embargo, es probable que la naturaleza no utilice la BP,[29] ya que el mecanismo resulta demasiado costoso para su implementación física.[30] Esto deja un amplio margen para mejoras futuras.

A pesar de los avances algorítmicos mencionados, el entrenamiento de redes profundas seguía siendo complicado y computacionalmente costoso en las décadas de 1980 y 1990. A finales de la década de 1990, las máquinas de vectores de soporte (SVM) [CV95] habían ganado popularidad como una alternativa superior para las tareas de clasificación.[31] Las SVM ofrecían dos ventajas: un marco de aprendizaje estadístico riguroso conocido como la teoría de Vapnik–Chervonenkis (VC) y algoritmos eficientes de optimización convexa [BV04]. El auge de las máquinas de vectores de soporte marcó el inicio de un segundo invierno para las redes neuronales a principios de la década de 2000.

Autocodificación compresiva. A finales de la década de 1980 y a lo largo de la de 1990, las redes neuronales artificiales ya se usaban para aprender representaciones de baja dimensión para datos de alta dimensión como imágenes. Se había mostrado que las redes neuronales podían aprender PCA directamente a partir de los datos [**Oja1982SimplifiedNM**; **Baldi89**], en lugar de usar los métodos clásicos expuestos en la Sección 1.3.1. Durante ese período también se argumentó que, gracias a su capacidad para modelar transformaciones no lineales, las redes neuronales podían aprender representaciones de baja dimensión para datos con distribuciones no lineales. De manera similar al caso del PCA lineal, es posible aprender simultáneamente un codificador no lineal de reducción de dimensiones f y un decodificador g, ambos modelados mediante

[29] Como hemos comentado anteriormente, la naturaleza aprende casi en forma generalizada a corregir los errores mediante la retroalimentación en bucle cerrado.

[30] La BP de extremo a extremo resulta computacionalmente inmanejable para estructuras neuronales tan complejas como el cerebro, especialmente si las actualizaciones deben realizarse en tiempo real. En cambio, las actualizaciones localizadas en pequeñas secciones del circuito neuronal son mucho más factibles desde el punto de vista biológico. Existen relativamente pocos trabajos sobre la aplicación de estas reglas de «aprendizaje local» al entrenamiento de redes profundas, eludiendo la BP [**baldi2016theory**; **millidge2022predictive**; **li2025noprop**]. Creemos que esta es una oportunidad emocionante para mejorar la escalabilidad del entrenamiento de redes profundas.

[31] Ideas similares para problemas de clasificación se remontan a la tesis doctoral que Thomas Cover defendió en 1964, que se resumió y publicó en un artículo ese mismo año [Cov64].

una red neuronal profunda [**Rumelhart1986**; **Kramer1991NonlinearPC**]:

$$\boldsymbol{X} \xrightarrow{f} \boldsymbol{Z} \xrightarrow{g} \hat{\boldsymbol{X}}. \tag{1.3.30}$$

Al garantizar la consistencia entre los datos decodificados $\hat{\boldsymbol{X}}$ y los originales $\boldsymbol{X}$—por ejemplo, minimizando un error de reconstrucción del tipo MSE[32]:

$$\min_{f,g} \|\boldsymbol{X} - \hat{\boldsymbol{X}}\|_2^2, \qquad \text{donde} \quad \hat{\boldsymbol{X}} = g(f(\boldsymbol{X})), \tag{1.3.31}$$

un autocodificador puede aprenderse directamente a partir de los datos $\boldsymbol{X}$.

¿Pero cómo podemos garantizar que una autocodificación de este tipo realmente capture las verdaderas estructuras de baja dimensión en $\boldsymbol{X}$ en lugar de producir uona representación trivial redundante? Por ejemplo, uno podría simplemente elegir que f y g sean mapeos de identidad y definir $\boldsymbol{Z} = \boldsymbol{X}$. Para asegurar que la autocodificación valga la pena, la representación resultante debería ser compresiva, según alguna medida calculable de complejidad. En 1993, Geoffrey Hinton y sus colegas propusieron usar la longitud de código como tal medida, transformando la meta de autocodificación en la búsqueda de representación que minimice la longitud de código [**Hinton-1993**]. Este trabajo estableció una conexión fundamental entre el principio de longitud mínima de descripción[**Rissanen-1978**] y la minimización de la energía libre (Helmholtz). Los trabajos posteriores del grupo de Hinton [**Hinton504**] demostraron empíricamente que esa autocodificación es capaz de aprender representaciones significativas de baja dimensión para imágenes del mundo real. Pierre Baldi presentó un estudio exhaustivo sobre los autocodificadores en 2011 [**Baldi2011**], justo antes de que las redes profundas alcanzaran una gran popularidad. Analizaremos las medidas de complejidad y la autocodificación con más detalle en la Sección 1.4 y presentaremos un estudio sistemático de la autocodificación compresiva en el **??** desde una perspectiva más unificada.

Las redes neuronales profundas modernas

Durante casi 30 años —desde la década de 1980 hasta la de 2010— las redes neuronales no fueron tomadas en serio por la mayor parte de la comunidad del aprendizaje automático. Las primeras redes profundas, como LeNet, mostraron un rendimiento prometedor en problemas de clasificación a pequeña escala, como el reconocimiento de dígitos, pero su diseño era en gran medida empírico, los conjuntos de datos disponibles eran ínfimos y la retropropagación resultaba computacionalmente prohibitiva para el hardware de la época. Estos factores provocaron una disminución del interés y un estancamiento en los avances y solo un puñado de investigadores siguió adelante.

[32] Se sabe que los errores de tipo MSE son problemáticos para datos de imágenes con estructuras no lineales complejas. Como veremos en breve, gran parte de la investigación reciente sobre métodos generativos, incluidas las GAN, se ha centrado en encontrar mejores funciones de distancia sustitutivas entre los datos originales $\boldsymbol{X}$ y los datos regenerados $\hat{\boldsymbol{X}}$.

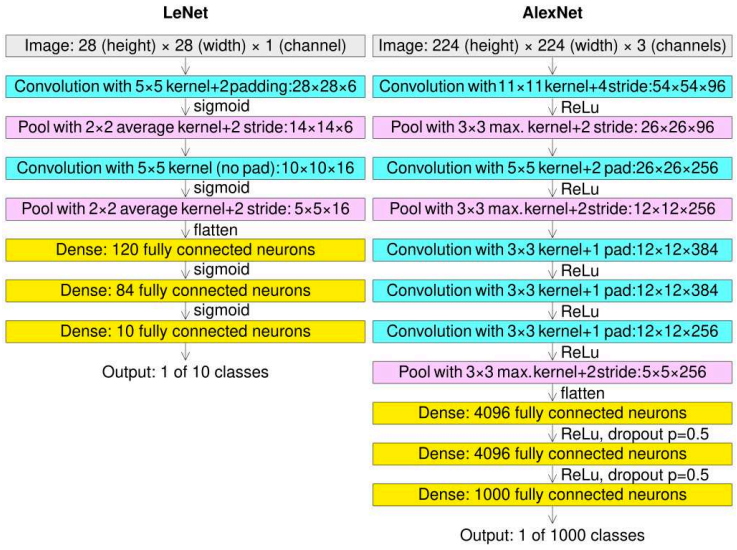


Figura 1.20: Las arquitecturas de LeNet [**LeCun-1989**] versus AlexNet [**krizhevsky2012imagenet**].

Clasificación y reconocimiento. El enorme potencial de las redes profundas solo pudo ponerse de manifiesto una vez que se dispuso de datos y potencia computacional suficientes. En la década de 2010 habían surgido grandes conjuntos de datos como ImageNet y las GPU se habían hecho suficientemente poderosas como para que la retropropagación resultara viable incluso para redes mucho más grandes que LeNet. En 2012, la red convolucional profunda AlexNet —que debe su nombre a Alex Krizhevsky, uno de los autores [**krizhevsky2012imagenet**]— atrajo amplia atención al superar a los métodos de clasificación existentes operando sobre ImageNet por un margen significativo. [33] En la Figura 1.20 se compara a AlexNet con LeNet. AlexNet retiene muchas características de LeNet — simplemente es más grande y reemplaza las activaciones por sigmoide de LeNet por ReLUs. Este trabajo contribuyó a que se le otorgara el Premio Turing de 2018 a Geoffrey Hinton.

Este éxito temprano inspiró a la comunidad de la inteligencia artificial a explorar nuevas arquitecturas para redes neuronales, así como variaciones y mejoras. Empíricamente, se descubrió que las redes más grandes y más profundas arrojan un mejor desempeño en tareas como la clasificación de imágenes. Entre las notables arquitecturas que surgieron se cuentan VGG[**Simonyan15**], GoogLeNet [**Szegedy2014GoingDW**], ResNet [**He2016-lc**] y, más recientemente, los transformers[**vaswani2017attention**]. Pese al rápido mejoramiento empírico del desempeño, las explicaciones teóricas

[33] Las redes profundas ya habían logrado resultados de vanguardia en tareas de reconocimiento de voz, pero estos éxitos recibieron mucha menos atención que el salto cualitativo logrado en la clasificación de imágenes.

de esas arquitecturas —y, de existir, las relaciones entre ellas— siguieron siendo escasas. Una meta de este libro consiste en desvelar el objetivo común que estas redes persiguen optimizar y en explicar sus características compartidas, tales como múltiples capas de operadores lineales intercalados con activaciones no lineales (véase el Capítulo 5).

Aprendizaje por refuerzo. Los primeros éxitos de las redes profundas se dieron principalmente en tareas de clasificación supervisada, como el reconocimiento del habla y de imágenes. Más tarde, se empezaron a utilizar las redes profundas para aprender políticas de toma de decisiones o de control para los juegos. En este contexto, las redes profundas modelan la política óptima de decisión/control (es decir, una distribución de probabilidad sobre las acciones que se deben realizar para maximizar la recompensa esperada) o la función de valor óptimo asociada (una estimación de la recompensa esperada partiendo de un estado dado), tal y como se muestra en la Figura 1.21. Los parámetros de la red se optimizan de manera incremental[34] en función de las recompensas obtenidas por el éxito o el fracaso al jugar la partida con la política actual. Este paradigma de aprendizaje se denomina *aprendizaje por refuerzo* [**Sutton-Barto**]; tiene su origen en la práctica de los sistemas de control de finales de la década de 1960 [**Waltz1965AHA**; **Mendel1970ReinforcementlearningCA**] y se remonta, a través de una larga y rica historia, a la *programación dinámica* de Richard Bellman [**Bellman-DP**] y el *aprendizaje por ensayo y error* [**Minsky-1954**] de Marvin Minsky en la década de 1950.

Desde el punto de vista de la implementación, la combinación de redes profundas y aprendizaje por refuerzo resultó ser potente: las redes profundas pueden aproximar políticas de control y funciones de valor para entornos del mundo real que resultan difíciles de modelar analíticamente. Esto culminó en el sistema AlphaGo de DeepMind, que dejó atónito al mundo en 2016 al derrotar al destacado jugador de Go Lee Sedol y, en 2017, al campeón mundial Jie Ke.[35]

El éxito de AlphaGo sorprendió a la comunidad informática, la que durante largo tiempo había considerado que el espacio de estados de ese juego era demasiado vasto para cualquier solución eficiente en términos de cálculo y tamaño de muestra. La única explicación plausible es que el valor óptimo y la función de política del Go poseen una estructura favorable significativa: cualitativamente, sus dimensiones intrínsecas son suficientemente bajas como para que puedan ser bien aproximadas por redes neuronales que se pueden entrenar a partir de un número manejable de muestras.

Generación y predicción. Desde cierta perspectiva, las primeras aplicaciones de las redes profundas en la década de 2010 se centraban en

[34] Normalmente mediante retropropagación (BP)

[35] En 1996, Deep Blue, de IBM, hizo historia al derrotar al gran maestro ruso Garry Kasparov en una partida de ajedrez utilizando técnicas tradicionales de aprendizaje automático como la búsqueda y poda de árboles, métodos que han demostrado ser menos escalables y poco eficaces para juegos más complejos como el Go.

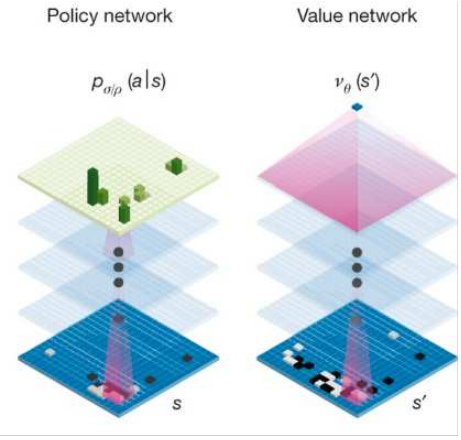


Figura 1.21: AlphaGo: el uso de redes neuronales profundas para modelar la función óptima de política o de valor para el juego Go.

extraer información relevante de los datos $\boldsymbol{X}$ y codificarla en una representación específica para cada tarea $\boldsymbol{Z}$ (supongamos que $\boldsymbol{Z}$ denota las etiquetas de clase en tareas de clasificación [36]):

$$\boldsymbol{X} \xrightarrow{f} \boldsymbol{Z}. \tag{1.3.32}$$

En este caso, la función de mapeo aprendida f no necesita conservar la mayor parte de la información distributiva sobre $\boldsymbol{X}$; basta con retener solo las estadísticas suficientes para la tarea. Por ejemplo, una muestra $\boldsymbol{x} \in \boldsymbol{X}$ podría ser la imagen de una manzana, a la que f asigna la etiqueta de clase $\boldsymbol{z} =$ "manzana".

En muchos entornos modernos—como los denominados grandes modelos de propósito general («fundamentales»)—es posible que también tengamos que decodificar $\boldsymbol{Z}$ para recuperar el $\boldsymbol{X}$ correspondiente con una precisión preestablecida:

$$\boldsymbol{Z} \xrightarrow{g} \hat{\boldsymbol{X}}. \tag{1.3.33}$$

Como $\boldsymbol{X}$ típicamente representa datos observados en el mundo exterior, un buen decodificador nos permite simular o predecir lo que sucede en el mundo. En una tarea de «texto a imagen» o «texto a video», por ejemplo, $\boldsymbol{z}$ es el texto que describe la imagen deseada $\boldsymbol{x}$, y el decodificador debe generar un $\hat{\boldsymbol{x}}$ cuyo contenido concuerde con $\boldsymbol{x}$. Dada una clase de objetos $\boldsymbol{z} =$ "manzana", el decodificador g debería producir una imagen $\hat{\boldsymbol{x}}$ que se vea como una manzana, aunque no necesariamente idéntica al original $\boldsymbol{x}$.

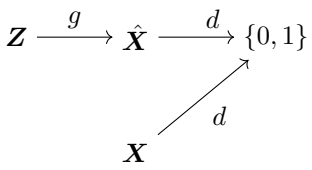
[36] Esta es una notación muy atípica para las etiquetas — la notación habitual es $\boldsymbol{y}$ — pero resulta útil para nuestros propósitos considerar las etiquetas como representaciones muy comprimidas y esparsas de los datos. Véase el Ejemplo 1.2 para más detalles.

Generación por la vía de enfoques discriminativos. Para que las imágenes generadas $\hat{\boldsymbol{X}}$ parezcan imágenes naturales verdaderas $\boldsymbol{X}$, debemos ser capaces de evaluar y minimizar alguna distancia:

$$\min \operatorname{dist}(\boldsymbol{X}, \hat{\boldsymbol{X}}). \tag{1.3.34}$$

Resulta que la mayoría de las distancias motivadas por la teoría son extremadamente difíciles —si no imposibles— de calcular y optimizar para distribuciones en espacios de alta dimensión con baja dimensión intrínseca.[37]

En 2007, Zhuowen Tu, decepcionado por los primeros intentos analíticos de modelar y generar imágenes naturales, decidió probar un enfoque radicalmente diferente. En un artículo publicado en CVPR 2007 [**Tu-2007**], sugirió por primera vez el aprendizaje de un modelo generativo para imágenes mediante un enfoque discriminativo. La idea es simple: si la evaluación de la distancia $\operatorname{dist}(\boldsymbol{X}, \hat{\boldsymbol{X}})$ resulta difícil, se puede, en su lugar, entrenar un discriminador d para separar $\hat{\boldsymbol{X}}$ de $\boldsymbol{X}$:



donde los rótulos 0 y 1 indican si una imagen es generada o real. Intuitivamente, mientras más difícil se hace separar $\hat{\boldsymbol{X}}$ de $\boldsymbol{X}$, más probablemente similares son.

El trabajo de Tu [**Tu-2007**] demostró por primera vez la viabilidad de aprender un modelo generativo a partir de un enfoque discriminativo. Sin embargo, el trabajo utilizaba métodos tradicionales para la generación de imágenes y la clasificación de la distribución (como el boosting), que resultaron ser lentos y difíciles de implementar. A partir de 2012, las redes neuronales profundas ganaron popularidad para la clasificación de imágenes. En 2014, Ian Goodfellow y sus colegas volvieron a proponer la generación de imágenes naturales mediante un enfoque discriminativo [**Goodfellow-2014**]. Sugirieron modelar tanto el generador g como el discriminador d mediante redes neuronales profundas. Además, propusieron aprender g y d a través de un juego minimax:

$$\min_{g} \max_{d} \ell(\boldsymbol{X}, \hat{\boldsymbol{X}}), \tag{1.3.35}$$

donde $\ell(\cdot)$ es alguna función de pérdida natural asociada a la tarea de clasificación. En esta formulación, el discriminador d maximiza su éxito al separar $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$, mientras el generador g hace lo contrario. Este enfoque se

[37] Esto sigue siendo cierto incluso cuando se especifica una familia paramétrica de distribuciones para $\boldsymbol{X}$. La distancia suele volverse mal condicionada o mal definida para distribuciones con soportes de baja dimensión. Peor aún, la familia elegida podría aproximar en forma deficiente la verdadera distribución de interés.

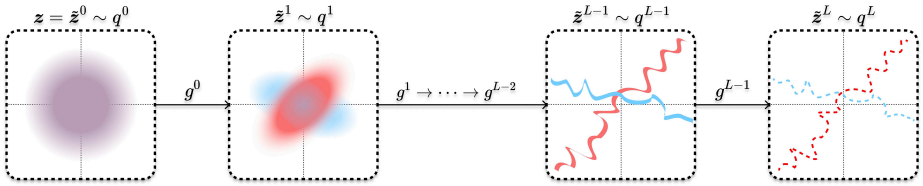


Figura 1.22: Ilustración de un proceso iterativo de eliminación de ruido y compresión que, a partir de una distribución gaussiana $q^0 = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, converge a una distribución arbitraria de datos de baja dimensión $q^L = p$.

denomina *redes generativas adversativas* (GANs). Se demostró que, una vez entrenadas con un gran conjunto de datos, las GAN pueden, de hecho, generar imágenes fotorrealistas. En parte debido a la influencia de este trabajo, Yoshua Bengio recibió el Premio Turing de 2018.

El enfoque discriminativo parece sortear hábilmente una dificultad fundamental en el aprendizaje de distribuciones. Sin embargo, hablando rigurosamente, este enfoque no resuelve plenamente la dificultad fundamental. En [**Goodfellow-2014**] se demuestra que, si se elige adecuadamente la función de pérdida, la formulación minimax resulta matemáticamente equivalente a minimizar la distancia de Jensen-Shannon (véase [**Cover-Thomas**]) entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$. Esto sigue siendo un problema difícil para dos distribuciones de baja dimensión en un espacio de alta dimensión. Por lo tanto, las GAN suelen basarse en numerosas heurísticas y trucos de ingeniería, y a menudo adolecen de problemas de inestabilidad, como el colapso de modos.[38]

Generación por vía de la eliminación de ruido y la difusión. En 2015, poco después de que se introdujeran las redes GAN y ganaran popularidad, Surya Ganguli y sus estudiantes propusieron que un proceso iterativo de eliminación de ruido modelado por una red profunda podría utilizarse para aprender una distribución general, como la de las imágenes naturales [**Sohl-Dickstein2015**]. Su método se inspiró en las propiedades de unos procesos gaussianos y binomiales especiales estudiados por William Feller en 1949 [**Feller1949OnTT**].[39]

Poco después, se demostró que los operadores de eliminación de ruido basados en la función de *score* [**hyvarinen05a**], presentados brevemente en Sección 1.3.1, eran más generales y unificaban los procesos y algoritmos de eliminación de ruido y difusión [**song2019**; **song2020score**; **ho2020denoising**]. La Figura 1.22 ilustra el proceso que transforma una distribución gaussiana genérica $q^0 = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ en una distribución empírica (arbitraria) $p(\boldsymbol{x})$ ejecutando una secuencia de operaciones iterativas de

[38] No obstante, tal formulación minimax ofrece una aproximación práctica de la distancia. Simplifica la implementación y evita ciertas dificultades que surgen al calcular directamente la distancia.

[39] ¡De nuevo, en la mágica época de los 40!

eliminación de ruido (o compresión):

$$\boldsymbol{z}^0 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}) \xrightarrow{g^0} \boldsymbol{z}^1 \xrightarrow{g^1} \cdots \xrightarrow{g^{L-1}} \boldsymbol{z}^L \sim p(\boldsymbol{x}). \tag{1.3.36}$$

A estas alturas, la eliminación de ruido (y la difusión) ha sustituido a las GAN y se ha convertido en el método predominante para el aprendizaje de distribuciones de imágenes y vídeos, lo que ha dado lugar a populares motores comerciales de generación de imágenes como Midjourney y Stability.ai. En el Capítulo 3 presentaremos y estudiaremos de manera sistemática el método de eliminación de ruido y difusión para aprender una distribución general de baja dimensión.

1.4 Un enfoque unificador

Hasta ahora, hemos ofrecido una breve exposición del objetivo principal y la historia de la inteligencia artificial, junto con muchas ideas y enfoques importantes relacionados con ella. En los últimos años, tras el éxito empírico de las redes neuronales profundas, se han realizado considerables esfuerzos para desarrollar marcos teóricos que nos ayuden a comprender las redes profundas diseñadas empíricamente, ya sea en lo que se refiere a componentes específicas que parecen necesarios (por ejemplo, el dropout [**srivastava2014dropout**], la normalización [**ioffe2015batch**; **ba2016layer**], la atención [**vaswani2017attention**]) o en cuanto a sus comportamientos globales (por ejemplo, el doble descenso [**belkin2019reconciling**], el colapso neuronal [**papyan2020prevalence**]).

Motivado en parte por esta tendencia, este libro persigue varios objetivos importantes y ambiciosos:

- Desarrollar un marco teórico que proporcione interpretaciones matemáticas rigurosas de las redes neuronales profundas.
- Asegurar la corrección de la distribución de los datos aprendida y su coherencia con la representación aprendida.
- Demostrar que el marco permite crear arquitecturas de alto rendimiento y sirve de guía para futuras mejoras prácticas.

En los últimos años, cada vez hay más indicios de que estos objetivos pueden en efecto alcanzarse aprovechando la teoría y las soluciones de los modelos analíticos clásicos de baja dimensión examinados con anterioridad (que se tratan con mayor detalle en el Capítulo 2) e integrando ideas fundamentales de campos relacionados, a saber, la teoría de la información y la codificación, la teoría de control y de juegos, así como la optimización. El objetivo de este libro es ofrecer una introducción sistemática a este nuevo enfoque.

1.4.1 Aprendizaje de representaciones parsimoniosas

Una condición necesaria para que cualquier tarea de aprendizaje resulte factible es que las secuencias de interés sean *calculables*, al menos en el sentido de Alan Turing [**Turing-1936**]. Es decir, una secuencia puede calcularse mediante un programa en una computadora común.[40] Además de ser computable, exigimos que el cálculo sea *manejable*.[41] Es decir, el costo computacional (espacial y temporal) de aprender y calcular la secuencia no debería crecer exponencialmente con la longitud. Además, tal y como vemos en la naturaleza (y en la práctica moderna de la inteligencia artificial), para la mayoría de las tareas prácticas, un sistema inteligente necesita aprender qué es lo predecible a partir de grandes volúmenes de datos en un espacio de muy alta dimensión, como los datos de visión, sonido y tacto. Por lo tanto, para hablar de inteligencia no es necesario considerar todas las secuencias y estructuras computables y manejables; conviene centrarnos únicamente en las secuencias y estructuras predecibles que admiten realizaciones *escalables* de sus algoritmos de aprendizaje y computación:

$$\textbf{computable} \implies \textbf{manejable} \implies \textbf{escalable}. \tag{1.4.1}$$

Esto es porque cualquier algoritmo que los seres inteligentes usen para aprender información útil éste tiene que ser *escalable*. Más concretamente, la complejidad computacional de los algoritmos debería escalar en forma adecuada —por lo general, de manera lineal o incluso sublineal— en función del tamaño y la dimensión de los datos. A nivel técnico, esto requiere que las operaciones en las que se basan los algoritmos para aprender sólo pueden utilizar información de oráculo que pueda calcularse de manera eficiente a partir de los datos. Más específicamente, cuando la dimensión es alta y la escala es grande,el único oráculo que se puede permitir calcular es o bien la información geométrica de primer orden sobre los datos[42] o bien la información estadística de segundo orden[43]. El objetivo principal de este libro es desarrollar un marco teórico y computacional en el que podamos desarrollar sistemáticamente soluciones o algoritmos eficientes y eficaces con tales oráculos y operaciones escalables para *aprender* estructuras de baja dimensión a partir de los datos muestreados y, posteriormente, la función predictiva.

Búsqueda de baja dimensiionalidad mediante compresión. A partir de los ejemplos de secuencias que presentamos en la Sección 1.2.1, queda claro que algunas secuencias son fáciles de modelar y calcular, mientras que otras son más

[40] De hecho, hay secuencias bien definidas que no son computables.

[41] No es necesario que nos planteemos predecir cosas cuya complejidad computacional sea inmanejable, por ejemplo, que crezca exponencialmente con la longitud o la dimensión de la secuencia.

[42] como aproximar una estructura no lineal localmente con subespacios lineales y calcular el gradiente de una función objetivo.

[43] como la covarianza o la correlación de los datos o sus características. Como veremos, las arquitecturas profundas más populares, como los Transformers, se basan en este tipo de información.

difíciles. El costo computacional de una secuencia depende de la complejidad de la función predictora f. Cuanto mayor sea el grado de regresión d, más costoso resulta calcularlo. La función f podría ser una simple función lineal, o bien una función no lineal cuya especificación y cálculo resulten arbitrariamente difíciles.

Es razonable pensar que si una secuencia —según cualquier criterio que elijamos— es más difícil de especificar y calcular, entonces también será más difícil de aprender a partir de sus segmentos muestreados. Sin embargo, para cualquier secuencia predecible, existen de hecho muchas formas, a menudo infinitas, de especificarla. Por ejemplo, para la secuencia simple $x_{n+1} = ax_n$, también podríamos definir la misma secuencia como $x_{n+1} = ax_n + bx_{n-1} - bx_{n-1}$. Por lo tanto, sería muy útil desarrollar un concepto objetivo y riguroso de «complejidad» para cualquier secuencia computable dada.

Andrey Kolmogorov, un matemático ruso, fue uno de los primeros en definir la complejidad para cualquier secuencia computable.[44] Propuso que, entre todos los programas que calculan la misma secuencia, la longitud del programa más corto mide su complejidad. Esto concuerda con la navaja de Occam: *elegir la teoría más simple que explique la misma observación.* Formalmente, sea p un programa que genera una secuencia $\boldsymbol{x}$ en la máquina universal $\mathcal{U}$. La complejidad de Kolmogorov de $\boldsymbol{x}$ es:

$$K(\boldsymbol{x}) = \min_{p\,:\,\mathcal{U}(p)=\boldsymbol{x}} L(p). \tag{1.4.2}$$

Por lo tanto, la complejidad mide el grado de parsimonia con el que podemos especificar o calcular la secuencia. Esta definición es conceptualmente importante e históricamente inspiró profundos estudios en complejidad computacional e informática teórica.

La longitud del programa más corto representa la compresión extrema de la secuencia, lo que cuantifica el beneficio que obtenemos al aprender su mecanismo generativo. Sin embargo, la complejidad de Kolmogorov es, por lo general, *incomputable* [**Cover-Thomas**] y su aproximación resulta prácticamente imposible. Por lo tanto, tiene poca utilidad práctica: no permite predecir las dificultades del aprendizaje ni evaluar lo bien que hemos aprendido.

Una medida computable de parsimonia. Para fines prácticos, necesitamos una medida eficientemente computable de complejidad para secuencias generadas por la misma función predictiva.[45] Cabe señalar que parte de la razón por la que la complejidad de Kolmogorov no es computable se debe en parte a que su definición no es constructiva.

Para introducir una medida computable de la complejidad, podemos adoptar un enfoque más constructivo, tal y como proponía Claude Shannon en el

[44] Muchos contribuyeron a esta noción de complejidad de secuencias, entre los que destacan Ray Solomonoff y Greg Chaitin. Los tres desarrollaron la teoría algorítmica de la información por separado: Solomonoff en 1960, Kolmogorov en 1965 [**Kolmogorov1998OnTO**] y Chaitin alrededor de 1966 [**Chaitin-1966**].

[45] En la práctica, lo que típicamente nos interesa es aprender la función predictiva f, en lugar de cualquier secuencia particular generada por f.

marco de la teoría de la información [**Shannon-1948**; **Cover-Thomas**].[46] Suponiendo que la secuencia $\boldsymbol{x}$ se extrae de una distribución probabilística $p(\boldsymbol{x})$, la entropía de la distribución:[47]

$$h(\boldsymbol{x}) \doteq -\int p(s)\log p(s)\mathrm{d}s \tag{1.4.3}$$

proporciona una medida natural de su complejidad. Esta medida también tiene una interpretación natural como número medio de bits binarios que se requieren para codificar tal secuencia, como veremos en el Capítulo 3.

Para ilustrar las principales ideas de este enfoque, tomemos un gran número de segmentos largos de secuencia:

$$\{x_1, x_2, \ldots, x_i, \ldots, x_N\} \subset \mathbb{R}^D, \tag{1.4.4}$$

generado por una función predictiva f. Sin pérdida de generalidad, suponemos que todas las secuencias tienen la misma longitud D, de modo que cada secuencia puede ser vista como vector en $\mathbb{R}^D$. Introducimos un esquema de codificación (con un libro de códigos), denotado $\mathcal{E}$, que mapea cada segmento x_i a una secuencia única de bits binarios $\mathcal{E}(x_i)$ y un esquema de decodificación correspondiente $\mathcal{D}$ que mapea las cadenas de bits binarios de vuelta a segmentos de secuencias (digamos que sujetos a un error máximo de $\epsilon \geq 0$ ya que tal esquema de codificación puede tener pérdidas). El esquema más sencillo consiste en llenar el espacio abarcado por todos los segmentos con bolas de radio ϵ, tal como se muestra en Figura 1.23. A continuación, numeramos las bolas, y cada secuencia se codifica como el índice binario de la bola más cercana, mientras que cada índice binario se decodifica de nuevo para obtener el centro de la bola correspondiente. Por lo tanto, cada segmento puede recuperarse con una precisión de ϵ a partir de su secuencia de bits.

Luego, para una codificación $\mathcal{E}$, la complejidad de la función predictiva f puede evaluarse como la longitud de codificación media $L(\mathcal{E}(x))$ de todas las secuencias x que genera, conocida como la tasa de codificación:[48]

$$R(x \mid \mathcal{E}) = \mathbb{E}[L(\mathcal{E}(x))] \approx \frac{1}{N}\sum_{i=1}^{N} L(\mathcal{E}(x_i)). \tag{1.4.5}$$

La medida de la tasa de codificación cambiará si utilizamos un esquema de codificación (o libro de códigos) diferente. En la práctica, cuanto mejor conozcamos la estructura de baja dimensión en torno a la cual se distribuyen los segmentos, más eficiente será el libro de códigos que podamos diseñar, tal como se ilustra en la Figura 1.23. Los lectores más perspicaces se darán cuenta

[46] Este marco ha guiado con éxito la práctica ingenieril en la industria de las comunicaciones durante más de 80 años.

[47] Aquí consideramos la entropía diferencial, ya que suponemos que la secuencia está formada por variables continuas. Para variables discretas, utilizaríamos $H(\boldsymbol{x}) = -\sum_i p(s_i)\log p(s_i)$.

[48] Se puede hacer esto más preciso tomando $R(\boldsymbol{x} \mid \mathcal{E})$ como la longitud de codificación esperada para todos los segmentos $\boldsymbol{x}$ de longitud D.

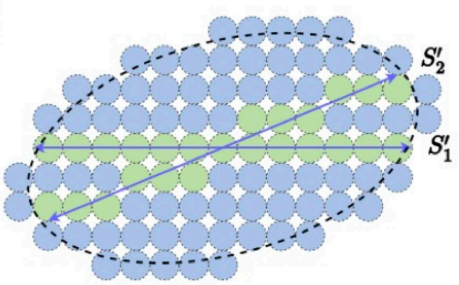


Figura 1.23: Comparación de dos esquemas de codificación. Imaginemos que la distribución verdadera de los datos se ubica a lo largo de las dos líneas con flechas S_1' and S_2'. Las muestras de estas líneas pueden codificarse con un libro de códigos de todas las bolas azules, o bien con un libro de códigos formado únicamente por las bolas verdes. El segundo esquema arroja una longitud/tasa de codificación mucho menor para la misma precisión.

de que, desde el punto de vista conceptual, el proceso de eliminación de ruido ilustrado en la Figura 1.22 se asemeja mucho al paso del esquema de codificación con las bolas azules al de las bolas verdes.

Dados dos esquemas de codificación $\mathcal{E}_1$ y $\mathcal{E}_2$ para los segmentos, si la diferencia entre las tasas de codificación es positiva:

$$R(x \mid \mathcal{E}_1) - R(x \mid \mathcal{E}_2) > 0, \tag{1.4.6}$$

podemos decir que el esquema de codificación $\mathcal{E}_2$ es mejor. Esta diferencia puede verse como una medida de cuánta más información tiene $\mathcal{E}_2$ en comparación con $\mathcal{E}_1$ acerca de la distribución de los datos. En gran medida, el objetivo de aprender la distribución de los datos es equivalente al de hallar el esquema de codificación y decodificación más eficiente que minimiza la tasa de codificación con una precisión deseada:

$$\min_{\mathcal{E},\mathcal{D}} R(x \mid \mathcal{E}) \quad \text{sujeto a} \quad \text{dist}(x, \mathcal{D}(\mathcal{E}(x))) \leq \epsilon. \tag{1.4.7}$$

Como veremos en el Capítulo 4, la tasa mínima que se puede lograr está estrechamente relacionada con la noción de entropía $H(x)$ (1.4.3).

Nota 1.2. La idea de medir la complejidad de los datos mediante esquemas de codificación explícitos ha motivado varios criterios de aprendizaje propuestos para revisar la complejidad de Kolmogorov y mejorar su computabilidad [**WallaceC1999**], entre ellos la longitud mínima del mensaje (MML) propuesta en 1968 [**WallaceC1968**] y la longitud mínima de descripción (MDL) en 1978 [**Rissanen-1978**; **HansenM2001**]. Estos objetivos incluyen la longitud del esquema de codificación $\mathcal{E}$ en sí mismo (incluido su libro de códigos) además de los datos vx de interés: $L(\mathcal{E}(x)) + L(\mathcal{E})$. Sin embargo, si el objetivo es aprender un libro de códigos de tamaño finito y aplicarlo a un gran número de secuencias,

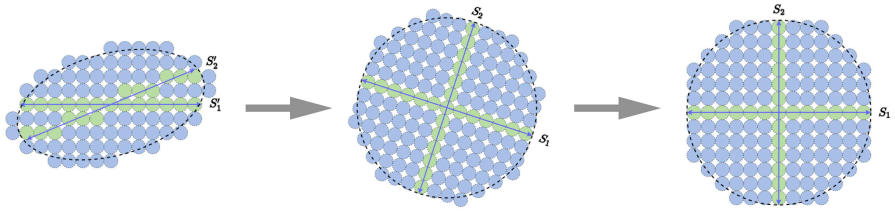


Figura 1.24: Transformación de la distribución de baja dimensión de los datos identificada —aquí sobre dos subespacios lineales— en una representación más informativa y estructurada, en la que los dos subespacios se vuelven más incoherentes.

el costo amortizado del libro de códigos puede ignorarse, ya que

$$\frac{1}{N}\left(L(\mathcal{E})+\sum_{i=1}^{N}L(\mathcal{E}(x_i))\right)\approx\frac{1}{N}\sum_{i=1}^{N}L(\mathcal{E}(vx_i)) \tag{1.4.8}$$

a medida que N se hace grande.

Una vez más, se puede considerar que el esquema de codificación óptimo resultante es aquel que logra la mejor compresión de los datos observados. En general, en comparación con la complejidad de Kolmogorov, la longitud de codificación dada por cualquier esquema de codificación siempre será mayor:

$$K(\boldsymbol{x}) < L(\mathcal{E}(\boldsymbol{x})). \tag{1.4.9}$$

Por lo tanto, minimizar la tasa o longitud de codificación equivale, en esencia, a minimizar una cota superior de la complejidad de Kolmogorov, que de otro modo sería no calculable.

1.4.2 Aprendizaje de representaciones informativas

Si el objetivo simplemente fuese comprimir los datos proporcionados por el solo hecho de hacerlo, los códigos óptimos que se acercan a la complejidad de Kolmogorov se convertirían, en teoría, en secuencias casi aleatorias o carentes de estructura [**Chaitin-1966**].[49] Sin embargo, nuestro verdadero objetivo al aprender la distribución de datos de las secuencias S es poder utilizarla repetidamente y con facilidad en predicciones futuras. Por lo tanto, aunque la compresión nos permite identificar la distribución de baja dimensión en los datos, nos gustaría codificar esta distribución de una manera *estructurada y organizada* para que la representación resultante sea informativa y eficiente de utilizar[50], a modo de «memoria». La Figura 1.24 ilustra de manera intuitiva por qué es deseable dicha transformación.

[49] Cualquier código con estructura puede comprimirse aún más.

[50] Por ejemplo, para obtener muestras de la distribución bajo diferentes condiciones.

Como veremos en el Capítulo 4, estas estructuras deseadas en la representación final pueden promoverse de manera precisa al elegir una medida natural de *ganancia de información* basada en las tasas de codificación de los esquemas de codificación elegidos.[51] A lo largo de este libro, un enfoque de codificación tan explícito y constructivo ofrece un potente marco computacional para aprender buenas representaciones de estructuras de baja dimensión en datos del mundo real. En muchos casos de importancia práctica, la función de longitud de codificación puede calcularse de manera eficiente o aproximarse con precisión. En algunos casos bien comportados, incluso podemos obtener fórmulas en forma cerrada, por ejemplo, modelos de subespacio y gaussianos.

Además, este marco computacional da lugar a un enfoque basado en principios que revela en forma natural el papel que desempeñan las redes profundas en este proceso de aprendizaje. Como derivaremos sistemáticamente en el Capítulo 5, las capas de una red profunda realizan operaciones que optimizan en forma incremental la función objetivo de interés. Desde esta perspectiva, el papel de las redes profundas puede interpretarse precisamente como la emulación de un determinado algoritmo de optimización iterativa —por ejemplo, el descenso por gradiente—para optimizar el objetivo de ganancia de información. Las capas de las arquitecturas profundas resultantes pueden dotarse de interpretaciones estadísticas y geométricas precisas, es decir, realizar operaciones incrementales de codificación y decodificación compresivas. En consecuencia, las redes profundas derivadas se convierten en «cajas blancas» transparentes que son totalmente explicables desde el punto de vista matemático.

1.4.3 Aprendizaje de representaciones consistentes

Para resumir lo que hemos comentado hasta ahora, denotemos los datos como

$$\boldsymbol{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_i, \ldots, \boldsymbol{x}_N\} \subset \mathbb{R}^D, \tag{1.4.10}$$

y sea $\boldsymbol{Z} = \mathcal{E}(\boldsymbol{X})$ el código de $\boldsymbol{X}$ mediante algún codificador $\mathcal{E}$:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z}. \tag{1.4.11}$$

En el contexto del aprendizaje automático, a $\boldsymbol{Z}$ se le suele llamar las «características» de $\boldsymbol{X}$. Nótese que sin conocer la distribución subyacente de $\boldsymbol{X}$, no sabemos qué codificador $\mathcal{E}$ se debe utilizar para conservar la información más útil sobre la distribución de $\boldsymbol{X}$. En la práctica, la gente suele empezar probando un esquema de codificación compacto que se adapte bien a una tarea concreta.

[51] La ganancia de información es un concepto muy extendido en la teoría algorítmica de la información y se ha incorporado en varios enfoques de este tema, siendo la más reciente la teoría de la información $\mathcal{V}$ [**xu2020theory**]. Esta instanciación concreta, al igual que muchas otras medidas de complejidad que hemos comentado, sufre de un **condicionamiento cada vez peor** a medida que el soporte de los datos se vuelve más compacto y de baja dimensión, por lo que no resulta adecuada para nuestras necesidades.

En concreto, se intenta entrenar un codificador que optimice una determinada medida (empírica) de parsimonia para la representación aprendida:

$$\min \rho(\boldsymbol{Z}). \tag{1.4.12}$$

Ejemplo 1.2. Por ejemplo, la clasificación de imágenes es un caso de este tipo: asignamos a todas las imágenes de una misma clase un único código y a las imágenes de diferentes clases códigos distintos, es decir, vectores «one-hot»:

$$\boldsymbol{x} \mapsto \boldsymbol{z} \in \{[1,0,0,\ldots,0,0],\ [0,1,0\ldots,0,0],\ \ldots,\ [0,0,0,\ldots,0,1].\} \tag{1.4.13}$$

Ahora bien, un clasificador $f(\cdot)$ puede modelarse como una función que predice la probabilidad de pertenencia de un $\boldsymbol{x}$ dado a cada una de las k clases: $\hat{\boldsymbol{z}} = f(\boldsymbol{x}) \in \mathbb{R}^K$. Entonces, la «calidad» de un clasificador se puede medir mediante la denominada *entropía cruzada*:[52]

$$\ell(\hat{\boldsymbol{z}}, \boldsymbol{z}) = \sum_{k=1}^{K} -z_k \log \hat{z}_k, \tag{1.4.14}$$

donde z_k indica la k-ésima entrada del vector $\boldsymbol{z}$. Como indicaron los primeros trabajos sobre redes profundas [**krizhevsky2012imagenet**], si se dispone de suficientes datos, un esquema de codificación de este tipo a menudo puede representarse mediante una red profunda y aprenderse en forma integral mediante la optimización de la entropía cruzada. ■

La pérdida de entropía cruzada $\ell(\hat{\boldsymbol{z}}, \boldsymbol{z})$ puede considerarse como una medida especial de parsimonia $\rho(\boldsymbol{z})$ asociada con una familia particular de esquemas de codificación que son apropiados para la clasificación. Sin embargo, una codificación de este tipo obviamente tiene *grandes pérdidas*. El $\boldsymbol{z}$ aprendido no contiene ninguna otra información acerca de $\boldsymbol{x}$ excepto su tipo de clase. Por ejemplo, al asignarle a una imagen (un código que representa) la clase «manzana», ya no sabemos qué tipo específico de manzana se representa en la imagen original con base sólo en esa etiqueta.

Por supuesto, el otro extremo es exigir que el esquema de codificación sea *sin pérdida*. Es decir, que exista una correspondencia biunívoca entre $\boldsymbol{x}$ y su código $\boldsymbol{z}$. Sin embargo, como veremos en Capítulo 4, la codificación sin pérdida (o compresión) resulta sre impráctica, a menos que $\boldsymbol{x}$ sea discreta. En el caso de una variable aleatoria continua, solo podemos considerar esquemas de codificación con pérdida, de modo que la longitud de codificación de los datos sea finita. Es decir, solo codificamos los datos hasta una precisión predeterminada. Como desarrollaremos con más detalle en Capítulo 4, la codificación con pérdida no es solo una opción práctica, sino que desempeña un papel fundamental a la hora de hacer posible el aprendizaje de la distribución continua subyacente a partir de muestras finitas de dicha distribución.

[52] La entropía cruzada puede considerarse como una medida de distancia entre la distribución real de $\boldsymbol{z}$ y la de la predicción $\hat{\boldsymbol{z}}$. También puede considerarse como la longitud de codificación esperada de $\boldsymbol{z}$ si utilizamos el libro de códigos óptimo para $\hat{\boldsymbol{z}}$ a fin de codificar $\boldsymbol{z}$. La entropía cruzada alcanza su mínimo cuando $\boldsymbol{z}$ y $\hat{\boldsymbol{z}}$ tienen la misma distribución.

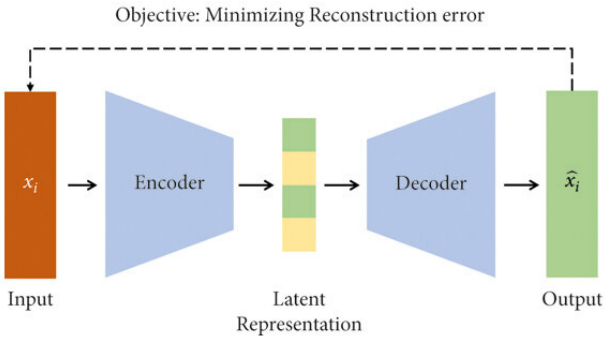


Figura 1.25: Ilustración de la arquitectura de un autocodificador.

Para muchos fines de aprendizaje, queremos que la característica $\boldsymbol{z}$, aunque *con pérdidas*, conserve más información sobre $\boldsymbol{x}$ que simplemente su tipo de clase. En este libro, presentaremos una medida de parsimonia más general basada en la longitud o tasa de codificación asociada a una familia más general de esquemas de codificación: la codificación con una mezcla de subespacios o gaussianas. Esta familia tiene la capacidad de aproximar con gran exactitud distribuciones arbitrarias del mundo real hasta un cierto grado de precisión. Como veremos en el Capítulo 4 y Capítulo 5, dicha medida no solo conservará la mayor parte de la información sobre la distribución de $\boldsymbol{X}$, sino que también favorecerá ciertas estructuras geométricas y estadísticas deseables para la representación aprendida $\boldsymbol{Z}$.

Autocodificación bidireccional para la consistencia. En un contexto de aprendizaje más amplio, el objetivo principal de un esquema de codificación compresiva $\mathcal{E}$ es identificar las estructuras de baja dimensión en los datos $\boldsymbol{X}$ para utilizarlas en la realización de predicciones en el espacio de datos original. Esto requiere que el esquema de codificación aprendido $\mathcal{E}$ admita un esquema de decodificación eficiente, denotado por $\mathcal{D}$, que mapea la representación codificada $\boldsymbol{Z}$ de vuelta al espacio de datos:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z} \xrightarrow{\mathcal{D}} \hat{\boldsymbol{X}}. \tag{1.4.15}$$

A este par de codificación-decodificación $(\mathcal{E}, \mathcal{D})$ lo denominamos *autocodificación*. La Figura 1.25 ilustra este proceso.

Idealmente, la decodificación es aproximadamente una «inversa» de la codificación, de modo que los datos (la distribución) $\hat{\boldsymbol{X}}$ decodificados a partir de $\boldsymbol{Z}$ se asemejan a los datos originales (la distribución) $\boldsymbol{X}$ en cierta medida.[53] Si esto se cumple, podemos recuperar o predecir a partir de $\boldsymbol{Z}$ lo que ocurre en el espacio de datos original. En este caso, decimos que el par $(\mathcal{E}, \mathcal{D})$ proporciona

[53] Más adelante aclararemos qué entendemos por «similar».

una autocodificación *consistente*. Para la mayoría de los efectos prácticos, no solo necesitamos que exista tal decodificación, sino que también sea posible llevarla a cabo de manera eficiente y implementarla físicamente. Por ejemplo, si $\boldsymbol{x}$ es una variable de valor real, se requiere la cuantización para que cualquier esquema de decodificación sea realizable en una máquina de estados finitos, tal como explicaremos en el Capítulo 4. Por lo tanto, en general, los esquemas de codificación y decodificación viables conllevan necesariamente pérdidas. Una cuestión fundamental es cómo aprender una representación compacta (con pérdidas) $\boldsymbol{Z}$ que, aun así, pueda predecir bien $\boldsymbol{X}$.

En términos generales, tanto el codificador como el decodificador pueden modelarse y implementarse mediante redes profundas y aprenderse resolviendo un problema de optimización de la siguiente forma:

$$\min_{f,g}[\ell(\boldsymbol{X}, \hat{\boldsymbol{X}}) + \rho(\boldsymbol{Z})], \qquad \text{where} \quad \boldsymbol{Z} = f(\boldsymbol{X}), \quad \hat{\boldsymbol{X}} = g(\boldsymbol{Z}) \tag{1.4.16}$$

donde $\ell(\cdot, \cdot)$ es una función de distancia que promueve la similitud entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$[54] y $\rho(\boldsymbol{Z})$ es una medida que favorece la parsimonia y la riqueza de información de $\boldsymbol{Z}$. El análisis de componentes principales clásico (PCA) [**JolliffeI2002**] es un ejemplo típico de autocodificación consistente, que estudiaremos con gran detalle en el Capítulo 2, como precursor de estructuras de baja dimensión más generales. En **??**, estudiaremos cómo aprender una autocodificación consistente para distribuciones generales (por ejemplo, no lineales) de baja dimensión.

1.4.4 Aprendizaje de representaciones autoconsistentes

En el objetivo de autocodificación anterior, hay que evaluar cuán cerca están los datos decodificados $\hat{\boldsymbol{X}}$ de los originales $\boldsymbol{X}$. Esto suele requerir una supervisión externa o el conocimiento de una medida de similitud adecuada. El cálculo de la similitud entre $\hat{\boldsymbol{X}}$ y $\boldsymbol{X}$ puede resultar prohibitivamente costoso, o incluso imposible. [55] En la naturaleza, los animales aprenden de manera autónoma sin comparar su estimación $\hat{\boldsymbol{X}}$ con los datos reales $\boldsymbol{X}$ en el espacio de datos; por lo general, carecen de esa opción.[56]

¿Cómo, entonces, puede un sistema aprender sin supervisión externa ni comparación? ¿Cómo puede saber que $\hat{\boldsymbol{X}}$ es consistente con $\boldsymbol{X}$ sin una comparación directa? Esto nos lleva a la idea de «cerrar el bucle». Bajo condiciones suaves (especificadas con precisión en el **??**), garantizar la consistencia entre $\boldsymbol{X}$ y $\hat{\boldsymbol{X}}$ se reduce a codificar $\hat{\boldsymbol{X}}$ como $\hat{\boldsymbol{Z}}$ y verificar la consistencia entre $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$. A este concepto lo denominamos *autoconsistencia*, como se ilustra a continuación:

$$\boldsymbol{X} \xrightarrow{\mathcal{E}} \boldsymbol{Z} \xrightarrow{\mathcal{D}} \hat{\boldsymbol{X}} \xrightarrow{\mathcal{E}} \hat{\boldsymbol{Z}}, \tag{1.4.17}$$

[54] Ya sea a nivel de muestras o a nivel de distribución, dependiendo de la elección de ℓ.

[55] Por ejemplo, minimizar la distancia entre distribuciones que existe entre las dos variables aleatorias resulta difícil tanto desde el punto de vista estadístico como computacional.

[56] Tanto en los animales como en los seres humanos, los ojos procesan la información visual mucho antes de que llegue al cerebro; el cerebro no cuenta con un mecanismo para procesar la información visual en bruto y debe basarse en lo que extraen los ojos para realizar algún aprendizaje.

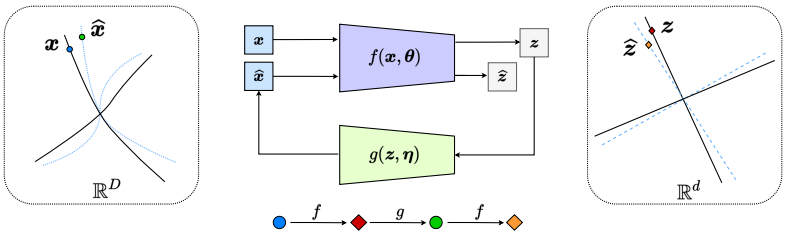


Figura 1.26: Ilustración de una transcripción en bucle cerrado. Aquí usamos un mapeo f para representar al codificador $\mathcal{E}$ y g para representar al decodificador $\mathcal{D}$.

A este proceso lo denominamos *transcripción en bucle cerrado*,[57] tal y como se muestra en la Figura 1.26.

Podría decirse que es cierto que cualquier ser inteligente autónomo solo necesita aprender una representación autoconsistente $\boldsymbol{Z}$ de los datos observados $\boldsymbol{X}$, ya que verificar la consistencia en el espacio de datos original —lo que a menudo significa en el mundo externo—resulta demasiado costoso o incluso físicamente inviable. La formulación en bucle cerrado permite aprender una codificación óptima $f(\cdot,\boldsymbol{\theta})$ y una decodificación $g(\cdot,\boldsymbol{\eta})$ mediante un juego min-max que depende únicamente de la característica interna (aprendida) $\boldsymbol{Z}$:

$$\max_{\boldsymbol{\theta}} \min_{\boldsymbol{\eta}} [\ell(\boldsymbol{Z},\hat{\boldsymbol{Z}}) + \rho(\boldsymbol{Z})], \qquad \text{where} \quad \boldsymbol{Z} = f(\boldsymbol{X}), \quad \hat{\boldsymbol{X}} = g(\boldsymbol{Z}), \quad \hat{\boldsymbol{Z}} = f(\hat{\boldsymbol{X}}) \tag{1.4.18}$$

donde $\ell(\boldsymbol{Z},\hat{\boldsymbol{Z}})$ es una función de pérdida basada en las tasas de codificación de las características $\boldsymbol{Z}$ y $\hat{\boldsymbol{Z}}$, lo cual, como veremos, puede resultar mucho más fácil de calcular. Una vez más, $\rho(\boldsymbol{Z})$ es alguna medida que favorece la parsimonia y la riqueza de información de $\boldsymbol{Z}$. De manera algo sorprendente, como veremos en el **??**, bajo condiciones bastante moderadas, como que $\boldsymbol{X}$ sea de dimensión suficientemente baja, ¡la autoconsistencia entre $(\boldsymbol{Z},\hat{\boldsymbol{Z}})$ implica consistencia en $(\boldsymbol{X},\hat{\boldsymbol{X}})$! Además, también veremos que un sistema en bucle cerrado nos permitirá aprender la distribución de manera *continua e incremental*,[58] sin sufrir problemas como el olvido catastrófico asociado a los modelos de bucle abierto.

1.5 Tender un puente entre teoría y práctica en inteligencia artificial

Hasta ahora, hemos presentado varios marcos relacionados para aprender una representación compacta y estructurada $\boldsymbol{Z}$ de una distribución de datos dada $\boldsymbol{X}$:

- la codificación en bucle abierto (1.4.11);

[57] Inspirado en la transcripción entre el ADN y el ARN u otras proteínas.
[58] Es decir, aprender una clase a la vez o incluso una muestra a la vez.

- la autocodificación bidireccional (1.4.15);
- la transcripción en bucle cerrado (1.4.17).

En este libro, estudiaremos de manera sistemática los tres marcos, uno tras otro:

$$\textbf{circuito abierto} \implies \textbf{bidireccional} \implies \textbf{en bucle cerrado}, \tag{1.5.1}$$

en el Capítulo 5, la **??** y la **??**, respectivamente.

Como veremos en este libro, estos marcos, incluidos muchos de los ya discutidos en este capítulo, pueden considerarse ahora como intentos de abordar *parcialmente* el problema de aprender una representación *completa y computable* de la distribución de los datos $\boldsymbol{X}$:

$$\boldsymbol{X} \longleftrightarrow \boldsymbol{Z} \longleftrightarrow \boldsymbol{G}, \tag{1.5.2}$$

en los que se incluyen dos conjuntos de mapeos:

1. una autocodificación entre los datos $\boldsymbol{X}$ y la representación $\boldsymbol{Z}$ de sus características informativas,
2. un proceso de difusión y eliminación de ruido entre la representación aprendida $\boldsymbol{Z}$ y una distribución simple $\boldsymbol{G}$, por ejemplo, gaussiana.

Este último proceso nos permite acceder de las características estructuradas de baja dimensión aprendidas $\boldsymbol{Z}$ y muestrearlas para tareas como la regeneración de los datos originales $\hat{\boldsymbol{X}}$.[59] El panorama geométrico general asociado a estos dos procesos (de transformación) se ilustra en la Figura 1.27.

Como veremos en el Capítulo 5, la función de las redes profundas modernas puede interpretarse precisamente como la realización de los operadores[60] necesarios para llevar a cabo las transformaciones anteriores entre distribuciones. Las arquitecturas populares de redes neuronales profundas (como las CNN, ResNet o Transformers) que se desarrollaron de manera empírica se interpretarán matemáticamente, se justificarán y se mejorarán mediante este enfoque.

En los últimos años, se han propuesto muchos marcos teóricos para ayudar a comprender las redes profundas. Sin embargo, la mayoría no ha conseguido ofrecer soluciones escalables que igualen el rendimiento de los métodos empíricos aplicados a datos y tareas del mundo real. Muchas teorías ofrecen poca orientación práctica para seguir mejorando empíricamente los resultados. El **??** y el **??** demostrarán cómo el marco presentado en este libro puede salvar la brecha entre la teoría y la práctica. El **??** mostrará cómo utilizar la distribución aprendida y su representación para realizar inferencias (bayesianas) en tareas prácticas relacionadas con la generación (condicional), la estimación

[59] Nótese que, incluso si las características aprendidas se encuentran en un único subespacio lineal de baja dimensión, la forma más eficaz de identificar dicho subespacio es, en esencia, mediante un proceso de eliminación de ruido: el método de potencias que desarrollaremos en el 2.

[60] mediante optimización desenrollada, como veremos

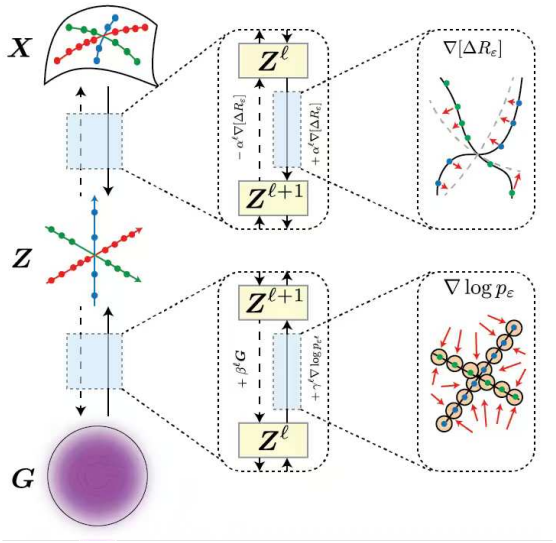


Figura 1.27: Ilustración de una representación aprendida, informativa y estructurada de características $\boldsymbol{Z}$ para los datos $\boldsymbol{X}$ a la que se puede acceder de manera eficiente por medio de un proceso de eliminación de ruido a partir de $\boldsymbol{G}$.

y la predicción. El **??** aportará pruebas experimentales convincentes de que las redes y los sistemas diseñados a partir de principios fundamentales pueden alcanzar un rendimiento competitivo —y potencialmente superior— en tareas como el aprendizaje de representaciones visuales, la clasificación de imágenes, su completamiento, su segmentación y la generación de texto.

De vuelta a la inteligencia. Como se mencionó al principio, una tarea fundamental de cualquier ser inteligente es adquirir información predecible a partir de los datos percibidos. Ahora comprendemos la naturaleza computacional de esta tarea y reconocemos que se trata de un proceso interminable por dos razones:

- Es poco probable que la memoria o el conocimiento adquirido a partir de datos —por ejemplo, mediante esquemas de codificación y decodificación— sea correcto u óptimo. La inteligencia debe ser capaz de perfeccionarse cuando las predicciones de nuevas observaciones contengan errores.

- Los datos observados aún no abarcan toda la información predecible. La inteligencia debe reconocer cuándo la memoria o el conocimiento actuales son inadecuados y adquirir nueva información cada vez que esté disponible.

Por lo tanto, la inteligencia *no* consiste en recopilar todos los datos

informativos por adelantado y entrenar un modelo para que memorice su distribución (de baja dimensión). En cambio, requiere mecanismos computacionales capaces de mejorar continuamente el conocimiento actual de la distribución y de adquirir nueva información cuando sea necesario. Una característica fundamental de cualquier ser o sistema inteligente —ya sea un animal, un ser humano, una civilización o toda la comunidad científica— es *la capacidad de mejorar continuamente o de adquirir nueva información o conocimientos por sí mismo* a través de un bucle cerrado de retroalimentación sobre la eficacia con la que su memoria o sus conocimientos actuales logran explicar o predecir sus observaciones. En consecuencia, en cierta medida, podemos expresar simbólicamente la relación entre la inteligencia y el conocimiento[61] de la siguiente manera:

$$\begin{aligned} \text{Inteligencia}(t) &= \frac{\mathrm{d}}{\mathrm{d}t}\,\text{Conocimiento}(t), \\ \text{Conocimiento}(t) &= \int_0^t \text{Inteligencia}(s)\mathrm{d}s. \end{aligned}$$

Un marco de retroalimentación en bucle cerrado es un mecanismo universal para cualquier sistema capaz de superarse a sí mismo y de aprender por sí mismo —siendo el refuerzo una forma primitiva, como en la selección natural— o de competir, siendo la investigación científica su forma más avanzada a través de la formulación de hipótesis o conjeturas y la falsificación o verificación. Todos los seres y sistemas inteligentes de la naturaleza utilizan este tipo de mecanismos de bucle cerrado para aprender en todos los niveles y a todas las escalas. Esta omnipresencia inspiró los primeros intentos de modelar la inteligencia mediante máquinas autónomas u ordenadores semiautónomos, en particular el movimiento cibernético iniciado y promovido por Norbert Wiener en la década de 1940.

Esperamos que este libro ayude a los lectores a comprender mejor los objetivos científicos, los principios matemáticos y los mecanismos computacionales que subyacen a la inteligencia, al menos en lo que respecta a la formación de la memoria útil o el conocimiento empírico. Creemos que este trabajo no solo tiende un puente entre la teoría y la práctica, sino que también conecta el presente con el futuro: aclara lo que hemos hecho hasta ahora y, al mismo tiempo, revela lo que no hemos hecho. Por lo tanto, sienta las bases para el estudio futuro de posibles nuevos mecanismos a la base de la inteligencia de alto nivel propia del ser humano: la verdadera «inteligencia artificial» a la que se refería el programa de Dartmouth de 1956. Dichos mecanismos pueden crear conceptos abstractos y realizar inferencias deductivas que, en nuestra opinión, van más allá de la compresión de datos empíricos para la memoria, así como realizar inferencias inductivas que se estudian y explican en este libro y que se están aprovechando en la tecnología actual de la «IA». Analizaremos los problemas pendientes para comprender y, posiblemente, hacer realidad una

[61] Aquí entendemos «conocimiento» en el sentido más amplio posible, incluyendo tanto la memoria empírica como el conocimiento científico.

inteligencia más avanzada en el **??** final.

Capítulo 2

El aprendizaje de estructuras lineales e independientes

> «*El arte de hacer matemáticas consiste en encontrar ese caso particular que contiene todos los gérmenes de la generalidad.*»
>
> – David Hilbert

Los datos reales tienen una estructura de baja dimensión. Para entender por qué esto es así, consideremos el humilde caso del ruido estático en un televisor cuando el satélite no funciona. En cada fotograma (aproximadamente cada $\frac{1}{30}$ segundos), el ruido estático RGB en una pantalla de tamaño $H \times W$ se muestrea, a grandes rasgos, en forma independiente a partir de una distribución uniforme en $[0,1]^{3\times H\times W}$. En teoría, la imagen estática *podría* convertirse en una imagen natural en cualquier fotograma, pero aunque alguien pase mil años mirando la pantalla del televisor, eso no sucederá. Esta discrepancia se explica por el hecho de que el conjunto de $H \times W$ imágenes naturales ocupa una fracción infinitesimal del hipercubo $[0,1]^{3\times H\times W}$. En particular, es de dimensión extremadamente baja, en comparación con la dimensión del espacio que lo rodea. Se producen fenómenos similares con todos los demás tipos de datos naturales, como texto, audio o vídeo. Por lo tanto, cuando diseñamos sistemas y métodos para procesar datos naturales y aprender su estructura o distribución, esta es una propiedad fundamental de los datos naturales que debemos tener en cuenta.

Por lo tanto, nuestra tarea principal consiste en aprender una distribución que tenga baja dimensión intrínseca en un espacio de alta dimensión. En el resto de este capítulo, analizaremos varios métodos *clásicos* para llevar a cabo esta tarea en varios modelos algo *idealizados* de la distribución a saber, modelos geométricamente lineales o estadísticamente independientes. Aunque estos modelos y métodos son importantes y útiles por sí mismos, los analizamos

aquí porque motivan, inspiran y sirven como precursores o análogos de métodos más modernos para distribuciones más generales que recurren al aprendizaje profundo (de representaciones).

Nuestro enfoque principal (y la formulación general del problema) se puede resumir así:

> **Problema:** *Dadas una o varias observaciones (con ruido o incompletas) de una verdad de referencia (la muestra verdadera) de la distribución de los datos reales, obtenga una estimación de dicha muestra.*

Este enfoque sustenta varios métodos clásicos de procesamiento de datos, que abordamos en este capítulo.

- Sección 2.1 — Análisis de componentes principales (PCA): Dadas unas muestras ruidosas procedentes de una distribución con soporte en *un subespacio de baja dimensión*, obtener una estimación de la muestra verdadera que se sitúa en este subespacio.
- Sección 2.2 — Aprendizaje de diccionario completo y análisis de componentes independientes (ICA): Dadas unas muestras ruidosas procedentes de una distribución definida en *una unión (que no es el espacio generado) de unos cuantos subespacios de baja dimensión*, obtener una estimación de las muestras verdaderas.
- Sección 2.3 — Codificación esparsa y aprendizaje de diccionarios: Dadas unas muestras ruidosas procedentes de *una distribución definida sobre combinaciones de unos pocos vectores incoherentes*, como por ejemplo los ejes de coordenadas, obtener una estimación de la muestra verdadera, que también tenga esa propiedad.

En este capítulo, tal y como se ha descrito anteriormente, formulamos supuestos simplificadores de modelado que, en esencia, suponen que (el soporte de) la distribución de los datos presenta estructuras geométricamente (casi, por tramos) lineales y componentes estadísticamente independientes, tal y como se ilustra en la Figura 2.1. En el Capítulo 1 nos hemos referido a este tipo de modelos de datos como «modelos analíticos». Estas hipótesis de modelado nos permiten derivar algoritmos eficientes con garantías demostrables de eficiencia, tanto en términos de datos como de complejidad computacional, para el procesamiento de datos a gran escala. Sin embargo, se trata de modelos imperfectos para las distribuciones de datos del mundo real, que suelen ser complejas, por lo que sus supuestos subyacentes solo se cumplen de manera aproximada. Esto significa que las garantías que ofrece el análisis detallado de estos algoritmos también solo se cumplen de manera aproximada en el caso de datos reales.

No obstante, las técnicas que se analizan en este capítulo son útiles por sí mismas y, además de eso, sirven como «caso especial con gérmenes de

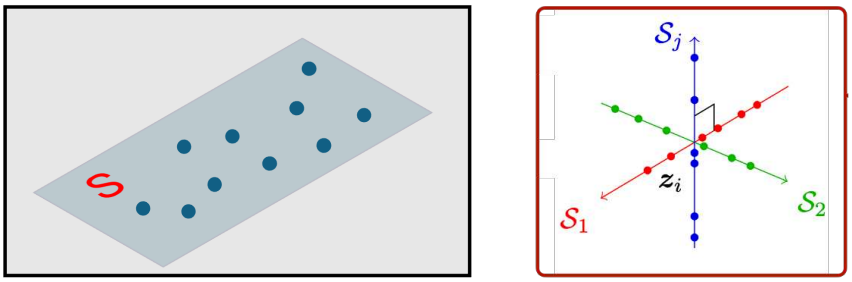


Figura 2.1: Datos sobre un único subespacio de baja dimensión (izquierda), por ejemplo, $\mathcal{S} = \text{col}(\boldsymbol{U})$ o una mezcla de subespacios de baja dimensión (derecha), por ejemplo, $\mathcal{S}_j = \text{col}(\boldsymbol{U}_j)$.

generalidad», por así decirlo, en el sentido de que ofrecen una motivación e intuición orientativas para los paradigmas más generales en el ámbito del aprendizaje (profundo) de distribuciones más generales que abordaremos más adelante. Como veremos en capítulos posteriores, en la era del aprendizaje profundo, los enfoques modernos «basados en datos» o «no paramétricos» para aprender las distribuciones de datos (de baja dimensión) adoptan, en esencia, los mismos conceptos e ideas para el aprendizaje.

Una nota sobre la notación. Recomendamos al lector que consulte el capítulo Notación, en particular la sección «Notación de modelado y optimización», al comenzar a leer este capítulo. A lo largo del libro empleamos las mismas convenciones de notación para el modelado de datos y la optimización de los parámetros de estos modelos. Los modelos sencillos y explícitos que estudiamos en este capítulo ayudan a aclarar el contenido «semántico» de estas convenciones.

2.1 Subespacio de baja dimensión

2.1.1 Análisis de componentes principales (PCA)

Quizás la hipótesis de modelado más simple posible para una estructura de baja dimensión sea la denominada hipótesis de «bajo rango». Sea D la dimensión de nuestro espacio de datos; supongamos que nuestros datos pertenecen a un subespacio de baja dimensión $d \ll D$, posiblemente con algunas pequeñas perturbaciones.[1] Esta suposición resulta ser prácticamente válida para algunos datos sorprendentemente complejos, como las imágenes de dígitos escritos a mano y los datos faciales [**BasriR2003-PAMI**], tal como se muestra en la

[1] Además, posiblemente, un desplazamiento no trivial con respecto a $\mathbf{0}$; en la práctica, tenemos en cuenta este desplazamiento restando la media empírica de las muestras de datos (calculada sobre las muestras de datos) a cada muestra y en el siguiente capítulo ya no le prestamos más atención.



Figura 2.2: Imágenes de rostros humanos alineados y números escritos a mano. A pesar de la aparente gran variedad de sus formas, cada conjunto de estos datos genera (aproximadamente) un subespacio (casi) lineal de muy baja dimensión. Esto se puede expresar con precisión técnica en la notación de (2.1.1): para una dimensión de subespacio relativamente pequeña d (digamos, ≈ 20, en comparación con la dimensión $D = 784$ de los dígitos de MNIST, los vectores de error $\boldsymbol{\varepsilon}_i$ son pequeños para cada $i \in [N]$ (donde $N = 6 \cdot 10^4$ para MNIST es el tamaño del conjunto de datos). Podemos *aprender* el subespacio óptimo de este tipo a partir de los datos.

Figura 2.2, pero, como veremos, se prestará exepcionalmente bien a un análisis exhaustivo.

Formulación del problema. Para expresarlo en notación matemática, representamos un subespacio $\mathcal{S} \subseteq \mathbb{R}^D$ de dimensión d mediante una matriz ortonormal $\boldsymbol{U} \in \mathsf{O}(D, d) \subseteq \mathbb{R}^{D \times d}$ tal que las columnas de $\boldsymbol{U}$ generan $\mathcal{S}$. Luego, decimos que nuestros datos $\{\boldsymbol{x}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ tienen una estructura (aproximada) de bajo rango si existen una matriz ortonormal $\boldsymbol{U} \in \mathsf{O}(D, d)$, vectores $\{\boldsymbol{z}_i\}_{i=1}^N \subseteq \mathbb{R}^d$ y *pequeños* vectores $\{\boldsymbol{\varepsilon}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ tales que

$$\boldsymbol{x}_i = \boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i, \quad \forall i \in [N]. \tag{2.1.1}$$

Aquí, $\boldsymbol{\varepsilon}_i$ son perturbaciones que impiden que los datos sean perfectamente de bajo rango; su presencia en nuestro modelo nos permite cuantificar hasta qué punto nuestro análisis sigue siendo relevante cuando se producen desviaciones respecto a nuestro modelo. El verdadero subespacio de soporte es $\mathcal{S} \doteq \operatorname{col}(\boldsymbol{U})$, el espacio generado por las columnas de $\boldsymbol{U}$. Para procesar toda la información posible a partir de estos datos, necesitamos recuperar $\mathcal{S}$; para ello basta con recuperar $\boldsymbol{U}$, también denominadas *componentes principales*. Afortunadamente, se trata de una tarea computacionalmente viable denominada **Análisis de Componentes Principales** (PCA por su sigla en inglés) y ahora veremos cómo resolverla.

Dados los datos $\{\boldsymbol{x}_i\}_{i=1}^N$ distribuidos como en (2.1.1), nuestro objetivo es recuperar el modelo $\boldsymbol{U}$. Un enfoque natural consiste en encontrar el subespacio

$\boldsymbol{U}^\star$ y los vectores latentes $\{\boldsymbol{z}_i^\star\}_{i=1}^N$ que proporcionen la mejor aproximación $\boldsymbol{x}_i \approx \boldsymbol{U}^\star \boldsymbol{z}_i^\star$. Es decir, nuestro objetivo es resolver el problema

$$\min_{\tilde{\boldsymbol{U}}, \{\tilde{\boldsymbol{z}}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2, \tag{2.1.2}$$

donde $\tilde{\boldsymbol{U}}$ está restringida a ser una matriz ortonormal, tal como se ha indicado anteriormente. Omitiremos esta restricción en las exposiciones similares que siguen, en aras de la brevedad.

Codificación y decodificación en subespacios mediante eliminación de ruido. Para simplificar este problema, para un valor fijo de $\tilde{\boldsymbol{U}}$, tenemos (la demostración como ejercicio):

$$\min_{\{\tilde{\boldsymbol{z}}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2 = \frac{1}{N} \sum_{i=1}^N \min_{\tilde{\boldsymbol{z}}_i} \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2 \tag{2.1.3}$$

$$= \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{U}}^\top \boldsymbol{x}_i\|_2^2. \tag{2.1.4}$$

Es decir, la solución óptima $(\boldsymbol{U}^\star, \{\boldsymbol{z}_i^\star\}_{i=1}^N)$ del problema de optimización anterior cumple que $\boldsymbol{z}_i^\star = (\boldsymbol{U}^\star)^\top \boldsymbol{x}_i$.

Ahora, podemos reformular el problema de optimización original en términos de $\tilde{\boldsymbol{U}}$ y $\{\tilde{\boldsymbol{z}}_i\}_{i=1}^N$ como un problema de optimización que se define únicamente sobre $\tilde{\boldsymbol{U}}$, es decir, para obtener la base $\boldsymbol{U}^\star$ y los códigos compactos $\{\boldsymbol{z}_i^\star\}_{i=1}^N$ basta con resolver cualquiera de los siguientes dos problemas equivalentes:

$$\min_{\tilde{\boldsymbol{U}}, \{\tilde{\boldsymbol{z}}_i\}_{i=1}^N} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{z}}_i\|_2^2 = \min_{\tilde{\boldsymbol{U}}} \frac{1}{N} \sum_{i=1}^N \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}} \tilde{\boldsymbol{U}}^\top \boldsymbol{x}_i\|_2^2. \tag{2.1.5}$$

Téngase en cuenta que el problema que aparece en el miembro derecho de (2.1.5) es un problema de *eliminación de ruido*: dadas observaciones ruidosas $\boldsymbol{x}_i$ de datos de bajo rango, nuestro objetivo es encontrar la copia *sin ruido* de $\boldsymbol{x}_i$, que bajo el modelo (2.1.1) es $\boldsymbol{U}\boldsymbol{z}_i$. Es decir, la entrada sin ruido $\hat{\boldsymbol{x}}_i = \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{x}_i$. Nótese que este es el punto del subespacio que se encuentra más cerca de $\boldsymbol{x}_i$, tal como se muestra en la Figura 2.3. Aquí, al resolver el problema equivalente de encontrar el mejor subespacio, parametrizado por la base aprendida $\boldsymbol{U}^\star$, aprendemos una aproximación al *denoiser*, es decir, la matriz de proyección $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top \approx \boldsymbol{U}\boldsymbol{U}^\top$ que proyecta el punto de datos ruidoso $\boldsymbol{x}_i$ sobre el subespacio $\mathcal{S}$.

Al reunir el proceso anterior, obtenemos básicamente un sencillo esquema de codificación-decodificación que mapea un punto de datos $\boldsymbol{x}$ en $\mathbb{R}^D$ a un espacio (latente) de menor dimensión $\mathbb{R}^d$ y, a continuación, de vuelta a $\mathbb{R}^D$:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}=(\boldsymbol{U}^\star)^\top} \boldsymbol{z} \xrightarrow{\mathcal{D}=\boldsymbol{U}^\star} \hat{\boldsymbol{x}}. \tag{2.1.6}$$

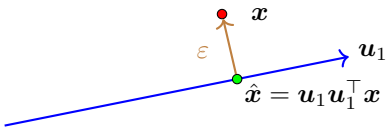


Figura 2.3: **La geometría de PCA.** Un punto de datos $\boldsymbol{x}$ (en rojo) se proyecta sobre el subespacio unidimensional aprendido, generado por el vector base unitario $\boldsymbol{u}_1$ (flecha azul). La proyección $\boldsymbol{U}\boldsymbol{U}^\top\boldsymbol{x} = \boldsymbol{u}_1\boldsymbol{u}_1^\top\boldsymbol{x}$ (en verde) es la versión sin ruido de $\boldsymbol{x}$ utilizando la estructura de baja dimensión dada por $\boldsymbol{u}_1$ y $\boldsymbol{\varepsilon}$ (flecha marrón) representa el residuo de la proyección, es decir, el ruido.

En este contexto, $\boldsymbol{z} \in \mathbb{R}^d$ se puede considerar como el código compacto de baja dimensión (o una representación latente) de un punto de datos $\boldsymbol{x} \in \mathbb{R}^D$, mientras que la base del subespacio aprendido $\boldsymbol{U}^\star$ se puede entender como el libro de códigos asociado, cuyas columnas son las palabras de código óptimas (aprendidas). El proceso logra eliminar el ruido de $\boldsymbol{x}$ mediante su proyección sobre el subespacio generado por $\boldsymbol{U}^\star$.

Cálculo de la base del subespacio. Por ahora, continuamos con nuestro cálculo. Sea $\boldsymbol{X} = \left[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\right] \in \mathbb{R}^{D\times N}$ la matriz cuyas columnas son las observaciones $\boldsymbol{x}_i$. Tenemos (demostración como ejercicio)

$$\arg\min_{\tilde{\boldsymbol{U}}} \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top\boldsymbol{x}_i\|_2^2 = \arg\max_{\tilde{\boldsymbol{U}}} \frac{1}{N}\sum_{i=1}^{N} \|\tilde{\boldsymbol{U}}^\top\boldsymbol{x}_i\|_2^2 \tag{2.1.7}$$

$$= \arg\max_{\tilde{\boldsymbol{U}}} \operatorname{tr}\left\{\tilde{\boldsymbol{U}}^\top\left(\frac{\boldsymbol{X}\boldsymbol{X}^\top}{N}\right)\tilde{\boldsymbol{U}}\right\}. \tag{2.1.8}$$

Por lo tanto, para calcular las componentes principales, se busca la matriz ortonormal $\tilde{\boldsymbol{U}}$ que maximice el término $\operatorname{tr}(\tilde{\boldsymbol{U}}^\top(\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{U}})$. Podemos demostrar por inducción que esta matriz $\boldsymbol{U}^\star$ tiene columnas que son los *primeros d vectores propios unitarios* de $\boldsymbol{X}\boldsymbol{X}^\top/N$. Dejamos toda la demostración al lector en el Ejercicio 2.1, pero aquí tratamos el caso base de la inducción. Supongamos que $d = 1$. Entonces solo tenemos que recuperar un único vector unitario $\boldsymbol{u}$, por lo que el problema anterior se reduce a

$$\max_{\tilde{\boldsymbol{u}}:\ \|\tilde{\boldsymbol{u}}\|_2=1} \tilde{\boldsymbol{u}}^\top(\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{u}}. \tag{2.1.9}$$

Este es el llamado *cociente de Rayleigh* de $\boldsymbol{X}\boldsymbol{X}^\top/N$. Aplicando el teorema espectral, diagonalizamos $\boldsymbol{X}\boldsymbol{X}^\top/N = \boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\top$, donde $\boldsymbol{Q}$ es ortogonal y $\boldsymbol{\Lambda}$ es diagonal con entradas no negativas. Por lo tanto

$$\tilde{\boldsymbol{u}}^\top(\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{u}} = \tilde{\boldsymbol{u}}^\top\boldsymbol{Q}\boldsymbol{\Lambda}\boldsymbol{Q}^\top\tilde{\boldsymbol{u}} = (\boldsymbol{Q}^\top\tilde{\boldsymbol{u}})^\top\boldsymbol{\Lambda}(\boldsymbol{Q}^\top\tilde{\boldsymbol{u}}). \tag{2.1.10}$$

Dado que $\boldsymbol{Q}$ es una transformación ortogonal invertible, $\boldsymbol{Q}^\top\tilde{\boldsymbol{u}}$ es un vector unitario y optimizar con respecto a $\tilde{\boldsymbol{u}}$ equivale a optimizar con respecto a $\tilde{\boldsymbol{w}} \doteq$

$\boldsymbol{Q}^\top \tilde{\boldsymbol{u}}$. Por lo tanto, podemos escribir

$$\tilde{\boldsymbol{u}}^\top (\boldsymbol{X}\boldsymbol{X}^\top/N)\tilde{\boldsymbol{u}} = \tilde{\boldsymbol{w}}^\top \boldsymbol{\Lambda}\tilde{\boldsymbol{w}}, \tag{2.1.11}$$

cuyas soluciones óptimas $\boldsymbol{w}^\star$ entre los vectores unitarios son vectores «one-hot» cuya única entrada distinta de cero (y, por lo tanto, unitaria) se encuentra en uno de los índices correspondientes al mayor valor propio de $\boldsymbol{X}\boldsymbol{X}^\top/N$. Esto significa que $\boldsymbol{u}^\star = \boldsymbol{Q}\boldsymbol{w}^\star$, la solución óptima del problema original, corresponde a un vector propio unitario de $\boldsymbol{X}\boldsymbol{X}^\top/N$ (es decir, una columna de $\boldsymbol{Q}$) que corresponde al mayor valor propio. Generalizando adecuadamente esto al caso $d > 1$ y resumiendo todo lo expuesto anteriormente, obtenemos el siguiente teorema informal.

Teorema 2.1. *Supongamos que nuestro conjunto de datos $\{\boldsymbol{x}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ se puede escribir como*

$$\boldsymbol{x}_i = \boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i, \qquad \forall i \in [N], \tag{2.1.12}$$

donde $\boldsymbol{U} \in \mathsf{O}(D,d)$ representa la estructura de bajo rango, $\{\boldsymbol{z}_i\}_{i=1}^N \subseteq \mathbb{R}^d$ son los códigos compactos de los datos y $\{\boldsymbol{\varepsilon}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ son pequeños vectores que indican la desviación de nuestros datos respecto al modelo de bajo rango. Entonces, las componentes principales $\boldsymbol{U}^\star \in \mathsf{O}(D,d)$ de nuestro conjunto de datos vienen dadas por los d primeros vectores propios de $\boldsymbol{X}\boldsymbol{X}^\top/N$, donde $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{D\times N}$ y corresponden aproximadamente al denoiser lineal óptimo: $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top \approx \boldsymbol{U}\boldsymbol{U}^\top$.

Aquí no indicamos tasas explícitas de aproximación, ya que pueden resultar bastante técnicas. En el caso especial en que $\boldsymbol{\varepsilon}_i = \boldsymbol{0}$ para todo i, el $\boldsymbol{U}^\star$ aprendido abarca el soporte de las muestras $\{\boldsymbol{x}_i\}_{i=1}^N$. Si, además, los $\boldsymbol{z}_i$ son lo suficientemente diversos (por ejemplo, abarcan todo el espacio $\mathbb{R}^d$), entonces tendríamos una recuperación perfecta: $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top = \boldsymbol{U}\boldsymbol{U}^\top$.

Nota 2.1. En algunas tareas de análisis de datos, la matriz de datos $\boldsymbol{X}$ se organiza de tal manera que cada punto de datos es una *fila* en lugar de una *columna*, como se muestra aquí. En este caso, las componentes principales son los d primeros vectores propios de $\boldsymbol{X}^\top\boldsymbol{X}/N$.

Nota 2.2 (Selección de base mediante valores propios de la eliminación de ruido). En muchos casos, nuestros datos no estarán distribuidos realmente según un modelo de subespacio más ruido, o bien no conoceremos la verdadera dimensión subyacente d del subespacio. En este caso, tenemos que elegir d; este problema se denomina *selección de modelos*. En el caso restringido del PCA, una forma de realizar la selección de modelos consiste en calcular $\boldsymbol{X}\boldsymbol{X}^\top/N$ y buscar casos en los que los valores propios adyacentes disminuyan bruscamente; esto es un indicador de que el índice del valor propio más grande es la «dimensión verdadera d» el resto de los valores propios de $\boldsymbol{X}\boldsymbol{X}^\top/N$ se deben al ruido o a las perturbaciones $\boldsymbol{\varepsilon}_i$. La selección de modelos es un problema difícil y, hoy en día, en la era del aprendizaje profundo, donde se conoce como «optimización de hiperparámetros», suele realizarse mediante el método de fuerza bruta o la optimización bayesiana, a menudo utilizando la pérdida en un conjunto de datos de «validación» como objetivo.

Nota 2.3 (Eliminación de ruido en muestras). La expresión del miembro derecho de (2.1.5), es decir,

$$\min_{\tilde{\boldsymbol{U}}} \frac{1}{N} \sum_{i=1}^{N} \|\boldsymbol{x}_i - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top \boldsymbol{x}_i\|_2^2, \tag{2.1.13}$$

es lo que se conoce como un *problema de eliminación de ruido*, llamado así porque se trata de un problema de optimización cuya solución *elimina el ruido de las muestras para que estas se ajusten al subespacio.* La eliminación de ruido —el aprendizaje de un mapa que elimina el ruido de muestras ruidosas para que se ajuste a la estructura de datos (como en (2.1.1), aunque quizá en forma más compleja)— es un método habitual para el aprendizaje de distribuciones que se analizará en lo que sigue y a lo largo de todo el manuscrito. Nótese que ya hemos tratado este concepto, pero vale la pena repetirlo debido a su gran importancia en los capítulos siguientes.

Nota 2.4 (Interpretación como red neuronal). Si realizamos un análisis de componentes principales (PCA), recuperamos aproximadamente el soporte de la distribución codificada por el parámetro $\boldsymbol{U}^\star$. El mapa de eliminación de ruido aprendido adopta entonces la forma $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top$. Además de ser un *denoiser*, esto puede considerarse como una *red neuronal lineal simple de dos capas con pesos vinculados* RN: la primera capa multiplica por $(\boldsymbol{U}^\star)^\top$ y la segunda multiplica por $\boldsymbol{U}^\star$, es decir

$$\text{reduce}_{\text{r}}\text{uido}(\boldsymbol{x}) = \boldsymbol{U}^\star \circ \underbrace{\underbrace{\text{id} \circ \underbrace{(\boldsymbol{U}^\star)^\top \boldsymbol{x}}_{\text{primera “capa”}}}_{\text{post-activación de la primera “capa”}}}_{\text{salida de la “RN”}} \tag{2.1.14}$$

Si lo comparamos con una red neuronal estándar de dos capas RN, observamos una similitud estructural:

$$\text{RN}(\boldsymbol{x}) = \boldsymbol{W}^\star \circ \underbrace{\underbrace{\text{ReLU} \circ \underbrace{(\boldsymbol{U}^\star)^\top \boldsymbol{x}}_{\text{primera capa}}}_{\text{post-activación de la primera capa}}}_{\text{salida de la RN}} \tag{2.1.15}$$

En concreto, el PCA puede interpretarse como *el aprendizaje de un autocodificador simple de dos capas de eliminación de ruido*,[2] uno de los ejemplos más simples de una red neuronal no trivial. En este marco, las **representaciones aprendidas** serían simplemente $(\boldsymbol{U}^\star)^\top \boldsymbol{x} \approx \boldsymbol{z}$. De este modo, el PCA sirve como problema modelo para el aprendizaje de representaciones (profundas), al que recurriremos más adelante en la monografía. Obsérvese que, en esta analogía, las representaciones reflejan —o son proyecciones de— los datos de entrada hacia una estructura aprendida de baja dimensión. Esta propiedad será especialmente relevante en el futuro.

[2] De hecho, como hemos mencionado en el capítulo anterior, el PCA fue uno de los primeros problemas en el que se utilizaron redes neuronales para resolverlo [**Oja1982SimplifiedNM**; **Baldi89**].

2.1.2 Búsqueda de estructuras de rango bajo por el método de las potencias

Existe una forma computacionalmente eficiente de estimar los vectores propios principales (asociados a los autovalores más grandes) de $\boldsymbol{X}\boldsymbol{X}^\top/N$ o de cualquier matriz simétrica semidefinida positiva $\boldsymbol{M}$, denominada *método de las potencias*. Este método constituye la base de varios enfoques algorítmicos para el análisis de datos de alta dimensión que trataremos más adelante en este capítulo, por lo que lo abordamos aquí.

Sea $\boldsymbol{M}$ una matriz simétrica semidefinida positiva. Existe una base ortonormal para $\mathbb{R}^D$ formada por los vectores propios $(\boldsymbol{w}_i)_{i=1}^D$ de $\boldsymbol{M}$, con los valores propios correspondientes $\lambda_1 \geq \cdots \geq \lambda_D \geq 0$. Por definición, cualquier vector propio $\boldsymbol{w}_i$ cumple que $\lambda_i \boldsymbol{w}_i = \boldsymbol{M}\boldsymbol{w}_i$. Por lo tanto, para cualquier $\lambda_i > 0$, $\boldsymbol{w}_i$ es un «punto fijo» de la siguiente ecuación:

$$\boldsymbol{w} = \frac{\boldsymbol{M}\boldsymbol{w}}{\|\boldsymbol{M}\boldsymbol{w}\|_2}. \tag{2.1.16}$$

Teorema 2.2 (Método de las potencias)**.** *Supongamos que $\lambda_1 > \lambda_i$ para todo $i > 1$. Si calculamos el punto fijo de (2.1.16) utilizando la siguiente iteración:*

$$\boldsymbol{v}_0 \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{1}), \qquad \boldsymbol{v}_{t+1} \leftarrow \frac{\boldsymbol{M}\boldsymbol{v}_t}{\|\boldsymbol{M}\boldsymbol{v}_t\|_2}, \tag{2.1.17}$$

entonces, en el límite, $\boldsymbol{v}_t$ convergerá al vector propio unitario de $\boldsymbol{M}$ asociado al mayor de los valores propios.

Demostración. En primer lugar, nótese que, para todo t, tenemos

$$\boldsymbol{v}_t = \frac{\boldsymbol{M}\boldsymbol{v}_{t-1}}{\|\boldsymbol{M}\boldsymbol{v}_{t-1}\|_2} = \frac{\boldsymbol{M}^2\boldsymbol{v}_{t-2}}{\|\boldsymbol{M}\boldsymbol{v}_{t-1}\|_2\|\boldsymbol{M}\boldsymbol{v}_{t-2}\|_2} = \cdots = \frac{\boldsymbol{M}^t\boldsymbol{v}_0}{\prod_{s=1}^t \|\boldsymbol{M}\boldsymbol{v}_s\|_2}. \tag{2.1.18}$$

Por lo tanto, $\boldsymbol{v}_t$ tiene la misma dirección que $\boldsymbol{M}^t\boldsymbol{v}_0$ y es de norma unitaria, por lo que podemos escribir

$$\boldsymbol{v}_t = \frac{\boldsymbol{M}^t\boldsymbol{v}_0}{\|\boldsymbol{M}^t\boldsymbol{v}_0\|_2}. \tag{2.1.19}$$

Dado que todos los vectores propios $\boldsymbol{w}_i$ de $\boldsymbol{M}$ forman una base ortonormal para $\mathbb{R}^D$, podemos escribir

$$\boldsymbol{v}_0 = \sum_{i=1}^D \alpha_i \boldsymbol{w}_i, \tag{2.1.20}$$

donde, dado que $\boldsymbol{v}_0$ es gaussiano, los α_i son todos distintos de cero con probabilidad 1. Por lo tanto, podemos utilizar nuestra expresión anterior para $\boldsymbol{v}_t$ y escribir

$$\boldsymbol{v}_t = \frac{\boldsymbol{M}^t\boldsymbol{v}_0}{\|\boldsymbol{M}^t\boldsymbol{v}_0\|_2} = \frac{\sum_{i=1}^D \lambda_i^t \alpha_i \boldsymbol{w}_i}{\|\sum_{i=1}^D \lambda_i^t \alpha_i \boldsymbol{w}_i\|_2} = \frac{\sum_{i=1}^D \lambda_i^t \alpha_i \boldsymbol{w}_i}{\sqrt{\sum_{i=1}^D \lambda_i^{2t} \alpha_i^2}}. \tag{2.1.21}$$

Ahora, consideremos el caso en el que $\lambda_1 > \lambda_2 \geq \cdots \geq \lambda_D \geq 0$. (El caso en el que se repiten los valores propios principales se resuelve de manera similar.) Entonces podemos escribir

$$\boldsymbol{v}_t = \frac{\alpha_1 \boldsymbol{w}_1 + \sum_{i=2}^{D} (\lambda_i/\lambda_1)^t \alpha_i \boldsymbol{w}_i}{\sqrt{\alpha_1^2 + \sum_{i=2}^{D} (\lambda_i/\lambda_1)^{2t} \alpha_i^2}}. \tag{2.1.22}$$

Dado que $\lambda_1 > \lambda_i$ para todo $i > 1$, los términos dentro de la suma tienden exponencialmente a 0 y el resultado es el límite

$$\lim_{t \to \infty} \boldsymbol{v}_t = \frac{\alpha_1}{|\alpha_1|} \boldsymbol{w}_1 = \text{sign}(\alpha_1) \boldsymbol{w}_1, \tag{2.1.23}$$

que es el vector propio unitario asociado al valor propio principal de $\boldsymbol{M}$. El valor propio principal λ_1 de $\boldsymbol{M}$ puede estimarse mediante $\boldsymbol{v}_t^\top \boldsymbol{M} \boldsymbol{v}_t$, que converge a λ_1 a una velocidad comparable. □

Nota 2.5 (El método de las potencias como proceso de eliminación de ruido). Nótese que el cálculo de la componente principal mediante el método de las potencias puede interpretarse como un proceso de «eliminación de ruido»: partiendo de un ruido aleatorio, converge iterativamente hacia un subespacio unidimensional. De hecho, como mostraremos más adelante en el Capítulo 3, si suponemos que *el soporte de la distribución de los datos es un único subespacio*, el operador de eliminación de ruido asociado (o la función de *score*) es exactamente el mismo que la iteración de potencias anterior. En el Capítulo 3, mostraremos cómo generalizar ese proceso de eliminación de ruido a distribuciones de estructuras de baja dimensión más generales. Se trata, en esencia, de un enfoque universal y muy eficaz para aprender cualquier distribución de baja dimensión.

Para hallar el segundo vector propio principal, aplicamos el algoritmo de las potencias a $\boldsymbol{M} - \lambda_1 \boldsymbol{v}_1 \boldsymbol{v}_1^\top$, que tiene vectores propios $(\boldsymbol{w}_i)_{i=2}^D$ y los valores propios correspondientes $(\lambda_i)_{i=2}^D$. Repitiendo este procedimiento d veces seguidas, podemos estimar de manera muy eficiente los d vectores propios principales de $\boldsymbol{M}$ para cualquier matriz simétrica semidefinida positiva $\boldsymbol{M}$. Por lo tanto, también podemos aplicarlo a $\boldsymbol{X}\boldsymbol{X}^\top/N$ para recuperar las primeras d componentes principales, que es lo que buscábamos desde un principio. Obsérvese que este enfoque recupera una componente principal a la vez; en secciones posteriores lo compararemos con otros enfoques algorítmicos, como el descenso por gradiente sobre todos los vectores propios.

2.1.3 Aprendizaje local e incremental mediante PCA en línea

Nótese que, en la construcción anterior, la transformación lineal $\boldsymbol{U}$ (es decir, la matriz de componentes principales) utilizada para la codificación y la decodificación se calcula con antelación «fuera de línea» a partir de todos los

datos de entrada. Una pregunta es si esta transformación se puede aprender «en línea» a medida que llegan los datos de entrada. Esta pregunta fue respondida por el trabajo de Oja en 1982 [**Oja1982SimplifiedNM**].

Ejemplo 2.1 (Esquema de aprendizaje hebbiano normalizado para PCA). Consideremos una secuencia de vectores aleatorios independientes e idénticamente distribuidos $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_i, \ldots \in \mathbb{R}^n$ con covarianza $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$. Sea $\boldsymbol{u}_0 \in \mathbb{R}^n$ y definamos la respuesta de un vector de entrada $\boldsymbol{x}_i$ ante un vector de pesos $\boldsymbol{u}_i$ como su producto interno:

$$\eta_i = \boldsymbol{u}_i^T \boldsymbol{x}_i \tag{2.1.24}$$

y actualicemos el vector de pesos según el siguiente esquema:

$$\boldsymbol{u}_{i+1} = \frac{\boldsymbol{u}_i + \gamma \eta_i \boldsymbol{x}_i}{\|\boldsymbol{u}_i + \gamma \eta_i \boldsymbol{x}_i\|_2} \tag{2.1.25}$$

para alguna ganancia pequeña $\gamma > 0$. Este esquema de actualización puede considerarse como un esquema normalizado de aprendizaje hebbiano, en el que los pesos de las conexiones entre neuronas se fortalecen si tanto la entrada $\boldsymbol{x}$ como la salida η (o sus productos) son fuertes. Se puede considerar que el vector de pesos $\boldsymbol{u}$ se «aprende» a partir de una forma de retroalimentación procedente de la salida η. Entonces, bajo supuestos razonables, Oja [**Oja1982SimplifiedNM**] ha demostrado que $\boldsymbol{u}_i$ converge al vector propio asociado al valor propio más grande de $\boldsymbol{\Sigma}$. ■

Para interpretar el esquema hebbiano normalizado (2.1.25), consideremos el objetivo de PCA para la muestra de datos (2.1.13) como una aproximación a la versión *poblacional* del objetivo, definida mediante una esperanza sobre la distribución de las muestras $\boldsymbol{x}$:

$$\min_{\boldsymbol{U}^\top \boldsymbol{U} = \boldsymbol{I}} \mathbb{E}_{\boldsymbol{x}} \left[\left\| \boldsymbol{x} - \boldsymbol{U}\boldsymbol{U}^\top \boldsymbol{x} \right\|_2^2 \right] \tag{2.1.26}$$

Profundizaremos en este objetivo en la siguiente sección. Entonces, el esquema hebbiano normalizado puede interpretarse como una aproximación de primer orden a un esquema de descenso por gradiente proyectado *estocástico* sobre la Ecuación (2.1.26) (con un tamaño de lote 1, y con el número de columnas de $\boldsymbol{U}$ igual a 1) siempre que $\|\boldsymbol{u}\|_2 = 1$, lo cual se mantiene mediante la operación de proyección en (2.1.25). Un *algoritmo en línea* de este tipo es un elemento básico útil para algoritmos más complejos que que procesan datos con estructura secuencial y causal, como veremos con más detalle en la **??** y la **??**. Curiosamente y en términos más generales, el sencillo algoritmo de PCA en línea sugiere que *algoritmos más sencillos que la retropropagación (de extremo a extremo) pueden lograr aprender pares de codificador y decodificador*, lo cual revisaremos brevemente en el **??** como un objetivo importante para el estudio futuro de la inteligencia.

2.1.4 La perspectiva estadística: PCA probabilístico

Nótese que la formulación anterior no establece ningún supuesto estadístico sobre el proceso generador de datos. Sin embargo, es habitual incluir elementos

estadísticos en un modelo de datos dado, ya que pueden aportar interpretaciones adicionales más esclarecedoras sobre el resultado del análisis. Por lo tanto, nos surge la pregunta natural: *¿Cuál es el análogo estadístico de una estructura de baja dimensión?* Nuestra respuesta es que una *distribución* de baja dimensión es aquella cuyo soporte se concentra en torno a una estructura geométrica de baja dimensión.

Para ilustrar este punto, abordamos el *análisis probabilístico de componentes principales (PPCA)* [**TippingM1999**; **DBLP:conf/nips/Roweis97**]. Esta formulación puede considerarse una variante estadística del PCA convencional. Desde el punto de vista matemático, consideramos ahora nuestros datos como muestras de una variable aleatoria $\boldsymbol{x}$ que toma valores en $\mathbb{R}^D$ (lo que a veces también se denomina un *vector aleatorio*). Decimos que $\boldsymbol{x}$ tiene una estructura estadística (aproximada) de bajo rango si y solo si existen una matriz ortonormal $\boldsymbol{U} \in \mathsf{O}(D, d)$, una variable aleatoria $\boldsymbol{z}$ que toma valores en $\mathbb{R}^d$, así como una *pequeña* variable aleatoria $\boldsymbol{\varepsilon}$ que toma valores en $\mathbb{R}^D$ tal que $\boldsymbol{z}$ y $\boldsymbol{\varepsilon}$ son independientes y

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}. \tag{2.1.27}$$

Nuestro objetivo es, una vez más, recuperar $\boldsymbol{U}$. Con este fin, planteamos el problema análogo al de la Sección 2.1.1, es decir, la optimización sobre los subespacios de soporte $\tilde{\boldsymbol{U}}$ y las variables aleatorias $\boldsymbol{z}$ para resolver el siguiente problema:

$$\min_{\tilde{\boldsymbol{U}}, \tilde{\boldsymbol{z}}} \mathbb{E}\, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{z}}\|_2^2. \tag{2.1.28}$$

Dado que estamos buscando la mejor variable aleatoria $\boldsymbol{z}$ de este tipo, podemos calcular su valor concreto $\boldsymbol{z}(\boldsymbol{x})$ por separado para cada valor de $\boldsymbol{x}$. Llevando a cabo los mismos cálculos que en Sección 2.1.1, obtenemos

$$\min_{\tilde{\boldsymbol{U}}, \tilde{\boldsymbol{z}}} \mathbb{E}\, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{z}}\|_2^2 = \min_{\tilde{\boldsymbol{U}}} \mathbb{E} \min_{\tilde{\boldsymbol{z}}(\boldsymbol{x})} \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{z}}(\boldsymbol{x})\|_2^2 = \min_{\tilde{\boldsymbol{U}}} \mathbb{E}\, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top \boldsymbol{x}\|_2^2, \tag{2.1.29}$$

reafirmando una vez más el hecho de que el subespacio estimado con componentes principales $\boldsymbol{U}^\star$ corresponde a un *denoiser* $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top$ que se proyecta sobre ese subespacio. Como antes, obtenemos

$$\arg\min_{\tilde{\boldsymbol{U}}} \mathbb{E}\, \|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top \boldsymbol{x}\|_2^2 = \arg\max_{\tilde{\boldsymbol{U}}} \mathbb{E}\, \|\tilde{\boldsymbol{U}}^\top \boldsymbol{x}\|_2^2 \tag{2.1.30}$$

$$= \arg\max_{\tilde{\boldsymbol{U}}} \operatorname{tr}(\tilde{\boldsymbol{U}}^\top\, \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]\tilde{\boldsymbol{U}}), \tag{2.1.31}$$

y la solución a este último problema consiste simplemente en las d primeras componentes principales de la matriz de momentos de segundo orden $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$. En realidad, los problemas anteriores son visualmente muy similares a las ecuaciones para calcular las componentes principales de la subsección anterior, salvo que $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ sustituye a $\boldsymbol{X}\boldsymbol{X}^\top/N$. De hecho, esta última cantidad es una estimación de la primera. Ambas formulaciones hacen efectivamente lo mismo y tienen la misma solución práctica: calcular los vectores singulares izquierdos de la matriz de datos $\boldsymbol{X}$, o, lo que es lo mismo, los vectores propios superiores

de la matriz de covarianza estimada $\boldsymbol{X}\boldsymbol{X}^\top/N$. Sin embargo, la formulación estadística tiene una interpretación adicional. Supongamos que $\mathbb{E}[\boldsymbol{z}] = \mathbf{0}$ y $\mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}$. Tenemos que

$$\mathbb{E}[\boldsymbol{x}] = \boldsymbol{U}\,\mathbb{E}[\boldsymbol{z}] + \mathbb{E}[\boldsymbol{\varepsilon}] = \mathbf{0}, \tag{2.1.32}$$

de modo que $\operatorname{Cov}[\boldsymbol{x}] = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$. Ahora, desarrollando $\operatorname{Cov}[\boldsymbol{x}]$ tenemos

$$\operatorname{Cov}[\boldsymbol{x}] = \boldsymbol{U}\operatorname{Cov}[\boldsymbol{z}]\boldsymbol{U}^\top + \operatorname{Cov}[\boldsymbol{\varepsilon}] = \boldsymbol{U}\,\mathbb{E}[\boldsymbol{z}\boldsymbol{z}^\top]\boldsymbol{U}^\top + \operatorname{Cov}[\boldsymbol{\varepsilon}]. \tag{2.1.33}$$

En particular, si $\operatorname{Cov}[\boldsymbol{\varepsilon}]$ es pequeño, se cumple que $\operatorname{Cov}[\boldsymbol{x}] = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ es aproximadamente una matriz de bajo rango, concretamente de rango d. Por lo tanto, los d vectores propios principales de $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ resumen esencialmente toda la matriz de covarianza. Pero también son las componentes principales, por lo que podemos interpretar el análisis de componentes principales como una descomposición de bajo rango de $\operatorname{Cov}[\boldsymbol{x}]$.

Nota 2.6. Al aplicar el enfoque probabilístico del PCA, logramos una comprensión más clara y cuantitativa de cómo se relaciona con la eliminación de ruido. En primer lugar, consideremos el problema de eliminación de ruido en (2.1.29), es decir

$$\min_{\tilde{\boldsymbol{U}}} \mathbb{E}\,\|\boldsymbol{x} - \tilde{\boldsymbol{U}}\tilde{\boldsymbol{U}}^\top\boldsymbol{x}\|_2^2. \tag{2.1.34}$$

No es muy difícil demostrar que si $\tilde{\boldsymbol{U}}$ tiene d columnas y si $\boldsymbol{\varepsilon}$ es una variable aleatoria gaussiana isotrópica, es decir, con distribución $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2\boldsymbol{I})$,[3], entonces para *cualquier* solución óptima $\boldsymbol{U}^\star$ a este problema, tenemos

$$\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top = \boldsymbol{U}\boldsymbol{U}^\top \tag{2.1.35}$$

y así, el verdadero subespacio de soporte, sea $\mathcal{S} \doteq \operatorname{col}(\boldsymbol{U})$, se obtiene como el espacio generado por las columnas de $\boldsymbol{U}^\star$, ya que

$$\mathcal{S} = \operatorname{col}(\boldsymbol{U}) = \operatorname{col}(\boldsymbol{U}\boldsymbol{U}^\top) = \operatorname{col}(\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top) = \operatorname{col}(\boldsymbol{U}^\star). \tag{2.1.36}$$

En concreto, el *mapa de eliminación de ruido* estimado $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top$ es una proyección ortogonal sobre $\mathcal{S}$, que empuja los puntos con ruido hacia el subespacio de soporte de la verdad de referencia (ground truth). Podemos obtener un resultado técnico similar en el caso de que solo dispongamos de un número finito de muestras como en el Teorema 2.1, pero esto requiere más esfuerzo y complejidad técnica. En resumen, tenemos el siguiente teorema informal.

Teorema 2.3. *Supongamos que la variable aleatoria $\boldsymbol{x}$ se puede escribir como*

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon} \tag{2.1.37}$$

donde $\boldsymbol{U} \in \mathsf{O}(D, d)$ captura la estructura de bajo rango, $\boldsymbol{z}$ es un vector aleatorio que toma valores en $\mathbb{R}^d$ y $\boldsymbol{\varepsilon}$ es un vector aleatorio que toma valores en $\mathbb{R}^D$, de tal

[3]Otras distribuciones también funcionan siempre y cuando abarquen todo $\mathbb{R}^D$, pero la gaussiana es la más fácil de manejar en este caso.

manera que $\boldsymbol{z}$ y $\boldsymbol{\varepsilon}$ son independientes y $\boldsymbol{\varepsilon}$ es pequeño. Entonces, las componentes principales $\boldsymbol{U}^\star \in \mathsf{O}(D,d)$ de nuestro conjunto de datos vienen dadas por los d primeros vectores propios de $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$ y corresponden aproximadamente al denoiser lineal óptimo: $\boldsymbol{U}^\star(\boldsymbol{U}^\star)^\top \approx \boldsymbol{U}\boldsymbol{U}^\top$.

2.1.5 Completamiento enmascarado de matrices de bajo rango

En las subsecciones anteriores, examinamos el problema de *aprender una distribución geométrica o estadística de bajo rango*, en la que los datos se obtuvieron mediante muestreo de un subespacio con ruido aditivo. Pero este no es el único tipo de perturbación derivada de una distribución de baja dimensión que vale la pena estudiar. En esta sección, presentamos otra clase de errores no aditivos que están cobrando cada vez más importancia en el aprendizaje profundo. Consideremos el caso en el que tenemos unos datos $\{\boldsymbol{x}_i\}_{i=1}^N$ generados según (2.1.1). Ahora los organizamos en una matriz $\boldsymbol{X} = \begin{bmatrix}\boldsymbol{x}_1, \dots, \boldsymbol{x}_N\end{bmatrix} \in \mathbb{R}^{D\times N}$. A diferencia del caso anterior, no observamos $\boldsymbol{X}$ directamente; en su lugar, imaginamos que nuestra observación se vio corrupta en el camino y que obtuvimos

$$\boldsymbol{Y} = \boldsymbol{M} \odot \boldsymbol{X}, \tag{2.1.38}$$

donde $\boldsymbol{M} \in \{0,1\}^{D\times N}$ es una *máscara* que conocemos y $\odot$ es la multiplicación elemento por elemento. En este caso, nuestro objetivo es recuperar $\boldsymbol{X}$ (a partir de ahí podemos utilizar el PCA para recuperar $\boldsymbol{U}$, etc.), partiendo únicamente de la observación corrupta $\boldsymbol{Y}$, la máscara $\boldsymbol{M}$ y el conocimiento de que $\boldsymbol{X}$ es (aproximadamente) de bajo rango. Esto se denomina *completamiento de matrices de bajo rango*.

Este y otros problemas similares generalizados de recuperación de estructuras de bajo rango se denominan problemas de «PCA robusta». Existen muchas fuentes excelentes que analizan algoritmos y enfoques para resolver estos problemas [**Wright-Ma-2022**]. Por lo tanto, no entraremos en detalle sobre el método de solución en este contexto. En su lugar, analizaremos bajo qué condiciones es *plausible* resolver este problema. Por un lado, en el caso más absurdo, supongamos que cada entrada de la matriz $\boldsymbol{X}$ se eligiera independientemente de todas las demás. Entonces no habría ninguna posibilidad de recuperar $\boldsymbol{X}$ con exactitud, incluso si solo faltara una entrada y tuviéramos $DN-1$ entradas. Por otro lado, supongamos que $\boldsymbol{X}$ tuviera exactamente rango 1, lo cual es una condición extremadamente fuerte sobre la estructura de baja dimensión de los datos y que estuviéramos trabajando con la máscara

$$\boldsymbol{M} = \begin{bmatrix} \mathbf{1}_{(D-1)\times 1} & \mathbf{0}_{(D-1)\times(N-1)} \\ 1 & \mathbf{1}_{1\times(N-1)} \end{bmatrix}. \tag{2.1.39}$$

Entonces sabemos que los datos se distribuyen en una recta y conocemos un vector en esa recta: no es más que la primera columna de la matriz $\boldsymbol{Y} = \boldsymbol{M}\odot\boldsymbol{X}$. A partir de la última coordenada de cada columna, que también nos revela la máscara, podemos resolver cada columna, ya que para cada coordenada final

solo hay un vector en la recta que pasa por esa coordenada. De este modo, podemos reconstruir $\boldsymbol{X}$ con total precisión, para lo que solo necesitamos un número lineal de observaciones $D + N - 1$.

En el mundo real, los problemas concretor se sitúan en algún punto intermedio entre los dos casos límite mencionados anteriormente. Sin embargo, las diferencias entre estos dos extremos, así como la discusión anterior sobre el PCA, revelan un núcleo general de verdad:

> *Cuanto menor sea la dimensión y más estructurada sea la distribución de los datos, tanto más fácil resulta procesarlos y menos observaciones se necesitan, siempre y cuando el algoritmo aproveche eficazmente esta estructura de baja dimensión.*

Como cabía esperar, nos encontraremos con este motivo una y otra vez en el resto del manuscrito, empezando ya en la siguiente sección.[4]

2.2 Una mezcla de subespacios completos de baja dimensión

Como hemos visto, los modelos de señales de bajo rango son lo suficientemente ricos en estructura como para ofrecer una visión global de la interacción entre la dimensionalidad baja de los datos y los algoritmos computacionales eficientes y escalables para la representación y recuperación en presencia de errores. Estos modelos implican un proceso de aprendizaje de representaciones *lineal* y simétrico (2.1.6):

$$\boldsymbol{z} = \mathcal{E}(\boldsymbol{x}) = \boldsymbol{U}^\top \boldsymbol{x}, \quad \hat{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z}) = \boldsymbol{U}\boldsymbol{z},$$

de la que se puede demostrar que se aprende a partir de un número finito de muestras de $\boldsymbol{x}$ mediante el análisis de componentes principales (resuelto de manera eficiente, por ejemplo, con el método de potencias) siempre que la distribución de $\boldsymbol{x}$ sea realmente lineal. Se trata de una hipótesis restrictiva, pues, tal y como objetó Harold Hotelling, el distinguido estadístico del siglo XX,[5] cuando George Dantzig presentó por primera vez su teoría de la programación lineal[**Dantzig2002-eh**],

> *"...todos sabemos que el mundo es no lineal."*

Aun teniendo en cuenta su elegancia y simplicidad, la suposición de bajo rango es demasiado restrictiva como para ser ampliamente aplicable al modelado

[4] En capítulos posteriores, veremos que es posible completar datos del mundo real, como imágenes, con solo una fracción de observaciones sin enmascarar; por ejemplo, la autocodificación enmascarada de imágenes en la **??**. Esto se debe también a que la distribución de las imágenes naturales presenta estructuras de dimensiones bastante bajas (aunque no lineales) en el espacio de todas las imágenes.

[5] Por casualidad, también famoso por sus contribuciones al desarrollo y la denominación del Análisis de Componentes Principales [**Hotelling1933**].

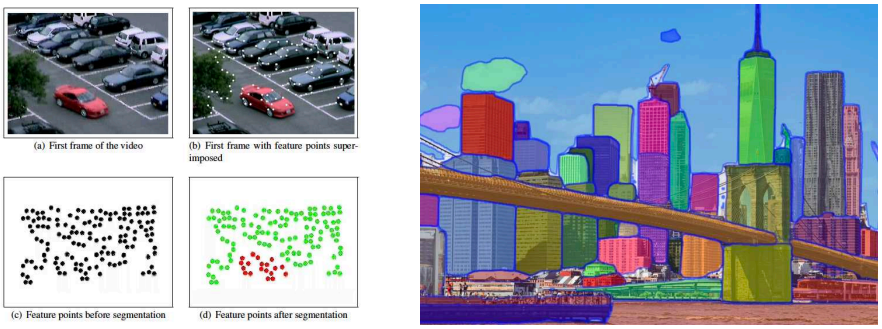


Figura 2.4: **Izquierda:** características rastreadas en objetos que se mueven en forma independiente dentro de una escena. **Derecha:** fragmentos de imagen asociados a diferentes regiones de una imagen.

de datos del mundo real. Una limitación clave es la suposición de un *único* subespacio lineal responsable de generar las observaciones estructuradas. En muchas aplicaciones prácticas, la estructura generada por una *mezcla* de subespacios distintos de baja dimensión ofrece un modelo más realista. Por ejemplo, consideremos una secuencia de vídeo que captura el movimiento de varios objetos distintos, cada uno de los cuales con su propio desplazamiento independiente (Figura 2.4 a la izquierda). Bajo supuestos adecuados sobre los movimientos individuales, cada objeto se convierte en responsable de un subespacio independiente de baja dimensión en la secuencia concatenada de fotogramas de vídeo [**VidalR2004-ECCV**]. Como otro ejemplo, pensemos en la modelación de imágenes naturales mediante el aprendizaje de un modelo para la distribución de *parches*, conjuntos de píxeles espacialmente contiguos dentro de una imagen (Figura 2.4 a la derecha). A diferencia del entorno simplificado de la Figura 2.2, donde las imágenes con poses correspondientes pueden aproximarse bien mediante un único subespacio de baja dimensión, el fragmento situado en una ubicación específica de una imagen natural puede corresponder a objetos con propiedades muy diferentes, por ejemplo, colores o formas distintos debido a los bordes de la oclusión. Por lo tanto, modelar la distribución de los parches con un único subespacio resulta estéril, pero una *mezcla* de subespacios, uno para cada región, ofrece unos resultados que funcionan sorprendentemente bien en la práctica, por ejemplo, para fines de segmentación o compresión [**Mobahi-IJCV2011**].[6] Veremos un ejemplo concreto en un capítulo posterior (Ejemplo 4.10).

En esta sección, comenzaremos examinando los fundamentos conceptuales y algorítmicos del aprendizaje de representaciones estructuradas cuando la distribución de los datos puede modelarse mediante *una mezcla de subespacios de baja dimensión*, tal y como se ilustra en la Figura 2.1. En este contexto,

[6] Volveremos sobre esta observación en el Capítulo 5, donde demostraremos que puede generalizarse de manera significativa para aprender representaciones de conjuntos de datos modernos a gran escala.

el mapeo del decodificador será casi tan simple como en el caso de un único subespacio: simplemente lo representamos mediante

$$\hat{\boldsymbol{x}} = \mathcal{D}(\boldsymbol{z}) = \left(\sum_{k=1}^{K} \pi_k(\boldsymbol{z}) \boldsymbol{U}_k \right) \boldsymbol{z}, \tag{2.2.1}$$

donde $\pi_k : \mathbb{R}^d \to \{0, 1\}$ es un conjunto de variables aleatorias de ponderación *esparsas*, de tal manera que en la suma solo se selecciona un único subespacio $\mathcal{S}_k = \text{col}(\boldsymbol{U}_k)$. Sin embargo, la tarea de codificar dichos datos $\boldsymbol{x}$ en representaciones adecuadas $\boldsymbol{z}$ y de aprender dicho par codificador-decodificador a partir de los datos, resultará más compleja.

Veremos cómo las ideas que provienen de la amplia bibliografía sobre *representación esparsa* y *análisis de componentes independientes* conducen a una reformulación natural de la arquitectura del decodificador anterior desde la perspectiva de la esparcidad, la arquitectura correspondiente del codificador (obtenida mediante un algoritmo similar al método de potencias, análogo al del análisis de componentes principales) y garantías sólidas de corrección y eficiencia para el aprendizaje de dichos pares codificador-decodificador a partir de los datos. En este sentido, el caso de la estructura lineal mixta de baja dimensión ya nos lleva a muchas de las componentes conceptuales clave del aprendizaje de representaciones estructuradas que desarrollaremos con una generalidad mucho mayor en este libro.

2.2.1 Mezclas de subespacios y diccionarios con codificación esparsa

Sea $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K$, cada uno de tamaño $D \times d$, una colección de bases ortonormales para K subespacios de dimensión d en $\mathbb{R}^D$. Decir que $\boldsymbol{x}$ sigue una distribución de mezcla de subespacios parametrizada por $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K$ significa, en términos geométricos, que

$$\boldsymbol{x} = \boldsymbol{U}_k \boldsymbol{z} \quad \text{for some } \; k \in [K], \;\; \boldsymbol{z} \in \mathbb{R}^d. \tag{2.2.2}$$

El análogo estadístico de este modelo geométrico, tal como vimos en el caso del PCA y la estructura lineal, es que $\boldsymbol{x}$ sigue una distribución de *mezcla de gaussianas*: es decir,

$$\boldsymbol{x} \sim \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k \boldsymbol{U}_k^\top), \quad \text{for some } \; \pi_k \geq 0, \;\; \sum_{k=1}^{K} \pi_k = 1. \tag{2.2.3}$$

En otras palabras, para cada $k \in [K]$, $\boldsymbol{x}$ es gaussiano en el subespacio de baja dimensión $\text{col}(\boldsymbol{U}_k)$ con probabilidad π_k.

Nota 2.7 (Mezcla de gaussianas vs superposición de gaussianas). Hay que tener en cuenta que el modelo anterior (2.2.3) es una *mezcla* de distribuciones

gaussianas, que no debe confundirse con una mezcla de variables gaussianas por superposición, por ejemplo

$$\boldsymbol{x} = \sum_{i=1}^{n} w_i \boldsymbol{x}_i, \quad \boldsymbol{x}_i \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_i \boldsymbol{U}_i^\top), \tag{2.2.4}$$

donde $\boldsymbol{x}_i$ son vectores aleatorios gaussianos independientes y w_i son un conjunto de pesos fijos. Como sabemos por las propiedades de los vectores gaussianos, dicha superposición $\boldsymbol{x}$ seguirá siendo una distribución gaussiana.

Por ahora, nos centramos en la perspectiva geométrica que ofrece (2.2.2). Existe una alternativa algebraicamente conveniente a esta representación condicional. Consideremos un vector de representación *elevado* $\boldsymbol{z} = [\boldsymbol{z}_1^\top, \ldots, \boldsymbol{z}_K^\top]^\top \in \mathbb{R}^{dK}$, tal que $\boldsymbol{z}$ es *d-esparso* y tiene su soporte en uno de los K bloques consecutivos no superpuestos de d coordenadas de entre las dK. Entonces, (2.2.2) puede escribirse de forma equivalente como

$$\boldsymbol{x} = \underbrace{\begin{bmatrix} | & \cdots & | \\ \boldsymbol{U}_1 & \cdots & \boldsymbol{U}_K \\ | & \cdots & | \end{bmatrix}}_{\boldsymbol{U}} \underbrace{\begin{bmatrix} \boldsymbol{z}_1 \\ \vdots \\ \boldsymbol{z}_K \end{bmatrix}}_{\boldsymbol{z}}, \quad \left\| \begin{bmatrix} \|\boldsymbol{z}_1\|_2 \\ \vdots \\ \|\boldsymbol{z}_K\|_2 \end{bmatrix} \right\|_0 = 1. \tag{2.2.5}$$

Aquí, la «norma» ℓ^0 $\|\cdot\|_0$ mide la esparcidad contando el número de entradas distintas de cero:

$$\|\boldsymbol{z}\|_0 = |\{i \mid z_i \neq 0\}|, \tag{2.2.6}$$

y la matriz $\boldsymbol{U} \in \mathbb{R}^{D \times Kd}$ se denomina *diccionario*, cuyos bloques de columnas $\{\boldsymbol{U}_i\}_{i=1}^K$ se llaman palabras de código. En general, si el número de subespacios en la mezcla K es lo suficientemente grande, no hay ningún límite en cuanto al número de columnas que contiene el diccionario $\boldsymbol{U}$. En el caso de que $Kd < D$, $\boldsymbol{U}$ se denomina *subcompleta*, cuando $Kd = D$, se denomina *completa*; y cuando $Kd > D$, se denomina *sobrecompleta*.

Ahora bien, (2.2.5) sugiere una relajación conveniente para la manejabilidad del análisis: en lugar de modelar $\boldsymbol{x}$ como procedente de una mezcla de K subespacios específicos $\boldsymbol{U}_1, \ldots, \boldsymbol{U}_K$, podríamos, en su lugar, comenzar con un diccionario $\boldsymbol{U} \in \mathbb{R}^{D \times m}$, donde m puede ser menor o mayor que D, y simplemente buscar representar $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z}$ con la esparcidad $\|\boldsymbol{z}\|_0$ suficientemente pequeña. Esto nos lleva al *modelo de diccionario esparso* para $\boldsymbol{x}$:

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}, \quad \|\boldsymbol{z}\|_0 \ll d, \tag{2.2.7}$$

donde $\boldsymbol{\varepsilon}$ representa un vector de ruido desconocido. Desde el punto de vista geométrico, esto implica que $\boldsymbol{x}$ se encuentra cerca del espacio generado por un subconjunto de $\|\boldsymbol{z}\|_0$ columnas de $\boldsymbol{U}$, lo que convierte esto en una instancia del modelo de mezcla de subespacios (2.2.2) con un valor muy elevado de K y correlaciones específicas entre los subespacios $\boldsymbol{U}_k$.

Diccionario ortogonal para una codificación esparsa. Ahora podemos formular el problema del aprendizaje de representaciones estructuradas para mezclas de subespacios de baja dimensión que estudiaremos en esta sección. Suponemos que disponemos de muestras $\boldsymbol{X} = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_N]$ procedentes de un modelo de diccionario esparso desconocido (2.2.7), posiblemente con ruido añadido $\boldsymbol{\varepsilon}_i$. Partamos de la suposición de que el diccionario $\boldsymbol{U}$ en el modelo de diccionario esparso (2.2.7) es completo y ortogonal,[7] y que cada vector de coeficientes $\boldsymbol{z}$ es d-esparso, con $d \ll D$. En este contexto, el aprendizaje de representaciones equivale a aprender correctamente el diccionario ortogonal $\boldsymbol{U}$ mediante optimización: entonces podemos tomar $\mathcal{E}(\boldsymbol{x}) = \boldsymbol{U}^\top \boldsymbol{x}$ como codificador y $\mathcal{D}(\boldsymbol{z}) = \boldsymbol{U}\boldsymbol{z}$ como decodificador y $\mathcal{D} = \mathcal{E}^{-1}$. En forma de diagrama:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}=\boldsymbol{U}^\top} \boldsymbol{z} \xrightarrow{\mathcal{D}=\boldsymbol{U}} \hat{\boldsymbol{x}}. \tag{2.2.8}$$

Observamos que el par $(\mathcal{E}, \mathcal{D})$ de autocodificadores para el aprendizaje completo de un diccionario es simétrico, al igual que en el caso de un único subespacio lineal, lo que hace que la tarea computacional de codificación y decodificación no sea más difícil que en el caso lineal. Por otro lado, la tarea de aprender el diccionario $\boldsymbol{U}$ es estrictamente más difícil que aprender un único subespacio lineal mediante el PCA. Para comprender por qué no podemos limitarnos a utilizar el PCA para aprender correctamente el diccionario ortogonal $\boldsymbol{U}$, observemos que la función de pérdida que dio origen al PCA, es decir, (2.1.5), es completamente invariante ante las rotaciones de las filas de la matriz $\boldsymbol{U}$: es decir, si $\boldsymbol{Q}$ es cualquier matriz ortogonal $d \times d$, entonces tanto $\boldsymbol{U}$ como $\boldsymbol{U}\boldsymbol{Q}$ son viables y tienen una pérdida idéntica para (2.1.5). El modelo de diccionario esparso no es, en absoluto, invariante ante tales transformaciones: si reemplazáramos $\boldsymbol{U}$ por $\boldsymbol{U}\boldsymbol{Q}$ y realizáramos la rotación correspondiente $\boldsymbol{Q}^\top \boldsymbol{z}$ de los coeficientes de representación $\boldsymbol{z}$, en general destruiríamos la estructura de esparcidad de $\boldsymbol{z}$, lo que violaría la suposición del modelo. Por lo tanto, necesitamos nuevos algoritmos para aprender diccionarios ortogonales.

2.2.2 Aprendizaje de diccionarios completos

En esta sección, derivaremos algoritmos para resolver el problema del aprendizaje de diccionarios ortogonales. Para ser más precisos, suponemos que el vector observado $\boldsymbol{x} \in \mathbb{R}^D$ sigue un modelo estadístico

$$\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}, \tag{2.2.9}$$

donde $\boldsymbol{U} \in \mathbb{R}^{D \times D}$ es un diccionario ortogonal desconocido, $\boldsymbol{z}$ es un vector de ruido aleatorio, cuyas componentes z_i son independientes entre sí, gaussianas y de pequeña magnitud, mientras $\boldsymbol{\varepsilon} \in \mathbb{R}^D$ es un vector aleatorio independiente de pequeños ruidos (gaussianos). El objetivo es recuperar $\boldsymbol{U}$ (y, por lo tanto, $\boldsymbol{z}$) a partir de muestras de $\boldsymbol{x}$.

[7] Se puede demostrar que, para el caso completo, no perdemos generalidad al hacer la suposición de ortogonalidad (Ejercicio 2.4).

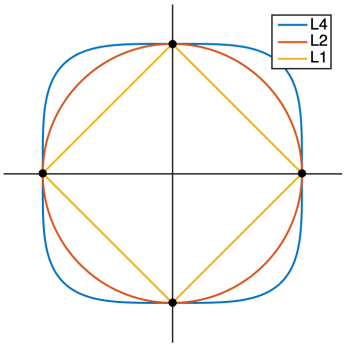


Figura 2.5: Maximizar la norma ℓ^4 o minimizar la norma ℓ^1 promueve la esparcidad (para vectores sobre la esfera).

Aquí suponemos que cada componente independiente z_i sigue una distribución de la forma

$$z_i \sim \mathrm{Ber}(\theta) \cdot \mathcal{N}(0, 1/\theta).$$

Es decir, es el producto de una variable aleatoria de Bernoulli con probabilidad θ de tomar el valor 1 y probabilidad $1-\theta$ de tomar el valor 0 y una variable aleatoria gaussiana independiente con varianza $1/\theta$. Esta distribución se conoce formalmente como distribución *de Bernoulli-Gauss*. La normalización se elige de tal manera que $\mathrm{Var}(z_i) = 1$ y, por lo tanto, $\mathbb{E}[\|\boldsymbol{z}\|_2^2] = d$. Este supuesto de modelado implica que el vector de componentes independientes $\boldsymbol{z}$ suele ser muy esparso: calculamos $\mathbb{E}\left[\|\boldsymbol{z}\|_0\right] = d\theta$, que es pequeño cuando θ es inversamente proporcional a d.

Nota 2.8 (La suposición de ortogonalidad). A primera vista, la suposición de que el diccionario $\boldsymbol{U}$ es ortogonal podría parecer algo restrictiva. Pero, en realidad, no hay ninguna pérdida de generalidad. Se puede considerar como un diccionario completo a cualquier matriz cuadrada invertible $\boldsymbol{U}$. Con muestras generadas a partir de este diccionario: $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{Z} \in \mathbb{R}^{D\times N}$, es fácil demostrar que, con cierto precondicionamiento de la matriz de datos $\boldsymbol{X}$:

$$\bar{\boldsymbol{X}} = \left(\frac{1}{N\theta}\boldsymbol{X}\boldsymbol{X}^\top\right)^{-\frac{1}{2}}\boldsymbol{X}, \tag{2.2.10}$$

entonces existe una matriz ortogonal $\boldsymbol{U}_o \in \mathsf{O}(D)$ tal que

$$\bar{\boldsymbol{X}} = \boldsymbol{U}_o\boldsymbol{Z}. \tag{2.2.11}$$

Véase el Ejercicio 2.4 o [**sun2017completeI**] para más detalles.

Aprendizaje de diccionarios mediante el algoritmo MSP Supongamos ahora que se nos da un conjunto de observaciones:

$$\boldsymbol{x}_i = \boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i,\ \forall i \in [N]. \tag{2.2.12}$$

Sea $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ y $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N]$. El objetivo es recuperar $\boldsymbol{U}$ a partir de los datos $\boldsymbol{X}$. Por lo tanto, para toda matriz ortogonal $\boldsymbol{A} \in \mathsf{O}(D)$,

$$\boldsymbol{A}\boldsymbol{x}_i = \boldsymbol{A}\boldsymbol{U}\boldsymbol{z}_i + \boldsymbol{A}\boldsymbol{\varepsilon}_i \tag{2.2.13}$$

sería casi esparsa si $\boldsymbol{A} = \boldsymbol{U}^\top$ (puesto que, según la hipótesis, el ruido $\boldsymbol{\varepsilon}_i$ es de magnitud pequeña).

Además, dado que $\boldsymbol{U}$ es ortogonal y que $\boldsymbol{\varepsilon}$ es pequeño, el vector $\boldsymbol{x}$ tiene una norma esperada predecible, es decir, $\mathbb{E}[\|\boldsymbol{x}\|_2^2] \approx \mathbb{E}[\|\boldsymbol{z}\|_2^2] = d$. Es un hecho conocido que, para vectores en una esfera, maximizar la norma ℓ^4 es equivalente a minimizar la norma ℓ^0 (para favorecer la esparcidad),

$$\arg\max_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_4 \quad \Leftrightarrow \quad \arg\min_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_0. \tag{2.2.14}$$

Esto se ilustra en la Figura 2.5.

Una matriz ortogonal $\boldsymbol{A}$ conserva la norma euclidiana (ℓ^2): $\|\boldsymbol{A}\boldsymbol{x}\|_2^2 = \|\boldsymbol{x}\|_2^2$. Por lo tanto, para hallar el diccionario ortogonal correcto $\boldsymbol{U}$ a partir de $\boldsymbol{X}$, podemos intentar resolver el siguiente programa de optimización:

$$\max_{\tilde{\boldsymbol{A}}\in\mathsf{O}(D)} \frac{1}{4}\left\|\tilde{\boldsymbol{A}}\boldsymbol{X}\right\|_4^4 = \frac{1}{4}\sum_{i=1}^{N}\left\|\tilde{\boldsymbol{A}}\boldsymbol{x}_i\right\|_4^4 \tag{2.2.15}$$

Esto se conoce como el problema de maximización de la norma ℓ^4 [**Zhai-2020**]. Una vez que encontramos la solución $\boldsymbol{A}^\star$, podemos calcular la transpuesta $\boldsymbol{U}^\star = (\boldsymbol{A}^\star)^\top$.

Nota 2.9. También se sabe que, para vectores en una esfera, minimizar la norma ℓ^1 equivale a minimizar la norma ℓ^0 (para favorecer la esparcidad),

$$\arg\min_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_1 \quad \Leftrightarrow \quad \arg\min_{\boldsymbol{z}\in\mathbb{S}^n} \|\boldsymbol{z}\|_0,$$

lo cual también se ilustra en la Figura 2.5. Este hecho también puede aprovecharse para aprender el diccionario $\boldsymbol{A}$ de manera eficaz y eficiente. En realidad, este enfoque ya se había explorado con anterioridad a la norma ℓ^4 empleada en este trabajo. Los lectores interesados pueden consultar el trabajo de [**qu2020findingsparsestvectorssubspace**].

Obsérvese que el problema anterior equivale al siguiente problema de optimización con restricciones:

$$\min -\frac{1}{4}\left\|\tilde{\boldsymbol{A}}\boldsymbol{X}\right\|_4^4 \quad \text{subject to} \quad \tilde{\boldsymbol{A}}^\top\tilde{\boldsymbol{A}} = \boldsymbol{I}. \tag{2.2.16}$$

Como se muestra en [**Wright-Ma-2022**], utilizando el método de los multiplicadores de Lagrange, se puede deducir que la solución óptima al problema debe satisfacer la siguiente condición de «punto fijo»:

$$\boldsymbol{A}^\star = \mathcal{P}_{\mathrm{O}(D)}[(\boldsymbol{A}^\star \boldsymbol{X})^{\circ 3} \boldsymbol{X}^\top], \tag{2.2.17}$$

donde $\mathcal{P}_{\mathrm{O}(D)}[\,\cdot\,]$ es la proyección sobre el espacio de matrices ortogonales $\mathrm{O}(D)$.[8]

Para calcular el punto fijo de la ecuación anterior, en forma similar a a la utilizada para calcular los vectores propios en el PCA (2.1.16), podemos utilizar la siguiente iteración de potencias:

$$\tilde{\boldsymbol{A}}_{t+1} = \mathcal{P}_{\mathrm{O}(D)}[(\tilde{\boldsymbol{A}}_t \boldsymbol{X})^{\circ 3} \boldsymbol{X}^\top]. \tag{2.2.18}$$

Esto se conoce como el algoritmo de *coincidencia, estiramiento y proyección* (MSP) propuesto por [**Zhai-2020**]. Se demostró que, en condiciones generales, un algoritmo voraz de este tipo converge efectivamente hacia la solución correcta a un ritmo superlineal.

Nota 2.10 (Optimalidad global de la maximización de ℓ^4). El problema de maximización con restricciones de ℓ^4 es un programa no convexo. En general, *no* se debe esperar que algún algoritmo voraz (por ejemplo, del tipo de descenso por gradiente) converja hacia la solución óptima global. Sorprendentemente, se puede demostrar que, a diferencia de los programas no convexos generales, el paisaje de la maximización de ℓ^4 sobre una esfera

$$\min -\frac{1}{4}\left\|\tilde{\boldsymbol{q}}^\top \boldsymbol{X}\right\|_4^4 \quad \text{subject to} \quad \tilde{\boldsymbol{q}}^\top \tilde{\boldsymbol{q}} = 1. \tag{2.2.19}$$

es muy benigno: todos los mínimos locales están cerca de los óptimos globales y todos los puntos críticos son puntos de silla con una dirección de curvatura negativa. Por lo tanto, cualquier método de descenso capaz de evitar los puntos de silla estrictos encuentra, de manera demostrable, soluciones óptimas globales. Para una formulación más rigurosa, se remite al lector interesado a [**Qu2020Geometric**].

Nota 2.11 (Red profunda lineal estable). El proceso iterativo anterior de cálculo del diccionario tiene una interpretación natural e incremental de «aprendizaje profundo». Definamos $\delta \boldsymbol{A}_{t+1} = \tilde{\boldsymbol{A}}_{t+1} \tilde{\boldsymbol{A}}_t^\top$ y $\boldsymbol{Z}_t = \tilde{\boldsymbol{A}}_t \boldsymbol{X}$, entonces es fácil demostrar que

$$\delta \boldsymbol{A}_{t+1} = \mathcal{P}_{\mathrm{O}(D)}[(\boldsymbol{Z}_t)^{\circ 3} \boldsymbol{Z}_t^\top].$$

Si $\tilde{\boldsymbol{A}}_t$ converge al diccionario correcto $\boldsymbol{U}$, entonces dicho proceso iterativo de codificación es esencialmente equivalente a una «red lineal profunda»:

$$\boldsymbol{Z} \longleftarrow \boldsymbol{Z}_{t+1} = \underbrace{\delta \boldsymbol{A}_{t+1} \delta \boldsymbol{A}_t \ldots \delta \boldsymbol{A}_1}_{\text{capas construidas en el paso directo}} \boldsymbol{X}.$$

[8] Para cualquier matriz $\boldsymbol{M}$ con SVD $\boldsymbol{M} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$, se tiene $\mathcal{P}_{\mathrm{O}(D)}[\boldsymbol{M}] = \boldsymbol{U}\boldsymbol{V}^\top$. Se deja esto como ejercicio para el lector.

Téngase en cuenta que el cálculo de las transformaciones de incremento $\delta \boldsymbol{A}_{t+1}$ en cada capa depende únicamente de la salida (de características) de la capa anterior $\boldsymbol{Z}_t$. La red es estable por naturaleza, ya que cada capa es una transformación ortogonal que conserva la norma. A pesar de su semejanza con una red profunda lineal, la retropropagación no es necesaria para el aprendizaje de cada capa. ¡Todas las capas se entrenan en una sola pasada hacia adelante!

2.2.3 Relación con ICA y la curtosis

En el modelo de Bernoulli-Gauss, las variables z_i son independientes y no gaussianas. Además, existe una clara correspondencia entre el aprendizaje de diccionarios y el análisis clásico de componentes independientes (ICA), hasta el punto de que los algoritmos diseñados para resolver un problema pueden utilizarse para resolver el otro.[9]

Con el fin de derivar un algoritmo basado en ICA, nos centramos en una función objetivo conocida como *curtosis*, que se utiliza en el ICA como consecuencia directa de la naturaleza no gaussiana de las componentes. La *curtosis*, o cumulante de cuarto orden, de una variable aleatoria X de media cero se define como

$$\operatorname{kurt}(X) = \mathbb{E}\, X^4 - 3(\mathbb{E}\, X^2)^2. \tag{2.2.20}$$

Si solo disponemos de un número finito de muestras de la variable aleatoria X organizadas en un vector $\boldsymbol{x} = [x_1, \ldots, x_N]$, definimos la curtosis a través de su media empírica, lo que da como resultado

$$\operatorname{kurt}(\boldsymbol{x}) = \frac{1}{N}\|\boldsymbol{x}\|_4^4 - \frac{3}{N^2}\|\boldsymbol{x}\|_2^4. \tag{2.2.21}$$

Por último, en el caso de los vectores aleatorios, definimos su curtosis como la suma de la curtosis escalar de cada componente. La curtosis es una función de pérdida natural para ICA, ya que, para una variable X gaussiana, la curtosis es cero; el lector puede comprobar además que la distribución de Bernoulli-Gauss tiene una curtosis positiva. Por lo tanto, un procedimiento natural para buscar componentes independientes no gaussianas consiste en buscar un conjunto de direcciones mutuamente ortogonales $\tilde{\boldsymbol{U}} \in \mathbb{R}^{d\times k}$, de tal manera que $\tilde{\boldsymbol{U}}^\top \boldsymbol{X}$ tenga la máxima curtosis, donde $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{Z} \in \mathbb{R}^{D\times N}$ es la matriz Bernoulli-gaussiana de datos ICA (de tal manera que $\tilde{\boldsymbol{U}}$ aproxima $\boldsymbol{U}$ salvo permutaciones de las columnas). Formalmente, nuestro objetivo es resolver el problema

$$\max_{\tilde{\boldsymbol{U}}^\top \tilde{\boldsymbol{U}} = \boldsymbol{I}} \operatorname{kurt}(\tilde{\boldsymbol{U}}^\top \boldsymbol{X}). \tag{2.2.22}$$

En un extremo, podemos establecer $k = D$ e intentar recuperar todo el diccionario $\boldsymbol{U}$ de una sola vez. Se puede demostrar que este problema se

[9] Analizamos este tema con mayor profundidad en el Ejercicio 2.3, donde se establece una relación entre la naturaleza no gaussiana de las componentes independientes y el concepto puramente geométrico de simetría. Este problema está relacionado con nuestra observación anterior de que el PCA no sirve para recuperar diccionarios ortogonales de uso esparso: en el contexto estadístico, puede relacionarse con la invariancia rotacional de la distribución gaussiana (Ejercicio 2.2).

puede resolver con el algoritmo MSP que hemos visto anteriormente. En el otro extremo, podemos fijar $k = 1$ y tratar de recuperar una sola dirección (columna de $\boldsymbol{U}$) a la vez, realizando una *deflación*, es decir, sustituyendo la matriz de datos $\boldsymbol{X}$ por $(\boldsymbol{I} - \boldsymbol{u}\boldsymbol{u}^\top)\boldsymbol{X}$, después de cada paso antes de buscar otra dirección. Existe una disyuntiva natural entre la escalabilidad del enfoque incremental $k = 1$ y la eficiencia y solidez del enfoque $k = D$.

ICA incremental: corrección y algoritmo FastICA. El algoritmo FastICA, propuesto por Hyvärinen y Oja [**hyvarinen-1997**], es un algoritmo rápido de punto fijo para resolver el esquema de maximización de la curtosis con $k = 1$ para ICA. El problema en cuestión es

$$\max_{\|\tilde{\boldsymbol{u}}\|_2^2=1} \operatorname{kurt}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}}). \tag{2.2.23}$$

En primer lugar, realizaremos un análisis muy básico de este objetivo para verificar su corrección. Obsérvese que, mediante el cambio de variables $\tilde{\boldsymbol{w}} = \boldsymbol{U}^\top \tilde{\boldsymbol{u}}$, este problema es equivalente a

$$\max_{\|\tilde{\boldsymbol{w}}\|_2^2=1} \operatorname{kurt}(\boldsymbol{Z}^\top \tilde{\boldsymbol{w}}).$$

Este objetivo es tan sencillo que podemos hacer afirmaciones contundentes sobre su corrección como formulación para recuperar el diccionario $\boldsymbol{U}$. Por ejemplo, en el contexto poblacional donde $N \to \infty$, podemos utilizar las propiedades de aditividad de la curtosis (Ejercicio 2.5) y nuestra normalización supuesta de las componentes independientes para expresar el problema anterior en forma equivalente como

$$\max_{\|\tilde{\boldsymbol{w}}\|_2^2=1} \sum_{i=1}^{d} \operatorname{kurt}(z_i)\tilde{w}_i^4. \tag{2.2.24}$$

Se puede mostrar que, bajo el supuesto Bernoulli-gaussiano, el paisaje de optimización de este problema es «benigno» (Ejercicio 2.7) — lo que significa que todos los máximos locales de la función objetivo corresponden a la recuperación de una de las componentes independientes.

En la práctica, no podemos calcular directamente el cambio de variables que lleva de $\tilde{\boldsymbol{u}}$ a $\tilde{\boldsymbol{w}}$, ya que requiere el diccionario de referencia $\boldsymbol{U}$, por lo que desarrollamos directamente algoritmos para la parametrización de $\tilde{\boldsymbol{u}}$ y la matriz de datos $\boldsymbol{X}$. Entonces, una forma eficaz y escalable de calcular uno de estos máximos es mediante algoritmos de optimización de primer orden, que siguen en forma iterativa el gradiente de la función objetivo y se proyectan sobre el conjunto de restricciones $\{\tilde{\boldsymbol{u}} \mid \|\tilde{\boldsymbol{u}}\|_2^2 = 1\}$. Dado que hemos supuesto que cada z_i cumple que $\operatorname{Var}(z_i) = 1$, tenemos que, para cualquier $\tilde{\boldsymbol{u}}$ y para un N suficientemente grande

$$\operatorname{kurt}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}}) \approx \tfrac{1}{N}\|\boldsymbol{X}^\top \tilde{\boldsymbol{u}}\|_4^4 - 3\|\tilde{\boldsymbol{u}}\|_2^4. \tag{2.2.25}$$

De este modo, podemos obtener una aproximación correspondiente al gradiente:

$$\nabla_{\tilde{\boldsymbol{u}}} \operatorname{kurt}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}}) \approx \tfrac{4}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 12\|\tilde{\boldsymbol{u}}\|_2^2 \tilde{\boldsymbol{u}}.$$

El algoritmo FastICA utiliza un método de punto fijo para calcular las direcciones de máxima curtosis. Se parte de las condiciones de optimalidad de primer orden para el problema de maximización de la curtosis, dadas la aproximación del gradiente anterior y el conjunto de restricciones, que son las siguientes:

$$\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} = \underbrace{\left\langle \tilde{\boldsymbol{u}}, \boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} \right\rangle}_{\lambda} \tilde{\boldsymbol{u}}, \tag{2.2.26}$$

donde el valor concreto de λ se determina utilizando la restricción de norma unitaria sobre $\tilde{\boldsymbol{u}}$. En el Ejercicio 2.6 se describen los fundamentos matemáticos necesarios para derivar estas condiciones de optimalidad a partir de los principios básicos. La Ecuación (2.2.26) es satisfecha por *cualquier* punto crítico del problema de maximización de la curtosis; queremos derivar una ecuación que sea satisfecha únicamente por los maximizadores. Tras observar que $\lambda = \|\boldsymbol{X}^\top \tilde{\boldsymbol{u}}\|_4^4$, reexpresamos en forma equivalente la ecuación (2.2.26) como la ecuación modificada

$$\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}} = \left(\frac{\lambda}{N} - 3\right)\tilde{\boldsymbol{u}}, \tag{2.2.27}$$

y constatamos que cualquier maximizador de (2.2.23) debe satisfacer $\lambda/N - 3 > 0$, suponiendo que N sea suficientemente grande. Por lo tanto, podemos *normalizar* ambos lados de (2.2.27), lo que nos da la siguiente ecuación de punto fijo que cumple todo maximizador de (2.2.23):

$$\frac{\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}}}{\left\|\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}}\right\|_2} = \tilde{\boldsymbol{u}}. \tag{2.2.28}$$

Al iterar el mapeo definido por el lado izquierdo de esta expresión de punto fijo, se obtiene el algoritmo FastICA de Hyvärinen y Oja [**hyvarinen-1997**]:

$$\begin{aligned} \tilde{\boldsymbol{v}}^+ &= \tfrac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \tilde{\boldsymbol{u}})^{\odot 3} - 3\tilde{\boldsymbol{u}}, \\ \tilde{\boldsymbol{u}}^+ &= \tilde{\boldsymbol{v}}^+ / \left\|\tilde{\boldsymbol{v}}^+\right\|_2 . \end{aligned} \tag{2.2.29}$$

Resulta que el algoritmo FastICA converge con extrema rapidez (de hecho, a una velocidad *cúbica*) hacia las columnas del diccionario $\boldsymbol{U}$; los lectores interesados pueden consultar [**hyvarinen-1997**] para obtener más detalles. Al comparar el algoritmo FastICA (2.2.29) con el método de potencias estudiado en la Sección 2.1.1 para el problema del PCA y el algoritmo MSP (2.2.18), observamos una sorprendente similitud. De hecho, FastICA es, en esencia, un método de potencias modificado, que utiliza el gradiente de la curtosis empírica en lugar de un simple gradiente lineal del objetivo del PCA.

2.3 Una mezcla de subespacios sobrecompletos de baja dimensión

Como hemos visto, el aprendizaje de diccionarios completos cuenta con una elegante teoría computacional en la que se mantiene una estructura de

autocodificación simétrica $\mathcal{E}(\boldsymbol{x}) = \boldsymbol{U}^\top \boldsymbol{x}$, $\mathcal{D}(\boldsymbol{z}) = \boldsymbol{U}\boldsymbol{z}$, junto con un algoritmo escalable similar al método de potencias (el algoritmo MSP) para aprender un diccionario/libro de códigos ortogonal $\boldsymbol{U}$ a partir de los datos. No obstante, para aprender representaciones de distribuciones generales de datos de alta dimensión, es necesario ampliar el tamaño del libro de códigos más allá del requisito de ortogonalidad, lo que significa que debemos tener $\boldsymbol{A} \in \mathbb{R}^{D\times m}$, con $m \gg D$, lo que corresponde al caso de un diccionario/libro de códigos *sobrecompleto*,[10] y el modelo de señal

$$\boldsymbol{x} = \boldsymbol{A}\boldsymbol{z} + \boldsymbol{\varepsilon}, \quad \|\boldsymbol{z}\|_0 = d \ll m. \tag{2.3.1}$$

Existen motivos tanto geométricos como físicos y de modelización para pasar al caso sobrecompleto. Desde el punto de vista geométrico, recordemos que en nuestra reducción original del modelo de datos de mezcla de subespacios al modelo de diccionario esparso, una mezcla de K subespacios en $\mathbb{R}^D$, cada uno de dimensión d, dio lugar a un diccionario de forma $\boldsymbol{A} \in \mathbb{R}^{D\times Kd}$. En otras palabras, los diccionarios sobrecompletos corresponden a mezclas *más ricas* de subespacios, con más modos distintos de variabilidad para modelar la distribución de datos de alta dimensión. En lo que respecta al modelado, podemos realizar un experimento computacional con datos del mundo real que ponga de manifiesto la capacidad de modelado adicional que aporta una representación sobrecompleta.

Ejemplo 2.2. A partir de una muestra de imágenes de dígitos escritos a mano, la Figura 2.6(a) muestra el resultado de ajustar un diccionario ortogonal al conjunto de datos. Por el contrario, la Figura 2.6(b) muestra el resultado de aplicar un algoritmo de optimización para el aprendizaje de diccionarios sobrecompletos (que presentaremos en detalle más adelante en este capítulo) a estas muestras. Nótese que las representaciones se vuelven mucho más esparsas y los libros de códigos mucho más interpretables: consisten en primitivas fundamentales para los trazos que componen los dígitos, es decir, aristas orientadas. ■

De hecho, el aprendizaje de diccionarios sobrecompletos se propuso originalmente como un algoritmo biológicamente plausible para la representación de imágenes, basado en evidencia empírica sobre cómo las primeras capas de la corteza visual representan los estímulos visuales [**Olshausen1996-ap**; **Olshausen1997-yv**].

En el resto de esta sección, expondremos los fundamentos conceptuales y computacionales del aprendizaje de diccionarios sobrecompletos. Suponiendo que el modelo (2.3.1) se cumple con códigos esparsos $\boldsymbol{z}$, un diccionario sobrecompleto $\boldsymbol{A}$ y un nivel de esparcidad d, y dadas las muestras $\boldsymbol{X} = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_N]$ de $\boldsymbol{x}$, queremos aprender un codificador $\mathcal{E} : \mathbb{R}^D \to \mathbb{R}^m$ que asigne a cada $\boldsymbol{x}$ su código esparso $\boldsymbol{z}$, y un decodificador $\mathcal{D}(\boldsymbol{z}) = \boldsymbol{A}\boldsymbol{z}$ que reconstruya cada $\boldsymbol{x}$ a partir de su código esparso. En forma de diagrama:

$$\boldsymbol{x} \xrightarrow{\mathcal{E}} \boldsymbol{z} \xrightarrow{\mathcal{D}=\boldsymbol{A}} \hat{\boldsymbol{x}}. \tag{2.3.2}$$

[10] Cambiamos la notación aquí de $\boldsymbol{U}$ a $\boldsymbol{A}$ para enfatizar la no ortogonalidad y la forma no cuadrada del diccionario sobrecompleto $\boldsymbol{A}$.

Comenzaremos por la construcción del codificador $\mathcal{E}$. Trabajaremos por etapas: en primer lugar, *dado el diccionario verdadero* $\boldsymbol{A}$, mostraremos cómo el problema de la *codificación esparsa* ofrece un algoritmo elegante, escalable y cuya corrección es demostrable para recuperar el código esparso $\boldsymbol{z}$ de $\boldsymbol{x}$. Aunque este problema es NP-difícil en el peor de los casos, puede resolverse de manera eficiente y escalable para diccionarios $\boldsymbol{A}$ que sean *incoherentes*, es decir, que tengan columnas que no estén demasiado correlacionadas. La arquitectura del codificador que resulta de esta solución ya no será simétrica: veremos que tiene la forma de una red profunda primitiva, que depende del diccionario $\boldsymbol{A}$.

A continuación, pasaremos a la tarea de aprender el decodificador $\mathcal{D}$, o lo que es lo mismo, el diccionario sobrecompleto $\boldsymbol{A}$. Derivaremos un algoritmo para el aprendizaje de diccionarios sobrecompletos que nos permita aprender simultáneamente el libro de códigos $\boldsymbol{A}$ y los códigos esparsos $\boldsymbol{z}$, utilizando ideas de la codificación esparsa. Por último, analizaremos una perspectiva más moderna sobre la codificación esparsa aprendible que nos lleva a una estructura de codificador-decodificador totalmente asimétrica, como alternativa a (2.3.2). En este caso, el decodificador corresponderá a una solución incremental al problema del aprendizaje de diccionarios esparsos y dará como resultado un par codificador-decodificador similar a una red profunda para el aprendizaje de diccionarios esparsos. Esta estructura anticipará muchos de los avances que se tratarán en el resto de la monografía, a medida que pasemos de los modelos analíticos a las redes neuronales modernas.

2.3.1 Codificación esparsa con diccionario sobrecompleto

En esta sección, analizaremos el modelo de datos (2.3.1), que da cabida a combinaciones lineales esparsas de numerosos motivos, o *átomos*. Dados los datos $\{\boldsymbol{x}_i\}_{i=1}^N \subseteq \mathbb{R}^D$ que satisfacen este modelo, es decir, que pueden expresarse como

$$\boldsymbol{x}_i = \boldsymbol{A}\boldsymbol{z}_i + \boldsymbol{\varepsilon}_i, \qquad \forall i \in [N] \tag{2.3.3}$$

para algún diccionario $\boldsymbol{A} \in \mathbb{R}^{D\times m}$ con m átomos, códigos esparsos $\boldsymbol{z}_i$ tales que $\|\boldsymbol{z}_i\|_0 \leq d$ y pequeños errores $\boldsymbol{\varepsilon}_i$, el problema de la codificación esparsa consiste en recuperar los códigos $\boldsymbol{z}_i$ con la mayor precisión posible a partir de los datos $\boldsymbol{x}_i$, dado el diccionario $\boldsymbol{A}$. Los algoritmos eficientes para resolver este problema tienen éxito cuando el diccionario $\boldsymbol{A}$ es *incoherente* en el sentido de que los productos internos $\boldsymbol{a}_i^\top \boldsymbol{a}_j$ son uniformemente pequeños, por lo que los átomos son casi ortogonales.[11]

Nótese que podemos agrupar los $\boldsymbol{x}_i$ en $\boldsymbol{X} = \big[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\big] \in \mathbb{R}^{D\times N}$, agrupar los $\boldsymbol{z}_i$ en $\boldsymbol{Z} = \big[\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N\big] \in \mathbb{R}^{d\times N}$ y agrupar los $\boldsymbol{\varepsilon}_i$ en $\boldsymbol{E} = \big[\boldsymbol{\varepsilon}_1, \ldots, \boldsymbol{\varepsilon}_N\big] \in \mathbb{R}^{D\times N}$, para reescribir (2.3.3) como

$$\boldsymbol{X} = \boldsymbol{A}\boldsymbol{Z} + \boldsymbol{E}. \tag{2.3.4}$$

[11] Resulta que, en un espacio de alta dimensión, es bastante fácil empaquetar un número de vectores casi ortogonales que es mucho mayor que la dimensión del espacio [**Wright-Ma-2022**].

Un enfoque natural para resolver el problema de la codificación esparsa consiste en buscar las señales más esparsas que sean consistentes con nuestras observaciones, lo que nos lleva naturalmente al siguiente problema de optimización:

$$\min_{\tilde{\boldsymbol{Z}} \in \mathbb{R}^{d \times N}} \left\{ \|\boldsymbol{X} - \boldsymbol{A}\tilde{\boldsymbol{Z}}\|_F^2 + \lambda \|\tilde{\boldsymbol{Z}}\|_1 \right\}, \tag{2.3.5}$$

donde se sabe que la norma ℓ^1 $\|\tilde{\boldsymbol{Z}}\|_1$, tomada elemento por elemento, fomenta la esparcidad de la solución [**Wright-Ma-2022**]. Este problema de optimización se conoce como el problema LASSO.

Sin embargo, a diferencia del PCA o del problema del aprendizaje de diccionarios completos, no existe un algoritmo claro del tipo de iteración de potencias para calcular el $\boldsymbol{Z}^\star$ óptimo. Una alternativa natural es plantearse resolver el problema de optimización anterior con algoritmos del tipo de descenso por gradiente. Sea $\mathcal{L}(\tilde{\boldsymbol{Z}}) = \|\boldsymbol{X} - \boldsymbol{A}\tilde{\boldsymbol{Z}}\|_2^2 + \lambda\|\tilde{\boldsymbol{Z}}\|_1$. En principio, podríamos intentar calcular $\boldsymbol{Z}^\star$ mediante las siguientes iteraciones:

$$\tilde{\boldsymbol{Z}}_{t+1} \leftarrow \tilde{\boldsymbol{Z}}_t + \eta \nabla \mathcal{L}(\tilde{\boldsymbol{Z}}_t). \tag{2.3.6}$$

Sin embargo, dado que el término asociado a la norma ℓ^1 $\|\tilde{\boldsymbol{Z}}\|_1$ no es suave, no podemos limitarnos a aplicar el método del descenso por gradiente. Para este tipo de función, necesitamos sustituir el gradiente $\nabla\mathcal{L}(\tilde{\boldsymbol{Z}})$ por algo que generalice el concepto de gradiente, conocido como subgradiente $\partial\mathcal{L}(\tilde{\boldsymbol{Z}})$:

$$\tilde{\boldsymbol{Z}}_{t+1} \leftarrow \tilde{\boldsymbol{Z}}_t + \eta \partial \mathcal{L}(\tilde{\boldsymbol{Z}}_t). \tag{2.3.7}$$

Sin embargo, se sabe que la convergencia del descenso por subgradiente suele ser muy lenta. Por lo tanto, para este tipo de problemas de optimización, lo habitual es utilizar un algoritmo denominado «descenso de gradiente proximal». Ofrecemos un resumen técnico de este método en la **??**.

La aplicación del gradiente proximal a la función objetivo de LASSO (2.3.5) da lugar al clásico *algoritmo iterativo de contracción y umbralización* (ISTA), que implementa la iteración

$$\tilde{\boldsymbol{Z}}_1 \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{2.3.8}$$

$$\tilde{\boldsymbol{Z}}_{t+1} = S_{\eta\lambda}\Big(\tilde{\boldsymbol{Z}}_t - 2\eta \boldsymbol{A}^\top(\boldsymbol{A}\tilde{\boldsymbol{Z}}_t - \boldsymbol{X})\Big), \quad \forall t \geq 1, \tag{2.3.9}$$

con un paso $\eta \geq 0$ y el operador de umbralización suave S_α definido para escalares mediante

$$S_\alpha(x) \doteq \begin{cases} x - \alpha, & x \geq \alpha, \\ 0, & -\alpha < x < \alpha, \\ x + \alpha, & x \leq -\alpha \end{cases} \tag{2.3.10}$$

$$= \operatorname{sign}(x) \max\{|x| - \alpha, 0\}, \tag{2.3.11}$$

y se aplica elemento por elemento a la matriz de entrada. Al tratarse de un algoritmo de descenso por gradiente proximal aplicado a un problema convexo,

la convergencia hacia un óptimo global está garantizada y es posible derivar directamente una tasa de convergencia precisa [**Wright-Ma-2022**].

Ahora nos detendremos un momento para comentar la forma funcional del operador de actualización en (2.3.9). Adopta la forma de

$$\tilde{\boldsymbol{Z}}_{t+1} = \texttt{nolinealidad}(\tilde{\boldsymbol{Z}}_t + \texttt{lineal}^\top(\texttt{lineal}(\tilde{\boldsymbol{Z}}_t) + \texttt{sesgo})). \tag{2.3.12}$$

Esta forma funcional es muy similar a la de una capa de red residual, es decir,

$$\tilde{\boldsymbol{Z}}_{t+1} = \tilde{\boldsymbol{Z}}_t + \texttt{lineal}_1^\top(\texttt{nolinealidad}(\texttt{lineal}_2(\tilde{\boldsymbol{Z}}_t) + \texttt{sesgo}_1) + \texttt{bias}_2, \tag{2.3.13}$$

simplemente desacoplando las transformaciones lineales (es decir, las multiplicaciones matriciales), añadiendo un sesgo y desplazando la no linealidad. La no linealidad en (2.3.9) es notablemente similar a la función de activación ReLU de uso común $x \mapsto \max\{x, 0\}$ en el aprendizaje profundo —como se ilustra en la Figura 2.7, el operador de umbral suave es como una activación ReLU «con signo», que además se desplaza mediante un sesgo. Por lo tanto, la ISTA puede considerarse como una pasada hacia adelante de una red neuronal primitiva (recurrente de una sola capa). En el Capítulo 5 defendemos que este tipo de operaciones son esenciales para el aprendizaje de representaciones profundas.

2.3.2 Aprendizaje de diccionarios sobrecompletos

Recordemos que tenemos el modelo de datos

$$\boldsymbol{X} = \boldsymbol{A}\boldsymbol{Z} + \boldsymbol{E}, \tag{2.3.14}$$

donde $\boldsymbol{Z}$ es un vector esparso y nuestro objetivo anterior era estimar $\boldsymbol{Z}$ a partir de los datos $\boldsymbol{X}$ y los átomos del diccionario $\boldsymbol{A}$. Ahora pasamos al caso más práctico y más difícil, en el que no conocemos ni $\boldsymbol{A}$ ni $\boldsymbol{Z}$ y tratamos de determinarlos a partir de un gran conjunto de datos.

Una generalización directa de la Ecuación (2.3.5) sugiere resolver el problema

$$\min_{\tilde{\boldsymbol{A}}, \tilde{\boldsymbol{Z}}} \left\{ \|\boldsymbol{X} - \tilde{\boldsymbol{A}}\tilde{\boldsymbol{Z}}\|_F^2 + \lambda \|\tilde{\boldsymbol{Z}}\|_1 \right\}. \tag{2.3.15}$$

Sin embargo, el término bilineal en la Ecuación (2.3.15) introduce una ambigüedad de escala que dificulta la convergencia: dado cualquier punto $(\tilde{\boldsymbol{A}}, \tilde{\boldsymbol{Z}})$ y cualquier constante $c > 0$, la sustitución $(c^{-1}\tilde{\boldsymbol{A}}, c\tilde{\boldsymbol{Z}})$ da lugar a un valor de pérdida

$$\|\boldsymbol{X} - \tilde{\boldsymbol{A}}\tilde{\boldsymbol{Z}}\|_F^2 + c\lambda \|\tilde{\boldsymbol{Z}}\|_1. \tag{2.3.16}$$

Es evidente que esta pérdida se minimiza respecto de c al tomar $c \to 0$, lo que equivale a hacer tender el diccionario reescalado $c^{-1}\tilde{\boldsymbol{A}}$ «al infinito». En particular, los iterados de cualquier algoritmo de optimización que resuelva (2.3.15) no convergerán.

Este problema con (2.3.15) se trata añadiendo una restricción a la escala de las columnas del diccionario $\tilde{\boldsymbol{A}}$. Por ejemplo, es habitual suponer que

cada columna $\boldsymbol{A}_j$ del diccionario $\boldsymbol{A}$ en (2.3.14) tiene una norma ℓ^2 acotada — digamos, sin pérdida de generalidad, por 1. A continuación, lo imponemos como una restricción, estableciendo el objetivo

$$\min_{\tilde{\boldsymbol{Z}}, \tilde{\boldsymbol{A}} : \|\tilde{\boldsymbol{A}}_j\|_2 \leq 1} \left\{ \|\boldsymbol{X} - \tilde{\boldsymbol{A}}\tilde{\boldsymbol{Z}}\|_F^2 + \lambda \|\tilde{\boldsymbol{Z}}\|_1 \right\}. \tag{2.3.17}$$

Los problemas de optimización con restricciones, como el de la ecuación (2.3.17), pueden resolverse mediante una gran variedad de algoritmos sofisticados [**nocedal2006numerical**]. Sin embargo, una solución simple y escalable es, de hecho, la que ofrece el mismo algoritmo de descenso por gradiente proximal que utilizamos para resolver el problema de la codificación esparsa en la sección anterior. Podemos codificar cada restricción como un término de regularización adicional, mediante la función característica del conjunto de restricciones—los detalles se exponen en el **??**. La aplicación del descenso por gradiente proximal al problema regularizado resultante es equivalente al *descenso por gradiente proyectado*, en el que, en cada iteración, los iterados, tras dar un paso de descenso por gradiente, se proyectan sobre el conjunto de restricciones.

Nota 2.12 (Maximización de ℓ^4 vs minimización de ℓ^1). Cabe señalar que la formulación del problema anterior se deriva naturalmente de la formulación LASSO (2.3.5) para la codificación esparsa. Promovemos la esparcidad de la solución mediante la norma ℓ^1. No obstante, si solo nos interesa recuperar el diccionario sobrecompleto $\boldsymbol{A}$, el esquema de maximización de ℓ^4 introducido en la Sección 2.2.2 también se generaliza al caso sobrecompleto, sin ninguna modificación significativa. Los lectores interesados pueden consultar el trabajo de [**Qu2020Geometric**].

El problema anterior (2.3.17), al que denominamos *aprendizaje de diccionarios sobrecompletos*, es no convexo, ya que en este caso tanto $\boldsymbol{A}$ como $\boldsymbol{Z}$ son desconocidos. No se puede resolver fácilmente con las herramientas estándar de optimización convexa. No obstante, dado que es interesante, fácil de formular y de gran relevancia práctica, se han dedicado numerosos trabajos importantes a los distintos algoritmos y análisis teóricos relacionados con este problema. En el marco de este trabajo, presentamos un enfoque idiomático para resolver este problema que se ajusta más al espíritu del aprendizaje profundo.

A partir de nuestra experiencia con el problema LASSO anterior, es fácil ver que, para las dos incógnitas $\boldsymbol{A}$ y $\boldsymbol{Z}$, si fijamos una y optimizamos la otra, cada subproblema es, de hecho, convexo y fácil de resolver. Esto sugiere, naturalmente, que podríamos intentar resolver el programa anterior (2.3.17) minimizando alternativamente con respecto a $\boldsymbol{Z}$ o $\boldsymbol{A}$, por ejemplo, utilizando el descenso por gradiente. Si a esto le sumamos una elección natural de inicialización, se obtiene el siguiente esquema iterativo:

$$\tilde{\boldsymbol{Z}}^{\ell+1} = S_{\eta\lambda}\Big(\tilde{\boldsymbol{Z}}^{\ell} - 2\eta \boldsymbol{A}_+^\top (\boldsymbol{A}_+ \tilde{\boldsymbol{Z}}^{\ell} - \boldsymbol{X})\Big), \quad \tilde{\boldsymbol{Z}}^1 = \boldsymbol{0}, \quad \forall \ell \in [L] \tag{2.3.18}$$

$$\boldsymbol{Z}^+ = \tilde{\boldsymbol{Z}}^L, \tag{2.3.19}$$

$$\tilde{\boldsymbol{A}}_{t+1} = \mathrm{proj}_{\|(\,\cdot\,)_j\|_2 \leq 1,\, \forall j} \left(\tilde{\boldsymbol{A}}_t - 2\nu (\tilde{\boldsymbol{A}}_t \boldsymbol{Z}^+ - \boldsymbol{X})(\boldsymbol{Z}^+)^\top \right),$$

$$\text{where} \quad (\tilde{\boldsymbol{A}}_1)_j \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{0}, \tfrac{1}{D}\boldsymbol{I}), \ \forall j \in [m], \qquad \forall t \in [T], \tag{2.3.20}$$

$$\boldsymbol{A}_+ = \tilde{\boldsymbol{A}}_T, \tag{2.3.21}$$

donde la operación de proyección en la actualización de $\boldsymbol{A}$ garantiza que cada columna tenga una norma ℓ^2 máxima no mayor que la unidad, mediante $\boldsymbol{A}_j \mapsto \boldsymbol{A}_j / \max\{\|\boldsymbol{A}_j\|_2, 1\}$ y donde $\boldsymbol{A}_+$ se inicializa con cada columna independiente e idénticamente distribuida $\mathcal{N}(\boldsymbol{0}, \frac{1}{D}\boldsymbol{I})$. Lo anterior consiste en un «bloque» de minimización alternada y se repiten dichos bloques —cada uno con inicializaciones independientes— hasta alcanzar la convergencia. Arriba, hemos utilizado dos índices distintos, $\{t\}$ y $\{\ell\}$ para indicar las iteraciones. Como veremos más adelante, esto nos permite interpretar las dos actualizaciones por separado en el contexto del aprendizaje profundo.

A pesar de que el problema del aprendizaje de diccionarios es un problema no convexo, se ha demostrado que los algoritmos de minimización alternada convergen efectivamente hacia la solución correcta, al menos localmente. Véase, por ejemplo, [**alekh-2016**]. A modo de demostración práctica, se utilizó el algoritmo anterior (con $L = T = 1$) para generar los resultados del aprendizaje de diccionarios sobrecompletos que se muestran en la Figura 2.6.

2.3.3 Codificación esparsa profunda aprendida

La idea principal del algoritmo de minimización alternada para el aprendizaje de diccionarios sobrecompletos en la sección anterior (Ecuaciones (2.3.18) y (2.3.20)) es observar que *cuando fijamos $\boldsymbol{A}$, la actualización ISTA para $\boldsymbol{Z}^\ell$ (2.3.18) se asemeja al paso hacia adelante de una red neuronal profunda con pesos dados por $\boldsymbol{A}$ (y $\boldsymbol{A}^\top$)*. Pero, en general, no conocemos el verdadero $\boldsymbol{A}$, y la estimación actual $\boldsymbol{A}_+$ podría ser errónea. Por lo tanto, es necesario actualizarla aún más utilizando (2.3.20) a partir del residuo obtenido al utilizar la estimación actual de los códigos esparsos $\boldsymbol{Z}^+$ para reconstruir $\boldsymbol{X}$. El algoritmo de minimización alternada repite estos dos procedimientos hasta alcanzar la convergencia. Pero, en su lugar, podemos extrapolar y diseñar otros procedimientos de aprendizaje combinando estas ideas con técnicas del aprendizaje profundo. Esto da lugar a arquitecturas de red más interpretables, lo cual será un tema recurrente a lo largo de este manuscrito.

ISTA aprendido. La interpretación anterior, en términos de redes profundas, de la minimización alternada es más conceptual que práctica, ya que el proceso podría resultar bastante ineficiente y requerir muchas capas o iteraciones para converger. Pero esto se debe principalmente a que intentamos deducir tanto $\boldsymbol{Z}$ como $\boldsymbol{A}$ a partir de $\boldsymbol{X}$. El problema puede simplificarse considerablemente y las iteraciones anteriores pueden hacerse mucho más eficientes en el contexto *supervisado*, donde disponemos de un conjunto de datos de pares de entrada y salida $(\boldsymbol{X}, \boldsymbol{Z})$ distribuidos según (2.3.14) y solo buscamos aprender $\boldsymbol{A}^\ell$ para las

iteraciones de codificación esparsa aprendible por capas (2.3.29):[12]

$$\boldsymbol{Z}^{\ell+1} = S_{\eta\lambda}\big(\boldsymbol{Z}^{\ell} - 2\eta(\boldsymbol{A}^{\ell})^{\top}(\boldsymbol{A}^{\ell}\boldsymbol{Z}^{\ell} - \boldsymbol{X})\big), \quad \forall \ell \in [L]. \tag{2.3.22}$$

Si denotamos el operador para cada iteración como $\boldsymbol{Z}^{\ell+1} = f(\boldsymbol{A}^{\ell}, \boldsymbol{Z}^{\ell})$, la iteración anterior puede ilustrarse mediante un diagrama:

$$\boldsymbol{X}, \boldsymbol{Z}^1 \xrightarrow{f(\boldsymbol{A}^1, \cdot)} \boldsymbol{Z}^2 \xrightarrow{f(\boldsymbol{A}^2, \cdot)} \boldsymbol{Z}^3 \xrightarrow{f(\boldsymbol{A}^3, \cdot)} \cdots \boldsymbol{Z}^L \xrightarrow{f(\boldsymbol{A}^L, \cdot)} \boldsymbol{Z}^{L+1} \approx \boldsymbol{Z}.$$

Por lo tanto, dada la arquitectura secuencial, para aprender el operador $\boldsymbol{A}^{\ell}$ en cada capa, resulta totalmente natural hacerlo, por ejemplo, mediante la retropropagación (BP),[13] minimizando el error entre el código final $\boldsymbol{Z}^L$ y la verdad de referencia (ground truth) $\boldsymbol{Z}$:

$$\min_{\{\boldsymbol{A}^{\ell}\}} \big\|\boldsymbol{Z}^L(\boldsymbol{A}^1, \ldots, \boldsymbol{A}^L) - \boldsymbol{Z}\big\|_2^2. \tag{2.3.23}$$

Esta es la base del algoritmo Learned ISTA (LISTA) [**gregor2010learning**], que puede considerarse como el algoritmo de aprendizaje de una red neuronal profunda, que intenta emular el proceso de codificación esparsa de $\boldsymbol{X}$ a $\boldsymbol{Z}$. En concreto, se puede considerar como un *algoritmo simple de aprendizaje de representaciones*. De hecho, esta misma metodología puede servir de base para comprender las representaciones calculadas en arquitecturas de red más potentes, como los transformers. Desarrollamos estas implicaciones en detalle en el Capítulo 5.

Autocodificadores esparsos. La motivación original del aprendizaje de diccionarios sobrecompletos era proporcionar un modelo generativo sencillo para datos de alta dimensión. Hemos visto con LISTA que, además, los algoritmos iterativos para el aprendizaje de diccionarios sobrecompletos utilizados de manera esparsa ofrecen una interpretación de las redes profundas de tipo ReLU, que generalizaremos en capítulos posteriores a distribuciones de datos más complejas que (2.3.14). Pero también cabe señalar que, incluso en la era moderna de los grandes modelos, el modelo de generación de datos (2.3.14) ofrece una base práctica y útil para *interpretar características en redes profundas preentrenadas a gran escala*, tales como los transformers, siguiendo la hipótesis de que las características (no interpretables, *a priori*) en estas redes consisten en «superposiciones» esparsas de características subyacentes, las cuales son a su vez interpretables [**elhage2022superposition**]. Estos paradigmas de aprendizaje *sin supervisión* suelen ser también más eficientes en cuanto al uso de datos que LISTA, que requiere grandes cantidades de pares etiquetados $(\boldsymbol{X}, \boldsymbol{Z})$ para el entrenamiento supervisado.

Podemos utilizar el desarrollo del algoritmo LISTA que acabamos de ver para comprender las prácticas habituales en este campo de investigación. En

[12] Omitimos los acentos de «tilde» aquí por motivos de concisión, ya que la indexación deja clara la dependencia de la optimización.

[13] Véase la **??** para una breve descripción de la retropropagación.

su aplicación más sencilla (véase [**huben2024sparse**; **gao2025scaling**]), se recopila un gran número de características de una red profunda preentrenada h a partir de diferentes entradas $\boldsymbol{x}_i$, las cuales se seleccionan en función de la tarea de interpretación deseada.[14] Para simplificar, utilizaremos h para denotar el mapa de características preseleccionado en cuestión, con características de dimensión D; dadas N entradas de muestra, sea $\boldsymbol{H} \in \mathbb{R}^{D\times N}$ la matriz completa de características de h. A continuación, se entrena un autocodificador esparso $f : \mathbb{R}^D \to \mathbb{R}^d$, con un decodificador $g : \mathbb{R}^d \to \mathbb{R}^D$, mediante la función de pérdida LASSO (2.3.5):

$$\min_{f,g} \|\boldsymbol{H} - g(f(\boldsymbol{H}))\|_F^2 + \lambda \|f(\boldsymbol{H})\|_1, \tag{2.3.24}$$

donde el autocodificador esparso f adopta la forma de una red neuronal de una sola capa, es decir, $f(\boldsymbol{h}_i) = \sigma(\boldsymbol{W}_{\text{enc}}(\boldsymbol{h}_i - \boldsymbol{b}) + \boldsymbol{b}_{\text{enc}})$, donde $\sigma(x) = \max\{x, 0\}$ es la función de activación ReLU y el decodificador g es lineal, de modo que $g(\boldsymbol{z}_i) = \boldsymbol{W}_{\text{dec}}\boldsymbol{z} + \boldsymbol{b}$.[15]

El procedimiento de parametrización y entrenamiento (2.3.24) puede parecer inicialmente una aplicación arbitraria del aprendizaje profundo al problema de la codificación esparsa, pero en realidad está muy en consonancia con los algoritmos que hemos estudiado anteriormente para la codificación esparsa por capas con un diccionario aprendido. En particular, recordemos la arquitectura LISTA $\boldsymbol{Z}^L = f(\boldsymbol{A}^L, f(\boldsymbol{A}^{L-1}, \cdots, f(\boldsymbol{A}^1, \boldsymbol{X})\cdots)$. En el caso especial de $L = 2$, tenemos

$$\boldsymbol{Z}^2 = f(\boldsymbol{A}^1, \boldsymbol{X}) = S_{\eta\lambda}\big(\boldsymbol{Z}^1 - 2\eta(\boldsymbol{A}^1)^\top(\boldsymbol{A}^1\boldsymbol{Z}^1 - \boldsymbol{X})\big). \tag{2.3.25}$$

Supongamos que los códigos esparsos $\boldsymbol{Z}$ en cuestión son no negativos, es decir, que $\boldsymbol{Z} \geq \boldsymbol{0}$.[16] Entonces (véase el **??**), podemos considerar una arquitectura LISTA equivalente obtenida a partir del objetivo de codificación esparsa con una restricción adicional de no negatividad sobre $\boldsymbol{Z}$ como

$$\boldsymbol{Z}^2 = f(\boldsymbol{A}^1, \boldsymbol{X}) = \max\left\{\boldsymbol{Z}^1 - 2\eta(\boldsymbol{A}^1)^\top(\boldsymbol{A}^1\boldsymbol{Z}^1 - \boldsymbol{X}) - \lambda\eta\boldsymbol{1}, 0\right\}, \tag{2.3.26}$$

[14] Por ejemplo, las entradas $\boldsymbol{x}_i$ podrían corresponder a textos que contengan fragmentos de código informático en diferentes lenguajes de programación, siendo nuestra tarea tratar de identificar características interpretables en un mapa de características del transformer h que correspondan a diferentes aspectos destacados de la entrada, tales como el lenguaje de programación específico (distinto entre las «clases» de la entrada) o la necesidad de insertar un paréntesis de cierre en la posición actual (común entre las «clases» de la entrada). Analizamos el uso de redes profundas, y en particular de los transformers, para el aprendizaje de representaciones de texto con mayor detalle en el **??**.

[15] Dado que f es un operador de esparcidad, lo denotamos de manera análoga al f utilizado anteriormente, por ejemplo, en LISTA, donde f denota un algoritmo de optimización para codificar en forma esparsa la entrada. A lo largo del libro, se utilizará f para referirse a esas transformaciones esparsas y estructuradas de otro modo, incluidas las de los *codificadores de redes profundas* (Capítulos 4 y 5).

[16] En el modelo de generación de datos (2.3.14), un par arbitrario de un diccionario y un código esparsos $(\boldsymbol{A}, \boldsymbol{Z})$ puede sustituirse por uno en el que $\boldsymbol{Z} \geq \boldsymbol{0}$ simplemente duplicando el número de columnas en $\boldsymbol{A}$, por lo que, desde el punto de vista del modelado, se trata de una suposición muy moderada.

y, tras algunas operaciones algebraicas, expresarlo como

$$\boldsymbol{Z}^2 = f(\boldsymbol{A}^1, \boldsymbol{X}) = \max\left\{2\eta(\boldsymbol{A}^1)^\top + \left(\boldsymbol{Z}^1 - 2\eta(\boldsymbol{A}^1)^\top \boldsymbol{A}^1 \boldsymbol{Z}^1 - \lambda\eta\mathbf{1}\right), 0\right\}. \tag{2.3.27}$$

Dada la posibilidad de modificar la inicialización del código esparso $\boldsymbol{Z}^1$ como un parámetro aprendible (que, en el marco actual, debe tener todas las columnas iguales al mismo vector aprendible), esto adopta la forma de una red neuronal ReLU con sesgo aprendible, ¡idéntica al autocodificador esparso f! Además, para *decodificar* los códigos esparsos aprendidos $\boldsymbol{Z}^2$, lo más natural es aplicar el diccionario aprendido $\boldsymbol{Z}^2 \mapsto \boldsymbol{A}^1\boldsymbol{Z}^2$. Entonces, la única diferencia entre esto y el decodificador SAE g es el sesgo adicional $\boldsymbol{b}$, que técnicamente puede absorberse en $\boldsymbol{H}$ y f en el objetivo de entrenamiento (2.3.24).

Por lo tanto, el procedimiento para parametrizar y entrenar SAE coincide con el entrenamiento de LISTA con $L = 1$ y un objetivo de entrenamiento modificado — utilizando el objetivo LASSO (2.3.5), que sigue siendo *no supervisado*, en lugar de la pérdida de reconstrucción supervisada (2.3.23) utilizada en el método LISTA estándar. En concreto, podemos entender la arquitectura SAE a partir de nuestra interpretación de la arquitectura LISTA en términos de codificación esparsa por capa, tal y como se describe en (2.3.29). Esta conexión sugiere una gran variedad de nuevas estrategias de diseño para mejorar la metodología de la interpretabilidad práctica, muchas de las cuales siguen sin haberse explorado, lo cual resulta tentador. Comenzamos a establecer algunas conexiones con la metodología más amplia de la autocodificación en el **??**.

¿Codificación esparsa aprendida por capa? En el contexto supervisado, LISTA ofrece un análogo de red neuronal profunda de la iteración de codificación esparsa, con diccionarios aprendidos por capas, inspirado en la minimización alternada; incluso en el contexto no supervisado, se puede aplicar la misma metodología al aprendizaje, al igual que con los autocodificadores esparsos. Pero la relación entre los algoritmos de optimización que buscan estructuras de baja dimensión y las arquitecturas de redes profundas es mucho más profunda que eso y sugiere un abanico de arquitecturas de aprendizaje neuronal escalables y naturales que podrían incluso utilizarse sin retropropagación.

A modo de ejemplo sencillo, retomamos las iteraciones de minimización alternadas (2.3.18) y (2.3.20). Este esquema reinicializa aleatoriamente el diccionario $\boldsymbol{A}_1$ en cada una de esas actualizaciones. Una mejora utiliza en su lugar el «inicio en caliente», en el que el residuo se genera utilizando la estimación anterior $\boldsymbol{A}_+$ para el diccionario. Si consideramos entonces cada actualización ISTA (2.3.18) como una capa y permitimos que el diccionario asociado, ahora acoplado a las actualizaciones del código esparso como $\boldsymbol{A}_\ell$, se actualice con el tiempo, esto da lugar a un esquema de codificación esparsa «aprendible por capas»:

$$\boldsymbol{Z}^1 = \mathbf{0}, \quad (\boldsymbol{A}_1)_j \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\mathbf{0}, \tfrac{1}{D}\boldsymbol{I}), \ \ \forall j \in [m], \tag{2.3.28}$$

$$\boldsymbol{Z}^{\ell+1} = S_{\eta\lambda}\big(\boldsymbol{Z}^\ell - 2\eta(\boldsymbol{A}_\ell)^\top(\boldsymbol{A}_\ell\boldsymbol{Z}^\ell - \boldsymbol{X})\big), \tag{2.3.29}$$

$$\boldsymbol{A}_{\ell+1} = \boldsymbol{A}_\ell - 2\nu(\boldsymbol{A}_\ell \boldsymbol{Z}^{\ell+1} - \boldsymbol{X})(\boldsymbol{Z}^{\ell+1})^\top. \tag{2.3.30}$$

Téngase en cuenta que esta iteración corresponde a un reetiquetamiento de (2.3.18) y (2.3.20) para $T = L = 1$, sobre un número infinito de bloques. Cada uno de los pasos «internos» que actualizan $\boldsymbol{Z}$ puede considerarse como una pasada hacia adelante de una sola capa, mientras que cada uno de los pasos «externos» que actualizan $\boldsymbol{A}$ puede considerarse como una pasada hacia atrás de una sola capa, de una red neuronal profunda primitiva. En particular, este algoritmo constituye el caso más sencillo en el que se manifiesta una clara distinción entre la optimización hacia adelante y el aprendizaje hacia atrás. Esta distinción entre las actualizaciones hacia adelante de las «características» efímeras $\boldsymbol{Z}$ de los datos $\boldsymbol{X}$ y las actualizaciones hacia atrás de los parámetros persistentes $\boldsymbol{A}$ sigue observándose en las redes neuronales y los autocodificadores actuales; hablaremos mucho más sobre ello en el Capítulo 5 y el **??**.

Obsérvese que el esquema por capas anterior también sugiere una alternativa plausible a la actual estrategia de optimización de extremo a extremo, que se basa principalmente en la retropropagación (BP), tal como se detalla en el **??**. Liberar el entrenamiento de redes grandes de la dependencia de BP sería uno de los mayores retos y oportunidades del futuro, tal y como analizaremos con más detalle al final del libro en el **??**.

2.4 Resumen y notas

Mensajes clave. Los modelos idealizados que hemos presentado en este capítulo —el PCA, el ICA y el aprendizaje de diccionarios— se desarrollaron a lo largo del Siglo XX. Muchos libros se han dedicado a estos métodos, por lo que aquí solo hemos intentado ofrecer una visión general de los conceptos y resultados clave. Como veremos en los próximos capítulos, aunque podría decirse que se trata de modelos bastante idealizados y sencillos para distribuciones de datos de baja dimensión, las ideas básicas e incluso los algoritmos resultantes para identificar tales distribuciones pueden generalizarse o ampliarse de manera natural a cualquier distribución de baja dimensión: esa es la tesis de este libro. Además, como veremos en los próximos capítulos, estos modelos básicos (lineales) también pueden servir, en gran medida, como prototipos locales para aproximar de manera eficaz distribuciones generales con estructuras no lineales de baja dimensión.

Más antecedentes y referencias. Jolliffe [**Jolliffe1986**] atribuye el análisis de componentes principales a Pearson [**Pearson1901**] y, en forma independiente, a Hotelling [**Hotelling1933**]. En matemáticas, el resultado principal sobre el problema relacionado de la aproximación de bajo rango en normas unitariamente invariantes se atribuye a Eckart y Young [**Eckart1936-ep**], así como a Mirsky en su forma más general [**Mirsky1960-ek**]. El PCA sigue desempeñando un papel importante en la investigación, siendo quizás el problema modelo más simple para el aprendizaje

no supervisado de representaciones: ya en la década de 1980, trabajos como **Oja1982SimplifiedNM** y **Baldi89** utilizaron el problema para comprender el aprendizaje en redes neuronales primitivas y, más recientemente, ha servido como herramienta para comprender marcos de aprendizaje de representaciones más complejos, como los modelos de difusión [**wang2024diffusion**].

El análisis de componentes independientes fue propuesto por **Ans-1985** y impulsado de manera pionera por Aapo Hyvärinen en la década de 1990 y principios de la década del 2000 en una serie de trabajos de gran influencia: véase **Hyvrinen-2000** para un resumen. Como modelo sencillo para la estructura que surge en los datos reales, inicialmente se utilizó ampliamente en aplicaciones como la separación ciega de fuentes, donde cada componente independiente z_i representa una fuente independiente (como el sonido asociado a un instrumento distinto en una grabación musical) que se superpone para producir la observación $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z}$.

El problema del aprendizaje de diccionarios puede considerarse, en el caso completo u ortogonal, como uno de los problemas fundamentales del procesamiento de señales del siglo XX, especialmente en la teoría de los sistemas lineales, donde la base de Fourier desempeña un papel clave; a partir de la década de 1980, el campo del análisis armónico computacional se consolidó en torno al estudio de dichos diccionarios alternativos para clases de señales en las que la aproximación óptima solo podía realizarse en una base distinta de la de Fourier (por ejemplo, las ondículas) [**Donoho1998-zf**]. Sin embargo, la importancia del caso de las bases redundantes, o los diccionarios sobrecompletos, solo se puso de relieve tras el trabajo pionero de Olshausen y Field [**Olshausen1996-ap**; **Olshausen1997-yv**]. Los primeros trabajos en este campo establecieron las bases conceptuales y algorítmicas para el aprendizaje de diccionarios sobrecompletos de uso esparso, a menudo destinados a representar imágenes naturales [**Donoho2001-tl**; **DonohoD2003-PNAS**; **Elad2006-yi**; **Murray2007-cw**; **aharon2006k**; **Mairal2014-nq**; **Gribonval2015-fy**]. Más tarde, el gran interés teórico que suscitó el problema, como un problema modelo importante y no trivial para el aprendizaje de representaciones sin supervisión, llevó a que fuera estudiado por las comunidades de procesamiento de señales, aprendizaje automático teórico y informática teórica, centrándose en particular en las condiciones bajo las cuales el problema podía resolverse de manera demostrable y eficiente. Una lista no exhaustiva de trabajos destacados en esta línea incluye los de **Spielman2012-tl** y **sun2017complete_a** en el caso completo; **pmlr-v40-Arora15**; **Barak2015-uu**; y **Qu2020Geometric**. Muchas cuestiones teóricas profundas sobre este problema, que es fácil de formular, siguen sin resolverse, quizá en parte debido a una tensión con la dificultad NP del problema en el peor caso (véase, por ejemplo, **Tillmann2015-up**).

Un aspecto que deseamos destacar del estudio de estos modelos analíticos clásicos para estructuras de baja dimensión es el papel común que desempeñan los diversos *métodos de potencia generalizados*: algoritmos que convergen muy rápidamente, al menos localmente, hacia diversos tipos de estructuras de baja dimensión. La terminología utilizada para esta clase de algoritmos se basa en

Tabla 2.1: Resumen de los métodos (generalizados) de potencias presentados en este capítulo.

Problema	Algoritmo	Iteración	Tipo de estructura impuesta
PCA	Método de la potencia	$\boldsymbol{u}_t = \frac{\boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{u}_t}{\|\boldsymbol{X}\boldsymbol{X}^\top \boldsymbol{u}_t\|_2}$	1-dim. subespacio (vector unitario)
ICA	FastICA	$\boldsymbol{u}_{t+1} = \frac{\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{u}_t)^{\odot 3} - 3\boldsymbol{u}_t}{\left\|\frac{1}{N}\boldsymbol{X}(\boldsymbol{X}^\top \boldsymbol{u}_t)^{\odot 3} - 3\boldsymbol{u}_t\right\|_2}$	1-dim. subespacio (vector unitario)
DL completo	Algoritmo MSP	$\boldsymbol{U}_{t+1} = \mathcal{P}_{\mathrm{O}(D)}[(\boldsymbol{U}_t\boldsymbol{X})^{\odot 3}\boldsymbol{X}^\top]$	D-dim. subespacio (matriz orto.)

el trabajo de **JourneeM2010**. A grandes rasgos, siguiendo el modelo de la iteración clásica de potencias para el cálculo del vector propio principal de una matriz semidefinida $\boldsymbol{A} \in \mathbb{R}^{n\times n}$, es decir

$$\boldsymbol{u}_{t+1} = \frac{\boldsymbol{A}\boldsymbol{u}_t}{\|\boldsymbol{A}\boldsymbol{u}_t\|_2}, \tag{2.4.1}$$

esta clase de algoritmos consiste en una operación de «potenciación» que involucra una matriz $\boldsymbol{A}$ asociada a los datos, junto con una operación de «proyección» que impone el tipo de estructura deseado. La Tabla 2.1 presenta un resumen de los algoritmos que hemos estudiado en este capítulo y que siguen esta estructura. El lector podrá apreciar la aplicabilidad de esta metodología a diferentes tipos de estructuras de baja dimensión y a diferentes funciones de pérdida (es decir, tanto la pérdida cuadrática del PCA como las pérdidas de tipo curtosis del ICA), así como la ausencia de un algoritmo de este tipo para el aprendizaje de diccionarios sobrecompletos, a pesar de la amplitud de la bibliografía existente sobre estos algoritmos. Consideramos que el desarrollo de métodos de potencias para otras familias de estructuras de baja dimensión, en particular aquellas relevantes para aplicaciones en las que predomina el aprendizaje profundo, constituye una de las cuestiones de investigación más importantes (y aún sin resolver) que se plantean en este capítulo.

La relación que establecemos en la Sección 2.2.1 entre la hipótesis de distribución de la mezcla geométrica de subespacios y la hipótesis del diccionario esparso, más conveniente desde el punto de vista analítico, ya ha sido mencionada en trabajos anteriores, especialmente en aquellos centrados en el análisis de componentes principales generalizado y en aplicaciones como la agrupación de subespacios; véase, por ejemplo, el trabajo de **GPCA**. La hipótesis de la mezcla de subespacios seguirá desempeñando un papel importante a lo largo de este trabajo, tanto como caso de prueba analítico para diferentes paradigmas algorítmicos y como base para derivar diferentes arquitecturas de redes profundas, tal como ocurre con LISTA en la Sección 2.3.3, pero que puede adaptarse a distribuciones de datos más complejas.

2.5 Ejercicios y ampliaciones

Ejercicio 2.1. Demuestre que, para cualquier matriz simétrica $\boldsymbol{A}$, la solución al problema $\max_{\boldsymbol{U}\in\mathsf{O}(D,d)} \operatorname{tr}\left(\boldsymbol{U}^\top \boldsymbol{A}\boldsymbol{U}\right)$ es la matriz $\boldsymbol{U}^\star$, cuyas columnas son los d

primeros vectores propios unitarios de $\boldsymbol{A}$.

Ejercicio 2.2. Sea $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$ una variable aleatoria gaussiana con componentes independientes, cada uno con varianza σ^2. Demuestre que, para cualquier matriz ortogonal $\boldsymbol{Q}$ (es decir, $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$), la variable aleatoria $\boldsymbol{Qz}$ tiene la misma distribución que $\boldsymbol{z}$. *(Pista: recuerde la fórmula de la función de densidad de probabilidad gaussiana y la fórmula de la densidad de una función lineal de una variable aleatoria.)*

Ejercicio 2.3. El concepto —o noción— de identificabilidad estadística analizado anteriormente puede relacionarse con las *simetrías* de la clase de modelos, lo que permite entender la estimación de una manera puramente determinista sin ningún supuesto estadístico.

Consideremos el modelo $\boldsymbol{X} = \boldsymbol{UZ}$ para matrices $\boldsymbol{X}, \boldsymbol{U}, \boldsymbol{Z}$ de tamaños compatibles.

1. Muestre que si $\boldsymbol{A}$ es cualquier matriz cuadrada invertible de tamaño compatible, entonces el par $(\boldsymbol{UA}, \boldsymbol{A}^{-1}\boldsymbol{Z})$ también produce $\boldsymbol{X}$ según el mismo modelo. A esto lo llamamos una *simetría de* $\mathsf{GL}(d)$.

2. Supongamos que cada columna de $\boldsymbol{Z}$ es una observación independiente e identicamente distribuida procedente de un modelo estadístico común $\boldsymbol{z}$, que además tiene media cero y componentes independientes z_i con varianza positiva. Muestre que, para cualquier matriz cuadrada invertible $\boldsymbol{A}$, si $\boldsymbol{Az}$ tiene componentes no correlacionadas, entonces $\boldsymbol{A}$ puede escribirse como $\boldsymbol{D}_1\boldsymbol{Q}\boldsymbol{D}_2$, donde $\boldsymbol{Q}$ es una matriz ortogonal y $\boldsymbol{D}_1$ y $\boldsymbol{D}_2$ son matrices diagonales. *Esto vincula la suposición de «independencia» en el ICA con un efecto de «ruptura de simetría», que solo permite simetrías de escala y rotacionales.*

Ejercicio 2.4. Consideremos el modelo $\boldsymbol{x} = \boldsymbol{Uz}$, donde $\boldsymbol{U} \in \mathbb{R}^{D \times d}$ con $D \geq d$ es fijo y tiene rango d, mientras que $\boldsymbol{z}$ es una variable aleatoria de media cero. Sean $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N$ observaciones i.i.d. de este modelo.

1. Muestre que la matriz $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ tiene un rango no mayor que d y que por lo tanto existe una matriz ortonormal $\boldsymbol{V} \in \mathbb{R}^{D \times d}$ tal que $\boldsymbol{X} = \boldsymbol{VY}$, donde $\boldsymbol{Y} \in \mathbb{R}^{d \times N}$. (*Pista: usar el PCA.*)

2. Demuestre que la *matriz blanqueada* $(\boldsymbol{YY}^\top)^{-1/2}\boldsymbol{Y}$ existe en media siempre que $\mathrm{Cov}(\boldsymbol{z})$ sea no singular y que tiene una covarianza empírica igual a la matriz identidad.[17]

3. Muestre, utilizando la descomposición en valores singulares de $\boldsymbol{U}$, que la matriz $\boldsymbol{V}$ puede elegirse de tal manera que la matriz blanqueada satisfaga $(\boldsymbol{YY}^\top)^{-1/2}\boldsymbol{Y} = \boldsymbol{W}[\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N]$, donde $\boldsymbol{W}$ es una matriz ortonormal.

Ejercicio 2.5. Sean X e Y variables aleatorias independientes de media cero.

[17] En particular, se puede demostrar matemáticamente que esto basta para garantizar que la matriz blanqueada exista con alta probabilidad siempre que $\boldsymbol{z}$ satisfaga una desigualdad de concentración adecuada y N sea suficientemente grande.

1. Demuestre que $\operatorname{kurt}(X+Y) = \operatorname{kurt}(X) + \operatorname{kurt}(Y)$.

2. Para cualquier $\alpha \in \mathbb{R}$, demuestre que $\operatorname{kurt}(\alpha X) = \alpha^4 \operatorname{kurt}(X)$.

Ejercicio 2.6. Sea $f : \mathbb{R}^d \to \mathbb{R}$ una función objetivo dada, dos veces continuamente diferenciable. Consideremos el problema de optimización con restricciones esféricas

$$\max_{\|\boldsymbol{u}\|_2^2=1} f(\boldsymbol{u}). \tag{2.5.1}$$

En este ejercicio, derivaremos las expresiones que proporcionamos en la derivación de FastICA para maximizar la curtosis sobre la esfera mediante un algoritmo de ascenso por gradiente. Estas expresiones son casos especiales de una amplia teoría del cálculo y la optimización en variedades, de las cuales la esfera es un ejemplo concreto. Un estudio técnico en profundidad de este campo queda fuera del alcance de nuestro trabajo, por lo que solo mencionamos dos referencias clave para el lector interesado: el libro de texto pionero de Absil, Mahony y Sepulchre [**Absil2009-nc**] y un tratado introductorio más reciente de Boumal [**Boumal2023-rj**].

1. Para cualquier conjunto de restricciones $\mathcal{M}$ que sea una subvariedad diferenciable de $\mathbb{R}^d$, el *espacio tangente* en un punto $\boldsymbol{u} \in \mathcal{M}$ es, en términos informales, la mejor aproximación lineal local a la variedad $\mathcal{M}$ en el punto $\boldsymbol{u}$. En el importante caso especial en el que $\mathcal{M}$ se define localmente en $\boldsymbol{u}$ como un conjunto de nivel de una función $F : \mathbb{R}^d \to \mathbb{R}$, es decir

   $$U \cap \mathcal{M} = F^{-1}(\{0\})$$

   para un conjunto abierto $U \subset \mathcal{M}$ tal que $\boldsymbol{u} \in U$, el espacio tangente a $\mathcal{M}$ en $\boldsymbol{u}$ se puede calcular mediante la derivación:

   $$T_{\boldsymbol{u}}\mathcal{M} = \operatorname{Ker}(DF_{\boldsymbol{u}}).$$

   Es fácil ver, que la esfera tiene la ecuación definitoria $F(\boldsymbol{u}) = \|\boldsymbol{u}\|_2^2 - 1$. Demuestre, utilizando estos datos, que el espacio tangente a la esfera en $\boldsymbol{u}$ viene dado por

   $$T_{\boldsymbol{u}}\mathbb{S}^{d-1} = \{\boldsymbol{v} \in \mathbb{R}^d \mid \langle \boldsymbol{v}, \boldsymbol{u} \rangle = 0\},$$

   y que la proyección ortogonal sobre este subespacio es $\boldsymbol{P}_{\boldsymbol{u}}^{\perp} = \boldsymbol{I} - \boldsymbol{u}\boldsymbol{u}^{\top}$.

2. El campo vectorial

   $$\operatorname{grad} f(\boldsymbol{u}) = \boldsymbol{P}_{\boldsymbol{u}}^{\perp} \nabla f \tag{2.5.2}$$

   se conoce como el *gradiente riemanniano* de la función f restringida a la esfera. Las *condiciones de optimalidad de primer orden* para el problema de optimización (2.5.1) pueden expresarse en términos del gradiente riemanniano:

   $$\operatorname{grad} f(\boldsymbol{u}) = \boldsymbol{0}.$$

Desde el punto de vista geométrico, esto significa que el gradiente euclidiano de f en $\boldsymbol{u}$ debe ser ortogonal al espacio tangente a la esfera en $\boldsymbol{u}$. Supongamos ahora que $\boldsymbol{v} \in \mathbb{R}^d$ es distinto de cero. Demuestre que

$$\operatorname{proj}_{\mathbb{S}^{d-1}}(\boldsymbol{v}) \doteq \min_{\|\boldsymbol{u}\|_2^2=1} \|\boldsymbol{u}-\boldsymbol{v}\|_2 = \frac{\boldsymbol{v}}{\|\boldsymbol{v}\|_2},$$

utilizando las condiciones de optimalidad de primer orden.

3. Al optimizar en $\mathbb{R}^d$, se comprueban las condiciones de optimalidad de segundo orden (para determinar si un punto crítico es un punto máximo, un punto mínimo o un punto de silla) utilizando la *matriz hessiana* $\nabla^2 f(\boldsymbol{u})$. Muestre, derivando el gradiente riemanniano $\operatorname{grad} f(\boldsymbol{u})$ para la esfera con respecto a $\boldsymbol{u}$, tal como se hizo en la primera parte de este ejercicio, que el objeto correspondiente para determinar las condiciones de optimalidad de segundo orden en la optimización con restricciones esféricas es el *hessiano riemanniano*, definido como

$$\operatorname{Hess} f(\boldsymbol{u}) = \boldsymbol{P}_{\boldsymbol{u}}^{\perp}\left(\nabla^2 f(\boldsymbol{u}) - \langle \nabla f(\boldsymbol{u}), \boldsymbol{u}\rangle \boldsymbol{I}\right)\boldsymbol{P}_{\boldsymbol{u}}^{\perp}. \tag{2.5.3}$$

Ejercicio 2.7. En este ejercicio, esbozamos un argumento conocido en la literatura como *análisis de paisaje* para el problema de maximización de la curtosis de una población con restricción esférica (2.2.24). Demostraremos que, cuando hay al menos un componente independiente con curtosis positiva, sus maximizadores globales conducen efectivamente a la recuperación de una columna del diccionario $\boldsymbol{U}$. Para simplificar, supondremos que $\operatorname{kurt}(z_i) \neq 0$ para cada $i = 1, \ldots, d$.

1. Utilizando los resultados de la Parte 1 del Ejercicio 2.6, demuestre que la condición de optimalidad de primer orden para (2.2.24) es

$$\left(\sum_{i=1}^{d} \operatorname{kurt}(z_i) w_i^4\right)\boldsymbol{w} = \operatorname{kurt}(\boldsymbol{z}) \odot \boldsymbol{w}^{\odot 3}, \tag{2.5.4}$$

donde la curtosis se calcula elemento por elemento, $\odot$ denota la multiplicación elemento por elemento de vectores y $\boldsymbol{w}^{\odot 3}$ denota el cubo elemento por elemento de su argumento.

2. Demuestra que los vectores $\boldsymbol{w}$ de norma unitaria que además satisfacen (2.5.4) adoptan todos la siguiente forma. Sea $S^+ = \{i \in [d] \mid \operatorname{kurt}(z_i) > 0\}$ y $S^- = \{i \in [d] \mid \operatorname{kurt}(z_i) < 0\}$. Sea S un subconjunto de S^+ o de S^-. Entonces

$$\boldsymbol{w}_S = \sum_{i \in S} \pm \sqrt{\frac{1}{\operatorname{kurt}(z_i)\sum_{j \in S} \frac{1}{\operatorname{kurt}(z_j)}}}\,\boldsymbol{e}_i \tag{2.5.5}$$

satisface (2.5.4), donde $\boldsymbol{e}_i$ es el vector que contiene un 1 en la posición i y 0 en todas las demás y el signo $\pm$ indica la elección de un signo positivo o negativo.

3. Supongamos que existe al menos un i tal que $\operatorname{kurt}(z_i) > 0$. Utilizando los resultados de la Parte 2 del Ejercicio 2.6, demuestre que los únicos máximos locales de la función objetivo de (2.2.24) son los vectores con un solo elemento distinto de cero y signo $\pm \boldsymbol{e}_i$, donde $i \in S^+$. Se concluye que los maximizadores globales de (2.2.24) son los vectores con un solo elemento distinto de cero y signo, correspondientes a las componentes con máxima curtosis. (*Pista: cuente el número de valores propios positivos y negativos de la matriz hessiana riemanniana* (2.5.3) *en cada punto crítico.*)

4. Supongamos ahora que $\operatorname{kurt}(z_j) < 0$ para todo $j = 1, \ldots, d$. Esto corresponde a una instanciación «sobredeflacionada» del problema de maximización de la curtosis. Utilizando de nuevo los resultados de la Parte 2 del Ejercicio 2.6, muestre que los únicos máximos locales de la función objetivo de (2.2.24) son los vectores densos con signo $\sum_{i=1}^{d} \pm \boldsymbol{e}_i$. Esto demuestra que la formulación de optimización (2.2.24) no puede aplicarse sin más en forma ingenua.

Ejercicio 2.8. Este ejercicio sigue la estructura y el formalismo presentados en el Ejercicio 2.6, pero los aplica en este caso al grupo ortogonal $\mathsf{O}(d) = \{\boldsymbol{U} \in \mathbb{R}^{d\times d} \mid \boldsymbol{U}^\top \boldsymbol{U} = \boldsymbol{I}\}$. Consulte la descripción del Ejercicio 2.6 para obtener los fundamentos conceptuales necesarios; el formalismo se aplica de manera idéntica al caso en el que el espacio envolvente es el conjunto de matrices $d \times d$, siempre y cuando se tenga en cuenta que el producto interno pertinente para matrices es $\langle \boldsymbol{X}, \boldsymbol{Y} \rangle = \operatorname{tr}(\boldsymbol{X}^\top \boldsymbol{Y})$. Una excelente referencia general sobre la optimización en el grupo ortogonal es la obra de Edelman, Arias y Smith [**Edelman1998-lg**].

Sea $f : \mathbb{R}^{d\times d} \to \mathbb{R}$ una función objetivo dada que sea dos veces continuamente diferenciable. Consideremos el problema de optimización con restricciones ortogonales

$$\max_{\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}} f(\boldsymbol{Q}). \tag{2.5.6}$$

1. Es fácil ver que el grupo ortogonal tiene la ecuación definitoria $F(\boldsymbol{Q}) = \boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$. Demuestre, a partir de este hecho, que el espacio tangente al grupo ortogonal en $\boldsymbol{Q}$ viene dado por

$$T_{\boldsymbol{Q}}\mathsf{O}(d) = \{\boldsymbol{Q}\boldsymbol{\Omega} \in \mathbb{R}^{d\times d} \mid \boldsymbol{\Omega}^\top = -\boldsymbol{\Omega}\},$$

y que la proyección ortogonal sobre este subespacio es

$$\mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)}(\boldsymbol{\Delta}) = \boldsymbol{Q} \operatorname{skew}(\boldsymbol{Q}^\top \boldsymbol{\Delta}),$$

donde $\operatorname{skew}(\boldsymbol{\Delta}) = \frac{1}{2}(\boldsymbol{\Delta} - \boldsymbol{\Delta}^\top)$ es la proyección ortogonal sobre el conjunto de matrices antisimétricas. El campo vectorial

$$\operatorname{grad} f(\boldsymbol{Q}) = \mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)} \left(\nabla f(\boldsymbol{Q})\right) \tag{2.5.7}$$

se conoce como el *gradiente riemanniano* de la función f restringida al grupo ortogonal. Las *condiciones de optimalidad de primer orden* para el problema de optimización (2.5.6) pueden expresarse en términos del gradiente riemanniano:

$$\operatorname{grad} f(\boldsymbol{Q}) = \boldsymbol{0}.$$

2. Demuestre, derivando el gradiente riemanniano $\operatorname{grad} f(\boldsymbol{Q})$ para el grupo ortogonal con respecto a $\boldsymbol{Q}$, tal como se hizo en la primera parte de este ejercicio, que la *Hessiana riemanniana* viene dada por

$$\operatorname{Hess} f(\boldsymbol{Q}) = \mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)} \left(\nabla^2 f(\boldsymbol{Q}) - \operatorname{sym}(\boldsymbol{Q}^\top \nabla f(\boldsymbol{Q})) \otimes \boldsymbol{I} \right) \mathcal{P}_{T_{\boldsymbol{Q}}\mathsf{O}(d)}, \tag{2.5.8}$$

donde $\operatorname{sym}(\boldsymbol{\Delta}) = \frac{1}{2}(\boldsymbol{\Delta}+\boldsymbol{\Delta}^\top)$ denota la proyección ortogonal sobre el conjunto de matrices simétricas y $\otimes$ denota el producto de Kronecker de matrices. Tenga cuidado de interpretar los operadores que aparecen en la expresión anterior como *transformaciones lineales sobre matrices* $d \times d$, **y no** como matrices $d \times d$ en sí mismas. Las *condiciones de optimalidad de segundo orden* para el problema de optimización (2.5.6) pueden expresarse en términos de la hessiana riemanniana:

$$\operatorname{Hess} f(\boldsymbol{Q}) \preceq \boldsymbol{0}.$$

En un problema de minimización, el signo se invierte.

(*Pista: La clave está en manipular los cálculos para obtener la forma* (2.5.8)*, que es la más compacta posible. Para ello, utiliza el siguiente isomorfismo del producto de Kronecker: si* $\boldsymbol{A}$*,* $\boldsymbol{X}$ *y* $\boldsymbol{B}$ *son matrices de tamaños compatibles, entonces se tiene que*

$$(\boldsymbol{B}^\top \otimes \boldsymbol{A}) \operatorname{vec}(\boldsymbol{X}) = \operatorname{vec}(\boldsymbol{A}\boldsymbol{X}\boldsymbol{B}),$$

donde $\operatorname{vec}(\boldsymbol{X})$ *denota el «apilamiento de izquierda a derecha» de las columnas del argumento matricial en un vector. Utilizamos este isomorfismo en* (2.5.8) *con el fin de definir el producto de Kronecker de dos matrices como un operador sobre matrices en forma canónica.*)

3. Supongamos ahora que $\boldsymbol{X} \in \mathbb{R}^{d \times d}$ es de rango completo. En esta y la siguiente parte del ejercicio, consideramos la proyección de $\boldsymbol{X}$ sobre el grupo ortogonal:

$$\operatorname{proj}_{\mathsf{O}(d)}(\boldsymbol{X}) \doteq \min_{\boldsymbol{Q} \in \mathsf{O}(d)} \|\boldsymbol{Q} - \boldsymbol{X}\|_F^2. \tag{2.5.9}$$

Demostraremos que la solución a este problema está dada por

$$\operatorname{proj}_{\mathsf{O}(d)}(\boldsymbol{X}) = \boldsymbol{U}\boldsymbol{V}^\top,$$

donde $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top$ es una descomposición en valores singulares de $\boldsymbol{X}$.

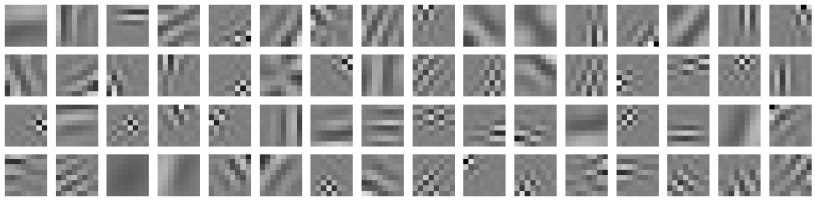
1. Utilizando las condiciones de optimalidad de primer y segundo orden, demuestre que todo minimizador local $\boldsymbol{Q}$ de (2.5.9) satisface

$$\begin{aligned} \left(\boldsymbol{Q}^\top \boldsymbol{X}\right)^\top &= \boldsymbol{Q}^\top \boldsymbol{X}, \\ \boldsymbol{Q}^\top \boldsymbol{X} &\succeq \boldsymbol{0}. \end{aligned}$$

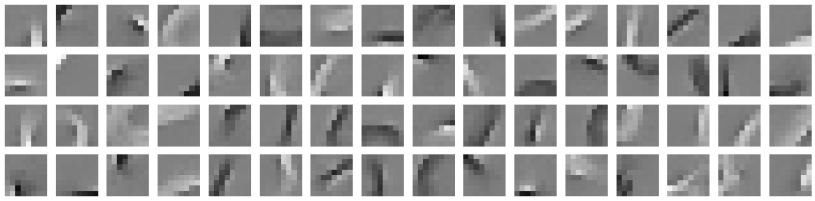
(*Pista: utilizar la linealidad del producto de Kronecker en cualquiera de sus dos argumentos cuando el otro es fijo.*)

2. A partir de estas condiciones, muestre que para todo mínimo local $\boldsymbol{Q}$ de (2.5.9), se cumple que $\boldsymbol{Q}^\top \boldsymbol{X} = (\boldsymbol{X}^\top \boldsymbol{X})^{1/2}$. (*Pista: Utilice el siguiente resultado clásico de álgebra lineal: si* $\boldsymbol{S} \succeq \boldsymbol{0}$ *es una matriz simétrica semidefinida positiva, entonces* $(\boldsymbol{S}^\top \boldsymbol{S})^{1/2} = \boldsymbol{S}$.)
3. Utilizando la descomposición en valores singulares $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top$, concluya que

$$\boldsymbol{U}\boldsymbol{V}^\top = \mathrm{proj}_{\mathsf{O}(d)}(\boldsymbol{X}).$$



(a)



(b)

Figura 2.6: Comparación de los átomos de diccionario aprendidos para diccionarios completos (ortogonales) y sobrecompletos, entrenados para reconstruir fragmentos de 8×8 tomados de los dígitos del MNIST. Ambos diccionarios se entrenan durante 6000 épocas en 10 000 fragmentos aleatorios con contenido no trivial y los códigos esparsos se calculan con el objetivo LASSO y $\lambda = 0.1$ (véase (2.3.5)). Las escalas de colores utilizan el negro para los valores negativos y el blanco para los valores positivos. **En la parte superior:** Un diccionario ortogonal aprendido con el algoritmo MSP (2.2.18) está limitado a tener un máximo de 64 átomos; Los átomos aprendidos se corresponden aproximadamente con un diccionario de tipo «pico y placa», y alcanzan niveles relativamente bajos de esparcidad en la reconstrucción en los datos de prueba no utilizados (los códigos son aproximadamente 17-esparsos en promedio, con respecto a un umbral de 10^{1}). **En la parte inferior:** Por el contrario, un diccionario sobrecompleto (en este caso, con 8^3 átomos; visualizamos un subconjunto aleatorio de 64) aprende átomos de diccionario semánticamente significativos correspondientes a aristas orientadas con signo, que pueden ensamblarse para crear fragmentos de dígitos y lograr niveles superiores de reconstrucción y esparcidad. Los códigos son aproximadamente 20-esparsos en promedio, aunque son 8 veces más grandes que los del diccionario ortogonal. Para calcular el diccionario, utilizamos un optimizador basado en la minimización linealizada alternante proximal sobre una versión adecuadamente regularizada de (2.3.17).

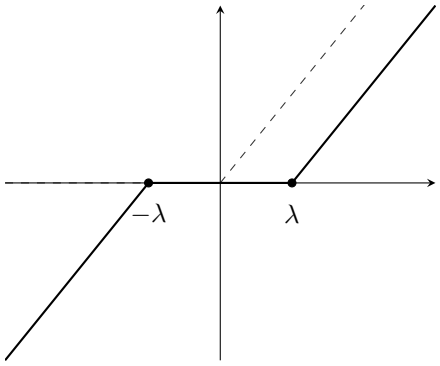


Figura 2.7: Gráfico de la función de umbralización suave S_λ (línea continua) comparada con la función de activación ReLU $x \mapsto \max\{x, 0\}$ (línea de trazos). Entre $-\lambda$ y $+\lambda$, el valor de S_λ se recorta a 0, mientras que el valor (absoluto) de las entradas grandes simplemente se reduce en λ. A diferencia de ReLU, S_λ es simétrica y actúa tanto sobre las entradas positivas como sobre las negativas.

Capítulo 3

Búsqueda de distribuciones de baja dimensión mediante eliminación de ruido

"*La información es la resolución de la incertidumbre.*"

—Claude Shannon, 1948

En el Capítulo 2, mostramos cómo se aprenden clases simples de distribuciones cuyos soportes se supone que son un único subespacio de baja dimensión, una mezcla de tales subespacios o gaussianas de bajo rango. Para simplificar, se considera que los diferentes modos lineales o gaussianos (ocultos) son ortogonales o independientes,[1] tal y como se ilustra en la Figura 2.1. Para estas distribuciones especiales, se pueden derivar algoritmos de aprendizaje sencillos y eficaces con garantías de corrección y eficiencia, siendo claras las interpretaciones geométricas y estadísticas de las operaciones algorítmicas asociadas.

En la práctica, tanto la linealidad como la independencia son supuestos idealizados que las distribuciones de datos reales de alta dimensión *rara vez* cumplen. Lo único que podemos suponer es que la dimensión intrínseca de la distribución es muy baja en comparación con la dimensión del espacio ambiente en el que los datos están incrustados. Por lo tanto, en este capítulo mostramos cómo aprender una clase más general de distribuciones de baja dimensión en un espacio de alta dimensión que *no* es necesariamente (ni por tramos) lineal.

Las distribuciones del mundo real suelen contener múltiples componentes o modos, por ejemplo, correspondientes a diferentes clases de objetos en las imágenes. Es posible que estos modos no sean estadísticamente independientes e incluso puedan tener dimensiones intrínsecas diferentes. Por lo tanto,

[1] O pueden reducirse a tales casos idealizados en condiciones favorables.

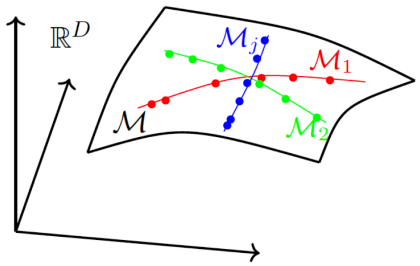


Figura 3.1: Los datos $\boldsymbol{X}$ se distribuyen sobre una mezcla de subvariedades de baja dimensión $\cup_j \mathcal{M}_j$ en un espacio envolvente de muy alta dimensión, por ejemplo, $\mathbb{R}^D$. Este es un caso mucho más general que los casos especiales estudiados en el Capítulo 2, donde se suponía que las subvariedades eran lineales por tramos y, por lo tanto, admitían formas paramétricas y soluciones analíticas.

generalmente suponemos que los datos se distribuyen sobre una mezcla de subvariedades no lineales de baja dimensión incrustadas en un espacio de alta dimensión; la Figura 3.1 ilustra este tipo de distribución. Otra limitación es que, en situaciones del mundo real, normalmente solo tenemos acceso a un número finito de muestras de la distribución de datos, en lugar de disponer de un conocimiento previo de la distribución exacta en sí.

Para aprender dicha distribución bajo estas condiciones, debemos abordar tres preguntas fundamentales:

- ¿Cuál es un enfoque general para aprender y representar una distribución de baja dimensión en un espacio de alta dimensión?
- ¿Cómo medimos la complejidad de la distribución para poder aprovechar eficazmente su baja dimensionalidad en el aprendizaje?
- ¿Cómo podemos hacer que el proceso de aprendizaje sea computacionalmente manejable y escalable, especialmente cuando la dimensión del entorno es elevada y el tamaño de la muestra es grande?

Como veremos, la idea fundamental de la *compresión*, o *reducción de dimensiones*, cuya eficacia ya hemos demostrado en el caso lineal/independiente, sigue siendo un principio rector para el desarrollo de modelos y métodos computacionales para distribuciones no lineales generales de baja dimensión.

Dada su importancia teórica y práctica, analizamos con mayor detalle cómo se materializa este marco basado en la compresión cuando la distribución de interés puede modelarse o aproximarse adecuadamente mediante una mezcla de subespacios de baja dimensión o gaussianas de bajo rango. Como se muestra en los capítulos siguientes, estas clases de modelos, aparentemente restrictivas, sirven como bloques básicos para la construcción de algoritmos que realizan la compresión en distribuciones generales.

3.1 Minimización de la entropía y compresión

3.1.1 Entropía y tasas de codificación

En el Capítulo 1 mencionamos que el objetivo del aprendizaje es encontrar la forma más sencilla de generar un conjunto de datos dado. En términos conceptuales, la complejidad de Kolmogorov fue concebida para proporcionar una medida de complejidad de este tipo, pero no es computable y no está asociada a ningún esquema implementable que pueda reproducir realmente los datos. Por lo tanto, necesitamos una medida de complejidad alternativa, computable y aplicable. Esto nos lleva al concepto de *entropía*, introducido por Shannon en 1948 [**Shannon-1948**].

Para ilustrar el carácter constructivo de la entropía, comencemos con el caso más sencillo. Supongamos que tenemos una variable aleatoria discreta que toma N valores distintos, o *tokens*, $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ con una probabilidad uniforme de $1/N$. Podemos codificar cada token $\boldsymbol{x}_i$ utilizando la representación binaria de $\log_2 N$ bits del número i. Este esquema de codificación puede generalizarse para codificar distribuciones discretas arbitrarias [**Cover-Thomas**]: dada una distribución p tal que $\sum_{i=1}^{N} p(\boldsymbol{x}_i) = 1$, se puede asignar a cada token $\boldsymbol{x}_i$ con probabilidad de $p(\boldsymbol{x}_i)$ un código binario de longitud $\log_2[1/p(\boldsymbol{x}_i)] = -\log_2 p(\boldsymbol{x}_i)$ bits. Por lo tanto, el número promedio de bits, o *la tasa de codificación*, necesario para codificar cualquier muestra de la distribución $p(\cdot)$ viene dado por la expresión[2]:

$$H(p) \doteq \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})] = -\sum_{i=1}^{N} p(\boldsymbol{x}_i) \log p(\boldsymbol{x}_i). \tag{3.1.1}$$

Esto se conoce como la *entropía* de la distribución (discreta) $p(\cdot)$. Obsérvese que esta entropía es siempre no negativa y es cero si y solo si $p(\boldsymbol{x}_i) = 1$ para algún $\boldsymbol{x}_i$ con $i \in [N]$.[3] Para una variable aleatoria $\boldsymbol{x}$, solemos escribir $H(\boldsymbol{x})$ para denotar la entropía de la distribución marginal de $\boldsymbol{x}$.

3.1.2 Entropía diferencial

Cuando la variable aleatoria $\boldsymbol{x} \in \mathbb{R}^D$ es continua y tiene una densidad de probabilidad p, se puede interpretar dicho límite (3.1.1) como relacionado con una integral:

$$h(p) \doteq \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})] = -\int_{\mathbb{R}^D} p(\boldsymbol{\xi}) \log p(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi}. \tag{3.1.2}$$

Más precisamente, dada una distribución de probabilidad continua p en $\mathbb{R}^D$, podemos discretizar $\mathbb{R}^D$ en una unión de cubos disjuntos C_i^ϵ de longitud de arista $\epsilon^{1/D}$, cada uno (digamos) centrado en $\boldsymbol{x}_i$, y definir la distribución discretizada

[2] Según la convención de la teoría de la información [**Cover-Thomas**], el log aquí tiene base 2. Por lo tanto, la entropía se mide en bits (binarios).

[3] Aquí utilizamos el hecho de que $\lim_{p\to 0} p \log p = 0$.

p^ϵ mediante $p^\epsilon(\boldsymbol{x}_i) = \mathbb{P}_{\boldsymbol{x}\sim p}[\boldsymbol{x} \in C_i^\epsilon]$. Entonces se puede demostrar que $H(p^\epsilon) + \log \epsilon \approx h(p)$. Tomando ϵ suficientemente pequeño, observamos que la entropía diferencial $h(p)$ puede ser negativa. Los lectores interesados pueden consultar [**Cover-Thomas**] para obtener una explicación más detallada. Al igual que con la entropía discreta, a menudo escribiremos $h(\boldsymbol{x})$ para referirnos a la entropía de la distribución (marginal) de $\boldsymbol{x}$.

Ejemplo 3.1 (Entropía de las distribuciones gaussianas). Mediante un cálculo directo, se puede mostrar que la entropía de una variable aleatoria gaussiana $x \sim \mathcal{N}(\mu, \sigma^2)$ es

$$h(x) = \frac{1}{2}\log(2\pi\sigma^2) + \frac{1}{2}. \tag{3.1.3}$$

La entropía de una variable aleatoria gaussiana multivariante $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ en $\mathbb{R}^D$ es

$$h(\boldsymbol{x}) = \frac{D}{2}(1 + \log(2\pi)) + \frac{1}{2}\operatorname{logdet}(\boldsymbol{\Sigma}). \tag{3.1.4}$$

■

Al igual que ocurre con la entropía de una distribución discreta, nos gustaría que la entropía diferencial estuviera relacionada con la tasa de codificación de algún esquema de codificación viable. Por ejemplo, como se mencionó anteriormente, si discretizamos el dominio de la distribución con una malla de tamaño $\epsilon > 0$ con centros $\boldsymbol{x}_i$, un posible esquema de codificación consiste en asignar cada $\boldsymbol{x}$ a su centro más cercano $\boldsymbol{x}_i$ y la entropía de la distribución discreta resultante se aproxima a la entropía diferencial [**Cover-Thomas**].

Existen ciertas salvedades asociadas a la definición de la entropía diferencial. En el caso de una distribución en un espacio de alta dimensión, cuando su soporte se vuelve degenerado (de baja dimensión), su entropía diferencial diverge hacia $-\infty$. Este hecho se demuestra en el **??**, pero incluso en el caso explícito y sencillo de las distribuciones gaussianas (3.1.4), cuando la covarianza $\boldsymbol{\Sigma}$ es singular, podemos ver que $\operatorname{logdet}(\boldsymbol{\Sigma}) = -\infty$, por lo que tenemos $h(\boldsymbol{x}) = -\infty$ (Nota: Según el **??**, las distribuciones gaussianas tienen la mayor entropía entre todas las distribuciones con la misma media y covarianza). En una situación así, no resulta evidente cómo cuantificar o codificar adecuadamente dicha distribución. Sin embargo, las distribuciones (gaussianas) degeneradas son precisamente los casos más sencillos posibles, y podría decirse que los más importantes, de distribuciones de baja dimensión en un espacio de alta dimensión. En el próximo capítulo, abordaremos una solución completa a esta aparente dificultad relacionada con la degeneración.

3.1.3 Minimización de la tasa de codificación

El problema de aprendizaje consiste en recuperar una distribución (potencialmente continua) $p(\boldsymbol{x})$ a partir de muestras $\{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\}$ extraídas de ella. Para facilitar la exposición, escribimos $\boldsymbol{X} = \big[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\big] \in \mathbb{R}^{D\times N}$. Dado que las distribuciones de interés son (casi) de baja dimensión, esperamos que su entropía (diferencial) sea muy pequeña. Sin embargo, a diferencia de

las situaciones estudiadas en el capítulo anterior, desconocemos la familia de modelos (analíticos) de baja dimensión a la que pertenece $p(\boldsymbol{x})$. Por lo tanto, comprobar si la entropía es baja parece ser la única pauta en la que podemos basarnos para identificar y modelar la distribución.

Ahora bien, si solo se tienen las muestras sin saber cuáles $p(\boldsymbol{x})$, en teoría podrían interpretarse como muestras de cualquier distribución genérica. En particular, podrían interpretarse como cualquiera de las siguientes:

1. como muestras de la propia distribución empírica $p^{\boldsymbol{X}}$, que asigna una probabilidad de $1/N$ a cada una de las N muestras $\boldsymbol{x}_i, i \in [N]$;
2. como muestras de una distribución normal estándar $\boldsymbol{x}^n \sim p^n \doteq \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I})$ con una varianza σ^2 lo suficientemente grande (digamos, mayor que los cuadrados de las normas de las muestras);
3. como muestras de una distribución normal $\boldsymbol{x}^e \sim p^e \doteq \mathcal{N}(\mathbf{0}, \hat{\boldsymbol{\Sigma}})$, siendo la covarianza $\hat{\boldsymbol{\Sigma}} = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top$ la covarianza empírica de las muestras;
4. como muestras de una distribución $\hat{\boldsymbol{x}} \sim \hat{q}(\boldsymbol{x})$ que se aproxima mucho a la distribución verdadera p.

La pregunta es: ¿qué interpretación es mejor y en qué sentido? Supongamos que creemos que estos datos $\boldsymbol{X}$ provienen de una distribución concreta $q(\boldsymbol{x})$, que podría ser una de las anteriores. Entonces podríamos codificar los puntos de datos con el libro de códigos óptimo para la distribución $q(\boldsymbol{x})$. Entonces, a medida que el número de muestras N aumenta, la longitud media de codificación requerida (o tasa de codificación) viene dada por la denominada *entropía cruzada*:

$$\mathrm{CE}(p \parallel q) \doteq \mathbb{E}_{\boldsymbol{x}\sim p}[-\log q(\boldsymbol{x})]. \tag{3.1.5}$$

Si hemos identificado la distribución correcta $p(\boldsymbol{x})$, la tasa de codificación viene dada por la entropía $h(p) = \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})]$. Resulta que la longitud de codificación anterior $\mathrm{CE}(p \parallel q)$ es siempre mayor o igual que la entropía $h(\boldsymbol{x})$, a menos que $p = q$.

Por lo tanto, dado un conjunto de datos muestreados $\boldsymbol{X}$, para determinar qué distribución caracteriza mejor los datos entre p^n, p^e y $\hat{q}$, podemos comparar sus tasas de codificación para $\boldsymbol{X}$ y ver cuál ofrece la tasa más baja. Sabemos por lo anterior que la tasa de codificación (teóricamente alcanzable) de una distribución está estrechamente relacionada con su entropía. En general,

$$h(p^n) > h(p^e) > h(\hat{q}). \tag{3.1.6}$$

Si los datos $\boldsymbol{X}$ se codificaran mediante el libro de códigos asociado a cada una de estas distribuciones, la tasa de codificación de $\boldsymbol{X}$ disminuiría, por lo tanto, en el mismo orden:

$$\mathrm{CE}(p \parallel p^n) > \mathrm{CE}(p \parallel p^e) > \mathrm{CE}(p \parallel \hat{q}). \tag{3.1.7}$$

Esta observación ofrece una pauta general para buscar una distribución $p(\boldsymbol{x})$ que tenga una estructura de baja dimensión. Sugiere dos posibles enfoques:

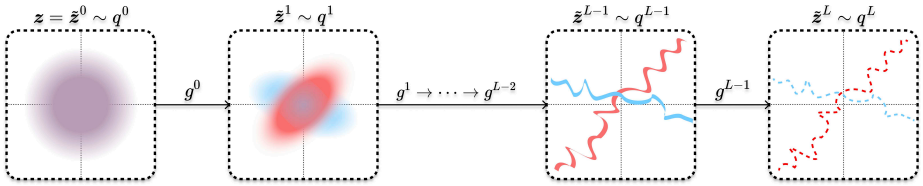


Figura 3.2: Ilustración de un proceso iterativo de eliminación de ruido que, partiendo de una distribución gaussiana isotrópica, converge hacia una distribución de datos arbitraria.

A1. Partiendo de una distribución general (por ejemplo, una distribución normal) con alta entropía, transformar gradualmente la distribución hacia la distribución (empírica) de los datos mediante la reducción de la entropía;

A2. Dentro de una amplia familia de distribuciones (paramétricas o no paramétricas) con esquemas de codificación explícitos que codifican los datos en cuestión, buscar progresivamente mejores esquemas de codificación que ofrezcan tasas de codificación más bajas.

A nivel conceptual, tanto el enfoque A1 como el A2 pretenden lograr lo mismo. Para A1, debemos asegurarnos de que exista tal ruta de transformación y de que sea computable. Para A2, es necesario que la familia elegida sea lo suficientemente amplia y pueda aproximarse de cerca (o abarcar) a la distribución verdadera. En cualquiera de los dos enfoques, debemos asegurarnos de que las soluciones con menor entropía o mejores tasas de codificación puedan calcularse de manera eficiente y converjan rápidamente hacia la distribución deseada.[4] Exploraremos ambos enfoques en las secciones restantes de este capítulo.

3.2 Compresión mediante eliminación de ruido

En esta sección describimos una forma *natural* y *computacionalmente tratable* de aprender una distribución $p_{\boldsymbol{x}}$ mediante el aprendizaje de una codificación paramétrica que minimice la entropía o la tasa de codificación de la representación y luego utilizando esta codificación para transformar muestras de alta entropía de una distribución gaussiana estándar en muestras de baja entropía de la distribución objetivo, tal como se ilustra en la Figura 3.2. Esta metodología combina los enfoques A1 y A2 mencionados anteriormente para aprender y obtener muestras de la distribución.

3.2.1 Procesos de difusión y de eliminación de ruido

En primer lugar, queremos un procedimiento que reduzca la entropía de una variable aleatoria muy ruidosa, generando una muestra de menor entropía a

[4] Por ejemplo, la distribución de datos del mundo real, como imágenes y textos.

partir de la distribución de los datos. Aquí describimos un posible enfoque, quizá el más natural. Primero, diseñamos un método para *aumentar gradualmente* la entropía de las muestras ya extraídas de la distribución de datos. A continuación, buscamos una *inversa aproximada* de este proceso. En general, el aumento de la entropía no tiene inverso, ya que parte de la información de la distribución original puede destruirse. Por lo tanto, nos limitamos a un caso especial en el que (1) la operación que aumenta la entropía es simple, computable y reversible, así como (2) contamos con una codificación (paramétrica) de la distribución de los datos, tal como se ha insinuado anteriormente. Como veremos, estas dos condiciones hacen que nuestro enfoque sea viable.

Procesos de difusión

Aumentaremos la entropía de lo que podría considerarse la forma más sencilla posible, es decir, *añadiendo ruido gaussiano isotrópico*. Más precisamente, dada la variable aleatoria $\boldsymbol{x}$, podemos considerar el *proceso estocástico* $(\boldsymbol{x}_t)_{t\in[0,T]}$ que le añade ruido en forma gradual, es decir,

$$\boldsymbol{x}_t \doteq \boldsymbol{x} + t\boldsymbol{g}, \qquad \forall t \in [0,T], \tag{3.2.1}$$

donde $T \in [0,\infty)$ es un horizonte temporal y $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ se extrae en forma independiente de $\boldsymbol{x}$. Este proceso es un ejemplo de *proceso de difusión*, llamado así porque distribuye (es decir, *difunde*) la masa de probabilidad por todo $\mathbb{R}^D$ a medida que pasa el tiempo, aumentando la entropía. Esta intuición queda confirmada gráficamente en la Figura 3.3, y de manera rigurosa, mediante el siguiente teorema.

Teorema 3.1 (Versión simplificada del **??**)**.** *Supongamos que* $(\boldsymbol{x}_t)_{t\in[0,T]}$ *sigue el modelo* (3.2.1)*. Para cualquier* $t \in (0,T]$*, la variable aleatoria* $\boldsymbol{x}_t$ *tiene una densidad de probabilidad* p_t *y una entropía diferencial* $h(\boldsymbol{x}_t) > -\infty$*. Además, bajo ciertas condiciones técnicas sobre* $\boldsymbol{x}$*,*

$$\frac{\mathrm{d}}{\mathrm{d}t} h(\boldsymbol{x}_t) > 0, \qquad \forall t \in (0,T], \tag{3.2.2}$$

es decir, la entropía de la distribución de $\boldsymbol{x}_t$ *aumenta con el paso del tiempo* t*.*

La demostración es elemental, pero bastante extensa, por lo que la posponemos hasta la **??**. La principal consecuencia, aún no expresada, de este resultado es que $h(\boldsymbol{x}_t) > h(\boldsymbol{x}_s)$ para todo $t > s \geq 0$. Para comprobarlo, nótese que si $s = 0$ y $h(\boldsymbol{x}) = h(\boldsymbol{x}_0) = -\infty$, entonces $h(\boldsymbol{x}_t) > -\infty$ para todo $t > 0$, por lo que la conclusión es verdadera; de lo contrario, $h(\boldsymbol{x}_t) = h(\boldsymbol{x}_s) + \int_s^t \left[\frac{\mathrm{d}}{\mathrm{d}u} h(\boldsymbol{x}_u)\right] \mathrm{d}u > h(\boldsymbol{x}_s)$ por el teorema fundamental del cálculo, por lo que en ambos casos $h(\boldsymbol{x}_t) > h(\boldsymbol{x}_s)$ para todo $t > s \geq 0$.

Ejemplo 3.2 (Añadir ruido a una mezcla de distribuciones gaussianas). En este ejemplo, consideramos añadir ruido a una mezcla de K distribuciones gaussianas. Elegimos K vectores de medias $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K \in \mathbb{R}^D$, K matrices de covarianza $\boldsymbol{\Sigma}_1, \ldots, \boldsymbol{\Sigma}_K \in \mathsf{PSD}(D)$ y probabilidades $\pi_1, \ldots, \pi_K \in [0,1]$ tales que $\sum_{k=1}^K \pi_k = 1$. Entonces, $\boldsymbol{x}$ se muestrea de la siguiente manera.

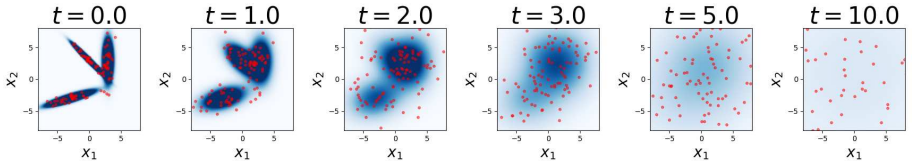


Figura 3.3: **Difusión de una mezcla de distribuciones gaussianas $\boldsymbol{x}$.** De izquierda a derecha, observamos la evolución de la densidad p_t de $\boldsymbol{x}_t$ a medida que t crece de 0 a 10, junto con algunas muestras representativas (en rojo). El plano se colorea según la densidad de probabilidad p_t; las regiones de alta densidad aparecen en un azul más oscuro. Observamos que la masa de probabilidad se vuelve menos concentrada a medida que aumenta t, lo que indica que la entropía aumenta.

- En primer lugar, se elige un índice (o *etiqueta*) $y \in [K]$ de tal manera que $y = k$ con probabilidad π_k.
- En segundo lugar, $\boldsymbol{x}$ se obtiene mediante muestreo de la distribución gaussiana $\mathcal{N}(\boldsymbol{\mu}_y, \boldsymbol{\Sigma}_y)$; es decir, si $y = k$, entonces $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

Por lo tanto, $\boldsymbol{x}$ sigue una distribución de mezcla gaussiana

$$\boldsymbol{x} \sim \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \tag{3.2.3}$$

Los modelos de mezcla gaussiana son relevantes porque pueden aproximar muchas distribuciones: dada cualquier distribución para $\boldsymbol{x}$, existe una mezcla gaussiana que la aproxima con precisión arbitraria. Por este motivo, los modelos de mezcla gaussiana son una alternativa popular y analíticamente manejable para las distribuciones de datos multimodales y complejas. En este libro los utilizamos ampliamente como «problema modelo» para comprender y desarrollar casi todos los conceptos. Por ahora, representamos los datos $\boldsymbol{x}$ mediante una mezcla gaussiana. Tras añadir ruido, la distribución de $\boldsymbol{x}_t$ vuelve a ser una mezcla gaussiana:

$$\boldsymbol{x}_t = \boldsymbol{x} + t\boldsymbol{g} \sim \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k + t^2\boldsymbol{I}). \tag{3.2.4}$$

Para ver con un ejemplo cómo evoluciona la entropía, supongamos que $K = 1$, de modo que $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Según (3.1.4), la entropía de $\boldsymbol{x}_t$ es

$$h(\boldsymbol{x}_t) = \frac{D}{2}(1 + \log(2\pi)) + \frac{1}{2}\operatorname{logdet}(\boldsymbol{\Sigma} + t^2\boldsymbol{I}). \tag{3.2.5}$$

Al derivar con respecto al tiempo (véase el **??**) se obtiene

$$\frac{\mathrm{d}}{\mathrm{d}t}h(\boldsymbol{x}_t) = t\,\operatorname{tr}((\boldsymbol{\Sigma} + t^2\boldsymbol{I})^{-1}) = \sum_{i=1}^{D} \frac{t}{\lambda_i(\boldsymbol{\Sigma}) + t^2} > 0, \tag{3.2.6}$$

donde $\lambda_i(\boldsymbol{\Sigma})$ son los valores propios (no negativos) de $\boldsymbol{\Sigma}$. Dado que esta derivada es positiva, la entropía $h(\boldsymbol{x}_t)$ de la distribución ruidosa aumenta con el tiempo. ■

Procesos de eliminación de ruido

La operación inversa a la adición de ruido se conoce como *eliminación o eliminación de ruido*. Se trata de un tema clásico y ampliamente estudiado en el procesamiento de señales y la teoría de sistemas, que se materializa (por ejemplo) en el filtro de Wiener y el filtro de Kalman (véase la Sección 1.3.1). Varios problemas tratados en el Capítulo 2, como el PCA, el ICA y el aprendizaje de diccionarios, son casos concretos del problema de la eliminación de ruido. Para un t fijo y el modelo de ruido gaussiano aditivo (3.2.1), el problema de eliminación de ruido puede formularse como el intento de aprender una función $\bar{\boldsymbol{x}}^\star(t,\cdot)$ que constituya la mejor aproximación posible (en esperanza) de la variable aleatoria verdadera $\boldsymbol{x}$, dados tanto t como $\boldsymbol{x}_t$:

$$\bar{\boldsymbol{x}}^\star(t,\cdot) \in \arg\min_{\bar{\boldsymbol{x}}(t,\cdot)} \mathbb{E}\,\|\boldsymbol{x} - \bar{\boldsymbol{x}}(t,\boldsymbol{x}_t)\|_2^2. \tag{3.2.7}$$

La solución a este problema, al optimizar $\bar{\boldsymbol{x}}(t,\cdot)$ sobre todas las funciones posibles (cuadráticamente integrables), es el denominado *filtro óptimo de eliminación de ruido de Bayes* y resulta ser la esperanza condicional:

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{\xi}) \doteq \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t = \boldsymbol{\xi}]. \tag{3.2.8}$$

Esta expresión justifica la notación $\bar{\boldsymbol{x}}$, que calcula una esperanza condicional (es decir, la media condicional o el promedio condicional). En resumen, intenta eliminar el ruido de la entrada ruidosa y genera la mejor estimación posible (en términos de esperanza y con respecto a la distancia ℓ^2) de la variable aleatoria original sin ruido.

Ejemplo 3.3 (Eliminación del ruido gaussiano en una mezcla de distribuciones gaussianas). En este ejemplo calculamos el filtro óptimo de eliminación de ruido de Bayes para una clase importante de distribuciones: el modelo de mezcla gaussiana del Ejemplo 3.2. Definamos $\varphi(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma})$ como la densidad de probabilidad de $\mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})$ evaluada en $\boldsymbol{x}$. Con esta notación, la densidad de $\boldsymbol{x}_t$ es

$$p_t(\boldsymbol{x}_t) = \sum_{k=1}^{K} \pi_k \varphi(\boldsymbol{x}_t;\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k + t^2\boldsymbol{I}). \tag{3.2.9}$$

Dado y, las variables siguen una distribución gaussiana multivariada: si escribimos $\boldsymbol{x} = \boldsymbol{\mu}_y + \boldsymbol{\Sigma}_y^{1/2}\boldsymbol{u}$, donde $(\cdot)^{1/2}$ es la raíz cuadrada matricial y $\boldsymbol{u} \sim \mathcal{N}(\boldsymbol{0},\boldsymbol{I})$ independientemente de y (y de $\boldsymbol{g}$), entonces tenemos

$$\begin{bmatrix}\boldsymbol{x}\\ \boldsymbol{x}_t\end{bmatrix} = \begin{bmatrix}\boldsymbol{\mu}_y\\ \boldsymbol{\mu}_y\end{bmatrix} + \begin{bmatrix}\boldsymbol{\Sigma}_y^{1/2} & \boldsymbol{0}\\ \boldsymbol{\Sigma}_y^{1/2} & t\boldsymbol{I}\end{bmatrix}\begin{bmatrix}\boldsymbol{u}\\ \boldsymbol{g}\end{bmatrix}. \tag{3.2.10}$$

Esto muestra que $\boldsymbol{x}$ y $\boldsymbol{x}_t$ siguen una distribución gaussiana multivariada (condicionada en y), tal como se afirmaba. Por lo tanto, podemos escribir

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{x}_t \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_y \\ \boldsymbol{\mu}_y \end{bmatrix}, \begin{bmatrix} \boldsymbol{\Sigma}_y & \boldsymbol{\Sigma}_y \\ \boldsymbol{\Sigma}_y & \boldsymbol{\Sigma}_y + t^2 \boldsymbol{I} \end{bmatrix} \right). \tag{3.2.11}$$

Por lo tanto, la esperanza condicional de $\boldsymbol{x}$ dada $\boldsymbol{x}_t$ (es decir, el filtro óptimo de eliminación de ruido de Bayes dado y) es, como es bien sabido, (véase también el Ejercicio 3.2)

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y] = \boldsymbol{\mu}_y + \boldsymbol{\Sigma}_y (\boldsymbol{\Sigma}_y + t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \boldsymbol{\mu}_y). \tag{3.2.12}$$

Para hallar el *denoiser* óptimo de Bayes global, utilizamos la ley de las esperanzas iteradas, obteniendo

$$\begin{aligned} \bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) &= \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] && (3.2.13) \\ &= \mathbb{E}[\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y] \mid \boldsymbol{x}_t] && (3.2.14) \\ &= \sum_{k=1}^{K} \mathbb{P}[y = k \mid \boldsymbol{x}_t] \, \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y = k]. && (3.2.15) \end{aligned}$$

La probabilidad puede abordarse de la siguiente manera. Sea $p_{t|y}$ la densidad de probabilidad de $\boldsymbol{x}_t$ condicionada a y. Entonces

$$\begin{aligned} \mathbb{P}[y = k \mid \boldsymbol{x}_t] &= \frac{p_{t|y}(\boldsymbol{x}_t \mid k) \pi_k}{p_t(\boldsymbol{x}_t)} && (3.2.16) \\ &= \frac{\pi_k \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})}{\sum_{i=1}^{K} \pi_i \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i + t^2 \boldsymbol{I})}. && (3.2.17) \end{aligned}$$

Por otro lado, la esperanza condicional es la ya descrita:

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t, y = k] = \boldsymbol{\mu}_k + \boldsymbol{\Sigma}_k (\boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \boldsymbol{\mu}_k). \tag{3.2.18}$$

Para resumir todo lo anterior, el verdadero filtro de eliminación de ruido óptimo de Bayes es

$$\begin{aligned} \bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) = \sum_{k=1}^{K} \Bigg\{ & \frac{\pi_k \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})}{\sum_{i=1}^{K} \pi_i \varphi(\boldsymbol{x}_t; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i + t^2 \boldsymbol{I})} && (3.2.19) \\ & \cdot \left(\boldsymbol{\mu}_k + \boldsymbol{\Sigma}_k (\boldsymbol{\Sigma}_k + t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \boldsymbol{\mu}_k) \right) \Bigg\}. && (3.2.20) \end{aligned}$$

Este ejemplo es especialmente importante, y varios casos particulares nos proporcionarán una gran perspectiva conceptual más adelante. Por ahora, intentemos extraer alguna intuición geométrica a partir de la forma funcional del filtro óptimo de eliminación de ruido (3.2.19).

Para comprender (3.2.19) de manera intuitiva, fijemos primero $K = 1$ (es decir, una sola gaussiana) de tal manera que $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Diagonalicemos

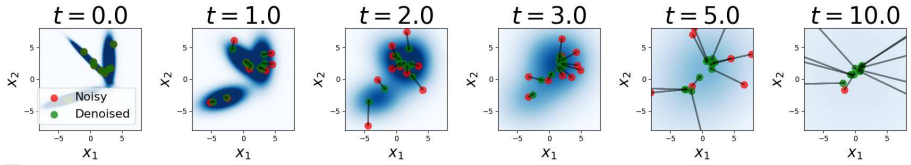


Figura 3.4: ***Denoiser* óptimo de Bayes y *score* de un modelo de mezcla gaussiana.** En el mismo contexto que en la Figura 3.3, mostramos el efecto del {textitdenoiser óptimo de Bayes $\bar{\boldsymbol{x}}^\star$ graficando $\boldsymbol{x}_t$ (rojo) y $\bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t)$ (verde) para un valor arbitrario de t y $\boldsymbol{x}_t$. Según la fórmula de Tweedie, Teorema 3.2, el residuo entre ambos es proporcional a l *score* (Hyvärinen) $\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)$. Podemos observar que el *score* apunta hacia los modos de la distribución de $\boldsymbol{x}_t$.

entonces $\boldsymbol{\Sigma} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^\top$. Entonces, el filtro óptimo de eliminación de ruido de Bayes es

$$\bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) = \boldsymbol{\mu} + \boldsymbol{\Sigma}(\boldsymbol{\Sigma} + t^2\boldsymbol{I})^{-1}(\boldsymbol{x}_t - \boldsymbol{\mu}) \tag{3.2.21}$$

$$= \boldsymbol{\mu} + \boldsymbol{V} \begin{bmatrix} \lambda_1/(\lambda_1 + t^2) & & \\ & \ddots & \\ & & \lambda_D/(\lambda_D + t^2) \end{bmatrix} \boldsymbol{V}^\top(\boldsymbol{x}_t - \boldsymbol{\mu}), \tag{3.2.22}$$

donde $\lambda_1, \ldots, \lambda_D$ son los valores propios de $\boldsymbol{\Sigma}$. Se observa que este filtro de eliminación de ruido consta de tres pasos:

- Trasladar la entrada $\boldsymbol{x}_t$ por $\boldsymbol{\mu}$.
- Contraer la entrada (trasladada) $\boldsymbol{x}_t - \boldsymbol{\mu}$ en cada dirección de vector propio por un factor de $\lambda_i/(\lambda_i + t^2)$. Si la entrada trasladada es de bajo rango y algunos valores propios de $\boldsymbol{\Sigma}$ son cero, el filtro de eliminación de ruido contrae inmediatamente estas direcciones hasta 0, lo que garantiza que la salida de la contracción sea igualmente de bajo rango.
- Trasladar el resultado hacia atrás por $\boldsymbol{\mu}$.

Esta es la interpretación geométrica del filtro de eliminación de ruido de una *única* gaussiana. El *denoiser* global del modelo de mezcla gaussiana (3.2.19) utiliza K *denoisers* de este tipo, ponderando su salida mediante las probabilidades a posteriori $\mathbb{P}[y = k \mid \boldsymbol{x}_t]$. Si las medias de las distribuciones gaussianas están bien separadas, estas probabilidades a posteriori son muy cercanas a 0 o 1 cerca de cada media o grupo. En este régimen, el *denoiser* global (3.2.19) tiene la misma interpretación geométrica que el *denoiser* gaussiano único mencionado anteriormente. ■

Intuitivamente y como podemos ver en el Ejemplo 3.3, el *denoiser* óptimo de Bayes $\bar{\boldsymbol{x}}^\star(t, \cdot)$ debería desplazar su entrada $\boldsymbol{x}_t$ hacia los modos de la distribución de $\boldsymbol{x}$. Resulta que podemos cuantificar esto mostrando que el *denoiser* óptimo de Bayes *da un paso de ascenso por gradiente* en la log-densidad de $\boldsymbol{x}_t$, que (recordemos) hemos denotado como p_t. Es decir, aplicar el filtro de eliminación

de ruido implica pasar de la iteración de entrada a una región de mayor probabilidad dentro de esta distribución (perturbada). Para valores pequeños de t, la perturbación es pequeña, por lo que nuestra intuición inicial es, por lo tanto, casi totalmente correcta. La imagen se muestra en la Figura 3.4 y se formula rigurosamente como la fórmula de Tweedie [**Robbins1956AnEB**].

Teorema 3.2 (La fórmula de Tweedie)**.** *Supongamos que* $(\boldsymbol{x}_t)_{t\in[0,T]}$ *obedece a* (3.2.1)*. Sea* p_t *la densidad de* $\boldsymbol{x}_t$ *(tal como se ha definido anteriormente). Entonces*

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] = \boldsymbol{x}_t + t^2 \nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t). \tag{3.2.23}$$

Demostración. Para demostrarlo, supongamos que $\boldsymbol{x}$ tiene una densidad,[5] y llamemos a esta densidad p. Sean $p_{0|t}$ y $p_{t|0}$ las densidades condicionales de $\boldsymbol{x} = \boldsymbol{x}_0$ dado $\boldsymbol{x}_t$ y de $\boldsymbol{x}_t$ dado $\boldsymbol{x}$, respectivamente. Sea $\varphi(\boldsymbol{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ la densidad de $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ evaluada en $\boldsymbol{x}$, de modo que $p_{t|0}(\boldsymbol{x}_t \mid \boldsymbol{x}) = \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I})$. Entonces, un cálculo sencillo nos da

$$\begin{aligned}
\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t) &= \frac{\nabla_{\boldsymbol{x}_t} p_t(\boldsymbol{x}_t)}{p_t(\boldsymbol{x}_t)} && (3.2.24)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \nabla_{\boldsymbol{x}_t} \int_{\mathbb{R}^D} p(\boldsymbol{x}) p_{t|0}(\boldsymbol{x}_t \mid \boldsymbol{x}) \mathrm{d}\boldsymbol{x} && (3.2.25)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \nabla_{\boldsymbol{x}_t} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \mathrm{d}\boldsymbol{x} && (3.2.26)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) [\nabla_{\boldsymbol{x}_t} \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I})] \mathrm{d}\boldsymbol{x} && (3.2.27)\\
&= \frac{1}{p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \left[-\frac{\boldsymbol{x}_t - \boldsymbol{x}}{t^2} \right] \mathrm{d}\boldsymbol{x} && (3.2.28)\\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) [\boldsymbol{x} - \boldsymbol{x}_t] \mathrm{d}\boldsymbol{x} && (3.2.29)\\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \boldsymbol{x} \mathrm{d}\boldsymbol{x} && (3.2.30)\\
&\quad - \frac{\boldsymbol{x}_t}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) \varphi(\boldsymbol{x}_t; \boldsymbol{x}, t^2\boldsymbol{I}) \mathrm{d}\boldsymbol{x} \\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p(\boldsymbol{x}) p_{t|0}(\boldsymbol{x}_t \mid \boldsymbol{x}) \boldsymbol{x} \mathrm{d}\boldsymbol{x} - \frac{\boldsymbol{x}_t}{t^2 p_t(\boldsymbol{x}_t)} p_t(\boldsymbol{x}_t) && (3.2.31)\\
&= \frac{1}{t^2 p_t(\boldsymbol{x}_t)} \int_{\mathbb{R}^D} p_t(\boldsymbol{x}_t) p_{0|t}(\boldsymbol{x} \mid \boldsymbol{x}_t) \boldsymbol{x} \mathrm{d}\boldsymbol{x} - \frac{\boldsymbol{x}_t}{t^2 p_t(\boldsymbol{x}_t)} p_t(\boldsymbol{x}_t) && (3.2.32)\\
&= \frac{1}{t^2} \int_{\mathbb{R}^D} p_{0|t}(\boldsymbol{x} \mid \boldsymbol{x}_t) \boldsymbol{x} \mathrm{d}\boldsymbol{x} - \frac{\boldsymbol{x}_t}{t^2} && (3.2.33)\\
&= \frac{1}{t^2} \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] - \frac{\boldsymbol{x}_t}{t^2} && (3.2.34)
\end{aligned}$$

[5] Obsérvese que el teorema es cierto sin esta suposición y que puede demostrarse mediante una modificación mínima de esta demostración para utilizar medidas de probabilidad en lugar de densidades.

$$= \frac{\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] - \boldsymbol{x}_t}{t^2}. \tag{3.2.35}$$

Una simple reorganización de la igualdad anterior demuestra el teorema. □

Este resultado establece una conexión entre la eliminación de ruido y la optimización: el *denoiser* óptimo de Bayes da un único paso de ascenso por gradiente en la densidad de datos perturbada p_t, y el tamaño del paso se reduce en forma adaptativa (es decir, da pasos más precisos) a medida que la perturbación de la distribución de datos se va reduciendo. La cantidad $\nabla_{\boldsymbol{x}_t} \log p_t(\boldsymbol{x}_t)$ se denomina *puntuación (de Hyvärinen)* y aparece con frecuencia en discusiones sobre la eliminación de ruido, etc.; apareció por primera vez en un artículo de Aapo Hyvärinen en el contexto del ICA [**hyvarinen05a**].

De manera similar a cómo un solo paso de descenso por gradiente casi nunca es suficiente para minimizar un objetivo en la práctica cuando se inicia lejos del óptimo, la salida del *denoiser* óptimo de Bayes $\bar{\boldsymbol{x}}^\star(t, \cdot)$ casi nunca se encuentra en una región de alta probabilidad de la distribución de datos cuando t es grande, *especialmente* cuando los datos tienen una estructura de baja dimensión. Ilustramos este punto de manera explícita en el siguiente ejemplo.

Ejemplo 3.4 (Eliminación de ruido en una mezcla de dos puntos). Sea x una variable aleatoria uniforme en el conjunto de dos puntos $\{-1, +1\}$ y sea $(x_t)_{t\in[0,T]}$ una variable aleatoria que sigue la ecuación (3.2.1). Se trata precisamente de un modelo de mezcla gaussiana degenerada con distribuciones a priori iguales a $\frac{1}{2}$, medias $\{-1, +1\}$ y covarianzas ambas iguales a 0. Para un valor fijo de $t > 0$, podemos utilizar el cálculo del *denoiser* óptimo de Bayes en (3.2.19) para obtener (la demostración se deja como ejercicio)

$$\bar{x}^\star(t, x_t) = \frac{\varphi(x_t; +1, t^2) - \varphi(x_t; -1, t^2)}{\varphi(x_t; 1, t^2) + \varphi(x_t; -1, t^2)} = \tanh\Big(\frac{x_t}{t^2}\Big). \tag{3.2.36}$$

Para valores de t cercanos a 0, esta cantidad se aproxima a $\{-1, +1\}$ para casi todas las entradas $\bar{x}^\star(t, x_t)$. Sin embargo, para t grande, esta cantidad no se encuentra necesariamente ni siquiera aproximadamente en el soporte original de x, que, como recordaremos, es $\{-1, +1\}$. En particular, para $x_t \approx 0$ se cumple que $\bar{x}^\star(t, x_t) \approx 0$, lo cual se encuentra completamente entre los dos puntos posibles. Por lo tanto, $\bar{x}^\star$ *no dará como resultado un* x *«realista»*. Más rigurosamente, la distribución de $\bar{x}^\star(t, x_t)$ es muy diferente de la distribución de x. ■

Eliminación de ruido incremental

Por lo tanto, para eliminar el ruido de la muestra muy ruidosa $\boldsymbol{x}_T$ (donde, recordemos, T es el tiempo máximo), no podemos utilizar el filtro de eliminación de ruido solo una vez. En cambio, debemos aplicarlo muchas veces, en forma análoga al descenso por gradiente con tamaños de paso decrecientes, para converger hacia un punto estacionario $\hat{\boldsymbol{x}}$. Es decir, utilizamos el filtro de eliminación de ruido para pasar de $\boldsymbol{x}_T$ a $\hat{\boldsymbol{x}}_{T-\delta}$, que se aproxima a $\boldsymbol{x}_{T-\delta}$, luego

de $\hat{\boldsymbol{x}}_{T-\delta}$ a $\hat{\boldsymbol{x}}_{T-2\delta}$, y así sucesivamente, desde $\hat{\boldsymbol{x}}_\delta$ hasta $\hat{\boldsymbol{x}} = \hat{\boldsymbol{x}}_0$. Cada paso de eliminación de ruido hace que la acción del filtro de eliminación de ruido se parezca más a un paso de gradiente sobre la densidad (logarítmica) original.

De manera más formal, discretizamos uniformemente el intervalo $[0, T]$ en $L+1$ pasos temporales $0 = t_0 < t_1 < \cdots < t_L = T$, es decir,

$$t_\ell = \frac{\ell}{L} T, \qquad \ell \in \{0, 1, \ldots, L\}. \tag{3.2.37}$$

A continuación, para cada $\ell \in [L] = \{1, 2, \ldots, L\}$, procediendo de $\ell = L$ a $\ell = 1$, ejecutamos la iteración

$$\begin{aligned}
\hat{\boldsymbol{x}}_{t_{\ell-1}} &= \mathbb{E}[\boldsymbol{x}_{t_{\ell-1}} \mid \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}] && (3.2.38)\\
&= \mathbb{E}[\boldsymbol{x} + t_{\ell-1}\boldsymbol{g} \mid \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}] && (3.2.39)\\
&= \mathbb{E}\left[\boldsymbol{x} + t_{\ell-1} \cdot \frac{\boldsymbol{x}_{t_\ell} - \boldsymbol{x}}{t_\ell} \;\middle|\; \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}\right] && (3.2.40)\\
&= \frac{t_{\ell-1}}{t_\ell}\hat{\boldsymbol{x}}_{t_\ell} + \left(1 - \frac{t_{\ell-1}}{t_\ell}\right)\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_{t_\ell} = \hat{\boldsymbol{x}}_{t_\ell}] && (3.2.41)\\
&= \left(1 - \frac{1}{\ell}\right)\hat{\boldsymbol{x}}_{t_\ell} + \frac{1}{\ell}\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}). && (3.2.42)
\end{aligned}$$

Al inicio de la iteración, cuando ℓ es grande, apenas confiamos en la salida del *denoiser* y, en su mayor parte, conservamos el iterado actual. Esto tiene sentido, ya que el filtro de eliminación de ruido puede presentar una enorme varianza (véase el Ejemplo 3.4). Cuando ℓ es pequeño, el *denoiser* se «fija» en los modos de la distribución de los datos, ya que un paso de la eliminación de ruido es, en esencia, un paso de gradiente sobre la log-densidad de la distribución verdadera y podemos confiar en que no generará muestras poco verosímiles; por lo tanto, la etapa de eliminación de ruido está determinada por la salida del *denoiser*. Cuando $\ell = 1$, incluso descartamos el iterado actual y conservamos únicamente la salida del *denoiser*.

Visualizamos el proceso de convergencia en $\mathbb{R}^3$ en la Figura 3.5. Aunque más adelante desarrollaremos algunos resultados rigurosos sobre la convergencia, ahora ofrecemos un ejemplo en forma cerrada del proceso iterativo de eliminación de ruido aplicado a la eliminación de ruido de una distribución gaussiana de datos.

Ejemplo 3.5 (Eliminación de ruido iterativa para una sola gaussiana). Para comprender de manera intuitiva el procedimiento iterativo de reducción de ruido descrito anteriormente, la aplicamos para eliminar el ruido de $\boldsymbol{x}_T$, definido en (3.2.1), donde $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$. Utilizando (3.2.42), calculamos

$$\begin{aligned}
\hat{\boldsymbol{x}}_{t_{\ell-1}} - \boldsymbol{\mu} &= \left(1 - \frac{1}{\ell}\right)\hat{\boldsymbol{x}}_{t_\ell} + \frac{1}{\ell}\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}) - \boldsymbol{\mu} && (3.2.43)\\
&= \left(1 - \frac{1}{\ell}\right)(\hat{\boldsymbol{x}}_{t_\ell} - \boldsymbol{\mu}) + \frac{1}{\ell}(\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}) - \boldsymbol{\mu}). && (3.2.44)
\end{aligned}$$

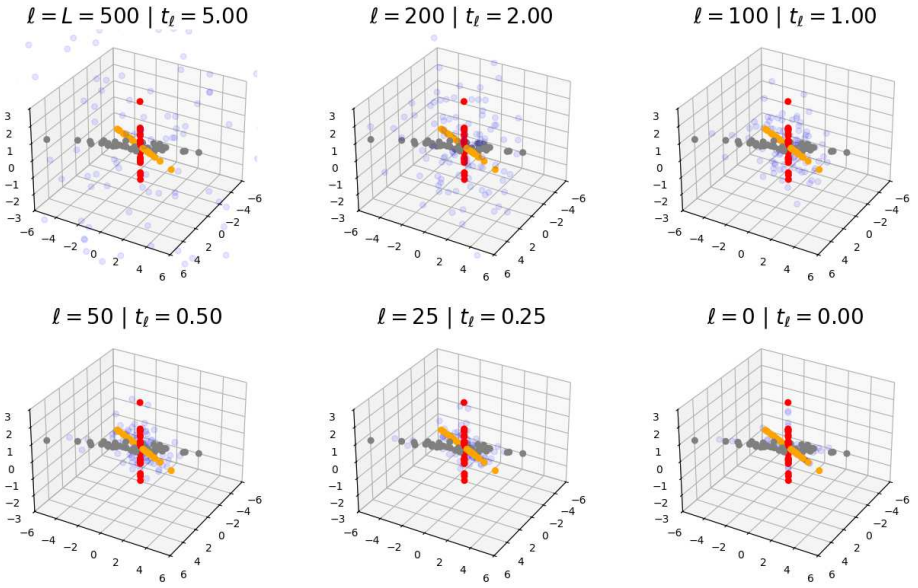


Figura 3.5: **Eliminación del ruido en una mezcla de gaussianas de rango bajo.** Cada figura muestra muestras de la distribución real de los datos (gris, naranja, rojo) y muestras sometidas al proceso de eliminación de ruido (3.2.82) (celeste). En la parte superior izquierda, el proceso acaba de comenzar y el ruido es muy intenso. A medida que avanza el proceso, el ruido se desplaza cada vez más hacia el soporte de la distribución de datos de rango bajo. Por último, en la parte inferior derecha, las muestras generadas se alinean perfectamente con el soporte de los datos y se asemejan mucho a las muestras extraídas del modelo de mezcla gaussiana de rango bajo.

A continuación, al sustituir la expresión de $\bar{\boldsymbol{x}}^\star$ que obtenemos de (3.2.22) y realizar algunas operaciones tediosas de álgebra matricial, obtenemos

$$\boldsymbol{x}_{t_{\ell-1}} - \boldsymbol{\mu} = \boldsymbol{V} \begin{bmatrix} 1 - \frac{\ell T^2}{\lambda_1 L^2 + \ell^2 T^2} & & \\ & \ddots & \\ & & 1 - \frac{\ell T^2}{\lambda_D L^2 + \ell^2 T^2} \end{bmatrix} \boldsymbol{V}^\top (\boldsymbol{x}_{t_\ell} - \boldsymbol{\mu}). \quad (3.2.45)$$

Obsérvese que todas las entradas de esta matriz diagonal están en el intervalo $(0, 1)$. Si tomamos la norma ℓ^2 y utilizamos una cota sencilla pero ajustada, se cumple que

$$\|\boldsymbol{x}_{t_{\ell-1}} - \boldsymbol{\mu}\|_2 \leq \underbrace{\max_{i \in [D]} \left\{ 1 - \frac{\ell T^2}{\lambda_i L^2 + \ell^2 T^2} \right\}}_{c_{\ell, L}} \cdot \|\boldsymbol{x}_{t_\ell} - \boldsymbol{\mu}\|_2. \quad (3.2.46)$$

A primera vista, esta desigualdad parece implicar que la iteración de eliminación de ruido es, conceptualmente, muy similar a un mapeo de contracción, como la

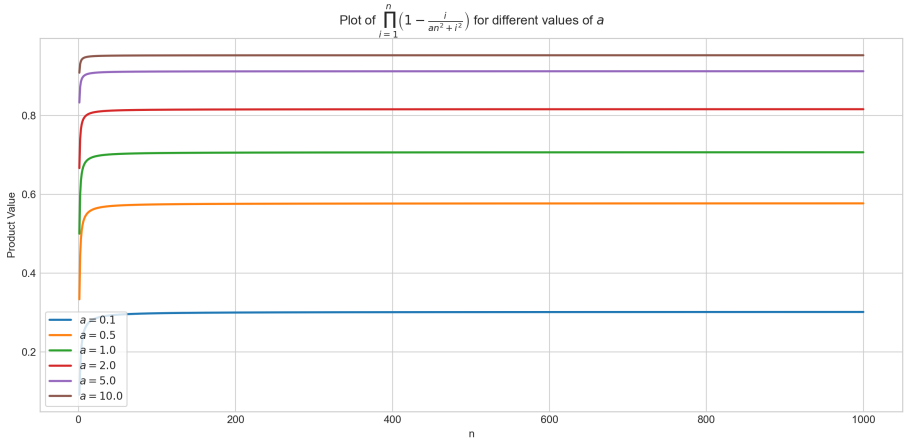


Figura 3.6: **La iteración de eliminación de ruido en el Ejemplo 3.5 no es un mapeo de contracción.** Gráfico de los productos de los coeficientes de contracción $\prod_{\ell=1}^{L} c_{\ell,L}$ en el caso de que los datos se encuentran en $D = 1$, el horizonte temporal $T = 1$ y la varianza de los datos $\lambda_1 = a$, para distintos valores de a. Si la iteración de eliminación de ruido fuera (conceptualmente similar a) una aplicación de contracción, estos productos tenderían a 0, pero, en cambio, convergen hacia valores distintos de cero.

iteración de potencias (2.1.17), que lleva el iterado $\boldsymbol{x}_t$ a la media $\boldsymbol{\mu}$, ya que todos los valores de $c_{\ell,L}$ pertenecen a $(0, 1)$ para cualquier ℓ y L. Sin embargo, esto no es así. Para verlo, obsérvese que, al iterar (3.2.46), se cumple que

$$\|\boldsymbol{x}_0 - \boldsymbol{\mu}\|_2 \leq \prod_{\ell=1}^{L} c_{\ell,L} \cdot \|\boldsymbol{x}_T - \boldsymbol{\mu}\|_2. \tag{3.2.47}$$

Si cada actualización de eliminación de ruido tuviera propiedades similares a las de un mapeo de contracción, entonces $\prod_{\ell=1}^{L} c_{\ell,L}$ tendería a 0 cuando $L \to \infty$ y la iteración convergería a $\boldsymbol{\mu}$ extremadamente rápido, lo que haría que el muestreador fuera completamente inútil o vacuo. Pero, en realidad, la iteración de eliminación de ruido *no es, en realidad, un mapeo de contracción*, ya que $c_{\ell,L}$ varía con ambos índices ℓ y L, de modo que $\prod_{\ell=1}^{L} c_{\ell,L}$ no tiende a 0 cuando $L \to \infty$, como se demuestra en la Figura 3.6 (también es posible calcular el límite analíticamente). En realidad, el muestreador garantiza que las iteraciones finales se dispersen a lo largo del soporte de la distribución de datos, lo que permite un muestreo real a partir de la distribución de datos, algo muy distinto del objetivo de la iteración de potencias (por ejemplo). ■

Recordemos que queríamos construir el proceso iterativo de eliminación de ruido para reducir la entropía de los iterados. Aunque lo hicimos por un rodeo, invirtiendo un proceso que añade entropía, ahora es el momento de pagar las consecuencias y confirmar que nuestro proceso iterativo de eliminación de ruido reduce la entropía.

Teorema 3.3 (Versión simplifiada del **??**). *Supongamos que* $(\boldsymbol{x}_t)_{t\in[0,T]}$ *obedece a* (3.2.1). *Entonces, bajo ciertas condiciones técnicas sobre* $\boldsymbol{x}$, *para todo* $s < t$ *con* $s, t \in (0, T]$,

$$h(\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]) < h(\boldsymbol{x}_t). \tag{3.2.48}$$

El enunciado completo del teorema y la propia demostración requieren cierto grado de tecnicismo, por lo que se posponen hasta la **??**.

Cálculo del *denoiser*

Lo último que comentamos aquí es que, en muchas ocasiones, *no podremos calcular* $\bar{\boldsymbol{x}}^\star(t,\cdot)$ para ningún t, ya que no disponemos de la distribución p_t. Pero podemos intentar *aprender una a partir de los datos*. Recordemos que el *denoiser* $\bar{\boldsymbol{x}}^\star$ se define en (3.2.7) como la minimización del error cuadrático medio $\mathbb{E}\,\|\bar{\boldsymbol{x}}(t,\boldsymbol{x}_t)-\boldsymbol{x}\|_2^2$. Podemos utilizar este error cuadrático medio como función de pérdida o función objetivo para entrenar el filtro de eliminación de ruido. Por ejemplo, podemos parametrizar $\bar{\boldsymbol{x}}(t,\cdot)$ mediante una red neuronal, escribiéndola como $\bar{\boldsymbol{x}}_\theta(t,\cdot)$ y optimizar la pérdida en el espacio de parámetros Θ:

$$\min_{\theta\in\Theta} \mathbb{E}\,\|\bar{\boldsymbol{x}}_\theta(t,\boldsymbol{x}_t) - \boldsymbol{x}\|_2^2. \tag{3.2.49}$$

La solución a este problema de optimización, implementada mediante descenso por gradiente o un algoritmo similar, nos dará un $\bar{\boldsymbol{x}}_{\theta^\star}(t,\cdot)$ que es una buena aproximación a $\bar{\boldsymbol{x}}^\star(t,\cdot)$ (al menos si el entrenamiento funciona) y que utilizaremos como nuestro *denoiser*.

¿Cuál sería una buena arquitectura para esta red neuronal $\bar{\boldsymbol{x}}_{\theta^\star}(t,\cdot)$? Para responder a esta pregunta, intentaremos primero comprender el caso en que los datos $\boldsymbol{x}$ son de *bajo rango*, es decir, que tienen un soporte de bajo rango. La forma más canónica de representar datos de rango bajo es mediante una *distribución gaussiana de rango bajo* , por lo que comenzamos nuestro análisis suponiendo que $\boldsymbol{x}$ se extrae (aproximadamente) de una distribución gaussiana de rango bajo:

$$\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}\boldsymbol{U}^\top), \tag{3.2.50}$$

donde $\boldsymbol{U} \in \mathsf{O}(D,P)$ es una matriz ortogonal alta con $P \le D$ (utilizando una media nula esencialmente por conveniencia notacional). El *denoiser* óptimo bajo (3.2.50) es (según el Ejemplo 3.3):

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{x}_t) = \boldsymbol{U}\boldsymbol{U}^\top(\boldsymbol{U}\boldsymbol{U}^\top + t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t. \tag{3.2.51}$$

La matriz $\boldsymbol{U}\boldsymbol{U}^\top + t^2\boldsymbol{I}$ es una perturbación de bajo rango de la matriz de rango completo $t^2\boldsymbol{I}$, por lo que se presta a una simplificación mediante la *identidad de Sherman-Morrison-Woodbury*, es decir, para matrices $\boldsymbol{A},\boldsymbol{C},\boldsymbol{U},\boldsymbol{V}$ tales que $\boldsymbol{A}$ y $\boldsymbol{C}$ son invertibles,

$$(\boldsymbol{A}+\boldsymbol{U}\boldsymbol{C}\boldsymbol{V})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{U}(\boldsymbol{C}^{-1}+\boldsymbol{V}\boldsymbol{A}^{-1}\boldsymbol{U})^{-1}\boldsymbol{V}\boldsymbol{A}^{-1}. \tag{3.2.52}$$

Demostramos esta identidad en el Ejercicio 3.3. Por ahora, aplicamos esta identidad con $\boldsymbol{A} = t^2\boldsymbol{I}$, $\boldsymbol{U} = \boldsymbol{U}$, $\boldsymbol{V} = \boldsymbol{U}^\top$ y $\boldsymbol{C} = \boldsymbol{I}$, obteniendo

$$(\boldsymbol{U}\boldsymbol{U}^\top + t^2\boldsymbol{I})^{-1} = \frac{1}{t^2}\boldsymbol{I} - \frac{1}{t^4}\boldsymbol{U}\left(\boldsymbol{I} + \frac{1}{t^2}\boldsymbol{U}^\top\boldsymbol{U}\right)^{-1}\boldsymbol{U}^\top \tag{3.2.53}$$

$$= \frac{1}{t^2}\boldsymbol{I} - \frac{1}{t^4\left(1+\frac{1}{t^2}\right)}\boldsymbol{U}\boldsymbol{U}^\top \tag{3.2.54}$$

$$= \frac{1}{t^2}\left(\boldsymbol{I} - \frac{1}{1+t^2}\boldsymbol{U}\boldsymbol{U}^\top\right). \tag{3.2.55}$$

Esto implica

$$\boldsymbol{U}\boldsymbol{U}^\top(\boldsymbol{U}\boldsymbol{U}^\top + t^2\boldsymbol{I})^{-1} = \frac{1}{t^2}\boldsymbol{U}\boldsymbol{U}^\top\left(\boldsymbol{I} - \frac{1}{1+t^2}\boldsymbol{U}\boldsymbol{U}^\top\right) \tag{3.2.56}$$

$$= \frac{1}{t^2}\left(1 - \frac{1}{1+t^2}\right)\boldsymbol{U}\boldsymbol{U}^\top \tag{3.2.57}$$

$$= \frac{1}{1+t^2}\boldsymbol{U}\boldsymbol{U}^\top. \tag{3.2.58}$$

Nuestro filtro de eliminación de ruido óptimo es una proyección lineal (reescalada) sobre el subespacio de rango bajo que sustenta la distribución de nuestros datos. Esto es básicamente lo mismo que el PCA probabilístico del Capítulo 2 (véase Sección 2.1.4). Podemos formalizar esta intuición considerando una clase de *denoisers* parametrizados por subespacios:

$$\bar{\boldsymbol{x}}_{\boldsymbol{V}}(t, \boldsymbol{x}_t) = \frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{x}_t, \tag{3.2.59}$$

y mostrando que el problema de eliminación de ruido $\min_{\boldsymbol{V}\in\mathsf{O}(D,P)} \mathbb{E}\,\|\bar{\boldsymbol{x}}_{\boldsymbol{V}}(t,\boldsymbol{x}_t) - \boldsymbol{x}\|_2^2$ obtiene la misma solución que el PCA probabilístico (Sección 2.1.4). De hecho, al sustituir nuestro *denoiser* por subespacios en la función de pérdida de entrenamiento (3.2.7), obtenemos

$$\min_{\boldsymbol{V}\in\mathsf{O}(D,P)} \mathbb{E}\,\|\bar{\boldsymbol{x}}_{\boldsymbol{V}}(t,\boldsymbol{x}_t) - \boldsymbol{x}\|_2^2 = \mathbb{E}\left\|\frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top(\boldsymbol{x}+t\boldsymbol{g}) - \boldsymbol{x}\right\|_2^2, \tag{3.2.60}$$

donde la igualdad se debe a (3.2.1). Condicionando respecto a $\boldsymbol{x}$ (integrando así sobre $\boldsymbol{g}$), calculamos

$$\mathbb{E}_{\boldsymbol{g}}\left\|\frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top(\boldsymbol{x}+t\boldsymbol{g}) - \boldsymbol{x}\right\|_2^2 \tag{3.2.61}$$

$$= \left\|\frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{x} - \boldsymbol{x}\right\|_2^2 - \frac{t}{1+t^2}\,\mathbb{E}_{\boldsymbol{g}}\left\langle \boldsymbol{x} - \frac{1}{1+t^2}\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{x}, \boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{g}\right\rangle \tag{3.2.62}$$

$$+ \frac{t^2}{(1+t^2)^2}\,\mathbb{E}_{\boldsymbol{g}}\left\|\boldsymbol{V}\boldsymbol{V}^\top\boldsymbol{g}\right\|_2^2 \tag{3.2.63}$$

$$= \left\| \frac{1}{1+t^2} \boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{x} - \boldsymbol{x} \right\|_2^2 + \frac{t^2 P}{(1+t^2)^2} \tag{3.2.64}$$

donde la segunda igualdad se deduce de $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ y del hecho de que $\boldsymbol{V} \in \mathsf{O}(D, P)$, lo que implica que $\mathbb{E}_{\boldsymbol{g}} \left\| \boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{g} \right\|_2^2 = \mathbb{E}_{\boldsymbol{g}} \left\| \boldsymbol{V}^\top \boldsymbol{g} \right\|_2^2 = P$. Por lo tanto, el problema de eliminación de ruido es equivalente a

$$\min_{\boldsymbol{V} \in \mathsf{O}(D,P)} \left\{ \mathbb{E} \left\| \frac{1}{1+t^2} \boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{x} - \boldsymbol{x} \right\|_2^2 \right. \tag{3.2.65}$$

$$\left. = \mathbb{E} \left\| \boldsymbol{x} \right\|_2^2 + \left(\frac{1}{(1+t^2)^2} - \frac{2}{1+t^2} \right) \mathbb{E} \left\| \boldsymbol{V}^\top \boldsymbol{x} \right\|_2^2 \right\}, \tag{3.2.66}$$

lo cual, a su vez, equivale del problema

$$\max_{\boldsymbol{V} \in \mathsf{O}(D,P)} \mathbb{E} \left\| \boldsymbol{V}^\top \boldsymbol{x} \right\|_2^2 \tag{3.2.67}$$

que, como se demostró en la Sección 2.1.1, es equivalente al problema

$$\min_{\boldsymbol{V} \in \mathsf{O}(D,P)} \mathbb{E} \left\| \boldsymbol{x} - \boldsymbol{V}\boldsymbol{V}^\top \boldsymbol{x} \right\|_2^2, \tag{3.2.68}$$

es decir, la formulación probabilística del PCA (2.1.30).

Por lo tanto, si supusiéramos que nuestros datos procedieran de una distribución gaussiana de rango bajo, nuestro algoritmo de eliminación de ruido consistiría básicamente en aplicar sucesivamente muchas iteraciones de PCA a datos cada vez menos ruidosos. Aunque este método parece muy sencillo, en realidad puede generar comportamientos de eliminación de ruido complejos y relativamente potentes cuando se aplica a datos reales, tal y como ha demostrado PCANet [**chan2015pcanet**]. No obstante, para obtener un operador de eliminación de ruido más general que pueda ser aplicable a con distribuciones de datos complejas y multimodales, consideraremos una clase más amplia de distribuciones de datos: la omnipresente clase de *modelos de mezcla gaussiana de bajo rango*, cuyo *denoiser* calculamos en el Ejemplo 3.2 y el Ejemplo 3.3. En el contexto de la eliminación de ruido, dado que los modelos de mezcla gaussiana pueden aproximar una clase muy amplia de distribuciones con una precisión arbitraria, en la práctica, la optimización dentro de la clase de *denoisers* para modelos de mezcla gaussiana puede, potencialmente, proporcionarnos algo cercano al *denoiser* óptimo para la distribución real de los datos.

Formalmente, supongamos ahora que $\boldsymbol{x}$ se extrae de un modelo de mezcla gaussiana de bajo rango $\frac{1}{K} \sum_{k=1}^{K} \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k \boldsymbol{U}_k^\top)$, donde $\boldsymbol{U}_k \in \mathsf{O}(D, P)$ son matrices ortogonales altas cuyas columnas abarcan los subespacios de dimensión P que soportan las K componentes gaussianas de rango bajo. Escribimos

$$\boldsymbol{x} \sim \frac{1}{K} \sum_{k=1}^{K} \mathcal{N}(\boldsymbol{0}, \boldsymbol{U}_k \boldsymbol{U}_k^\top). \tag{3.2.69}$$

Entonces, el *denoiser* óptimo según (3.2.1) es (de acuerdo al Ejemplo 3.3)

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{x}_t) = \sum_{k=1}^{K} \left\{ \frac{\varphi(\boldsymbol{x}_t;\mathbf{0},\boldsymbol{U}_k\boldsymbol{U}_k^\top + t^2\boldsymbol{I})}{\sum_{i=1}^{K}\varphi(\boldsymbol{x}_t;\mathbf{0},\boldsymbol{U}_i\boldsymbol{U}_i^\top + t^2\boldsymbol{I})} \right. \tag{3.2.70}$$

$$\left. \cdot \left(\boldsymbol{U}_k\boldsymbol{U}_k^\top(\boldsymbol{U}_k\boldsymbol{U}_k^\top + t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t\right) \right\}. \tag{3.2.71}$$

Al aplicar la misma identidad de Sherman-Morrison-Woodbury que en (3.2.55), obtenemos

$$(\boldsymbol{U}_k\boldsymbol{U}_k^\top + t^2\boldsymbol{I})^{-1} = \frac{1}{t^2}\left(\boldsymbol{I} - \frac{1}{1+t^2}\boldsymbol{U}_k\boldsymbol{U}_k^\top\right). \tag{3.2.72}$$

Podemos entonces calcular las probabilidades a posteriori de la siguiente manera. Dado que los $\boldsymbol{U}_k$ son ortogonales, $\det(\boldsymbol{U}_k\boldsymbol{U}_k^\top + t^2\boldsymbol{I})$ es igual para todo k. Por lo tanto

$$\frac{\varphi(\boldsymbol{x}_t;\mathbf{0},\boldsymbol{U}_k\boldsymbol{U}_k^\top + t^2\boldsymbol{I})}{\sum_{i=1}^{K}\varphi(\boldsymbol{x}_t;\mathbf{0},\boldsymbol{U}_i\boldsymbol{U}_i^\top + t^2\boldsymbol{I})} \tag{3.2.73}$$

$$= \frac{\exp\left(-\frac{1}{2}\boldsymbol{x}_t^\top(\boldsymbol{U}_k\boldsymbol{U}_k^\top + t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t\right)}{\sum_{i=1}^{K}\exp\left(-\frac{1}{2}\boldsymbol{x}_t^\top(\boldsymbol{U}_i\boldsymbol{U}_i^\top + t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t\right)} \tag{3.2.74}$$

$$= \frac{\exp\left(-\frac{1}{2t^2}\boldsymbol{x}_t^\top\left(\boldsymbol{I} - \frac{1}{1+t^2}\boldsymbol{U}_k\boldsymbol{U}_k^\top\right)\boldsymbol{x}_t\right)}{\sum_{i=1}^{K}\exp\left(-\frac{1}{2t^2}\boldsymbol{x}_t^\top\left(\boldsymbol{I} - \frac{1}{1+t^2}\boldsymbol{U}_i\boldsymbol{U}_i^\top\right)\boldsymbol{x}_t\right)} \tag{3.2.75}$$

$$= \frac{\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2 + \frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2 + \frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)} \tag{3.2.76}$$

$$= \frac{\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2\right)\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\exp\left(-\frac{1}{2t^2}\|\boldsymbol{x}_t\|_2^2\right)\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)} \tag{3.2.77}$$

$$= \frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)}. \tag{3.2.78}$$

Se trata de una operación softmax ponderada por la proyección de $\boldsymbol{x}_t$ sobre cada subespacio, cuantificada por $\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2$ (con temperatura $2t^2(1+t^2)$). Por su parte, los *denoisers* por componente pueden expresarse en forma análoga a (3.2.58) para obtener:

$$\boldsymbol{U}_k\boldsymbol{U}_k^\top(\boldsymbol{U}_k\boldsymbol{U}_k^\top + t^2\boldsymbol{I})^{-1}\boldsymbol{x}_t = \frac{1}{1+t^2}\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{x}_t. \tag{3.2.79}$$

Combinando todo esto, obtenemos

$$\bar{\boldsymbol{x}}^\star(t,\boldsymbol{x}_t) = \frac{1}{1+t^2}\sum_{k=1}^{K}\frac{\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_k^\top\boldsymbol{x}_t\|_2^2\right)}{\sum_{i=1}^{K}\exp\left(\frac{1}{2t^2(1+t^2)}\|\boldsymbol{U}_i^\top\boldsymbol{x}_t\|_2^2\right)}\boldsymbol{U}_k\boldsymbol{U}_k^\top\boldsymbol{x}_t, \tag{3.2.80}$$

es decir, una proyección de $\boldsymbol{x}_t$ sobre cada uno de los K subespacios, ponderada por una función softmax de una función cuadrática de $\boldsymbol{x}_t$. ¡Esta forma funcional es similar a un *mecanismo de atención* en una arquitectura de transformer! Como veremos en el Capítulo 5, esto no es en absoluto una coincidencia; el profundo vínculo entre la eliminación de ruido y la compresión con pérdida (que se tratará en la Sección 4.1) hace que los *denoisers* basados en transformers sean tan eficaces en la práctica. Por lo tanto, nuestra teoría del modelo de mezcla gaussiana justifica el uso de redes neuronales tipo transformador para la eliminación de ruido.

En general, la arquitectura del *denoiser* aprendido corresponde a un esquema de codificación (paramétrico implícito) de los datos dados, ya que puede utilizarse para eliminar el ruido o proyectar sobre la distribución de los datos. Entrenar un *denoiser* equivale a encontrar un mejor esquema de codificación, lo que implementa parcialmente el enfoque A2 (es decir, buscar una distribución mediante la exploración de esquemas con mejores tasas de codificación) al final de la Sección 3.1.3. A continuación, discutimos cómo implementar el otro enfoque.

3.2.2 Muestreo de una distribución mediante eliminación iterativa de ruido

Recordemos que, al final de la Sección 3.1.3, analizamos dos enfoques para obtener una distribución con estructura de baja dimensión. El primer enfoque (A1) comienza con una distribución normal de alta entropía y reduce gradualmente su entropía hasta alcanzar la distribución de los datos. A este procedimiento lo llamamos *muestreo*, ya que generamos nuevas muestras. Ha llegado el momento de analizar cómo poner esto en práctica con el conjunto de herramientas que hemos creado.

Sabemos cómo eliminar el ruido de muestras muy ruidosas $\boldsymbol{x}_T$ para obtener aproximaciones $\hat{\boldsymbol{x}}$ cuyas distribuciones se asemejan a la de la variable aleatoria objetivo $\boldsymbol{x}$. Sin embargo, el procedimiento de muestreo nos obliga a partir de una distribución modelo que *no* se vea influida por la distribución de $\boldsymbol{x}$ y a utilizar el *denoiser* para guiar las iteraciones hacia la distribución de $\boldsymbol{x}$. ¿Cómo podemos hacerlo? Una forma de hacerlo se justifica a continuación:

$$\frac{\boldsymbol{x}_T}{T} = \frac{\boldsymbol{x} + T\boldsymbol{g}}{T} = \frac{\boldsymbol{x}}{T} + \boldsymbol{g} \xrightarrow{T \to +\infty} \boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}). \tag{3.2.81}$$

Por lo tanto, $\boldsymbol{x}_T \approx \mathcal{N}(\boldsymbol{0}, T^2\boldsymbol{I})$. Esta aproximación es bastante buena para casi todas las distribuciones prácticas, y se muestra en la Figura 3.7.

Así pues, al discretizar $[0, T]$ en $0 = t_0 < t_1 < \cdots < t_L = T$ de manera uniforme utilizando $t_\ell = T\ell/L$ (como en la sección anterior), una forma posible de obtener muestras a partir de ruido puro es:

- Muestrear $\hat{\boldsymbol{x}}_T \sim \mathcal{N}(\boldsymbol{0}, T^2\boldsymbol{I})$ (independientemente de todo lo demás).

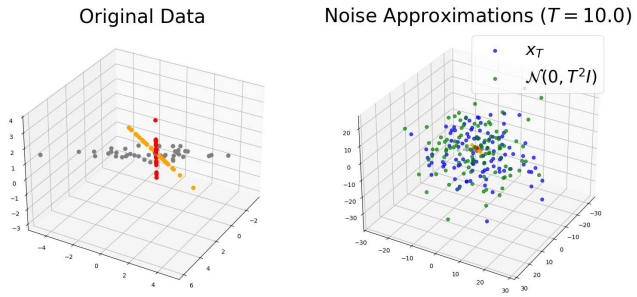


Figura 3.7: **Visualización de $\boldsymbol{x}_T$ vs $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$.** *Izquierda:* Gráfico de los datos del modelo de mezcla gaussiana $\boldsymbol{x}$. *Derecha:* Gráfico de $\boldsymbol{x}$, así como de $\boldsymbol{x}_T$ y una muestra independiente de $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$, para $T = 10$. En el gráfico de la derecha, $\boldsymbol{x}$ se representa con los mismos colores que en el de la izquierda; sin embargo, las muestras de $\boldsymbol{x}_T$ y $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$ son, en promedio, mucho más grandes que las muestras de $\boldsymbol{x}$, por lo que este último parece mucho más pequeño debido a la escala. A pesar de esta gran ampliación, observamos claramente las similitudes en las distribuciones de $\boldsymbol{x}_T$ y $\mathcal{N}(\mathbf{0}, T^2\boldsymbol{I})$.

- Ejecutar la iteración de eliminación de ruido tal y como se indica en la Sección 3.2.1, es decir,

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} = \left(1 - \frac{1}{\ell}\right)\hat{\boldsymbol{x}}_{t_\ell} + \frac{1}{\ell}\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}). \tag{3.2.82}$$

- Output $\hat{\boldsymbol{x}} = \hat{\boldsymbol{x}}_0$.

Conceptualmente, esto es todo lo que hay detrás de los *modelos de difusión*, que transforman el ruido en muestras de datos de acuerdo con la primera propiedad deseada. Sin embargo, aún quedan algunos pasos por dar antes de obtener modelos que puedan realmente tomar muestras de distribuciones de datos reales, como imágenes, bajo restricciones prácticas de recursos. En lo que sigue, presentamos y justificamos varios de esos pasos.

Paso 1: Diferentes discretizaciones. El primer paso se basa en la observación de que *no necesitamos realizar tantas iteraciones de eliminación de ruido* para valores grandes de t. La Figura 3.5 muestra que las primeras 200 o 300 iteraciones de las 500 iteraciones del proceso de muestreo se dedican a reducir el ruido hacia la distribución de los datos en su conjunto, antes de que las iteraciones restantes empujen las muestras hacia un subespacio. Dado un número fijo de iteraciones L, esto sugiere que deberíamos asignar más pasos temporales t_ℓ cerca de $t = 0$ que cerca de $t = T$. Durante el muestreo (y el entrenamiento), podemos, por lo tanto, utilizar una discretización alternativa de $[0, T]$ en $0 \leq t_0 < t_1 < \cdots < t_L \leq T$, como por ejemplo una *discretización exponencial*:

$$t_\ell = C_1(e^{C_2\ell} - 1), \qquad \forall \ell \in \{0, 1, \ldots, L\} \tag{3.2.83}$$

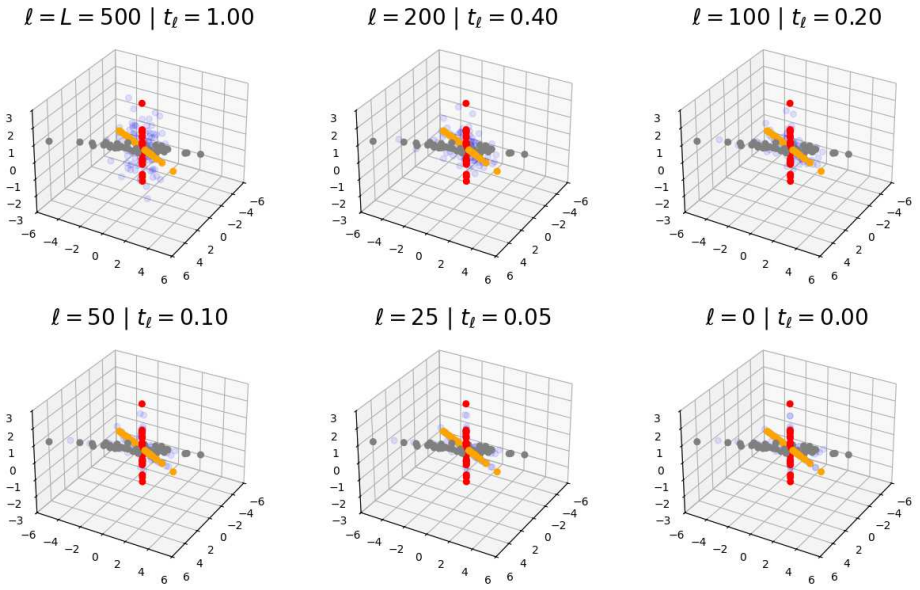


Figura 3.8: **Eliminación del ruido en una mezcla de distribuciones gaussianas mediante el proceso de difusión VP.** Utilizamos la misma configuración de la figura y la misma distribución de datos que en la Figura 3.5. Nótese que, en comparación con la Figura 3.5, la distribución del ruido está mucho más concentrada alrededor del origen.

donde $C_1, C_2 > 0$ son constantes que pueden ajustarse para obtener un rendimiento óptimo en la práctica; los análisis teóricos suelen especificar también dichas constantes óptimas. La iteración de eliminación de ruido/muestreo se convierte entonces en

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} \doteq \frac{t_{\ell-1}}{t_\ell}\hat{\boldsymbol{x}}_{t_\ell} + \left(1 - \frac{t_{\ell-1}}{t_\ell}\right)\bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}), \tag{3.2.84}$$

con $\hat{\boldsymbol{x}}_{t_L} \sim \mathcal{N}(\mathbf{0}, t_L^2 \boldsymbol{I})$.

Paso 2: Modelos de ruido diferentes. El segundo paso consiste en considerar modelos ligeramente distintos a los de (3.2.1). La motivación fundamental para esto es la siguiente. En la práctica, la distribución del ruido $\mathcal{N}(\mathbf{0}, t_L^2 \boldsymbol{I})$ se convierte en una estimación cada vez menos precisa de la covarianza real en dimensiones elevadas; es decir, (3.2.81) se convierte en una aproximación cada vez menos precisa, especialmente con datos anisotrópicos de alta dimensión. El aumento de la distancia entre $\mathcal{N}(\mathbf{0}, t_L^2 \boldsymbol{I})$ y la distribución real de $\boldsymbol{x}_{t_L}$ puede hacer que el *denoiser* funcione peor en tales circunstancias. En teoría, $\boldsymbol{x}_{t_L}$ nunca converge a ninguna distribución a medida que t_L aumenta, por lo que esta configuración es difícil de analizar de extremo a extremo. En

este caso, nuestra solución consiste en *añadir ruido y reducir la contribución de* $\boldsymbol{x}$ *al mismo tiempo, de modo que* $\boldsymbol{x}_T$ *converja cuando* $T \to \infty$. La tasa de ruido añadido se denota por $\sigma\colon [0,T] \to \mathbb{R}_{\geq 0}$ y la tasa de contracción se denota por $\alpha\colon [0,T] \to \mathbb{R}_{\geq 0}$, de modo que σ es *creciente* y α es (no estrictamente) *decreciente*, y

$$\boldsymbol{x}_t \doteq \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{g}, \qquad \forall t \in [0,T]. \tag{3.2.85}$$

En la configuración anterior, $\alpha_t = 1$ y $\sigma_t = t$ y esto se conoce como el *proceso de varianza explosiva (VE)*. Una opción habitual que reduce la contribución de $\boldsymbol{x}$, como lo describimos inicialmente, es aquella en la que $T = 1$ (de modo que $t \in [0,1]$), $\alpha_t = \sqrt{1-t^2}$ y $\sigma_t = t$; este es el *proceso de preservación de la varianza (VP)*. Nótese que, en el proceso VP, $\boldsymbol{x}_1 \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ exactamente, por lo que basta con tomar muestras de esta distribución estándar y aplicar el proceso de eliminación de ruido en forma iterativa. Como resultado, el proceso VP es mucho más fácil de analizar teóricamente y más estable empíricamente.[6]

Con este planteamiento más general, la fórmula de Tweedie (3.2.23) se convierte en

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] = \frac{1}{\alpha_t}\left(\boldsymbol{x}_t + \sigma_t^2 \nabla \log p_t(\boldsymbol{x}_t)\right). \tag{3.2.86}$$

La iteración de eliminación de ruido (3.2.84) se transforma en

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} = \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \hat{\boldsymbol{x}}_{t_\ell} + \left(\alpha_{t_{\ell-1}} - \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \alpha_{t_\ell}\right) \bar{\boldsymbol{x}}^\star(t_\ell, \hat{\boldsymbol{x}}_{t_\ell}). \tag{3.2.87}$$

Ejemplo 3.6 (El modelo de mezcla gaussiana). Para un modelo de mezcla gaussiana (véase el Ejemplo 3.2), su filtro de eliminación de ruido (3.2.19) se convierte en

$$\bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) = \sum_{k=1}^{K} \left\{ \frac{\pi_k \varphi(\boldsymbol{x}_t; \alpha_t \boldsymbol{\mu}_k, \alpha_t^2 \boldsymbol{\Sigma}_k + \sigma_t^2 \boldsymbol{I})}{\sum_{i=1}^{K} \pi_i \varphi(\boldsymbol{x}_t; \alpha_t \boldsymbol{\mu}_i, \alpha_t^2 \boldsymbol{\Sigma}_i + \sigma_t^2 \boldsymbol{I})} \right. \tag{3.2.88}$$
$$\left. \cdot \left(\boldsymbol{\mu}_k + \alpha_t \boldsymbol{\Sigma}_k (\alpha_t^2 \boldsymbol{\Sigma}_k + \sigma_t^2 \boldsymbol{I})^{-1} (\boldsymbol{x}_t - \alpha_t \boldsymbol{\mu}_k)\right) \right\}.$$

La Figura 3.8 muestra iteraciones del procedimiento de muestreo. ■

Cabe señalar que la iteración de eliminación de ruido (3.2.87) da lugar a un algoritmo de muestreo denominado DDIM («Denoising Diffusion Implicit Model») [**song2020denoising**], y es uno de los algoritmos de muestreo más populares utilizados actualmente en los modelos de difusión. Lo resumimos en el Algoritmo 3.1.

[6] ¿Por qué utilizar toda la configuración de α, σ? Como veremos en el Ejercicio 3.5, esta encapsula y unifica muchos de los procesos propuestos, incluido el recientemente popular proceso denominado *flow matching*. A pesar de ello, los procesos VE y VP siguen siendo los más populares tanto desde el punto de vista empírico como teórico (hasta ahora), por lo que los abordaremos en esta sección.

Algoritmo 3.1: Muestreo mediante *denoiser*.

Input: Una lista ordenada de pasos de tiempo $0 \leq t_0 < \cdots < t_L \leq T$ que se utilizarán para el muestreo.
Input: Un filtro de eliminación de ruido $\bar{\boldsymbol{x}}\colon \{t_\ell\}_{\ell=1}^L \times \mathbb{R}^D \to \mathbb{R}^D$.
Input: Funciones de escala y de nivel de ruido $\alpha, \sigma\colon \{t_\ell\}_{\ell=0}^L \to \mathbb{R}_{\geq 0}$.
Output: Una muestra $\hat{\boldsymbol{x}}$, aproximadamente de la distribución de $\boldsymbol{x}$.

function DDIMSampler($\bar{\boldsymbol{x}}, (t_\ell)_{\ell=0}^L$)
  Inicializar $\hat{\boldsymbol{x}}_{t_L} \sim$ la distribución aproximada de $\boldsymbol{x}_{t_L}$
  `# VP` $\implies \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, `VE` $\implies \mathcal{N}(\mathbf{0}, t_L^2 \boldsymbol{I})$.
  for $\ell = L, L-1, \ldots, 1$ **do**
    Calcular

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} \doteq \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \hat{\boldsymbol{x}}_{t_\ell} + \left(\alpha_{t_{\ell-1}} - \frac{\sigma_{t_{\ell-1}}}{\sigma_{t_\ell}} \alpha_{t_\ell} \right) \bar{\boldsymbol{x}}(t_\ell, \hat{\boldsymbol{x}}_{t_\ell})$$

  end for
  return $\hat{\boldsymbol{x}}_{t_0}$
end function

Paso 3: Optimización de los procesos de entrenamiento. Si usáramos el procedimiento prescrito en la Sección 3.2.1 para entrenar un *denoiser* independiente $\bar{\boldsymbol{x}}(t, \cdot)$ para cada instante t que se utilizará en el algoritmo de muestreo, *¡tendríamos que entrenar L denoisers independientes!* Esto es muy ineficiente: lo habitual es que tengamos que entrenar L redes neuronales distintas, lo que requiere L veces más tiempo de entrenamiento y memoria de almacenamiento y luego quedarnos atados a utilizar esos intervalos de tiempo para el muestreo de por siempre. En cambio, podemos *entrenar una sola red neuronal* para eliminar el ruido en todos los instantes t, tomando como entrada las variables continuas $\boldsymbol{x}_t$ y t (en lugar de solo $\boldsymbol{x}_t$ como antes). Mecánicamente, nuestra pérdida de entrenamiento se promedia a lo largo de t, es decir, resuelve el siguiente problema:

$$\min_\theta \mathbb{E} \, \|\bar{\boldsymbol{x}}_\theta(\tau, \boldsymbol{x}_\tau) - \boldsymbol{x}\|_2^2, \tag{3.2.89}$$

donde $\tau \sim \mathcal{U}([0, T])$, es decir, se extrae uniformemente del intervalo $[0, T]$. Al igual que en el Paso 1, donde utilizamos más intervalos de tiempo más cercanos a $t = 0$ para garantizar un mejor proceso de muestreo, es posible que queramos asegurarnos de que el *denoiser* sea de mayor calidad cuanto más cerca de $t = 0$ y por lo tanto *ponderar la pérdida* de modo que t cerca de 0 tenga mayor ponderación.[7] Si w_t es el peso en el momento t, la pérdida ponderada tendría

[7] En la práctica, la distribución de la t muestreada también puede ser no uniforme, con el fin de garantizar (junto con el esquema de ponderación) que la estimación de Monte Carlo de la pérdida tenga una varianza baja, lo que mejora considerablemente la estabilidad del entrenamiento.

la forma

$$\min_{\theta} \mathbb{E}[w_\tau \|\bar{\boldsymbol{x}}_\theta(\tau, \boldsymbol{x}_\tau) - \boldsymbol{x}\|_2^2]. \tag{3.2.90}$$

Una opción razonable para el peso en la práctica es $w_t = \alpha_t^2/\sigma_t^2$. La razón exacta se explicará en el siguiente párrafo, pero, en general, sirve para dar mayor peso a las pérdidas correspondientes a t cerca de 0, sin dejar de mantener una estabilidad numérica razonable. Además, por supuesto, en la práctica no podemos calcular la esperanza, por lo que utilizamos la media de Monte Carlo, que es la más sencilla, para estimarla. La serie de cambios que se han realizado aquí ofrece varias ventajas conceptuales y computacionales: no necesitamos entrenar múltiples *denoisers*, podemos entrenar con un solo conjunto de pasos de tiempo y realizar el muestreo utilizando un subconjunto (o incluso otros distintos), etc. El proceso completo se analiza en el Algoritmo 3.2. Sin duda, este proceso funciona en la práctica; comentamos algunos resultados en las **??????????**.

Paso 4: Cambio del objetivo de estimación. Es habitual, en su lugar, reorientar todo el proceso de eliminación de ruido en torno a los *predictores de ruido*, es decir, las estimaciones de $\mathbb{E}[\boldsymbol{g} \mid \boldsymbol{x}_t]$. En la práctica, los predictores de ruido son ligeramente más fáciles de entrenar porque su salida es (casi) siempre de un tamaño comparable al de una variable aleatoria gaussiana, por lo que el entrenamiento es más estable numéricamente, pero pueden requerir modelos más grandes, ya que predicen un objetivo de dimensión completa en lugar de datos potencialmente de baja dimensión [**li2025back**]. Por (3.2.85) tenemos

$$\boldsymbol{x}_t = \alpha_t \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t] + \sigma_t \mathbb{E}[\boldsymbol{g} \mid \boldsymbol{x}_t] \implies \mathbb{E}[\boldsymbol{g} \mid \boldsymbol{x}_t] = \frac{1}{\sigma_t}\left(\boldsymbol{x}_t - \alpha_t \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t]\right), \tag{3.2.91}$$

por lo que cualquier predictor de $\boldsymbol{x}$ puede transformarse en un predictor de $\boldsymbol{g}$ utilizando la relación anterior, es decir,

$$\bar{\boldsymbol{g}}(t, \boldsymbol{x}_t) = \frac{1}{\sigma_t}\boldsymbol{x}_t - \frac{\alpha_t}{\sigma_t}\bar{\boldsymbol{x}}(t, \boldsymbol{x}_t), \tag{3.2.92}$$

y viceversa. Por lo tanto, una buena red para estimar $\bar{\boldsymbol{g}}$ es lo mismo que una buena red para estimar $\bar{\boldsymbol{x}}$ *más una conexión residual* (como se ve, por ejemplo, en los transformers). Sus pérdidas son también las mismas que las del *denoiser*, salvo por el factor α_t/σ_t, es decir,

$$\mathbb{E}[w_\tau \|\boldsymbol{g} - \bar{\boldsymbol{g}}(\tau, \boldsymbol{x}_\tau)\|_2^2] = \mathbb{E}\left[w_\tau \frac{\alpha_\tau^2}{\sigma_\tau^2}\|\boldsymbol{x} - \bar{\boldsymbol{x}}(\tau, \boldsymbol{x}_\tau)\|_2^2\right], \tag{3.2.93}$$

recordando que $\tau \sim \mathcal{U}([0, T])$. Se han propuesto otros objetivos para diferentes tareas, por ejemplo, $\mathbb{E}[\boldsymbol{v}_t \mid \boldsymbol{x}_t]$ donde $\boldsymbol{v}_t = \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_t$ (denominada predicción v o predicción de velocidad), etc. Esto es equivalente a la eliminación de ruido y la predicción de ruido, pero también resulta fundamentalmente útil para generalizaciones de la difusión, como el flow matching, que analizaremos en la Nota 3.3.

Algoritmo 3.2: Aprender un *denoiser* a partir de datos.

Input: Conjunto de datos $\mathcal{D} \subseteq \mathbb{R}^D$.
Input: Una lista ordenada de pasos de tiempo $0 \leq t_0 < \cdots < t_L \leq T$ que se utilizarán para el muestreo.
Input: Una función de ponderación $w\colon \{t_\ell\}_{\ell=1}^L \to \mathbb{R}_{\geq 0}$.
Input: Funciones de escala y nivel de ruido $\alpha, \sigma\colon \{t_\ell\}_{\ell=0}^L \to \mathbb{R}_{\geq 0}$.
Input: Un espacio de parámetros Θ y una arquitectura de *denoiser* $\bar{\boldsymbol{x}}_\theta$.
Input: Un algoritmo de optimización para los parámetros.
Input: El número de iteraciones de optimización M.
Input: El número de extracciones de Monte Carlo N por paso de entrenamiento.
Output: Un *denoiser* entrenado $\bar{\boldsymbol{x}}_{\theta^\star}$.

function TrainDenoiser($\mathcal{D}, \Theta$)
  Inicializar $\theta^{(1)} \in \Theta$
  for $m \in [M]$ **do**
    for $i \in [N]$ **do**
      `# Extraer una muestra del conjunto de datos.`
      $\boldsymbol{x}^{(m,i)} \sim \mathcal{D}$
      `# Muestrear un intervalo de tiempo.`
      $t^{(m,i)} \overset{\text{i.i.d.}}{\sim} \mathcal{U}(\{t_\ell\}_{\ell=1}^L)$
      `# Calcular ponderaciones y escalas.`
      $(\alpha^{(m,i)}, \sigma^{(m,i)}, w^{(m,i)}) = (\alpha_{t^{(m,i)}}, \sigma_{t^{(m,i)}}, w_{t^{(m,i)}})$
      `# Muestrear un vector de ruido.`
      $\boldsymbol{g}^{(m,i)} \overset{\text{i.i.d.}}{\sim} \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$
      `# Calcular la muestra ruidosa.`
      $\boldsymbol{x}_t^{(m,i)} \doteq \alpha^{(m,i)} \boldsymbol{x}^{(m,i)} + \sigma^{(m,i)} \boldsymbol{g}^{(m,i)}$
    end for
    `# Calcular la pérdida de entrenamiento.`
    $\hat{\mathcal{L}}^{(m)} \doteq \dfrac{1}{N} \displaystyle\sum_{i=1}^{N} w^{(m,i)} \|\boldsymbol{x}^{(m,i)} - \bar{\boldsymbol{x}}_{\theta^{(m)}}(t^{(m,i)}, \boldsymbol{x}_t^{(m,i)})\|_2^2$
    `# Actualizar los parámetros del modelo.`
    $\theta^{(m+1)} \doteq \texttt{OptimUpdate}^{(m)}(\theta^{(m)}, \nabla_{\theta^{(m)}} \hat{\mathcal{L}}^{(m)})$
  end for
  `# Devolver el denoiser entrenado.`
  return $\bar{\boldsymbol{x}}_{\theta^{(M+1)}}$
end function

Tasa de error de muestreo del proceso de eliminación de ruido

Hemos introducido muchos cambios en nuestro proceso platónico original de generación y eliminación de ruido. Para asegurarnos de que el nuevo proceso sigue funcionando en la práctica, podemos calcular ejemplos numéricos (como en la Figura 3.8). Para asegurarnos de que es teóricamente sólido, podemos demostrar una *cota de la tasa de error* para el algoritmo de muestreo, lo que demuestra que la tasa de error es pequeña. A continuación, presentamos una cota de este tipo tomada de la literatura, que muestra que la distribución de los resultados del muestreador converge, en la denominada *distancia de variación total (TV)*, hacia la distribución verdadera. La distancia TV entre dos variables aleatorias $\boldsymbol{x}$ y $\boldsymbol{y}$ se define como:[8]

$$\mathsf{TV}(\boldsymbol{x}, \boldsymbol{y}) \doteq \sup_{A \subseteq \mathbb{R}^d} \left| \mathbb{P}[\boldsymbol{x} \in A] - \mathbb{P}[\boldsymbol{y} \in A] \right|. \tag{3.2.94}$$

Si $\boldsymbol{x}$ y $\boldsymbol{y}$ están muy cerca (uniformemente), entonces el supremo será pequeño. Así pues, la distancia TV mide la proximidad entre variables aleatorias. (Se trata, efectivamente, de una métrica, como sugiere su nombre; la demostración es un ejercicio.)

Teorema 3.4 ([**li2024d**] Teorema 1, simplificado)**.** *Supongamos que* $\mathbb{E}\,\|\boldsymbol{x}\|_2 < \infty$*. Si se elimina el ruido de* $\boldsymbol{x}$ *según el proceso VP con una discretización exponencial*[9] *como en* (3.2.83)*, la salida* $\hat{\boldsymbol{x}}$ *del Algoritmo 3.1 satisface la cota de variación total*

$$\mathsf{TV}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = \tilde{\mathcal{O}}\Bigg(\underbrace{\frac{D}{L}}_{\text{error de discretización}} + \underbrace{\sqrt{\frac{1}{L}\sum_{\ell=1}^{L} \frac{\alpha_{t_\ell}^2}{\sigma_{t_\ell}^4}\, \mathbb{E}\,\|\bar{\boldsymbol{x}}^\star(t_\ell, \boldsymbol{x}_{t_\ell}) - \bar{\boldsymbol{x}}(t_\ell, \boldsymbol{x}_{t_\ell})\|_2^2}}_{\text{error medio excesivo del denoiser}} \Bigg) \tag{3.2.95}$$

donde $\bar{\boldsymbol{x}}^\star$ *es el denoiser óptimo de Bayes para* $\boldsymbol{x}$ *y* $\tilde{\mathcal{O}}$ *es una versión de la notación «big-*$\mathcal{O}$*» que ignora los factores logarítmicos en* L*.*

La técnica de demostración de muy alto nivel consiste, como se ha comentado anteriormente, en limitar el error en cada paso, distinguir las fuentes de error (entre el error de discretización y el del *denoiser*) y asegurarse cuidadosamente de que los errores no se acumulen en exceso (o incluso se anulen entre sí).

Téngase en cuenta que si $L \to \infty$ y aprendemos correctamente el *denoiser* óptimo de Bayes $\bar{\boldsymbol{x}} = \bar{\boldsymbol{x}}^\star$ (de modo que el error excedente es 0), entonces

[8] El supremo aquí se toma únicamente sobre conjuntos A *medibles*, no a todos los conjuntos sin excepción. Al igual que con todas las demás reservas relacionadas con la teoría de la medida que aparecen en este libro, no dudes en ignorarlo a efectos prácticos.

[9] La definición exacta es bastante extensa en nuestra notación y solo se define hasta diversas constantes absolutas, por lo que la omitimos aquí por brevedad. Se encuentra en el artículo original [**li2024d**].

el proceso de muestreo en el Algoritmo 3.1 produce una *inversa perfecta (en distribución)* del proceso de adición de ruido, ya que la tasa de error en el Teorema 3.4 tiende a 0,[10] como se ha argumentado heurísticamente más arriba.

Nota 3.1. ¿Qué pasa si los datos son de baja dimensión, es decir, si se encuentran en un *subespacio de rango bajo* del espacio de alta dimensión $\mathbb{R}^D$? Si la distribución de los datos tiene soporte compacto —por ejemplo, si los datos se normalizan al hipercubo unitario, lo cual suele garantizarse como paso previo al procesamiento de datos reales, como las imágenes— es posible obtener mejores resultados. Concretamente, los autores de [**li2024d**] también definen una medida de la *dimensión intrínseca aproximada* κ utilizando la asintótica del denominado número de cobertura, que es extremadamente similar en su intuición (si no en su implementación) a la función de tasa-distorsión que se presenta en el siguiente capítulo. A continuación, muestran que, al aplicar una pequeña modificación concreta al muestreador DDIM del Algoritmo 3.1 (es decir, perturbando ligeramente los coeficientes de actualización), el error de discretización pasa a ser $\tilde{\mathcal{O}}(\kappa/L)$ en lugar de $\tilde{\mathcal{O}}(D/L)$ como en el Teorema 3.4. Por lo tanto, al utilizar este algoritmo modificado, L no tiene por qué ser demasiado grande, incluso cuando D alcanza valores de miles o millones, ya que los datos reales tienen una estructura de baja dimensión. Sin embargo, en la práctica utilizamos el muestreador DDIM, por lo que L debería tener una dependencia moderada de D para lograr tasas de error consistentes. La elección concreta de L supone un equilibrio entre la complejidad computacional (por ejemplo, el tiempo de ejecución o el consumo de memoria) del muestreo y la complejidad estadística de aprender un *denoiser* para estructuras de baja dimensión. El valor de L suele ser diferente en la fase de entrenamiento (donde un L mayor permite una mejor cobertura del intervalo $[0, T]$, lo que ayuda a la red a aprender una relación que se generaliza sobre t) y en la fase de muestreo (donde un L menor implica un muestreo más eficiente). Incluso se pueden seleccionar los intervalos de tiempo en forma adaptativa en el momento del muestreo para optimizar este equilibrio [**bao2022analytic**].

Nota 3.2. En otros trabajos, el proceso inverso se define como un desplazamiento hacia atrás en el índice temporal t mediante una ecuación de diferencias explícita, o una ecuación diferencial en el límite $L \to \infty$, o hacia adelante en el tiempo utilizando la transformación $\boldsymbol{y}_t = \boldsymbol{x}_{T-t}$, de tal manera que si t aumenta, entonces $\boldsymbol{y}_t$ se acerca a $\boldsymbol{x}_0$. Cada artículo utiliza su propia notación y a veces puede resultar confuso comprender las relaciones entre los resultados de los distintos artículos. En este trabajo nos esforzamos por mantener la coherencia: avanzamos en el tiempo para introducir ruido y retrocedemos en el tiempo para eliminarlo. Al leer otro trabajo en el que no queda claro el índice temporal, o al intentar implementar un algoritmo que presenta una ambigüedad similar, hay una forma de resolverla en todos los casos: el proceso de muestreo siempre debe tener un *coeficiente positivo* en el término del *denoiser* (o, equivalentemente, en la función de *score*) al dar un paso.

[10] Se obtienen resultados similares para los procesos VE; las herramientas teóricas utilizadas para las demostraciones difieren debido a la estructura de los procesos estocásticos subyacentes.

Nota 3.3. Los procesos de difusión y eliminación de ruido, junto con los modelos asociados, ofrecen una forma de transportar la distribución de los datos hacia y desde una distribución de alta entropía (gaussiana o aproximadamente gaussiana). Por otro lado, puede resultar útil transportar la distribución de los datos hacia y desde una distribución objetivo arbitraria, que podría ser gaussiana, una mezcla gaussiana o incluso otro tipo de distribución de datos. Si tenemos este requisito, podemos utilizar el marco de *flow matching* para lograrlo. Supongamos que la variable aleatoria objetivo es $\boldsymbol{y}$ y que nuestro flujo es

$$\boldsymbol{x}_t = \alpha_t \boldsymbol{x} + \sigma_t \boldsymbol{y}, \qquad \forall t \in [0,1]. \tag{3.2.96}$$

de tal manera que $\alpha_0 = \sigma_1 = 1$ y $\alpha_1 = \sigma_0 = 0$, de modo que $\boldsymbol{x}_0 = \boldsymbol{x}$ y $\boldsymbol{x}_1 = \boldsymbol{y}$. Nuestra estrategia no cambia en absoluto: podemos entrenar un *denoiser* $\bar{\boldsymbol{x}}_\theta$ para eliminar el ruido de $\boldsymbol{x}_t$ y volver a obtener $\boldsymbol{x}$ (o, lo que es lo mismo, un predictor de «ruido» para predecir $\boldsymbol{y}$) y luego aplicar nuestros algoritmos iterativos de eliminación de ruido para partir de una realización de $\boldsymbol{y}$ y obtener $\boldsymbol{x}$. Lo único que falta es la fórmula de Tweedie que relaciona el *score* $\nabla \log p_t$ del *denoiser* con el *denoiser* propiamente tal; dicha fórmula no existe en general.

En la práctica, inspirándose en el concepto de aprender un llamado *mapa de transporte* entre pares de datos $\boldsymbol{x}$ y $\boldsymbol{y}$, los profesionales suelen preferir entrenar un *predictor de velocidad* $\bar{\boldsymbol{v}}_\theta$ para predecir $\boldsymbol{v}_t = \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_t = \alpha_t' \boldsymbol{x} + \sigma_t' \boldsymbol{y}$ en lugar de un *denoiser* $\bar{\boldsymbol{x}}_\theta$, o más bien entrenar $\bar{\boldsymbol{v}}_\theta$ para predecir $\mathbb{E}[\boldsymbol{v}_t \mid \boldsymbol{x}_t]$. En este caso, lo más sencillo es elegir los coeficientes $\alpha_t = 1 - t$ y $\sigma_t = t$ (para $t \in [0,1]$), de modo que el objetivo de predicción sea $\boldsymbol{v}_t = \boldsymbol{y} - \boldsymbol{x}$ para cada instante t. Dado el estimador $\bar{\boldsymbol{v}}_\theta$, el algoritmo de muestreo estándar recorre el campo hacia atrás en el tiempo; es decir, partiendo de una realización de $\boldsymbol{y}$, podemos actualizar iterativamente el iterado de la siguiente manera:

$$\hat{\boldsymbol{x}}_{t_{\ell-1}} = \hat{\boldsymbol{x}}_{t_\ell} - \bar{\boldsymbol{v}}_\theta(t_\ell, \hat{\boldsymbol{x}}_{t_\ell})\Delta t, \tag{3.2.97}$$

donde $\Delta t = t_\ell - t_{\ell-1}$ es el intervalo de tiempo. Téngase en cuenta que la predicción de velocidad es algebraicamente equivalente a la eliminación de ruido (véase el Ejercicio 3.6). Las **????** contienen aplicaciones prácticas del flow matching.

Nota 3.4. En la práctica, la difusión suele requerir muchos pasos y muchas evaluaciones de la función del *denoiser*. Por lo tanto, es lógico preguntarse: «¿Podemos acelerar el proceso de muestreo utilizando un número menor de pasos y evaluaciones de la función?». La respuesta es afirmativa y a esos modelos se les suele llamar *modelos de consistencia* o, de manera más descriptiva, *modelos de pocos pasos*. La idea central consiste en utilizar un *denoiser* que tome dos veces $\bar{\boldsymbol{x}}_\theta(s,t,\boldsymbol{x}_t)$ y estime $\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]$, lo cual, en cierto sentido, es una tarea más sencilla, o bien utilizar un predictor de velocidad $\bar{\boldsymbol{v}}_\theta(s,t,\boldsymbol{x}_t)$ y estimar la velocidad media $\mathbb{E}[(\boldsymbol{x}_t - \boldsymbol{x}_s)/(t-s) \mid \boldsymbol{x}_t]$. Estos filtros de eliminación de ruido tienen condiciones de consistencia adicionales, tales como $\bar{\boldsymbol{x}}_\theta(u,t,\bar{\boldsymbol{x}}_\theta(s,u,\boldsymbol{\xi})) = \bar{\boldsymbol{x}}_\theta(s,t,\boldsymbol{\xi})$, lo que motiva el uso de pérdidas de entrenamiento auxiliares. Una vez que se aprende dicho predictor, el proceso de muestreo se vuelve muy simple

y eficiente, ya que solo requiere unos pocos pasos de la forma

$$\hat{\boldsymbol{x}}_s = \bar{\boldsymbol{x}}_\theta(s, t, \hat{\boldsymbol{x}}_t) = \hat{\boldsymbol{x}}_t - \bar{\boldsymbol{v}}_\theta(s, t, \hat{\boldsymbol{x}}_t) \cdot (t - s). \tag{3.2.98}$$

Entre las formulaciones relevantes se incluyen los mapas de flujo [**boffi2025build**] y los flujos medios [**geng2025mean**].

3.3 Memorización, generalización y tasas de codificación

El cálculo que se presenta al final de la Sección 3.2.1 parece indicar (en términos generales) que, en la práctica, utilizar una red de tipo transformer es una buena opción para entrenar o aproximar un *denoiser*. Esto es razonable, pero ¿qué problema hay en utilizar cualquier red neuronal corriente (como un perceptrón multicapa (MLP)) y simplemente escalarla hasta el infinito? Para responder a esta pregunta, consideremos una restricción que hasta ahora hemos ignorado o pasado por alto en este capítulo: solo disponemos de datos finitos con los que entrenar el *denoiser*. De hecho, consideremos el problema de entrenamiento (3.2.90), salvo que esta vez suponemos que nuestros datos $\boldsymbol{x}$ se extraen de la distribución empírica $p^{\boldsymbol{X}}$ sobre N muestras de entrenamiento $\boldsymbol{X} = [\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N)}]$. Dado que $p^{\boldsymbol{X}}$ está soportado en un subconjunto de dimensión 0 del espacio envolvente $\mathbb{R}^D$, tenemos que $h(p^{\boldsymbol{X}}) = -\infty$, por lo que, en cierto sentido, $p^{\boldsymbol{X}}$ es la distribución más simple (es decir, la de menor entropía) que podría generar nuestras observaciones $\boldsymbol{X}$. En resumen, ¡podría decirse que esta hipótesis *tiene el menor sesgo inductivo* respecto a la distribución de los datos! Muchas narrativas populares sobre el aprendizaje profundo y los modelos generativos quieren hacernos creer que, debido a esta propiedad, la mejor manera de resolver el problema del modelado generativo es escalar un *denoiser* genérico al máximo y aprovechar las ventajas correspondientes. Sin embargo, este consejo es (¡demostrablemente!) erróneo en este contexto. Para entender por qué, notemos que la distribución empírica $p^{\boldsymbol{X}}$ es, en realidad, una mezcla de distribuciones gaussianas con covarianzas nulas:

$$p^{[\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(N)}]} = \frac{1}{N} \sum_{i=1}^{N} \mathcal{N}(\boldsymbol{x}^{(i)}, \boldsymbol{0}). \tag{3.3.1}$$

En consecuencia, sabemos que la solución óptima (a nivel global) al problema de eliminación de ruido (3.2.90) con $\boldsymbol{x}$ extraído de la distribución empírica se expresa de la siguiente manera:

$$\min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^{N} \mathbb{E}[w_\tau \|\boldsymbol{x}^{(i)} - \bar{\boldsymbol{x}}_\theta(\tau, \boldsymbol{x}_\tau^{(i)})\|_2^2] \tag{3.3.2}$$

donde $\boldsymbol{x}_t^{(i)}$ es la versión con ruido de la muestra de entrenamiento $\boldsymbol{x}^{(i)}$. El optimizador de esta pérdida sobre todos los eliminadores de ruido

(cuadráticamente integrables) $\bar{\boldsymbol{x}}$ es el *denoiser* gaussiano óptimo asociado del Ejemplo 3.3, es decir,

$$\bar{\boldsymbol{x}}^{\star}(t, \boldsymbol{x}_t) = \sum_{i=1}^{N} \frac{\exp(-\|\boldsymbol{x}_t - \alpha_t \boldsymbol{x}^{(i)}\|_2^2/(2\sigma_t^2))}{\sum_{j=1}^{N} \exp(-\|\boldsymbol{x}_t - \alpha_t \boldsymbol{x}^{(j)}\|_2^2/(2\sigma_t^2))} \boldsymbol{x}^{(i)}. \tag{3.3.3}$$

Esta es una combinación convexa de los datos $\boldsymbol{x}^{(i)}$ y los coeficientes se vuelven más «agudos» (es decir, más cercanos a 0 o 1) a medida que $t \to 0$. En particular, para valores pequeños de t, el *denoiser* es casi exactamente igual a la muestra más cercana del conjunto de entrenamiento y la actualización acerca la iteración al punto más cercano, de modo que al final del muestreo, la iteración $\hat{\boldsymbol{x}}$ es (casi) igual a una muestra de entrenamiento $\boldsymbol{x}^{(i)}$. Reafirmemos una vez más este punto: *si utilizamos el denoiser* (3.3.3) *y un buen muestreador, nuestras muestras regenerarán los datos de entrenamiento.* Esto queda confirmado por nuestra garantía teórica sobre el muestreo Teorema 3.4, que, cuando se aplica en este contexto, establece que si llevamos el número de pasos de muestreo a ∞ y utilizamos el programa de pasos de tiempo prescrito, el modelo de difusión asociado al *denoiser* (3.3.3) siempre tomará muestras de la distribución empírica y, por lo tanto, siempre reproducirá una muestra de entrenamiento. Este fenómeno constituye una instanciación especialmente estricta de la *memorización* en el modelado generativo. [11]

Dado que, en la práctica, solo disponemos de un conjunto de datos fijo y finito con el que podemos entrenar nuestro *denoiser*, el problema de aprendizaje que pretendemos resolver en este caso es la pérdida de eliminación de ruido con muestra finita (3.3.2) (en comparación con la pérdida de eliminación de ruido con muestra infinita (3.2.90), por ejemplo). Por lo tanto, *si tenemos un modelo arbitrariamente grande y lo optimizamos lo mejor posible, aprenderá a memorizar y a regenerar el conjunto de entrenamiento.* En otras palabras, este modelo a gran escala no es mucho más interesante que una simple base de datos con los datos de entrenamiento.

El argumento anterior es válido para modelos de arbitrariamente potentes que puedan ajustar el (verdadero) *denoiser* memorizador. Por lo tanto, es pertinente preguntarse si esto se confirma realmente en la práctica: ¿los modelos muy grandes (en relación con el conjunto de datos) aprenden a memorizar los datos de entrenamiento? Resulta que esto es cierto desde el punto de vista empírico. Uno de los primeros estudios al respecto fue [**yoon2023diffusion**]. Para comprender sus resultados, definimos la razón de memorización como la probabilidad de memorización:

$$\text{TasaMem}(\bar{\boldsymbol{x}}) = \mathbb{P}_{\hat{\boldsymbol{x}} \sim \text{DM}(\bar{\boldsymbol{x}})}[\hat{\boldsymbol{x}} \text{ ha sido memorizado}], \tag{3.3.4}$$

donde $\text{DM}(\bar{\boldsymbol{x}})$ es la distribución inducida por el modelo de difusión que utiliza

[11] La memorización es un fenómeno mucho más amplio cuya caracterización exacta depende del dominio en el que se entrena el modelo (por ejemplo, texto, imágenes, vídeos, audio, etc.). Más adelante en esta sección analizaremos medidas más heurísticas de la memorización.



Figura 3.9: **Muestras generadas (izquierda) a partir de un modelo de difusión entrenado con** 6.000 **muestras y (derecha) sus puntos más cercanos en el conjunto de datos de entrenamiento (en la norma euclidiana).** Observamos que las muestras generadas son, en esencia, equivalentes a nivel de pixel a los datos de entrenamiento más similares. Ambos conjuntos de imágenes provienen de [**yoon2023diffusion**].

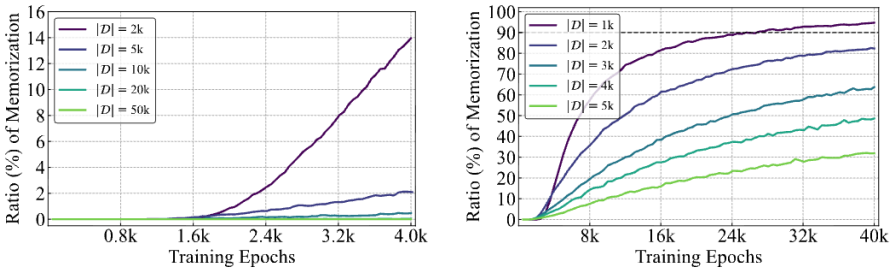


Figura 3.10: **Memorización frente al tamaño del conjunto de datos para un modelo fijo, con (izquierda) 4 000 épocas de entrenamiento y (derecha) 40 000 épocas de entrenamiento con conjuntos de datos más pequeños.** Observamos en el gráfico de la izquierda que los conjuntos de datos lo suficientemente grandes nunca se memorizarán (por ejemplo, de 10 000 a 50 000 muestras), mientras que los conjuntos de datos pequeños (de 1 000 a 5 000 muestras) se memorizarán tras un entrenamiento suficiente. Esto significa que, a medida que el conjunto de datos se vuelve más grande en relación con el modelo, la proporción de memorización al final del entrenamiento pasa de cerca de 1 a cerca de 0 con relativa rapidez, lo que indica la posible existencia de una transición de fase de la memorización a la generalización a medida que aumenta el tamaño del conjunto de datos. Tomada de [**gu2023memorization**].

el *denoiser* $\bar{\boldsymbol{x}}$, y la muestra $\hat{\boldsymbol{x}}$ se memoriza utilizando el siguiente criterio:

$$\hat{\boldsymbol{x}} \text{ está memorizado} \iff \frac{\|\hat{\boldsymbol{x}} - \boldsymbol{x}_{(1)}(\hat{\boldsymbol{x}})\|_2}{\|\hat{\boldsymbol{x}} - \boldsymbol{x}_{(2)}(\hat{\boldsymbol{x}})\|_2} < \frac{1}{3}, \tag{3.3.5}$$

donde $\boldsymbol{x}_{(i)}(\hat{\boldsymbol{x}})$ es el i-ésimo punto más cercano a $\hat{\boldsymbol{x}}$ en el conjunto de datos de entrenamiento (en la norma euclidiana) y $1/3$ es una constante que, según se ha demostrado empíricamente, concuerda bien con la percepción humana de la memorización. Nótese que esta definición se corresponde exactamente con nuestra definición intuitiva anterior de «memorización», según la cual cada uno de los puntos muestreados está mucho más cerca de una muestra de entrenamiento concreta que de todas las demás muestras de entrenamiento. Demostraron que este tipo de memorización se produce en la práctica cuando el modelo es mucho más grande (en términos del número de parámetros) en comparación con los datos; la Figura 3.9 muestra que, en este caso, las muestras generadas son, en esencia, copias perfectas, píxel a píxel, de las muestras de entrenamiento. Poco después, [**gu2023memorization**] llevó a cabo un estudio minucioso para caracterizar cómo diversos factores, tales como las arquitecturas y el tiempo de entrenamiento, influyen en la memorización, demostrando, entre otras cosas, que a medida que el tamaño del conjunto de datos aumenta en relación con un modelo fijo, el índice de memorización cambia rápidamente de un valor cercano a 1 a uno cercano a 0, es decir, el modelo deja de memorizar rápidamente. Esto indica la posibilidad de una transición de fase entre la memorización y la generalización a medida que el modelo se vuelve más pequeño en relación con los datos. [**buchanan2025edge**] investiga la existencia y el mecanismo teórico de esta transición de fase en datos sintéticos extraídos de un modelo de mezcla gaussiana, demostrando que existe y proporcionando una caracterización parcial del tamaño del modelo a partir del cual los modelos comienzan o dejan de memorizar. Otros trabajos teóricos, como [**niedoba2024towards**; **kamb2024analytic**; **george2025denoising**; **zhang2025generalization**] también intentan ofrecer una caracterización teórica de la transición de fase en función del tamaño del modelo y del conjunto de datos en diferentes contextos.

Con el fin de demostrar empíricamente la existencia de una transición de fase entre la memorización y la generalización en datos reales, [**pmlr-v235-zhang24cn**] replanteó la definición de memorización mediante el siguiente razonamiento. En el espacio de dimensiones muy elevadas de las imágenes naturales, las distancias euclidianas entre puntos se vuelven mucho más estrictas [**Wright-Ma-2022**], en el sentido de que, si dos imágenes no coinciden a la perfección a nivel de píxeles, sus distancias euclidianas se concentrarán en forma marcada en torno a un valor fijo (que escala con la dimensión de las imágenes). Esto no tiene en cuenta los casos en los que ciertos *conceptos* (como los personajes de dibujos animados protegidos por derechos de autor) se generan en posiciones novedosas, lo que dificulta enormemente establecer criterios de memorización eficaces y prácticos basados en la distancia euclidiana. Para solucionar esto, [**pmlr-v235-zhang24cn**] utiliza una función

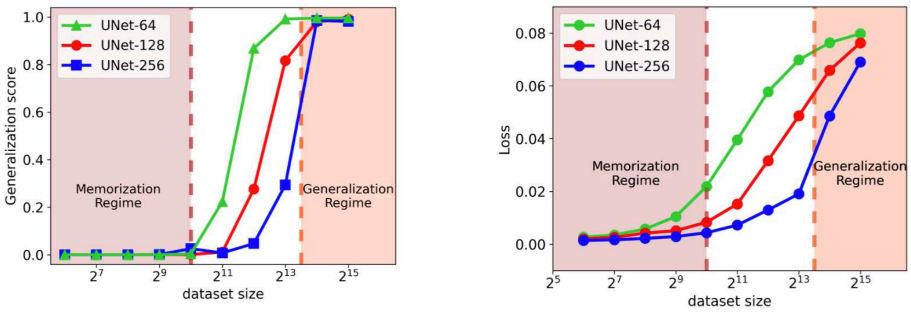


Figura 3.11: **La puntuación de generalización (izquierda) y la pérdida de entrenamiento (derecha) para un modelo fijo y un volumen de datos creciente**, extraído de [**pmlr-v235-zhang24cn**]. La puntuación de generalización es 1− la fracción de muestras que se memorizan, y las diferentes líneas corresponden a modelos de eliminación de ruido de distintos tamaños (UNet-64 es más pequeño que UNet-128, que a su vez es más pequeño que UNet-256). Las figuras muestran que, a medida que el tamaño del conjunto de datos aumenta en relación con el tamaño del modelo, la pérdida aumenta significativamente y deja de producirse la memorización, lo que muestra que nuestra teoría e intuición se aplican a modelos reales.

específica f_{SSCD} (una red neuronal preentrenada denominada «detector de copias autosupervisado» [**pizzi2022self**]) para mapear imágenes a un espacio euclidiano y luego toma el producto interno normalizado para obtener su similitud (coseno), de tal manera que se afirma que las imágenes que son similares en virtud de esta similitud coseno lo son también conceptualmente en el sentido necesario para la memorización. Entonces, [**pmlr-v235-zhang24cn**] afirma que una muestra $\hat{\boldsymbol{x}}$ ha sido memorizada si su similitud con cualquier punto de entrenamiento es $> 3/5$, es decir,

$$\hat{\boldsymbol{x}} \text{ está memorizada} \iff \max_{i\in[N]} \frac{f_{\text{SSCD}}(\hat{\boldsymbol{x}})^\top f_{\text{SSCD}}(\boldsymbol{x}^{(i)})}{\|f_{\text{SSCD}}(\hat{\boldsymbol{x}})\|_2 \|f_{\text{SSCD}}(\boldsymbol{x}^{(i)})\|_2} > \frac{3}{5} \tag{3.3.6}$$

La Figura 3.11 muestra el resultado de tomar numerosas muestras (alrededor de 10 000) de modelos de difusión entrenados con conjuntos de datos de entrenamiento de distintos tamaños, en cuanto a la estimación de la memorización y el cálculo de la pérdida de entrenamiento. El resultado concuerda con lo expuesto anteriormente: cuando el modelo es demasiado grande en relación con los datos de entrenamiento y la pérdida de entrenamiento es demasiado pequeña, se produce memorización y las muestras reproducen los datos de entrenamiento. Así pues, nuestra intuición se confirma en la práctica: *¡en el caso de los modelos generativos de difusión, escalar el modelo no es lo único que se necesita!*

Entonces, la pregunta es: ¿qué tipo de *denoiser* deberíamos usar? La clave es elegir una arquitectura de red que pueda aproximar el verdadero *denoiser* (por ejemplo, el que corresponde a una distribución de rango bajo,

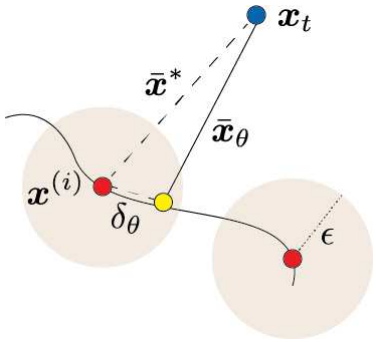


Figura 3.12: **Visualización del funcionamiento de un *denoiser* bien entrenado** como una perturbación del filtro empírico para *valores pequeños de* t, cuando el filtro memorizador actúa como una proyección sobre el punto más cercano del conjunto de entrenamiento.

como en (3.2.80)) sin aproximar el *denoiser* memorizador (3.3.3). Algunos trabajos han analizado esta delgada línea y por qué los modelos de difusión modernos, que utilizan arquitecturas basadas en transformers y convoluciones, pueden memorizar y generalizar en diferentes regímenes [**kamb2024analytic**; **niedoba2024towards**; **zhang2025generalization**].

Volvamos por fin a la premisa inicial de este capítulo, descrita en la Sección 3.1.3: encontrar la distribución de entropía mínima que modela los datos es la única forma de aprender la distribución. Esperamos que el análisis anterior haya demostrado que esta afirmación es falsa. Sin embargo, una afirmación relacionada, también defendida en la Sección 3.1.3 debido a su equivalencia en las situaciones que habíamos considerado entonces, podría seguir siendo cierta: encontrar la distribución con la *tasa de codificación* mínima es una forma de aprender la distribución. Supongamos por un momento que $\boldsymbol{x}$ tiene una densidad p; la tasa de codificación (es decir, la entropía cruzada) de la distribución empírica $p^{\boldsymbol{X}}$ con respecto a p es $\mathrm{CE}(p \mid p^{\boldsymbol{X}}) = \infty$. Es evidente que esta tasa de codificación no es mínima en ningún sentido; si utilizamos las mismas aproximaciones sucesivamente mejores p^n, p^e y $\hat{q}$ analizadas en la Sección 3.1.3, se cumple que

$$\infty = \mathrm{CE}(p \parallel p^{\boldsymbol{X}}) > \mathrm{CE}(p \parallel p^n) > \mathrm{CE}(p \parallel p^e) > \mathrm{CE}(p \parallel \hat{q}) \approx h(p). \tag{3.3.7}$$

Por lo tanto, la minimización de la tasa de codificación parece una forma sensata de aprender la distribución y, al mismo tiempo, evitar este problema. Sin embargo, ahora surge un problema de manejabilidad: ¿cómo podemos diseñar algoritmos que aprendan la distribución minimizando la tasa de codificación y evitar los inconvenientes asociados a una minimización excesiva de la entropía?

Una pista consiste en analizar qué ocurre con los modelos de difusión entrenados con éxito que no memorizan. Al expresar el *denoiser* entrenado $\bar{\boldsymbol{x}}_\theta$ como una perturbación del *denoiser* $\bar{\boldsymbol{x}}^\star$, es decir,

$$\bar{\boldsymbol{x}}_\theta(t, \boldsymbol{x}_t) = \bar{\boldsymbol{x}}^\star(t, \boldsymbol{x}_t) + \boldsymbol{\delta}_\theta(t, \boldsymbol{x}_t) \tag{3.3.8}$$

podemos hacernos una idea gráfica del funcionamiento de un filtro de eliminación de ruido bien entrenado, como se muestra en la Figura 3.12. A grandes rasgos, el *denoiser* entrenado calcula el filtro de memorización más una perturbación. Para un t pequeño (quizás incluso en el último paso de la eliminación de ruido), el *denoiser* memorizador apunta completamente en la dirección de una sola muestra de entrenamiento (la muestra de entrenamiento que se va a regenerar en la iteración actual). La distribución que se está aprendiendo, o respecto a la cual se elimina el ruido, es simplemente la distribución empírica con unas bolas de radio ϵ alrededor de cada punto,[12] donde ϵ es una cota superior de la magnitud de la perturbación, visualizado en la Figura 3.12 por las bolas de color naranja pálido.[13] En el caso de modelos de difusión cada vez más grandes y bien entrenados, el radio de perturbación ϵ se reduce a 0 y el modelo aprende la distribución empírica. Por otro lado, un valor de ϵ lo suficientemente grande como para que las bolas correspondientes a puntos diferentes se superpongan o estén muy cerca puede implicar que el modelo de difusión generalice bien.

Por lo tanto, nuestro modelo de difusión eficaz puede considerarse como una minimización de una *tasa de codificación con pérdida*, es decir, una tasa de codificación con precisión de ϵ (de modo que se puedan ignorar las bolas de radio ϵ alrededor de cada punto). Esta noción de la tasa de codificación con pérdida resulta ser lo que conviene minimizar, en principio, y será *extremadamente* útil. Exploraremos sus aplicaciones en el Capítulo 4 y a lo largo del resto de este libro.

3.4 Resumen y notas

Mensajes clave. En este capítulo, hemos mostrado que, para lograr una distribución general de baja dimensión, un enfoque computacional universal consiste en partir de una distribución genérica y comprimirla hasta alcanzar dicha distribución mediante la reducción (iterativa) del ruido o de la entropía. Como vimos en el capítulo anterior, esto ocurre incluso en el caso de las que podrían considerarse las distribuciones más simples de un único subespacio, como el cálculo del PCA mediante el algoritmo de iteración de potencias. Como veremos en los próximos capítulos, las funciones fundamentales de las redes profundas modernas no es ajustar distribuciones de datos o funciones arbitrarias (con ruido), sino más bien realizar o aproximar esos operadores de compresión o de eliminación de ruido (con pérdida). En capítulos posteriores derivaremos las formas precisas de esos operadores y las arquitecturas profundas asociadas.

[12] Más formalmente, esta distribución perturbada tiene su soporte en (un subconjunto de) la unión de bolas de radio ϵ centradas en cada muestra de entrenamiento. Esta visión está respaldada por trabajos anteriores, p. ej., [**song2025selective**].

[13] En dimensiones elevadas, se plantea la hipótesis de que un mejor modelo para estas perturbaciones no se describe mediante esferas, sino más bien mediante formas mucho más anisotrópicas y adaptadas a la geometría [**farghly2025diffusion**], aunque la intuición sigue siendo similar.

Más extensiones y referencias. El uso de la eliminación de ruido y la difusión para el muestreo tiene una larga trayectoria. Probablemente, el primer trabajo que trata específicamente de un modelo de difusión sea [**Sohl-Dickstein2015**], pero antes de este ya existían numerosos trabajos sobre la eliminación del ruido como problema computacional y estadístico. El más relevante de ellos es probablemente [**hyvarinen05a**], que utiliza explícitamente la función de *score* para eliminar el ruido y realizar un análisis de componentes independientes. Los trabajos posteriores más populares surgieron, en esencia, de manera prácticamente simultánea: [**ho2020denoising**; **song2019**]. Desde entonces, miles de artículos se han apoyado en los modelos de difusión; retomamos este tema en el **??**.

Muchos de estos trabajos utilizan un proceso estocástico distinto al de la simple combinación lineal (3.2.85). De hecho, todos los trabajos mencionados anteriormente hacen hincapié en la necesidad de añadir ruido gaussiano *independiente* al inicio de cada paso del proceso hacia adelante. Los trabajos de carácter teórico utilizan el movimiento browniano o ecuaciones diferenciales estocásticas para formular el proceso hacia adelante [**song2020score**]. Sin embargo, dado que las combinaciones lineales de distribuciones gaussianas siguen dando lugar a distribuciones gaussianas, las *distribuciones marginales* de dichos procesos siguen adoptando la forma de (3.2.85). La mayor parte de nuestro análisis solo requiere que las distribuciones marginales sean lo que son, por lo que nuestro modelo, aunque sea demasiado simplista, resulta suficiente para casi todo. El único caso en el que las distribuciones marginales no son suficientes es cuando derivamos una expresión para $\mathbb{E}[\boldsymbol{x}_s \mid \boldsymbol{x}_t]$ en términos de $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t]$. Los distintos procesos (de ruido) dan lugar a diferentes expresiones de este tipo, que pueden utilizarse para el muestreo (y, por supuesto, existen otras formas de obtener muestreadores eficientes, como el siempre popular muestreador DDPM). El proceso descrito en (3.2.85) es un proceso estocástico genuino, cuya iteración de eliminación de ruido «natural» adopta la forma del popular algoritmo DDIM [**song2020denoising**]. (Ni siquiera esta equivalencia es trivial; citamos [**de2025distributional**] como justificación.)

Además del trabajo teórico [**li2024d**] tratado en la Sección 3.2.2, y la línea de trabajo en la que se basa, que estudia la eficiencia de *muestreo* de los modelos de difusión cuando los datos tienen una estructura de baja dimensión, existe un amplio conjunto de trabajos que estudia la eficiencia de *entrenamiento* de los modelos de difusión cuando los datos tienen una estructura de baja dimensión. En concreto, **chen2023score** y **wang2024diffusion** caracterizaron el error de aproximación y estimación de los *denoisers* cuando los datos pertenecen a una mezcla de gaussianas de bajo rango, demostrando que el número de muestras de entrenamiento necesarias para aprender con precisión la distribución escala con la dimensión intrínseca de los datos, en lugar de la dimensión ambiente. Existe una considerable *labor metodológica* que intenta aprovechar la estructura de baja dimensión de los datos para realizar diversas tareas con modelos de difusión. Destacamos tres: la edición de imágenes [**chen2024exploring**], el marcado de agua [**li2024shallow**] y el «desaprendizaje» [**chen2025dual**], aunque esta lista no es exhaustiva.

3.5 Ejercicios y ampliaciones

Ejercicio 3.1. Mostrar que (3.2.8) es la solución óptima del problema (3.2.7).

Ejercicio 3.2. Consideremos vectores aleatorios $\boldsymbol{x} \in \mathbb{R}^D$ y $\boldsymbol{y} \in \mathbb{R}^d$, de tal manera que el par $(\boldsymbol{x}, \boldsymbol{y}) \in \mathbb{R}^{D+d}$ sea gaussiano en conjunto. Esto significa que

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{bmatrix} \sim \mathcal{N}\left(\begin{bmatrix} \boldsymbol{\mu}_{\boldsymbol{x}} \\ \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}, \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} \right),$$

donde los parámetros de media y covarianza se dan por

$$\boldsymbol{\mu}_{\boldsymbol{x}} = \mathbb{E}[\boldsymbol{x}], \quad \boldsymbol{\mu}_{\boldsymbol{y}} = \mathbb{E}[\boldsymbol{y}], \quad \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} = \mathbb{E}\left[\begin{bmatrix} \boldsymbol{x} - \mathbb{E}[\boldsymbol{x}] \\ \boldsymbol{y} - \mathbb{E}[\boldsymbol{y}] \end{bmatrix} \begin{bmatrix} \boldsymbol{x} - \mathbb{E}[\boldsymbol{x}] \\ \boldsymbol{y} - \mathbb{E}[\boldsymbol{y}] \end{bmatrix}^\top \right]$$

Supongamos que $\boldsymbol{\Sigma}_{\boldsymbol{y}}$ es positiva definida (y, por lo tanto, invertible); entonces, que la matriz de covarianza sea semidefinida positiva es equivalente a la condición del complemento de Schur $\boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top \succeq \mathbf{0}$.

En este ejercicio, demostraremos que la distribución condicional $p_{\boldsymbol{x}|\boldsymbol{y}}$ es gaussiana: es decir,

$$p_{\boldsymbol{x}|\boldsymbol{y}} \sim \mathcal{N}\left(\boldsymbol{\mu}_{\boldsymbol{x}} + \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} (\boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}}), \boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top\right). \tag{3.5.1}$$

Una vía directa para demostrar este resultado consiste en manipular el cociente definitoria de las densidades $p_{\boldsymbol{x},\boldsymbol{y}}/p_{\boldsymbol{y}}$. A continuación esbozamos un argumento algebraicamente conciso de este tipo.

1. Compruebe la siguiente identidad matricial para la covarianza:

$$\begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} = \begin{bmatrix} \boldsymbol{I}_D & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \\ \mathbf{0} & \boldsymbol{I}_d \end{bmatrix} \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix} \begin{bmatrix} \boldsymbol{I}_D & \mathbf{0} \\ \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{I}_d \end{bmatrix}. \tag{3.5.2}$$

   Esta identidad se obtiene aplicando dos rondas de eliminación gaussiana en bloques a la matriz de covarianza.

2. Partiendo de la identidad anterior, demuestre que

$$\begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix}^{-1} = \boldsymbol{P} \begin{bmatrix} \left(\boldsymbol{\Sigma}_{\boldsymbol{x}} - \boldsymbol{\Sigma}_{\boldsymbol{xy}} \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top\right)^{-1} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \end{bmatrix} \boldsymbol{P}^\top, \tag{3.5.3}$$

   donde

$$\boldsymbol{P} = \begin{bmatrix} \boldsymbol{I}_D & \mathbf{0} \\ -\boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{I}_d \end{bmatrix}, \tag{3.5.4}$$

   y siempre que las inversas pertinentes estén definidas.[14] Concluya que

$$\begin{bmatrix} \boldsymbol{x} - \boldsymbol{\mu}_{\boldsymbol{x}} \\ \boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}^\top \begin{bmatrix} \boldsymbol{\Sigma}_{\boldsymbol{x}} & \boldsymbol{\Sigma}_{\boldsymbol{xy}} \\ \boldsymbol{\Sigma}_{\boldsymbol{xy}}^\top & \boldsymbol{\Sigma}_{\boldsymbol{y}} \end{bmatrix}^{-1} \begin{bmatrix} \boldsymbol{x} - \boldsymbol{\mu}_{\boldsymbol{x}} \\ \boldsymbol{y} - \boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix} \tag{3.5.5}$$

[14] Cuando el término del complemento de Schur no sea invertible, se mantiene el mismo resultado sustituyendo su inversa por la pseudoinversa de Moore-Penrose. En particular, la distribución condicional (3.5.1) se reduce a una distribución gaussiana degenerada.

$$= \begin{bmatrix} \boldsymbol{x}-\boldsymbol{m} \\ \boldsymbol{y}-\boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}^{\top} \begin{bmatrix} \left(\boldsymbol{\Sigma}_{\boldsymbol{x}}-\boldsymbol{\Sigma}_{\boldsymbol{xy}}\boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1}\boldsymbol{\Sigma}_{\boldsymbol{xy}}^{\top}\right)^{-1} & \mathbf{0} \\ \mathbf{0} & \boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1} \end{bmatrix} \begin{bmatrix} \boldsymbol{x}-\boldsymbol{m} \\ \boldsymbol{y}-\boldsymbol{\mu}_{\boldsymbol{y}} \end{bmatrix}, \tag{3.5.6}$$

donde $\boldsymbol{m} = \boldsymbol{\mu}_{\boldsymbol{x}} + \boldsymbol{\Sigma}_{\boldsymbol{xy}}\boldsymbol{\Sigma}_{\boldsymbol{y}}^{-1}(\boldsymbol{y}-\boldsymbol{\mu}_{\boldsymbol{y}})$.

3. Dividiendo $p_{\boldsymbol{x},\boldsymbol{y}}/p_{\boldsymbol{y}}$, demuestre la Ecuación (3.5.1). (*Pista: Utilizando las identidades anteriores, solo debería ser necesario un mínimo de álgebra. Para la constante de normalización, utilice la Ecuación* (3.5.3) *para descomponer el determinante de manera similar.*)

Ejercicio 3.3. Demuestre la identidad de Sherman-Morrison-Woodbury, es decir, para matrices $\boldsymbol{A}$, $\boldsymbol{C}$, $\boldsymbol{U}$, $\boldsymbol{V}$ tales que $\boldsymbol{A}$, $\boldsymbol{C}$ y $\boldsymbol{A}+\boldsymbol{UCV}$ sean invertibles,

$$(\boldsymbol{A}+\boldsymbol{UCV})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{U}(\boldsymbol{C}^{-1}+\boldsymbol{V}\boldsymbol{A}^{-1}\boldsymbol{U})^{-1}\boldsymbol{V}\boldsymbol{A}^{-1}. \tag{3.5.7}$$

Ejercicio 3.4. Vuelva a derivar lo siguiente, suponiendo que $\boldsymbol{x}_t$ sigue el modelo de ruido generalizado (3.2.85).

- La fórmula de Tweedie: (3.2.86).
- La iteración del DDIM: (3.2.87).
- El *denoiser* óptimo de Bayes para un modelo de mezcla gaussiana: (3.2.88).

Ejercicio 3.5.

1. Implemente las fórmulas derivadas en el Ejercicio 3.4, creando un muestreador para mezclas gaussianas.
2. Reproduzca la Figura 3.4 y la Figura 3.8.
3. A continuación, presentamos *Flow Matching (FM)* (véase la Nota 3.3):

$$\alpha_t = 1-t, \qquad \sigma_t = t. \tag{3.5.8}$$

   Implemente este proceso utilizando el mismo marco y pruébelo para el muestreo en espacios de alta dimensión. ¿Qué proceso ofrece resultados mejores o más estables?

Ejercicio 3.6. Considere la interpolación, algo familiar

$$\boldsymbol{x}_t = \alpha_t\boldsymbol{x} + \sigma_t\boldsymbol{y}, \qquad \forall t \in [0,T]. \tag{3.5.9}$$

donde $\boldsymbol{y}$ es una variable aleatoria (no necesariamente gaussiana). Definimos la *velocidad* como

$$\boldsymbol{v}_t = \frac{\mathrm{d}}{\mathrm{d}t}\boldsymbol{x}_t = \alpha_t'\boldsymbol{x} + \sigma_t'\boldsymbol{y}, \qquad \forall t \in [0,T]. \tag{3.5.10}$$

Calcule una relación algebraica que exprese $\mathbb{E}[\boldsymbol{v}_t \mid \boldsymbol{x}_t]$ en términos de $\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{x}_t]$ y viceversa, demostrando que la predicción de la velocidad es algebraicamente equivalente a la eliminación de ruido y, por tanto, también a la predicción de ruido (véase la Nota 3.3).

Capítulo 4

Aprendizaje de representaciones mediante compresión con pérdidas

> "*Cuando puedes medir aquello de lo que hablas y expresarlo en números, sabes algo al respecto; pero cuando no puedes medirlo, cuando no puedes expresarlo en números, tu conocimiento es de un tipo exiguo e insatisfactorio: puede ser el comienzo del conocimiento, pero apenas has avanzado, en tus pensamientos, a la etapa de la ciencia, sea cual sea el tema de que se trate.*"
>
> – Lord Kelvin, 1883

Recapitulemos lo que hemos visto hasta ahora. Hemos mostrado que es posible identificar y acceder a una distribución con un soporte de dimensión arbitrariamente bajas, como la que se ilustra en la Figura 4.1 (izquierda), transformando gradualmente la distribución de ruido puro (alta entropía) en la distribución que se desea aprender mediante *eliminación iterativa de ruido*. Hemos mostrado que el *denoiser* en cada iteración está directamente relacionado con el gradiente de la log-densidad de la distribución mediante la fórmula de Tweedie (3.2.23). Demostramos que, en la práctica, ese filtro de eliminación de ruido $\bar{\boldsymbol{x}}_\theta$ puede estimarse utilizando un número finito de muestras. Por lo tanto, desarrollamos un método general para aprender o aproximarse a una distribución de datos.

No obstante, en esta metodología, la codificación de la distribución está implícita en la forma funcional y los parámetros del *denoiser*, si los tuviera. De hecho, los lectores más atentos habrán notado que, para una distribución general, nunca hemos especificado explícitamente cuál debería ser la forma funcional del *denoiser*. En la práctica, la gente suele modelarlo con alguna red neuronal profunda que tiene una arquitectura empíricamente diseñada.

Además, aunque sabemos que el proceso de eliminación de ruido anterior reduce la entropía, no sabemos en qué medida, ni conocemos la entropía de las distribuciones intermedias y finales. Por lo tanto, sería muy útil conocer la forma (óptima) de la operación de eliminación de ruido y cómo medir el cambio en la entropía. Por último, al final del capítulo anterior demostramos que si el proceso de eliminación de ruido reduce la entropía *en exceso*, esto conduce a que solo se aprendan los datos de entrenamiento, sin posibilidad de interpolación o extrapolación. La innovación metodológica que resuelve todas estas cuestiones se desarrollará en el próximo capítulo.

Medida de información. Recordemos que nuestro objetivo general es modelar datos procedentes de una distribución continua con un soporte de baja dimensión. Si queremos identificar el modelo más simple que genera los datos, podríamos considerar tres medidas típicas de parsimonia: la dimensión, el volumen o la entropía diferencial. Si usamos la dimensión, es obvio que el mejor modelo para un conjunto de datos dado es la propia distribución empírica, que es de dimensión cero.[1] Si, en cambio, utilizamos la entropía de los datos, entonces para todas las distribuciones con soporte de baja dimensión, la entropía diferencial es siempre negativa infinita. O bien, si utilizamos el volumen del espacio que abarcan los datos, el volumen del soporte de cualquier modelo de baja dimensión es siempre cero. Entonces, entre todas las distribuciones con un soporte de baja dimensión que podrían haber generado las mismas muestras de datos, ¿cómo podemos decidir cuál es la mejor basándonos en estas medidas de parsimonia que no permiten distinguirlas entre sí en absoluto?

En este capítulo, analizamos un marco que mitiga la dificultad técnica mencionada anteriormente al asociar la distribución aprendida con un esquema explícito y computable de codificación y decodificación, siguiendo y haciendo más explícito el segundo enfoque sugerido al final de la Sección 3.1.3. Como veremos, este enfoque nos permite, en esencia, aproximar con precisión la entropía de las distribuciones aprendidas en términos de una *tasa de codificación con pérdida* asociada al esquema de codificación, tal como se describe y justifica (aunque no se formaliza) al final de la Sección 3.3. A grandes rasgos, esta medida puede considerarse como una noción generalizada del volumen de la distribución de los datos empíricos. Con una medida de este tipo, podemos cuantificar con precisión en qué medida se reduce la entropía mediante una transformación de la distribución, por ejemplo, mediante un *denoiser*. Además, con una medida de este tipo, podemos obtener, al menos en principio, el operador óptimo de eliminación de ruido que reduce la entropía más rápidamente.

El aprendizaje de representaciones. Aunque el proceso de difusión y eliminación de ruido presentado en el capítulo anterior nos permite, en

[1] En la Introducción (Capítulo 1), sostuvimos que un objetivo importante de la inteligencia es buscar estructuras de baja dimensión en los datos observados. Sin embargo, desde el punto de vista técnico, no se debe interpretar el término «de baja dimensión» de manera literal. El objetivo de este capítulo es aclarar con precisión qué estructuras de baja dimensión debemos buscar y cómo hacerlo.

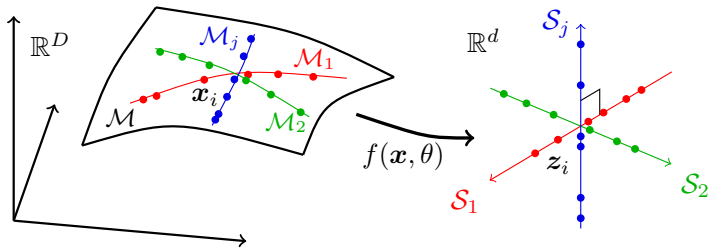


Figura 4.1: Aprender una representación estructurada (derecha) de la distribución de los datos (izquierda). Obsérvese que, además de aprender (y muestrear) la distribución de los datos originales como hicimos en el Capítulo 3 anterior, aquí queremos aprender una transformación $f(\boldsymbol{x}, \theta)$ para la distribución de los datos, de modo que se obtenga una representación con mejores propiedades geométricas o estadísticas deseadas, por ejemplo, lineal e incoherente.

principio, aprender una distribución arbitraria de baja dimensión, no ofrece una representación explícita de dicha distribución. Como veremos, se accederá a la distribución aprendida en repetidas ocasiones y se utilizará para tareas posteriores (por ejemplo, como distribución a priori para la inferencia bayesiana en el **??**). Por lo tanto, es conveniente que podamos transformar la distribución de los datos, por ejemplo mediante un mapeo $f(\boldsymbol{x}, \theta)$, a alguna forma estructurada estándar que facilite un acceso eficiente, como una mezcla de subespacios lineales (incoherentes) o gaussianas de bajo rango, tal y como se ilustra en la figura de la derecha de la Figura 4.1. En este capítulo formalizaremos este proceso como «aprendizaje de representaciones».

Como también veremos, para una distribución arbitraria, la medida de información anterior (tasa de codificación) no se puede calcular (fácilmente). Sin embargo, en el caso de una mezcla de subespacios o gaussianas de bajo rango, podemos obtener una fórmula cerrada precisa para la medida. Al optimizar esta medida, podremos encontrar la representación óptima. Como veremos en el siguiente Capítulo 5, la optimización de la medida también nos permite derivar formas explícitas del operador que realiza la transformación $f(\boldsymbol{x}, \theta)$ de la manera más eficiente. Esto dará lugar a una explicación basada en principios sobre la arquitectura de las redes profundas, así como a diseños de arquitectura profunda más eficientes.

4.1 Compresión mediante codificación con pérdidas

En esta sección, desarrollamos la *tasa de codificación con pérdida* como una medida computable de complejidad para distribuciones con un soporte de baja

dimensión, un contexto en el que las medidas clásicas como la entropía, la dimensión y el volumen no logran distinguir entre los modelos candidatos. En primer lugar, justificamos la necesidad de la codificación con pérdidas en relación con estas patologías (Sección 4.1.1); a continuación, demostramos que la función de tasa-distorsión relaciona esta medida de la teoría de la codificación con la geometría del soporte de datos por medio del recubrimiento esférico (Teorema 4.1). Esta conexión constituye el puente entre los enfoques basados en la teoría de la información y los enfoques geométricos para el aprendizaje de distribuciones de baja dimensión. Por último, desarrollamos una aproximación estrecha en forma cerrada de la tasa de codificación con pérdida para las distribuciones gaussianas (Sección 4.1.3), así como un algoritmo eficaz para agrupar mezclas de distribuciones gaussianas de baja dimensión (Sección 4.1.4), que se generalizará a distribuciones no lineales en la siguiente sección.

4.1.1 La necesidad de la codificación con pérdida

Ya hemos abordado en varias ocasiones una dificultad: si al final aprendemos la distribución a partir de un número finito de muestras y nuestra clase de funciones de *denoiser* contiene suficientes funciones, ¿cómo nos aseguramos de que estamos tomando muestras de la distribución *verdadera* (con soportes de baja dimensión), en lugar de cualquier otra distribución que pueda generar ese número finito de muestras con alta probabilidad? Expongamos algunas de las dificultades conceptuales y técnicas con ejemplos concretos.

Ejemplo 4.1 (Volumen, dimensión y entropía). Para el ejemplo que se muestra en la parte superior de la Figura 4.2, supongamos que hemos tomado algunas muestras de una distribución uniforme en una recta (por ejemplo, en un plano bidimensional). El volumen de la recta o de los conjuntos de muestras es cero. Desde el punto de vista geométrico, la distribución empírica sobre el conjunto finito de muestras obtenido es aquella de *dimensión mínima* capaz de generar dicho conjunto finito de muestras.[2] Pero esto parece estar en contraste con otra medida más de la complejidad: la entropía. La entropía (diferencial) de la recta es el infinito negativo, pero la entropía (discreta) de este conjunto de muestras es finita y positiva. Así pues, parece que nos encontramos ante una situación paradójica según estos criterios habituales de parsimonia o complejidad: no son capaces de diferenciar adecuadamente entre (los modelos de) distribuciones con soportes de baja dimensión y algunos parecen diferenciarlos incluso de maneras totalmente opuestas.[3] ■

Ejemplo 4.2 (Densidad). Consideremos los dos conjuntos de puntos de datos muestreados que se muestran en la Figura 4.2. Geométricamente, son esencialmente iguales: cada conjunto consta de ocho puntos y cada punto ha aparecido con una frecuencia igual a 1/8. La única diferencia es que, en el

[2] Un conjunto de muestras discretas tiene dimensión cero, mientras que la recta de soporte tiene dimensión uno.

[3] Por supuesto, en sentido estricto, la entropía diferencial y la entropía discreta no son directamente comparables.

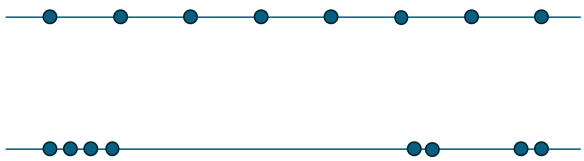


Figura 4.2: Ocho puntos situados en una línea con diferentes configuraciones geométricas; distribuidos uniformemente (arriba) o muy agrupados (abajo).

segundo conjunto de datos, algunos puntos están lo suficientemente «cerca» como para que se considere que tienen una mayor densidad alrededor de su respectivo «grupo». ¿Cuál de las dos es más relevante para la distribución real que podría haber generado las muestras? ¿Cómo podemos resolver esta ambigüedad a la hora de interpretar este tipo de distribuciones (empíricas)? ■

Existe una dificultad técnica adicional relacionada con la construcción de un esquema explícito de codificación y decodificación para un conjunto de datos. Dado un conjunto de datos muestreados en $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, ¿cómo se puede diseñar un esquema de codificación que sea implementable en máquinas con recursos computacionales y de memoria finitos? Nótese que incluso para representar un número real cualquiera se necesita un número infinito de dígitos o bits. Por lo tanto, cabe preguntarse si la entropía de una distribución es una medida directa de la complejidad de su esquema de codificación (óptimo). Analizamos este tema con otro ejemplo sencillo.

Ejemplo 4.3 (Precisión). Consideremos una distribución discreta $\boldsymbol{X} = [e, \pi]$ en la que hay una probabilidad igual de 1/2 de que que toma los valores del número de Euler $e \approx 2.71828$ y de $\pi \approx 3.14159$ **con probabilidad 1/2 cada uno. La entropía de esta distribución es $H = 1$, lo que sugiere que se pueden codificar los dos números mediante un único bit, 0 o 1, respectivamente. ¿Pero se puede implementar un esquema de decodificación para este código en una máquina de estados finitos? La respuesta es, de hecho, no, ya que se necesita un número infinito de bits para describir con precisión cualquiera de los dos números. ■

Por lo tanto, por lo general es imposible disponer de un esquema de codificación y decodificación capaz de reproducir con precisión muestras procedentes de una distribución arbitraria de valores reales.[4] Pero no tendría mucho valor práctico codificar una distribución sin poder decodificarla para muestras extraídas de la misma distribución.

Por lo tanto, para garantizar que cualquier esquema de codificación/decodificación sea computable y se pueda implementar con memoria y recursos computacionales finitos, debemos cuantificar la muestra $\boldsymbol{x}$ y codificarla solo hasta una cierta precisión, digamos $\epsilon > 0$. *Al hacerlo, en*

[4] Es decir, si se desea codificar dichas muestras con precisión, la única forma es memorizar cada una de las muestras.

esencia, consideramos que dos puntos de datos cualesquiera son equivalentes si su distancia es menor que ϵ. Más precisamente, nos gustaría considerar esquemas de codificación

$$\boldsymbol{x} \mapsto \hat{\boldsymbol{x}} \tag{4.1.1}$$

de tal manera que el error esperado causado por la cuantización esté acotado por ϵ. Es matemáticamente más conveniente, y conceptualmente prácticamente idéntico, acotar el error esperado *al cuadrado* por ϵ^2, es decir,

$$\mathbb{E}[d(\boldsymbol{x}, \hat{\boldsymbol{x}})^2] \leq \epsilon^2. \tag{4.1.2}$$

Por lo general, se elige que la distancia d sea la distancia euclidiana, o la norma L^2.[5] Adoptaremos esta opción en lo sucesivo.

4.1.2 La tasa-distorsión y la geometría de los datos

Por supuesto, de entre todos los esquemas de codificación que satisfacen la restricción anterior, nos gustaría elegir aquel que minimice la tasa de codificación resultante. Para una variable aleatoria dada $\boldsymbol{x}$ y una precisión ϵ, esta tasa se conoce como la *tasa-distorsión*, denotada como $\mathcal{R}_\epsilon(\boldsymbol{x})$.

Para comprender mejor esta tasa-distorsión, recurrimos a un teorema fundamental de la teoría de la información, demostrado originalmente en **shannon1959coding**, que relaciona esta tasa-distorsión con otras magnitudes de la teoría de la información. Para comprender este teorema, primero debemos definir una medida de *diferencia* entre dos distribuciones conocida como la divergencia de *Kullback-Leibler* (KL), o *entropía relativa*, que se denota por $\mathsf{KL}(\cdot \parallel \cdot)$ y se define en relación con la entropía y la entropía cruzada (véase la Sección 3.1.3) como

$$\begin{aligned} \mathsf{KL}(p \parallel q) &= \mathrm{CE}(p \parallel q) - h(p) = \mathbb{E}_{\boldsymbol{x}\sim p}[-\log q(\boldsymbol{x})] - \mathbb{E}_{\boldsymbol{x}\sim p}[-\log p(\boldsymbol{x})] && (4.1.3)\\ &= \mathbb{E}_{\boldsymbol{x}\sim p}[\log(p(\boldsymbol{x})/q(\boldsymbol{x}))]. && (4.1.4) \end{aligned}$$

La divergencia de KL siempre es no negativa y es igual a cero si y solo si $p = q$;[6] en el caso de que p y q tengan el mismo soporte y ambas sean densidades de probabilidad, esto se puede demostrar mediante el siguiente cálculo, basado en la desigualdad de Jensen y en el hecho de que la función $\log(\cdot)$ es estrictamente cóncava:

$$\begin{aligned} -\mathsf{KL}(p \parallel q) &= -\mathbb{E}_{\boldsymbol{x}\sim p}[\log(p(\boldsymbol{x})/q(\boldsymbol{x}))] = \mathbb{E}_{\boldsymbol{x}\sim p}[\log(q(\boldsymbol{x})/p(\boldsymbol{x}))] && (4.1.5)\\ &\leq \log \mathbb{E}_{\boldsymbol{x}\sim p}[q(\boldsymbol{x})/p(\boldsymbol{x})] = \log \int_{\mathbb{R}^D} \frac{q(\boldsymbol{\xi})}{p(\boldsymbol{\xi})} p(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} && (4.1.6)\\ &= \log \int_{\mathbb{R}^D} q(\boldsymbol{\xi}) \mathrm{d}\boldsymbol{\xi} = \log \mathbb{E}_{\boldsymbol{x}\sim q}[1] = \log 1 = 0. && (4.1.7) \end{aligned}$$

[5] De manera más general, podemos sustituir d^2 por cualquiera de las llamadas *divergencias*.

[6] Técnicamente, esta igualdad debe interpretarse en el sentido de «casi en todas partes», es decir, salvo, quizás, en un conjunto de medida (volumen) nula, ya que este conjunto no afectaría al valor de ninguna integral.

A continuación, definimos la *información mutua* $I(\cdot;\cdot)$, que es una medida de la *dependencia* entre dos variables aleatorias, como

$$I(\boldsymbol{x};\hat{\boldsymbol{x}}) = \mathsf{KL}(p(\boldsymbol{x},\hat{\boldsymbol{x}}) \parallel p(\boldsymbol{x})p(\hat{\boldsymbol{x}})), \tag{4.1.8}$$

Una vez establecidas las definiciones anteriores, podemos expresar la tasa-distorsión $\mathcal{R}_\epsilon$ en términos puramente probabilísticos de la siguiente forma:

$$\mathcal{R}_\epsilon(\boldsymbol{x}) = \min_{p(\hat{\boldsymbol{x}}|\boldsymbol{x}):\mathbb{E}[\|\boldsymbol{x}-\hat{\boldsymbol{x}}\|_2^2]\leq\epsilon^2} I(\boldsymbol{x};\hat{\boldsymbol{x}}). \tag{4.1.9}$$

Nótese que la minimización en (4.1.9) se realiza sobre todas las distribuciones condicionales $p(\hat{\boldsymbol{x}} \mid \boldsymbol{x})$ que satisfacen la restricción de distorsión $\mathbb{E}[\|\boldsymbol{x}-\hat{\boldsymbol{x}}\|_2^2] \leq \epsilon^2$. Cada una de estas distribuciones condicionales da lugar a una distribución conjunta $p(\boldsymbol{x},\hat{\boldsymbol{x}}) = p(\hat{\boldsymbol{x}} \mid \boldsymbol{x})p(\boldsymbol{x})$, que determina la información mutua (4.1.8). Muchas propiedades convenientes de la información mutua (y, por lo tanto, de la tasa-distorsión) se derivan de las propiedades correspondientes de la divergencia de KL. Por definición, sabemos que $\mathcal{R}_\epsilon(\boldsymbol{x})$ es una función *no creciente* en ϵ.

Ejemplo 4.4 (Tasa-distorsión de una fuente gaussiana (Teorema 13.3.3, [**Cover-Thomas**])). Consideremos una variable aleatoria gaussiana $\boldsymbol{x} \sim \mathcal{N}(0,\sigma^2)$. En este caso, es bien sabido que la función de tasa-distorsión se puede calcular en forma cerrada como:

$$\mathcal{R}_\epsilon(x) = \begin{cases} \frac{1}{2}\log\left(\sigma^2/\epsilon^2\right), & 0 \leq \epsilon \leq \sigma, \\ 0, & \epsilon > \sigma \end{cases}. \tag{4.1.10}$$

Esto se extiende de manera particularmente interesante al caso de D dimensiones. Supongamos que $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0},\boldsymbol{\Sigma})$, donde $\boldsymbol{\Sigma} \in \mathsf{PSD}(D)$ tiene valores propios $\lambda_1 \geq \cdots \geq \lambda_D \geq 0$. Entonces, la tasa-distorsión puede obtenerse mediante el «relleno de agua» como

$$\mathcal{R}_\epsilon(\boldsymbol{x}) = \frac{1}{2}\sum_{i=1}^{D}\log\left(\frac{\lambda_i}{\min\{\kappa,\lambda_i\}}\right), \tag{4.1.11}$$

donde el «nivel de agua» κ cumple que $\sum_{i=1}^{D}\min\{\kappa,\lambda_i\} = \epsilon^2$. Intuitivamente, las direcciones con una varianza λ_i mayor que κ se codifican de manera efectiva, mientras que las direcciones con una menor varianza se descartan, lo que garantiza que la restricción de distorsión se cumpla con una información mutua mínima. Consulte el Ejercicio 4.4 para ver una derivación detallada de esta fórmula cerrada. ■

Nota 4.1. Resulta que la tasa-distorsión es una aproximación viable a la entropía de $\boldsymbol{x}$ en el siguiente sentido. Supongamos que $\boldsymbol{x}$ y $\hat{\boldsymbol{x}}$ son vectores aleatorios continuos. Entonces, la información mutua se puede escribir como

$$I(\boldsymbol{x};\hat{\boldsymbol{x}}) = h(\boldsymbol{x}) - h(\boldsymbol{x} \mid \hat{\boldsymbol{x}}), \tag{4.1.12}$$

donde $h(\boldsymbol{x} \mid \hat{\boldsymbol{x}}) = -\mathbb{E}[\log_2 p(\boldsymbol{x} \mid \hat{\boldsymbol{x}})]$ es la *entropía condicional* de $\boldsymbol{x}$ dado $\hat{\boldsymbol{x}}$. Por lo tanto, la tasa de codificación mínima se alcanza cuando la diferencia entre la

entropía de $\boldsymbol{x}$ y la entropía condicional de $\boldsymbol{x}$ dado $\hat{\boldsymbol{x}}$ se minimiza entre todas las distribuciones que satisfacen la restricción: $\mathbb{E}[\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2^2] \leq \epsilon^2$.

De hecho, no es necesario suponer que $\boldsymbol{x}$ y $\hat{\boldsymbol{x}}$ sean continuos para llegar a una conclusión como la anterior. Por ejemplo, si ambos vectores aleatorios son discretos, tras una interpretación adecuada de la divergencia de KL para vectores aleatorios de valores discretos, tenemos que

$$I(\boldsymbol{x}; \hat{\boldsymbol{x}}) = H(\boldsymbol{x}) - H(\boldsymbol{x} \mid \hat{\boldsymbol{x}}), \tag{4.1.13}$$

donde $H(\boldsymbol{x})$ se define en la ecuación (3.1.1) de la Sección 3.1.3. En términos más generales, se pueden emplear conceptos matemáticos avanzados de la teoría de la medida para definir la información mutua (y, por lo tanto, la tasa-distorsión) para cualquier variable aleatoria $\boldsymbol{x}$ y $\hat{\boldsymbol{x}}$, incluidas aquellas con distribuciones de baja dimensión bastante exóticas; véase **Cover-Thomas**.

Nota 4.2. Dado un conjunto de puntos de datos en $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, siempre se pueden interpretar como muestras de una distribución discreta uniforme con una probabilidad igual a $1/N$ sobre estos N vectores. La entropía de dicha distribución es $H(\boldsymbol{X}) = \frac{1}{N} \log_2 N$.[7] Sin embargo, aunque $\boldsymbol{X}$ sea una distribución uniforme sobre sus muestras, la tasa de codificación $\mathcal{R}_\epsilon(\boldsymbol{X})$ alcanzable con un esquema de codificación con pérdida podría ser significativamente inferior a $H(\boldsymbol{X})$ si estas muestras no están tan uniformemente distribuidas y muchas se agrupan muy cerca unas de otras. Por lo tanto, para la segunda distribución que se muestra en la Figura 4.2, si se elige adecuadamente el error de cuantización ϵ, la tasa de codificación con pérdida que se puede alcanzar puede ser significativamente menor que si se codificara como una distribución uniforme.[8] Obsérvese también que, con el concepto de tasa-distorsión, la dificultad mencionada en el Ejemplo 4.3 también desaparece: podemos elegir dos números racionales que estén lo suficientemente cerca de cada uno de los dos números irracionales. El esquema de codificación resultante tendrá una complejidad finita.

Ejemplo 4.5. A veces, puede darse una situación opuesta, en la que queremos fijar primero la tasa de codificación e intentar encontrar un esquema de codificación que minimice la distorsión. Por ejemplo, supongamos que solo queremos utilizar un número fijo de códigos para los puntos muestreados de una distribución, y queremos saber cómo diseñar los códigos de tal manera que se minimice la distorsión promedio o máxima durante el esquema de codificación/decodificación. En el caso de una distribución uniforme en un cuadrado unitario, nos preguntamos con qué precisión podemos codificar los puntos extraídos de esta distribución, digamos, con n bits. Este problema equivale a preguntarse cuál es el radio mínimo (es decir, la distorsión) tal que podamos cubrir el cuadrado unitario con 2^n discos de ese radio. La Figura 4.3 muestra cubrimientos aproximadamente óptimos de un cuadrado con $n = 4, 6, 8,$

[7] Nótese, una vez más, que aunque podamos codificar estos vectores con esta tasa de codificación, no podemos decodificarlos con una precisión arbitraria.

[8] No obstante, para esta distribución uniforme discreta, cuando ϵ es lo suficientemente pequeño, siempre se cumple que $H(\boldsymbol{X}) \approx \mathcal{R}_\epsilon(\boldsymbol{X})$.

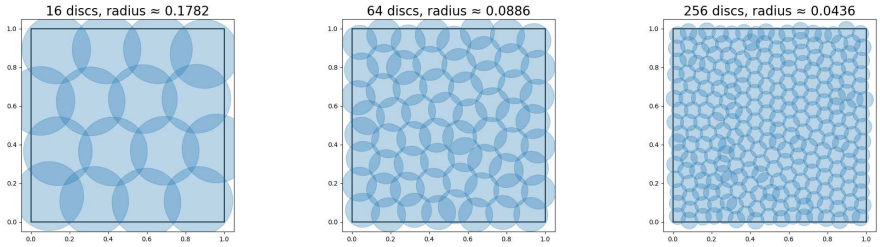


Figura 4.3: Aproximaciones a las soluciones óptimas para 2^4, 2^6 y 2^8 discos que cubren un cuadrado, junto con los radios correspondientes, calculados mediante un algoritmo heurístico de optimización.

de modo que $2^n = 16, 64, 256$ discos, respectivamente. Obsérvese que los radios óptimos de los discos disminuyen a medida que aumenta el número de discos 2^n. ■

Resulta ser un problema notoriamente difícil obtener expresiones en forma cerrada para la función de tasa-distorsión (4.1.9) para distribuciones generales $p(\boldsymbol{x})$. Sin embargo, tal y como sugiere el Ejemplo 4.5, existen casos especiales importantes en los que la *geometría* del soporte de la distribución $p(\boldsymbol{x})$ puede vincularse con la función de tasa-distorsión y, por lo tanto, con la tasa de codificación óptima para un nivel de distorsión ϵ. De hecho, este ejemplo puede generalizarse a cualquier contexto en el que el soporte de $p(\boldsymbol{x})$ sea un conjunto compacto suficientemente regular —incluidas las distribuciones de baja dimensión— y $p(\boldsymbol{x})$ esté uniformemente distribuída en su soporte. Esto abarca una gran cantidad de casos de interés práctico. Formalizamos esta noción en el siguiente resultado, que establece esta propiedad para un caso especial.

Teorema 4.1. *Supongamos que $\boldsymbol{x}$ es una variable aleatoria tal que su soporte $K \doteq \operatorname{Supp}(\boldsymbol{x})$ es un conjunto compacto. Definamos el número de cobertura $\mathcal{N}_\epsilon(K)$ como el número mínimo de bolas de radio ϵ que pueden cubrir K, es decir,*

$$\mathcal{N}_\epsilon(K) \doteq \min\left\{ n \in \mathbb{N} \colon \exists \boldsymbol{p}_1, \ldots, \boldsymbol{p}_n \in K \ \textit{s.t.}\ K \subseteq \bigcup_{i=1}^{n} B_\epsilon(\boldsymbol{p}_i) \right\}, \tag{4.1.14}$$

donde $B_\epsilon(\boldsymbol{p}) = \{\boldsymbol{\xi} \in \mathbb{R}^D \mid \|\boldsymbol{\xi} - \boldsymbol{p}\|_2 \leq \epsilon\}$ es la bola euclidiana de radio ϵ centrada en $\boldsymbol{p}$. Entonces se cumple que

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \leq \log_2 \mathcal{N}_\epsilon(K). \tag{4.1.15}$$

Si, además, $\boldsymbol{x}$ sigue una distribución uniforme en K y K es una unión de hiperesferas de baja dimensión mutuamente ortogonales,[9] *entonces se cumple una cota inferior correspondiente:*

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq \log_2 \mathcal{N}_\epsilon(K) - O(D). \tag{4.1.16}$$

[9] De hecho, es posible tratar casos de K altamente irregulares, como variedades compactas o

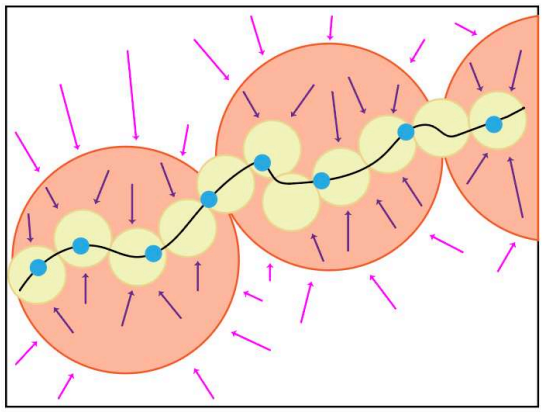


Figura 4.4: **La aproximación de una distribución de baja dimensión mediante bolas de radio ϵ.** Podemos observar que, a medida que el parámetro ϵ se reduce, la unión de las bolas de radio ϵ se aproxima cada vez mejor al soporte de la distribución real (negra). Además, los filtros de eliminación de ruido asociados (cuyo mapeo entrada-salida viene dada por las flechas proporcionadas) obtenidos al aproximar la distribución real mediante una mezcla de distribuciones gaussianas, cada una con covarianza $(\epsilon^2/D)\boldsymbol{I}$, se acercan cada vez más a los filtros de eliminación de ruido verdaderos. Para valores grandes de ϵ, estos eliminadores de ruido no se acercan en absoluto a la distribución real, mientras que para valores pequeños de ϵ se aproximan mucho a los eliminadores de ruido reales. El Teorema 4.1 establece que esta aproximación caracteriza la función de tasa-distorsión para distorsiones pequeñas ϵ, unificando los enfoques paralelos de minimización de la tasa de codificación y eliminación de ruido para el aprendizaje de distribuciones de baja dimensión sin patologías.

Demostración. La demostración de este teorema excede el alcance de este libro, por lo que la remitimos al **??**. □

La cota superior del Teorema 4.1 solo requiere que Supp($\boldsymbol{x}$) sea un conjunto compacto, mientras que nuestra demostración de la cota inferior requiere supuestos de regularidad más fuertes: concretamente, que $\boldsymbol{x}$ esté uniformemente distribuido sobre Supp($\boldsymbol{x}$) y que Supp($\boldsymbol{x}$) sea una mezcla de gaussianas de rango bajo (con la normalización adecuada).[10] Bajo estos supuestos, la implicación

incluso fractales, con un resultado paralelo, pero su formulación se vuelve mucho más técnica: véase Riegler et al. [**Riegler2018-jh**; **Riegler2023-rr**]. En el **??** ofrecemos una demostración sencilla que muestra el resultado para uniones de hiperesferas ortogonales, una suposición que equivale a una mezcla de distribuciones gaussianas de rango bajo con soportes ortogonales cuando la dimensión de cada subespacio es suficientemente grande. Véase el **??** para un análisis más detallado de esta hipótesis.

[10] Más adelante en este capítulo, en la Sección 4.2.3, veremos una función objetivo cuyos máximos son mezclas normalizadas de gaussianas de bajo rango que corresponden a los datos de entrada de cierta manera. El Capítulo 5 desarrollará aún más este marco para el aprendizaje profundo, demostrando que la suposición estructural que hacemos para demostrar la cota inferior no es excesivamente restrictiva.

del Teorema 4.1 puede resumirse de la siguiente manera: para una codificación suficientemente precisa de la distribución de $\boldsymbol{x}$ (es decir, a medida que $\epsilon \to 0$), el marco de codificación de mínima tasa-distorsión es equivalente al problema del empaquetamiento de bolas en el soporte de $\boldsymbol{x}$. Así pues, la tasa-distorsión puede considerarse como una forma «sensible a la probabilidad» de aproximar el soporte de la distribución de $\boldsymbol{x}$ mediante una mezcla de muchas bolas pequeñas (Figura 4.4).

Ahora analizaremos otra relación entre esto y la jerarquía de complejidad «eliminación de ruido-difusión-entropía» que hemos comentado anteriormente en este capítulo.

Nota 4.3. El ingrediente clave en la demostración de la cota inferior en el Teorema 4.1 es un resultado importante de la teoría de la información conocido como el *límite inferior de Shannon* para la tasa-distorsión, que lleva el nombre de Claude Shannon, quien lo derivó por primera vez en un caso especial [**shannon1959coding**]. En él se establece la siguiente estimación para la función de tasa-distorsión, para cualquier variable aleatoria $\boldsymbol{x}$ con una densidad $p(\boldsymbol{x})$ y una norma al cuadrado con esperanza finita [**Linder1994-ej**]:

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq h(\boldsymbol{x}) - \log_2 \operatorname{vol}(B_\epsilon) - C_D, \tag{4.1.17}$$

donde $C_D > 0$ es una constante que depende únicamente de D. Además, esta cota inferior es, de hecho, óptima cuando $\epsilon \to 0$: es decir,

$$\lim_{\epsilon \to 0} \mathcal{R}_\epsilon(\boldsymbol{x}) - [h(\boldsymbol{x}) - \log_2 \operatorname{vol}(B_\epsilon) - C_D] = 0. \tag{4.1.18}$$

Así pues, cuando la distorsión ϵ es pequeña, podemos pensar únicamente en términos de la cota inferior de Shannon, en lugar del problema de optimización (generalmente inmanejable) que define la relación tasa-distorsión (4.1.9).

La cota inferior de Shannon constituye el nexo entre la tasa de codificación, la minimización de la entropía / la eliminación de ruido y los enfoques de empaquetamiento geométrico de esferas para el aprendizaje de distribuciones de baja dimensión. Obsérvese que, en el caso especial de una densidad uniforme $p(\boldsymbol{x})$, (4.1.17) se convierte en

$$\mathcal{R}_\epsilon(\boldsymbol{x}) \geq -\int_K \frac{1}{\operatorname{vol}(K)} \log_2 \frac{1}{\operatorname{vol}(K)} \mathrm{d}\boldsymbol{\xi} - \log_2 \operatorname{vol}(B_\epsilon) - C_D \tag{4.1.19}$$

$$= \log_2 \operatorname{vol}(K) / \operatorname{vol}(B_\epsilon) - C_D. \tag{4.1.20}$$

La relación $\operatorname{vol}(K)/\operatorname{vol}(B_\epsilon)$ aproxima el número de bolas de radio ϵ necesarias para cubrir K mediante un argumento del peor caso, lo cual es preciso para conjuntos K suficientemente regulares cuando ϵ es pequeño (véase el **??** para más detalles). Mientras tanto, recordemos el modelo gaussiano de eliminación de ruido $\boldsymbol{x}_\epsilon = \boldsymbol{x} + \epsilon \boldsymbol{g}$ mencionado anteriormente en este capítulo, donde $\boldsymbol{g} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ es independiente de $\boldsymbol{x}$. Curiosamente, la entropía diferencial de la distribución conjunta $(\boldsymbol{x}, \epsilon \boldsymbol{g})$ se puede calcular como

$$h(\boldsymbol{x}, \epsilon \boldsymbol{g}) = -\int p(\boldsymbol{\xi}) p(\boldsymbol{\gamma}) \log_2 p(\boldsymbol{\xi}) p(\boldsymbol{\gamma}) \mathrm{d}\boldsymbol{\xi} \mathrm{d}\boldsymbol{\gamma} \tag{4.1.21}$$

$$= h(\boldsymbol{x}) + h(\epsilon \boldsymbol{g}). \tag{4.1.22}$$

Hemos visto la entropía gaussiana calculada en la Ecuación (3.1.4): cuando ϵ es pequeño, es igual, salvo por constantes aditivas, a la cantidad $-\log_2 \mathrm{vol}(B_\epsilon)$ que hemos visto en la cota inferior de Shannon. En ciertos casos especiales (por ejemplo, datos apoyados en subespacios incoherentes de bajo rango), cuando ϵ es pequeño y el soporte de $\boldsymbol{x}$ es suficientemente regular, la distribución de $\boldsymbol{x}_\epsilon$ admite incluso una buena aproximación local mediante el producto de las distribuciones $p(\boldsymbol{x})$ y $p(\boldsymbol{g})$, lo que justifica el cálculo anterior. Por lo tanto, el proceso de eliminación de ruido gaussiano ofrece otra interpretación más de la cota inferior de Shannon, ya que se deriva de la entropía de una versión con ruido de $\boldsymbol{x}$, con un nivel de ruido proporcional al nivel de distorsión ϵ.

Por lo tanto, este enfoque de tasa-distorsión finita mediante el recubrimiento de esferas revalida o generaliza todas las medidas previas de complejidad de la distribución, lo que nos permite diferenciar y clasificar las distintas distribuciones de manera unificada. Estas perspectivas interrelacionadas se visualizan en la Figura 4.4.

En el caso de una distribución general con niveles de distorsión finitos, suele ser imposible hallar su función de tasa-distorsión en forma analítica. A menudo hay que recurrir al cálculo numérico[11]. Sin embargo, como veremos, en nuestro contexto a menudo necesitamos conocer la tasa-distorsión como una función explícita de un conjunto de puntos de datos o de sus representaciones. Esto se debe a que queremos utilizar la tasa de codificación como medida de la calidad de las representaciones. Una forma analítica explícita facilita determinar cómo transformar la distribución de los datos para mejorar la representación. Por lo tanto, deberíamos trabajar con distribuciones cuyas funciones de tasa-distorsión adopten formas analíticas explícitas. Con este fin, comenzaremos con la familia de distribuciones más simple y, a la vez, la más importante.

4.1.3 Tasa de codificación con pérdida para una distribución gaussiana de baja dimensión

Supongamos ahora que disponemos de un conjunto de muestras de datos en $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$ procedente de cualquier distribución.[12] Nos gustaría elaborar un esquema constructivo que pueda codificar los datos con cierta precisión, digamos

$$\boldsymbol{x}_i \mapsto \hat{\boldsymbol{x}}_i, \quad \text{sujeto a} \quad \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|_2 \leq \epsilon. \tag{4.1.23}$$

Nótese que se trata de una condición suficiente, explícita e interpretable que garantiza que los datos se codifiquen de tal manera que $\frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|_2^2 \leq \epsilon^2$. Esta última desigualdad es precisamente la restricción de tasa-distorsión para la distribución empírica dada y su codificación. Por ejemplo, en el Ejemplo 4.5,

[11] Los lectores interesados pueden consultar [**Blahut-1972**] para conocer un algoritmo clásico que calcula numéricamente la función de tasa-distorsión para una distribución discreta.

[12] O bien, estos puntos de datos podrían considerarse ellos mismos como una distribución (empírica).

utilizamos este criterio simplificado para determinar la distorsión mínima y el esquema de codificación explícitos para una tasa de codificación dada.

Sin pérdida de generalidad, supongamos que la media de $\boldsymbol{X}$ es cero, es decir, $\frac{1}{N}\sum_{i=1}^{N}\boldsymbol{x}_i = \boldsymbol{0}$. Sin ningún conocimiento previo sobre la naturaleza de la distribución subyacente a $\boldsymbol{X}$, podemos considerar que $\boldsymbol{X}$ se ha obtenido a partir de una distribución gaussiana $\mathcal{N}(\boldsymbol{0}, \boldsymbol{\Sigma})$ con la covarianza[13]

$$\boldsymbol{\Sigma} = \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\top}. \tag{4.1.24}$$

Obsérvese que, desde el punto de vista geométrico, $\boldsymbol{\Sigma}$ caracteriza una región elipsoidal en la que se encuentran la mayoría de las muestras $\boldsymbol{x}_i$.

Podemos considerar $\hat{\boldsymbol{X}} = [\hat{\boldsymbol{x}}_1, \ldots, \hat{\boldsymbol{x}}_N]$ como una versión con ruido de $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$:

$$\hat{\boldsymbol{x}}_i = \boldsymbol{x}_i + \boldsymbol{w}_i, \tag{4.1.25}$$

donde $\boldsymbol{w}_i \sim \mathcal{N}(\boldsymbol{0}, \epsilon^2\boldsymbol{I}/D)$ es un ruido gaussiano independiente de $\boldsymbol{x}_i$. Entonces, la covarianza de $\hat{\boldsymbol{x}}_i$ viene dada por

$$\hat{\boldsymbol{\Sigma}} = \mathbb{E}\left[\hat{\boldsymbol{x}}_i\hat{\boldsymbol{x}}_i^{\top}\right] = \frac{\epsilon^2}{D}\boldsymbol{I} + \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\top}. \tag{4.1.26}$$

Nótese que el volumen de la región generada por los vectores $\boldsymbol{x}_i$ es proporcional a la raíz cuadrada del determinante de la matriz de covarianza

$$\text{volumen}(\hat{\boldsymbol{x}}_i) \propto \sqrt{\det\left(\hat{\boldsymbol{\Sigma}}\right)} = \sqrt{\det\left(\frac{\epsilon^2}{D}\boldsymbol{I} + \frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\top}\right)}. \tag{4.1.27}$$

El volumen engendrado por cada vector aleatorio $\boldsymbol{w}_i$ es proporcional a

$$\text{volumen}(\boldsymbol{w}_i) \propto \sqrt{\det\left(\frac{\epsilon^2}{D}\boldsymbol{I}\right)}. \tag{4.1.28}$$

Para codificar los vectores que se encuentran dentro de la región abarcada por $\hat{\boldsymbol{x}}_i$, podemos cubrir dicha región con bolas no superpuestas de radio ϵ, como se ilustra en la Figura 4.5. Cuando el volumen de la región abarcada por $\hat{\boldsymbol{x}}_i$ es significativamente mayor que el volumen de la bola de radio ϵ, el número total de bolas que necesitamos para cubrir la región es aproximadamente igual a el cociente entre ambos volúmenes:

$$\#\,\epsilon\text{-bolas} \approx \frac{\text{volumen}(\hat{\boldsymbol{x}}_i)}{\text{volumen}(\boldsymbol{w}_i)} = \sqrt{\det\left(\boldsymbol{I} + \frac{D}{N\epsilon^2}\boldsymbol{X}\boldsymbol{X}^{\top}\right)}. \tag{4.1.29}$$

Si utilizamos números binarios para etiquetar todas las bolas de radio ϵ en la región de interés, el número total de bits necesarios es, por lo tanto,

$$\mathcal{R}_{\epsilon}(\boldsymbol{X}) \approx \log_2(\#\,\epsilon\text{-bolas}) \approx R_{\epsilon}(\boldsymbol{X}) \doteq \frac{1}{2}\log\det\left(\boldsymbol{I} + \frac{D}{N\epsilon^2}\boldsymbol{X}\boldsymbol{X}^{\top}\right). \tag{4.1.30}$$

[13] Se sabe que, dada una varianza fija, la distribución gaussiana alcanza la máxima entropía. Es decir, establece un límite superior para lo que podría ser el peor de los casos en cuanto a la tasa de codificación posible.

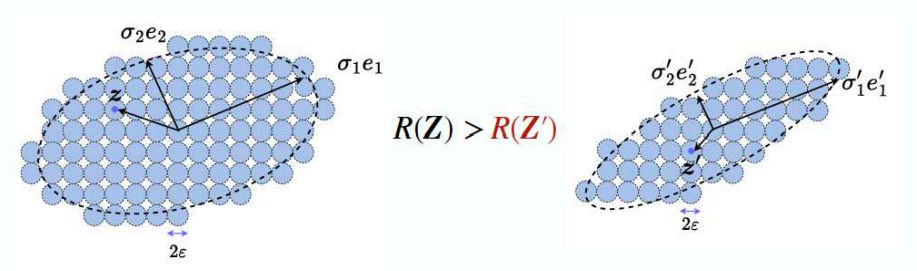


Figura 4.5: Cubrir la región abarcada por los vectores de datos utilizando bolas de radio ϵ. Cuanto mayor sea el volumen del espacio, más bolas se necesitarán y, por lo tanto, más bits se necesitarán para codificar y enumerar las bolas. Cada vector de valor real $\boldsymbol{x}$ puede codificarse como el índice de la bola en la que se encuentra ese vector.

Nótese que, a partir del Ejemplo 4.4, se puede mostrar que el lado derecho de esta fórmula coincide con la tasa-distorsión (hasta una precisión de ϵ) de una fuente gaussiana con matriz de covarianza $\frac{1}{N}\boldsymbol{X}\boldsymbol{X}^\top + (\epsilon^2/D)\boldsymbol{I}$ (por ejemplo, la matriz de covarianza empírica de los datos más un término de ruido). De este modo, un argumento geométrico sobre el volumen de los vectores encerrados y el empaquetamiento de esferas se traduce en un argumento basado en la teoría de la información sobre la tasa-distorsión de una fuente gaussiana perturbada.

Ejemplo 4.6. La Figura 4.5 muestra un ejemplo de una distribución bidimensional con un soporte elipsoidal, que se aproxima al soporte de una distribución gaussiana bidimensional. La región está cubierta por pequeñas bolas de tamaño ϵ. Todas las bolas están numeradas del 1 a, digamos, n. Entonces, dado cualquier vector $\boldsymbol{x}$ en esta región, solo necesitamos determinar cuál es la bola de radio ϵ cuyo centro está más cerca, lo que se denota como $\text{bola}_\epsilon(\boldsymbol{x})$. Para recordar $\boldsymbol{x}$, solo necesitamos recordar el número de esta bola, cuyo almacenamiento requiere $\log(n)$ bits. Si necesitamos decodificar $\boldsymbol{x}$ a partir de este número, simplemente tomamos $\hat{\boldsymbol{x}}$ como el centro de la bola. Esto da lugar a un esquema explícito de codificación y decodificación:

$$\boldsymbol{x} \longrightarrow \text{bola}_\epsilon(\boldsymbol{x}) \longrightarrow \hat{\boldsymbol{x}} = \text{centro de bola}_\epsilon(\boldsymbol{x}). \tag{4.1.31}$$

Estos centros de las bolas pueden denominarse «códigos» del libro de códigos (o diccionario) del esquema de codificación. Es fácil ver que la precisión de este esquema de codificación-decodificación (con pérdida) es aproximadamente el radio ϵ de las bolas. Es evidente que $\mathcal{R}_\epsilon(\boldsymbol{Z})$ es el número promedio de bits necesarios para codificar el número de bola de cada vector $\boldsymbol{z}$ con este esquema de codificación y, por lo tanto, puede denominarse la *tasa de codificación* asociada a este esquema. ■

A partir de la derivación anterior, sabemos que la tasa de codificación $\mathcal{R}_\epsilon(\boldsymbol{X})$ es (aproximadamente) alcanzable mediante un esquema explícito de codificación (y decodificación). Presenta dos propiedades interesantes:

- En primer lugar, se puede observar que $R_\epsilon(\boldsymbol{X})$ se asemeja mucho a la función de tasa-distorsión de una fuente gaussiana [**Cover-Thomas**] (véase el Ejemplo 4.4). De hecho, cuando ϵ es pequeño, la expresión anterior es una aproximación muy cercana a la tasa-distorsión de una fuente gaussiana, tal como se señala en [**MaY2007-PAMI**].

- En segundo lugar, la misma tasa de codificación en forma cerrada $R_\epsilon(\boldsymbol{X})$ puede derivarse como una aproximación de $\mathcal{R}_\epsilon(\boldsymbol{X})$ si se supone que los datos $\boldsymbol{X}$ proceden de un subespacio lineal. Esto se puede demostrar cuantificando adecuadamente la descomposición en valores singulares (SVD) de $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ y construyendo un esquema de codificación con pérdida para vectores en el subespacio generado por $\boldsymbol{U}$ [**MaY2007-PAMI**].

En nuestro contexto, la expresión en forma cerrada $R_\epsilon(\boldsymbol{X})$ es bastante fundamental: se trata de la tasa de codificación asociada a un esquema explícito y natural de codificación con pérdidas para datos extraídos de una distribución gaussiana o de un subespacio lineal. Como veremos en el próximo capítulo, esta fórmula desempeña un papel importante en la comprensión de la arquitectura de las redes neuronales profundas.

4.1.4 Agrupamiento de una mezcla de gaussianas de baja dimensión

Como ya hemos comentado anteriormente, el conjunto de datos dado $\boldsymbol{X}$ suele tener estructuras intrínsecas de baja dimensión. Por lo tanto, codificarlo como una distribución gaussiana general resultaría muy redundante. Si podemos identificar esas estructuras intrínsecas en $\boldsymbol{X}$, podríamos diseñar esquemas de codificación mucho mejores que ofrezcan tasas de codificación mucho más bajas. O, dicho de otra forma, los códigos utilizados para codificar el $\boldsymbol{X}$ en cuestión pueden comprimirse. Veremos que la compresión ofrece una forma computable y unificadora de identificar tales estructuras. En esta sección, ilustramos esta importante idea con la familia más básica de estructuras de baja dimensión: una mezcla de gaussianas (de baja dimensión) o subespacios.

Ejemplo 4.7. La Figura 4.6 muestra un ejemplo en el que los datos $\boldsymbol{X}$ se distribuyen alrededor de dos subespacios (o gaussianas de baja dimensión). Si se consideran y se codifican conjuntamente como una sola gaussiana, el libro de códigos discreto (con pérdida) asociado, representado por todas las bolas azules, es obviamente muy redundante. Podemos intentar identificar las ubicaciones de los dos subespacios, denotados por S_1 y S_2 y diseñar un libro de códigos que abarque únicamente esos dos subespacios, es decir, las bolas verdes. Si logramos particionar correctamente las muestras de los datos $\boldsymbol{X}$ en los dos subespacios: $\boldsymbol{X} = [\boldsymbol{X}_1, \boldsymbol{X}_2]\boldsymbol{\Pi}$ con $\boldsymbol{X}_1 \in S_1$ y $\boldsymbol{X}_2 \in S_2$, donde $\boldsymbol{\Pi}$ denota una matriz de permutación, entonces la tasa de codificación resultante para los datos será mucho menor. Esto proporciona una representación más parsimoniosa y, por lo tanto, más deseable de los datos. ■

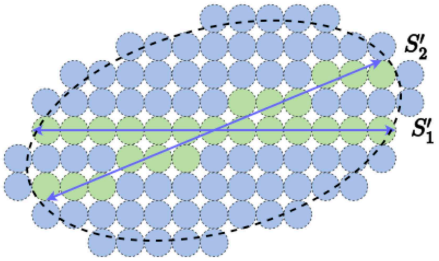


Figura 4.6: Comparación de dos esquemas de codificación con pérdidas para datos que se distribuyen alrededor de dos subespacios. El primero Uno consiste en empacar (azul) bolas de radio ϵ para todo el espacio generado por los dos subespacios; el segundo consiste en empacar bolas solo en una vecindad tabular alrededor de los dos subespacios. Este último, obviamente, tiene un libro de códigos mucho más pequeño y da como resultado una tasa de codificación mucho menor para las muestras en los subespacios.

Por lo tanto, en términos más generales, si los datos proceden de cualquier mezcla de subespacios o gaussianas de baja dimensión, sería deseable identificar esas componentes y codificar los datos basándose en las dimensiones intrínsecas de dichas componentes. Resulta que no perdemos mucha generalidad al suponer que los datos provienen de una mezcla de distribuciones gaussianas de baja dimensión. Esto se debe a que una mezcla de distribuciones gaussianas puede aproximar con gran precisión a la mayoría de las distribuciones generales [**borkar2016gaussian**].

El problema de agrupamiento. Ahora bien, en el caso de esta familia concreta de distribuciones, ¿cómo podemos identificar de manera eficaz y eficiente esas componentes de baja dimensión a partir de un conjunto de muestras

$$\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N]\,, \tag{4.1.32}$$

extraído de ellos? En otras palabras, dado el conjunto de datos completo $\boldsymbol{X}$, queremos dividirlo, o agruparlo, en varios subconjuntos, digamos K subconjuntos:

$$\boldsymbol{X\Pi} = [\boldsymbol{X}_1, \boldsymbol{X}_2, \ldots, \boldsymbol{X}_K], \tag{4.1.33}$$

donde cada subconjunto está formado por muestras extraídas de una sola gaussiana o subespacio de baja dimensión y $\boldsymbol{\Pi}$ es una matriz de permutación que indica la pertenencia a la partición. Cabe señalar que, dependiendo de la situación, la partición podría ser determinista o probabilística. Como se muestra en [**ma2007segmentation**], en el caso de una mezcla de distribuciones gaussianas, la partición probabilística no da lugar a una menor tasa de codificación. Por lo tanto, en aras de la simplicidad, aquí consideramos únicamente una partición determinista.

Agrupamiento mediante compresión con pérdidas. La principal dificultad para resolver el problema de agrupamiento mencionado anteriormente radica en que normalmente no conocemos el número de agrupamientos K, ni tampoco la dimensión de cada componente. El estudio de este problema de agrupamiento cuenta con una larga trayectoria. El libro de texto [**GPCA**] ofrece una exposición sistemática y exhaustiva de los diferentes enfoques de este problema. Para encontrar un enfoque eficaz para este problema, primero debemos comprender y aclarar por qué queremos agrupar los datos. En otras palabras, ¿qué ganamos exactamente al agrupar los datos, en comparación con no hacerlo? ¿Cómo medimos esa ganancia? Desde el punto de vista de la compresión de datos, una agrupación correcta debería dar lugar a un esquema de codificación (y decodificación) más eficiente.

Para cualquier conjunto de datos dado $\boldsymbol{X}$, ya existen dos esquemas de codificación obvios que sirven de referencia. Estos representan dos formas extremas de codificar los datos:

- Simplemente considerar todas las muestras juntas como si procedieran de una única distribución gaussiana. La tasa de codificación asociada, tal y como se ha deducido anteriormente, viene dada por:

$$\mathcal{R}_\epsilon(\boldsymbol{X}) \approx R_\epsilon(\boldsymbol{X}) = \frac{1}{2}\log\det\left(\boldsymbol{I} + \frac{D}{N\epsilon^2}\boldsymbol{X}\boldsymbol{X}^\top\right). \tag{4.1.34}$$

- Simplemente memorizar todas las muestras por separado asignando un número diferente a cada una. La tasa de codificación sería:

$$\mathcal{R}_0(\boldsymbol{X}) = \log(N). \tag{4.1.35}$$

Tengamos en cuenta que cualquiera de los dos esquemas de codificación puede convertirse en la solución «óptima» para una determinada elección (extrema) del error de cuantización ϵ:

1. *Régimen perezoso*: Si elegimos un valor de ϵ extremadamente grande, todas las muestras de $\boldsymbol{X}$ pueden quedar cubiertas por una sola bola. La tasa es $\lim_{\epsilon\to\infty}\mathcal{R}_\epsilon \to \frac{1}{2}\log\det(\boldsymbol{I}) = 0$.

2. *Régimen de memorización*: Si ϵ es extremadamente pequeño, cada muestra de $\boldsymbol{X}$ queda cubierta por una bola de ϵ diferente, por lo que el total es N. La tasa es $\lim_{\epsilon\to 0}\mathcal{R}_\epsilon \to \log(N)$.

Nótese que el primer esquema corresponde al caso en el que no nos interesa en absoluto ningún aspecto relevante de la distribución. No se desea reservar ningún bit para información relevante. A esto lo llamamos el «régimen perezoso». El segundo esquema corresponde al caso en el que se desea decodificar cada muestra con una precisión extremadamente alta. Por lo tanto, sería mejor «memorizar» cada muestra. A esto lo llamamos el «régimen de memorización».

Nótese que estas asintóticas en términos de ϵ son exactamente las mismas que se analizaron al presentar la tasa de codificación con pérdida como una

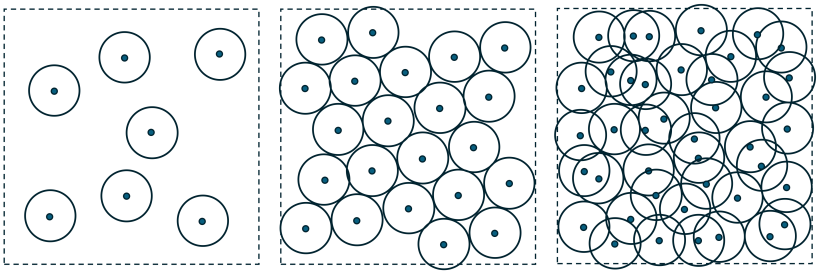


Figura 4.7: Varias de muestras aleatorias en un plano bidimensional. Consideremos un disco de radio ϵ asociado a cada muestra, con la muestra como centro. La densidad de las muestras aumenta de izquierda a derecha.

magnitud que los modelos de difusión entrenados de manera realista aproximan, tal como se muestra al final de la Sección 3.3. En la situación actual, codificamos *explícitamente* la distribución con una precisión de ϵ; en el caso de los modelos de difusión, codificamos *implícitamente* la distribución. Ciertas propiedades del esquema, como la memorización y la generalización, se manifiestan de manera similar en ambos casos y, de hecho, se puede estudiar un caso para comprender el otro.

Ejemplo 4.8. Para determinar cuándo es preferible o no el régimen de memorización, consideremos un número, digamos N, de muestras distribuidas aleatoriamente en un área unitaria sobre un plano bidimensional.[14] Imaginemos que intentamos diseñar un esquema de codificación con pérdida con un error de cuantización fijo ϵ. Esto equivale a colocar un disco de ϵ alrededor de cada muestra, como se muestra en la Figura 4.7. Cuando N es pequeño, la probabilidad de que todos los discos se superpongan entre sí es nula. En este caso, es necesario y óptimo un libro de códigos de tamaño N. Cuando N o la densidad alcanza un determinado valor crítico N_c, con alta probabilidad todos los discos comienzan a superponerse y a conectarse en un único grupo que cubre todo el plano; este fenómeno se conoce como «percolación» en el continuo [**Gilbert-1961**; **Mertens-Moore-2012**]. Cuando N supera este valor, los discos se superponen en gran medida. El número N de discos resulta muy redundante, ya que solo queremos codificar puntos en el plano con la precisión dada ϵ. El número de discos necesarios para cubrir todas las muestras es muy inferior a N.[15] ■

Tanto el régimen «perezoso» como el de memorización son en cierta medida triviales y quizás tengan poco interés teórico o práctico. Cualquiera de ambos esquemas estaría lejos de ser óptimo si se utilizara para codificar un gran número de muestras extraídas de una distribución que tenga un *soporte compacto y de baja dimensión*. El régimen interesante se encuentra entre estos dos.

[14] Supongamos que los puntos se generan mediante un proceso de Poisson con una densidad de N puntos por área unitaria.

[15] De hecho, existen algoritmos eficientes para encontrar dicha cobertura [**Booth-2001**].

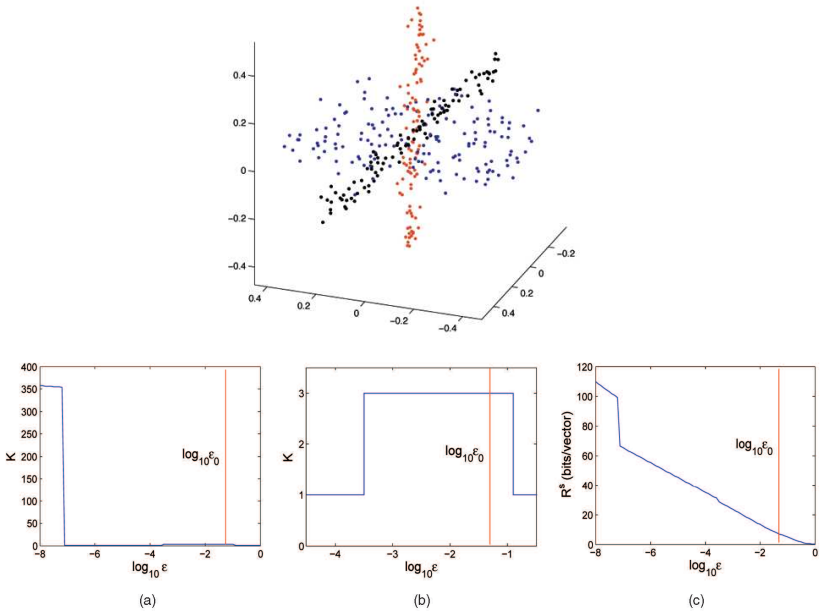


Figura 4.8: Parte superior: 358 muestras ruidosas extraídas de dos rectas y un plano en $\mathbb{R}^3$. Parte inferior: el efecto de variar ϵ sobre el resultado de la agrupación y la tasa de codificación. La línea roja marca la varianza ϵ_0 del ruido gaussiano añadido a las muestras. Obsérvese que los gráficos de la parte inferior izquierda y del centro muestran la misma magnitud con diferentes posiciones y escalas de los ejes.

Ejemplo 4.9. La Figura 4.8 muestra un ejemplo con muestras ruidosas extraídas de dos rectas y un plano en $\mathbb{R}^3$. Como se observa en la figura (c) de la derecha, la tasa de codificación óptima disminuye en forma monótona a medida que aumentamos ϵ, tal y como se esperaba a partir de la propiedad de la función de tasa-distorsión. Las figuras (a) y (b) muestran, al variar ϵ desde valores muy pequeños (cercanos a cero) hasta valores muy grandes (cercanos al infinito), el número óptimo de grupos cuando la tasa de codificación es mínima. Se pueden observar claramente el régimen «perezoso» y el régimen de memorización en los dos extremos de los gráficos. Pero también se puede observar en el figura (b) que, cuando el error de cuantización ϵ se elige en torno al nivel de la varianza real del ruido ϵ_0, el número óptimo de grupos es el número «correcto» tres, que representa dos líneas y un plano. Nos referimos informalmente a este régimen intermedio como el «régimen de generalización». Obsérvese que se produce una transición de fase brusca entre estos regímenes.[16] ■

A partir de la discusión y los ejemplos anteriores, vemos que, cuando el

[16]Hasta donde sabemos, no existe una justificación teórica rigurosa para estos comportamientos de transición de fase.

error de cuantización en relación con la densidad de la muestra[17] se encuentra dentro de un rango adecuado, minimizar la tasa de codificación con pérdida nos permitiría descubrir la distribución subyacente (de baja dimensión) de los datos muestreados. *Por lo tanto, la cuantización, que comenzó como una elección de carácter práctico, parece estar convirtiéndose en algo necesario para aprender una distribución continua a partir de su distribución empírica con un número finito de muestras.* Aunque sigue siendo esquiva una teoría rigurosa que explique este fenómeno, aquí, con fines de aprendizaje, nos interesa cómo aprovechar este fenómeno para diseñar algoritmos capaces de encontrar la distribución correcta.

Utilicemos el sencillo ejemplo que se muestra en la Figura 4.6 para ilustrar las ideas básicas. Si se pueden particionar todas las muestras en $\boldsymbol{X}$ en dos grupos en $\boldsymbol{X}_1$ y $\boldsymbol{X}_2$, con N_1 y N_2 muestras respectivamente, entonces la tasa de codificación asociada sería[18]

$$R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) = \frac{N_1}{N} R_\epsilon(\boldsymbol{X}_1) + \frac{N_2}{N} R_\epsilon(\boldsymbol{X}_2), \tag{4.1.36}$$

donde utilizamos $\boldsymbol{\Pi}$ para indicar la pertenencia a la partición. Si la partición respeta las estructuras de baja dimensión de la distribución — en este caso, la pertenencia de $\boldsymbol{X}_1$ y $\boldsymbol{X}_2$ a los dos subespacios, respectivamente, entonces la tasa de codificación resultante debería ser significativamente menor que la de los dos esquemas básicos anteriores:

$$R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) \ll R_\epsilon(\boldsymbol{X}), \quad R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) \ll R_0(\boldsymbol{X}). \tag{4.1.37}$$

En general, podemos reformular el problema de agrupamiento como un problema de optimización que minimiza la tasa de codificación:

$$\min_{\boldsymbol{\Pi}} \left\{ R_\epsilon^c(\boldsymbol{X} \mid \boldsymbol{\Pi}) \doteq \sum_{k=1}^{K} \frac{N_k}{N} R_\epsilon(\boldsymbol{X}_k) \right\}. \tag{4.1.38}$$

Estrategias de optimización para el agrupamiento. La pregunta que queda por resolver es cómo optimizar el objetivo de tasa de codificación anterior para encontrar los grupos óptimos. Existen tres enfoques naturales para abordar este objetivo:

1. Podemos empezar con el conjunto completo $\boldsymbol{X}$ como un único grupo (es decir, el régimen «perezoso») y luego buscar (por ejemplo, al azar) la forma de dividirlo de modo que se obtenga una tasa de codificación menor.

2. A la inversa, podemos partir de cada muestra $\boldsymbol{x}_i$ como un grupo independiente (es decir, el régimen de memorización) y buscar fusionar grupos que den lugar a una tasa de codificación menor.

[17] o la densidad de la muestra en relación con el error de cuantización

[18] Aquí ignoramos algunos bits de sobrecarga necesarios para codificar la pertenencia de cada muestra, por ejemplo, mediante la codificación de Huffman.

3. Por otra parte, si pudiéramos representar (o aproximar) la función de pertenencia $\mathbf{\Pi}$ mediante unos parámetros continuos, podríamos utilizar métodos de optimización como el descenso por gradiente (GD).

El primer enfoque no resulta tan atractivo desde el punto de vista computacional, ya que el número de particiones posibles que hay que probar es exponencial con respecto al número de muestras. Por ejemplo, el número de particiones de $\boldsymbol{X}$ en dos subconjuntos de igual tamaño es $\binom{N}{N/2}$, lo cual se dispara a medida que N aumenta. Analizaremos el tercer enfoque en el próximo Capítulo 5. Allí veremos cómo el papel de las redes neuronales profundas, en particular los transformers, está relacionado con el objetivo de la tasa de codificación.

El segundo enfoque fue propuesto originalmente en el trabajo de [**ma2007segmentation**]. Este trabajo demuestra las ventajas de poder evaluar la tasa de codificación de manera eficiente (por ejemplo, mediante una fórmula analítica). Gracias a ello, los grupos (de baja dimensión) de datos pueden identificarse en forma bastante eficiente y eficaz mediante el principio de minimización de la longitud de codificación (MCL). Nótese que, para un grupo $\boldsymbol{X}_k$ con N_k muestras, el número de bits binarios necesarios para codificar todas las muestras de $\boldsymbol{X}_k$ viene dada por:[19]

$$L(\boldsymbol{X}_k) = N_k R_\epsilon(\boldsymbol{X}_k). \tag{4.1.39}$$

Si tenemos dos grupos $\boldsymbol{X}_k$ y $\boldsymbol{X}_l$ y queremos codificar las muestras como dos grupos separados, el número de bits binarios necesarios es

$$L^c(\boldsymbol{X}_k, \boldsymbol{X}_l) = N_k R_\epsilon(\boldsymbol{X}_k) + N_l R_\epsilon(\boldsymbol{X}_l) - N_k \log \frac{N_k}{N_k + N_l} - N_l \log \frac{N_l}{N_k + N_l}.$$

Los dos últimos términos corresponden al número de bits necesarios para codificar la pertenencia de las muestras según el código de Huffman.

Entonces, dados cualesquiera dos grupos distintos $\boldsymbol{X}_1$ y $\boldsymbol{X}_2$, podemos decidir si fusionarlos o no basándonos en la diferencia entre las dos longitudes de codificación:

$$L(\boldsymbol{X}_k \cup \boldsymbol{X}_l) - L^c(\boldsymbol{X}_k, \boldsymbol{X}_l) \tag{4.1.40}$$

es positivo o negativo y $\boldsymbol{X}_k \cup \boldsymbol{X}_l$ denota la unión de los conjuntos de muestras en $\boldsymbol{X}_k$ y $\boldsymbol{X}_l$. Si es positivo, significa que la longitud de codificación se reduciría si fusionamos los dos grupos en uno solo. Este simple hecho da lugar al siguiente algoritmo de agrupamiento propuesto por [**ma2007segmentation**]:

Cabe señalar que este algoritmo es manejable, ya que el número total de comparaciones (por pares) y fusiones es de aproximadamente $O(N^2 \log N)$. Sin embargo, debido a su naturaleza voraz, no existe garantía teórica de que el proceso converja hacia la solución de agrupamiento óptima a nivel global. No obstante, según se señala en [**ma2007segmentation**], en la práctica,

[19] De hecho, una estimación más precisa de la longitud de codificación es $L(\boldsymbol{X}_k) = (N_k + D)R_\epsilon(\boldsymbol{X}_k)$ donde los bits adicionales se utilizan para codificar la base del subespacio [**ma2007segmentation**]. Aquí omitimos esta sobrecarga por simplicidad.

Algoritmo 4.1: Descenso más pronunciado por pares de longitud de codificación

Input: N puntos de datos $\{\boldsymbol{x}_i\}_{i=1}^N$
Output: Un conjunto $\mathcal{C}$ de grupos

procedure PairwiseSteepestDescentOfCodingLength($\{\boldsymbol{x}_i\}_{i=1}^N$)
  `# Inicializar` N `grupos` $\boldsymbol{X}_k$ `con un elemento cada uno`
  $\mathcal{C} \leftarrow \{\{\boldsymbol{x}_i\}\}_{i=1}^N$
  while $|\mathcal{C}| > 1$ **do**
    `# Si ninguna fusión ahorra bits`
    if $\min_{\boldsymbol{X}_k, \boldsymbol{X}_l \in \mathcal{C}} [L(\boldsymbol{X}_k \cup \boldsymbol{X}_l) - L^c(\boldsymbol{X}_k, \boldsymbol{X}_l)] \geq 0$ **then**
      `# Retorno anticipado: devuelve` $\mathcal{C}$ `y finaliza`
      return $\mathcal{C}$
    else
      `# Fusiona los grupos que ahorren más bits`
      $\boldsymbol{X}_{k^*}, \boldsymbol{X}_{l^*} \leftarrow \arg\min_{\boldsymbol{X}_k, \boldsymbol{X}_l \in \mathcal{C}} [L(\boldsymbol{X}_k \cup \boldsymbol{X}_l) - L^c(\boldsymbol{X}_k, \boldsymbol{X}_l)]$
      `# Eliminar grupos no fusionados y volver a agregar el grupo fusionado`
      $\mathcal{C} \leftarrow [\mathcal{C} \setminus \{\boldsymbol{X}_{k^*}, \boldsymbol{X}_{l^*}\}] \cup \{\boldsymbol{X}_{k^*} \cup \boldsymbol{X}_{l^*}\}$
    end if
  end while
  `# Si todas las fusiones generan ahorros, devolver un grupo`
  return $\mathcal{C}$
end procedure

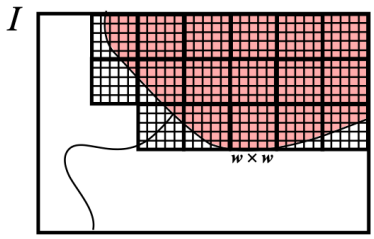


Figura 4.9: Fragmentos de imagen con un tamaño de $w \times w$ pixels.

este algoritmo, aparentemente sencillo, funciona extremadamente bien. Los resultados de la agrupación representados en la Figura 4.8 se calcularon, de hecho, mediante este algoritmo.

Ejemplo 4.10 (Segmentación de imágenes). La medida de longitud de codificación mencionada anteriormente y el algoritmo de agrupamiento asociado suponen que la distribución de los datos es una mezcla de distribuciones gaussianas (de baja dimensión). Aunque esto pueda parecer un tanto idealizado, la medida y el algoritmo pueden resultar muy útiles e incluso potentes en situaciones en las que el modelo sea (aproximadamente) válido.

Por ejemplo, una imagen natural suele estar compuesta por múltiples regiones con texturas casi homogéneas. Si tomamos muchas ventanas pequeñas de cada región, estas deberían parecerse a muestras extraídas de una distribución gaussiana (de baja dimensión), tal como se ilustra en la Figura 4.9. La Figura 4.10 muestra los resultados de la segmentación de imágenes obtenidos al aplicar el algoritmo de agrupamiento mencionado anteriormente directamente a los fragmentos de imagen. Se pueden encontrar más detalles técnicos sobre la adaptación del algoritmo al problema de la segmentación de imágenes en [**Mobahi-IJCV2011**]. ■

4.2 Transformación mediante maximización de la ganancia de información

Hasta ahora, en este capítulo hemos analizado cómo identificar una distribución con estructuras de baja dimensión mediante el principio de compresión. Como hemos visto en las dos secciones anteriores, la compresión computacional puede llevarse a cabo mediante la operación de eliminación de ruido o por vía de la agrupación en clústeres. La Figura 4.11 ilustra este concepto con nuestro ejemplo favorito.

Es importante señalar que el algoritmo de agrupamiento y las fórmulas de la tasa de codificación desarrollados en la sección anterior partían del supuesto de que la distribución de los datos es *ya* (aproximadamente) una mezcla de gaussianas de baja dimensión. En la práctica, los datos brutos —imágenes, audio, texto— rara vez cumplen esta hipótesis de manera directa:



Figura 4.10: Resultados de la segmentación basados en el algoritmo de agrupamiento aplicado a los fragmentos de imagen.

sus estructuras intrínsecas suelen ser *no lineales*. El objetivo de esta sección es desarrollar un marco para *transformar* los datos de tal manera que sus representaciones adopten la forma de una mezcla de subespacios incoherentes de baja dimensión, momento en el que las herramientas de la sección anterior se vuelven directamente aplicables. Es decir, la sección anterior responde a la pregunta «cómo comprimir y empacar datos que ya son gaussianos», mientras que esta sección responde a la pregunta «cómo transformar los datos para que se conviertan en gaussianos». Como veremos en el Capítulo 5, las redes profundas son precisamente el mecanismo computacional que permite llevar a cabo tales transformaciones y la arquitectura de la red se deriva de la optimización del objetivo de la tasa de codificación desarrollado aquí.

De hecho, el objetivo final de identificar una distribución de datos es utilizarla para facilitar ciertas tareas posteriores, como la segmentación, la clasificación o la generación (de imágenes). Por lo tanto, la forma en que se «representa» la distribución resultante es de vital importancia para determinar cómo se puede recuperar y utilizar de manera eficiente y eficaz la información relacionada con estas tareas posteriores. Esto plantea, naturalmente, una pregunta fundamental: *¿qué hace que una representación sea verdaderamente «buena» para su uso posterior?* A continuación, exploraremos las propiedades esenciales que debe poseer una representación significativa y útil y cómo estas propiedades pueden caracterizarse y perseguirse explícitamente mediante la maximización de la ganancia de información.

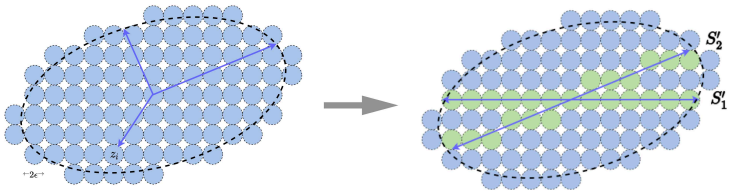


Figura 4.11: Identificación de una distribución de baja dimensión con dos subespacios (izquierda) mediante la eliminación de ruido o la agrupación, partiendo de una distribución gaussiana aleatoria genérica (derecha).

Cómo evaluar la calidad de las representaciones. Se puede considerar un conjunto de datos dado como un conjunto de muestras de un vector aleatorio $\boldsymbol{x}$ con una distribución determinada en un espacio de alta dimensión, por ejemplo, $\mathbb{R}^D$. Por lo general, la distribución de $\boldsymbol{x}$ tiene una dimensión intrínseca mucho menor que el espacio circundante. En términos generales, *aprender una representación* se refiere a aprender una aplicación continua, digamos $f(\cdot)$, que transforma $\boldsymbol{x}$ en lo que se denomina *vector de características* $\boldsymbol{z}$ en otro espacio (normalmente de menor dimensión), digamos $\mathbb{R}^d$, donde $d < D$. Se espera que a través de tal mapeo

$$\boldsymbol{x} \in \mathbb{R}^D \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z} \in \mathbb{R}^d, \tag{4.2.1}$$

las estructuras intrínsecas de baja dimensión de $\boldsymbol{x}$ se identifiquen y se representen mediante $\boldsymbol{z}$ de una manera más compacta y estructurada, con el fin de facilitar tareas posteriores como la clasificación o la generación. La característica $\boldsymbol{z}$ puede considerarse como un código compacto (aprendido) de los datos originales $\boldsymbol{x}$, por lo que la aplicación f también se denomina *codificador*. La pregunta fundamental del aprendizaje de representaciones es

¿Cuál sería una medida rigurosa y eficaz para evaluar la calidad de las representaciones?

Conceptualmente, la calidad de una representación $\boldsymbol{z}$ depende de qué tan bien identifique la información más relevante y suficiente de $\boldsymbol{x}$ para las tareas posteriores y qué tan eficientemente represente dicha información. Durante mucho tiempo, se creía y se defendía que la «suficiencia» o «calidad» de una representación de características aprendida debía definirse en función de una tarea específica. Por ejemplo, $\boldsymbol{z}$ solo tiene que ser suficiente para predecir la etiqueta de clase $\boldsymbol{y}$ en un problema de clasificación. A continuación, comencemos con el problema clásico de la clasificación de imágenes y analizaremos por qué ese concepto de «representación» específica para cada tarea es limitado y debe generalizarse.

4.2.1 Aprendizaje de representaciones a partir de datos etiquetados

Supongamos que $\boldsymbol{x} \in \mathbb{R}^D$ es un vector aleatorio extraído de una mezcla de K distribuciones (componentes) $\mathcal{D} = \{\mathcal{D}_k\}_{k=1}^K$. Dado un conjunto finito de muestras independientes e idénticamente distribuidas $\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{D\times N}$ del vector aleatorio $\boldsymbol{x}$, *buscamos una buena representación* mediante una aplicación continua $f(\boldsymbol{x}) : \mathbb{R}^D \to \mathbb{R}^d$ que capte las estructuras intrínsecas de $\boldsymbol{x}$ y facilite al máximo la tarea posterior de clasificación.[20] Para facilitar la tarea de aprender la distribución $\mathcal{D}$, en el popular contexto de la clasificación supervisada, se asigna a cada muestra $\boldsymbol{x}_i$ una etiqueta de clase verdadera (o una palabra de código para cada clase), representada normalmente por un vector one-hot $\boldsymbol{y}_i \in \mathbb{R}^K$.

Codificación de la información de clases mediante la entropía cruzada. Amplios estudios han mostrado que, para muchos conjuntos de datos prácticos (por ejemplo, imágenes, audio y lenguajes naturales), el mapeo (de codificación) entre los datos $\boldsymbol{x}$ y su etiqueta de clase $\boldsymbol{y}$ puede modelarse de manera eficaz mediante el entrenamiento de una red profunda,[21] aquí denotada

$$f(\boldsymbol{x}, \theta) : \boldsymbol{x} \mapsto \boldsymbol{y}$$

con parámetros de red $\theta \in \Theta$, donde Θ es el espacio de parámetros. Para que la salida $f(\boldsymbol{x}, \theta)$ coincida bien con la etiqueta $\boldsymbol{y}$, nos interesa minimizar la *pérdida de entropía cruzada* sobre un conjunto de entrenamiento $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^N$:[22]

$$\min_{\theta\in\Theta} \mathbb{E}[\mathrm{CE}(\boldsymbol{y} \parallel f(\boldsymbol{x}, \theta))] \approx -\frac{1}{N}\sum_{i=1}^{N} \langle \boldsymbol{y}_i, \log\left(f(\boldsymbol{x}_i, \theta)\right)\rangle. \tag{4.2.2}$$

Los parámetros óptimos θ de la red suelen obtenerse optimizando el objetivo anterior mediante un método eficiente de descenso por gradientes, en el que los gradientes se calculan por retropropagación, tal y como se describe en la Sección **??** del **??**.

A pesar de su eficacia y su enorme popularidad, este enfoque presenta dos limitaciones importantes: (1) Su único objetivo es predecir las etiquetas $\boldsymbol{y}$, incluso si estas pudieran estar mal etiquetadas. Los estudios empíricos muestran que las redes profundas, utilizadas como una «caja negra», pueden

[20] La clasificación es el ámbito en el que el aprendizaje profundo mostró sus primeros éxitos, lo que desató un interés explosivo por las redes profundas. Aunque nuestro estudio se centra en la clasificación, creemos que las ideas y los principios pueden generalizarse en forma natural a otros contextos, como la regresión.

[21] Por ahora, no nos preocupemos por qué red debemos usar aquí y por qué. El objetivo aquí es considerar cualquier red profunda probada empíricamente. Dejaremos la justificación de las arquitecturas de red para el próximo capítulo.

[22] Aquí interpretamos $\boldsymbol{y}$ como una distribución de probabilidad *one-hot* concreta sobre $[K] = \{1, 2, \ldots, K\}$, y $f(\boldsymbol{x}, \theta) \in \Delta([K]) \subseteq \mathbb{R}^K$ como una distribución de probabilidad sobre el mismo conjunto, que por lo tanto puede compararse mediante la entropía cruzada a pesar de estar representada computacionalmente como vectores en $\mathbb{R}^K$.

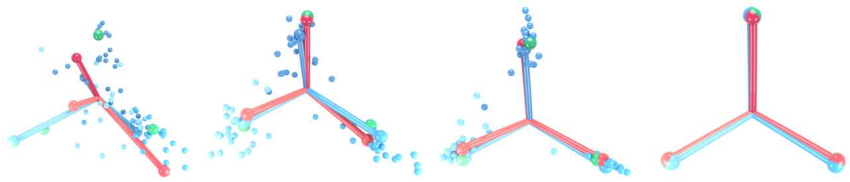


Figura 4.12: Evolución de las salidas de la penúltima capa de una red neuronal VGG13 cuando se entrena con el conjunto de datos CIFAR-10 utilizando 3 clases seleccionadas al azar. Figura tomada de [**papyan2020prevalence**].

incluso adaptarse a etiquetas aleatorias [**zhang2017understanding**]. (2) Con este tipo de ajuste de datos de extremo a extremo, a pesar de los numerosos esfuerzos empíricos por interpretar las características así aprendidas, no queda claro hasta qué punto las características intermedias aprendidas por la red captan las estructuras intrínsecas de los datos que hacen posible, en primer lugar, una clasificación significativa. Las propiedades geométricas y estadísticas precisas de las características aprendidas también suelen quedar ocultas, lo que da lugar a una falta de interpretabilidad y a la consiguiente ausencia de garantías de rendimiento (por ejemplo, generalización, transferibilidad y robustez, etc.) en el aprendizaje profundo. Por lo tanto, *uno de los objetivos de esta sección es abordar dichas limitaciones reformulando el objetivo para aprender representaciones explícitamente significativas y útiles para los datos* $\boldsymbol{x}$*, sin limitarse a la clasificación.*

Características discriminativas mínimas mediante cuello de botella de información. Un enfoque habitual para interpretar el papel de las redes profundas consiste en considerar que las salidas de las capas intermedias de la red seleccionan ciertas características latentes $\boldsymbol{z} = f(\boldsymbol{x}, \theta) \in \mathbb{R}^d$ de los datos que permiten distinguir entre varias clases. Las representaciones aprendidas $\boldsymbol{z}$ facilitan entonces la posterior tarea de clasificación para predecir la etiqueta de clase $\boldsymbol{y}$ mediante la optimización de un clasificador $g(\boldsymbol{z})$:

$$\boldsymbol{x} \xrightarrow{f(\boldsymbol{x},\theta)} \boldsymbol{z} \xrightarrow{g(\boldsymbol{z})} \boldsymbol{y}. \tag{4.2.3}$$

Sabemos por (4.1.13) que *la información mutua* entre dos variables aleatorias, digamos $\boldsymbol{x}, \boldsymbol{z}$, se define como

$$I(\boldsymbol{x}; \boldsymbol{z}) = H(\boldsymbol{x}) - H(\boldsymbol{x} \mid \boldsymbol{z}), \tag{4.2.4}$$

donde $H(\boldsymbol{x}|\boldsymbol{z})$ es la entropía condicional de $\boldsymbol{x}$ dado $\boldsymbol{z}$. La información mutua también se conoce como *la ganancia de información*: Mide en qué medida se puede reducir la entropía de la variable aleatoria $\boldsymbol{x}$ una vez que se conoce $\boldsymbol{z}$. O, dicho de otro modo, mide cuánta información contiene $\boldsymbol{z}$ sobre $\boldsymbol{x}$. La formulación del *cuello de botella de la información* (IB) [**Tishby-ITW2015**]

postula además que la función de la red es aprender $\boldsymbol{z}$ como la estadística mínima suficiente para predecir $\boldsymbol{y}$. Formalmente, busca maximizar la información mutua $I(\boldsymbol{z}, \boldsymbol{y})$ entre $\boldsymbol{z}$ y $\boldsymbol{y}$, al tiempo que minimiza la información mutua $I(\boldsymbol{x}, \boldsymbol{z})$ entre $\boldsymbol{x}$ y $\boldsymbol{z}$:

$$\max_{\theta \in \Theta} \text{IB}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}) \doteq I(\boldsymbol{z}; \boldsymbol{y}) - \beta I(\boldsymbol{x}; \boldsymbol{z}) \quad \text{s.t. } \boldsymbol{z} = f(\boldsymbol{x}, \theta), \tag{4.2.5}$$

donde $\beta > 0$.

Si se logran superar algunas limitaciones asociadas a este marco [**kolchinsky2018caveats-ICLR2018**], como por ejemplo cómo evaluar con precisión la información mutua con un número finito de muestras provenientes de distribuciones degeneradas, este marco puede resultar útil para explicar ciertos comportamientos de las redes profundas. Por ejemplo, un trabajo reciente [**papyan2020prevalence**] demuestra, de hecho, que las representaciones aprendidas mediante la pérdida de entropía cruzada (4.2.2) presentan un fenómeno de *colapso neuronal*. Es decir, las características de cada clase se mapean a un vector unidimensional, mientras que el resto de la información de la clase se suprime, tal y como se ilustra en la Figura 4.12.[23]

Nota 4.4 (Colapso Neuronal). El colapso neuronal se refiere a un fenómeno observado en redes neuronales profundas entrenadas para la clasificación, en el que las representaciones de características aprendidas y los pesos del clasificador muestran un comportamiento altamente simétrico y estructurado durante la fase final del entrenamiento [**papyan2020prevalence**; **zhu2021geometric**; **yaras2022neural**]. Concretamente, dentro de cada clase, las características colapsan hacia la media de su clase y entre las clases, estas medias se separan al máximo, formando una configuración equiangular en forma de simplex. El clasificador lineal se alinea con la media de la clase, salvo por un cambio de escala. Además, el clasificador de la última capa converge en elegir aquella clase cuya media de entrenamiento sea la más cercana. El colapso neuronal revela conexiones profundas entre la dinámica de la optimización, la generalización y las estructuras geométricas que surgen en el aprendizaje supervisado.

A partir del ejemplo de clasificación anterior, vemos que la representación así aprendida $\boldsymbol{z}$ de $\boldsymbol{x}$ da lugar a un codificador muy sencillo que, en esencia, asigna a cada clase de datos una sola palabra de código: el vector «one-hot» que representa cada clase. Desde el punto de vista de la compresión con pérdidas, un codificador de este tipo presenta demasiadas pérdidas como para conservar la información en la distribución de los datos. Otra información, como la que resulta útil para tareas tales como la segmentación de imágenes, el completamiento de imágenes o la generación de imágenes, se pierde en gran medida en un proceso de aprendizaje supervisado de este tipo.

Para remediar esta situación, queremos aprender un esquema de codificación diferente de tal manera que la representación de características resultante $\boldsymbol{z}$ pueda capturar tanta información como sea posible sobre la distribución de los

[23] Observaremos este fenómeno en experimentos con datos de imágenes reales más adelante en el Sección 4.3.1.

datos, sin limitarse únicamente a la información útil para la clasificación. Más concretamente, queremos que la representación $\boldsymbol{z} = f(\boldsymbol{x}, \theta)$ sea tal que exista un decodificador $g(\boldsymbol{z}, \eta)$ de modo que:

$$\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2 \leq \epsilon \quad \text{s.t.} \quad \hat{\boldsymbol{x}} = g(f(\boldsymbol{x})). \tag{4.2.6}$$

Introduciremos formalmente dicho esquema de codificación-decodificación como un «autocodificador» y dicha representación $\boldsymbol{z}$ como una representación «consistente» en el **??**. Por ahora, el lector puede considerarlo simplemente como una generalización del código de cuantización $\hat{\boldsymbol{x}}$ introducido en la sección anterior para la tasa-distorsión (en el dominio de datos nativos $\boldsymbol{x}$).

En la formulación del cuello de botella de información (4.2.5), no se requiere que la representación de características aprendida $\boldsymbol{z}$ sea consistente. Por lo tanto, como mínimo debería fijarse un objetivo de aprendizaje (consistente) de representaciones (RL):

$$\begin{aligned} &\max_{\theta,\eta} \ \mathrm{RL}(\boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z}) \doteq I(\boldsymbol{z}; \boldsymbol{y}) - \beta I(\boldsymbol{x}; \boldsymbol{z}) \\ &\text{s.t.} \quad \boldsymbol{z} = f(\boldsymbol{x}, \theta), \quad \|\boldsymbol{x} - g(\boldsymbol{z}, \eta)\|_2 \leq \epsilon. \end{aligned} \tag{4.2.7}$$

Resolver el problema anterior en el contexto más general es una tarea muy desalentadora. En general, ni siquiera sabemos qué clases de codificadores o decodificadores podrían garantizar la consistencia[24] Además, dado que desconocemos la distribución de la representación $\boldsymbol{z}$, calcular la información mutua $I(\boldsymbol{z}; \boldsymbol{y})$ puede resultar muy complicado, si no imposible.

Antes de estudiar cómo abordar este problema en contextos generales (en los capítulos siguientes), en el resto de este capítulo mostraremos cómo resolverlo, de manera limitada, en el caso particular en el que la distribución de los datos se compone de clases bien diferenciadas (de baja dimensión).

4.2.2 Representaciones discriminantes lineales

Que los datos proporcionados $\boldsymbol{X}$ de una distribución mixta $\mathcal{D}$ puedan clasificarse o agruparse de manera efectiva depende de cuán separables (o discriminantes) sean (o puedan hacerse) las distribuciones componentes $\mathcal{D}_k$. Una hipótesis de trabajo muy extendida es que la distribución de cada clase presenta estructuras intrínsecas de *dimensión* relativamente *baja*. Por lo tanto, podemos suponer que la distribución $\mathcal{D}_k$ de cada clase tiene su soporte en una subvariedad de baja dimensión, digamos $\mathcal{M}_k$ con dimensión $d_k \ll D$, y que la distribución $\mathcal{D}$ de $\boldsymbol{x}$ está definida sobre la mezcla de esas subvariedades, $\mathcal{M} = \cup_{k=1}^{K} \mathcal{M}_k$, en el espacio ambiental de alta dimensión $\mathbb{R}^D$, como se ilustró anteriormente en la Figura 4.1.

No solo necesitamos identificar la distribución de baja dimensionalidad, sino que también queremos representar la distribución de una forma que facilite al máximo las tareas posteriores, como la clasificación, la agrupamiento en

[24] En la práctica, las personas tienden a aprenderlas, al margen del objetivo mencionado anteriormente, mediante ensayo y error, como veremos con más detalle en el **??**.

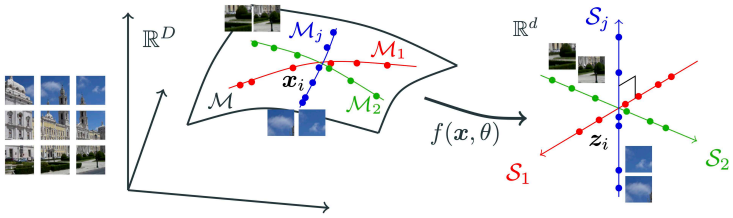


Figura 4.13: La distribución $\mathcal{D}$ de datos de alta dimensión $\boldsymbol{x} \in \mathbb{R}^D$ tiene su soporte en una variedad $\mathcal{M}$ y sus clases se encuentran en subvariedades de baja dimensión $\mathcal{M}_k$. Nuestro objetivo es aprender una aplicación $f(\boldsymbol{x}, \theta)$ parametrizada por θ tal que $\boldsymbol{z}_i = f(\boldsymbol{x}_i, \theta)$ se encuentre en una unión de subespacios con máxima descorrelación $\{\mathcal{S}_k\}$.

clústeres y la generación condicionada (como veremos más adelante). Para ello, necesitamos que nuestras representaciones de características aprendidas tengan las siguientes propiedades:

1. *Comprimibilidad intraclase:* Las características de las muestras de la misma clase deben estar fuertemente *correlacionadas*, en el sentido de que pertenezcan a un subespacio lineal de baja dimensión.

2. *Discriminación entre clases:* Las características de las muestras de diferentes clases deben estar muy *descorrelacionadas* y pertenecer a subespacios lineales de baja dimensión diferentes e incoherentes.

3. *Representación con máxima diversidad:* La dimensión (o varianza) de las características de cada clase debe ser *lo más grande posible*, siempre y cuando sean incoherentes con respecto a las de las demás clases.

Nos referimos a dicha representación como la *representación discriminativa lineal* (LDR).

Obsérvese que la primera propiedad se ajusta bien al el objetivo del clásico *análisis de componentes principales* (PCA) que hemos expuesto en la Sección 2.1.1. En esencia, esto significa que las características de las muestras que pertenecen a la misma clase[25] deben volverse o hacerse altamente correlacionadas. En otras palabras, la representación de características debe ser *contractiva* para las muestras que son similares.

La segunda propiedad exige que, en el caso de las muestras que pertenecen a clases diferentes o que se presumen que son distintas entre sí, sus características deben hacerse (geométricamente) incoherentes o (estadísticamente) independientes, tal y como se ilustra anteriormente en la Figura 4.1. La Figura 4.13 ilustra cuándo la distribución de los datos es, en realidad, una mezcla de subvariedades de baja dimensión. Mediante la compresión (eliminación de ruido o agrupación), identificamos en primer lugar la

[25] incluidas las instancias augmentadas de las mismas muestras

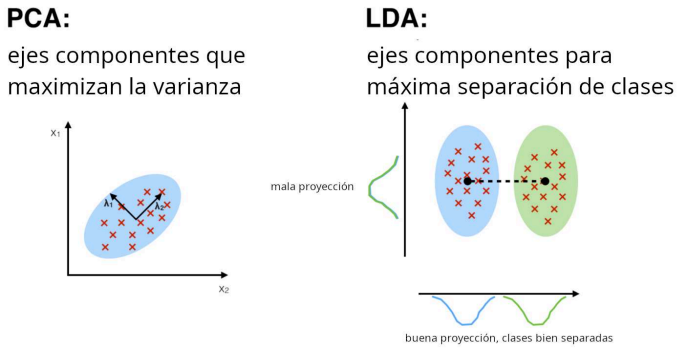


Figura 4.14: Comparación entre el PCA y el LDA. Figuras tomadas de https://sebastianraschka.com/Articles/2014_python_lda.html.

distribución real de los datos (izquierda). A continuación, queremos transformar la distribución de tal manera que las subvariedades acaben convirtiéndose en subespacios lineales mutuamente incoherentes o independientes (derecha).

Nota 4.5 (Análisis discriminante lineal). Se puede observar que el segundo criterio se asemeja al del clásico *análisis discriminante lineal* (LDA) [**HastieTiFr09**]. El análisis discriminante lineal (LDA) [**HastieTiFr09**] es una técnica supervisada de reducción de dimensionalidad cuyo objetivo es encontrar una proyección lineal de los datos que maximice la separabilidad de las clases. En concreto, a partir de datos etiquetados, el LDA busca una transformación lineal que proyecte las entradas de alta dimensión en un espacio de menor dimensión en el que las clases estén separadas al máximo. Tengamos en cuenta que el PCA es un método no supervisado que proyecta los datos en direcciones de máxima varianza sin tener en cuenta las etiquetas de clase. Mientras que el PCA se centra exclusivamente en preservar la estructura global de la varianza, el LDA aprovecha explícitamente la información de las etiquetas para mejorar el poder discriminativo; véase la comparación en la Figura 4.14. Pero el LDA solo realiza una proyección lineal de los datos originales. Como veremos, el LDR puede ser más general: busca una transformación, probablemente no lineal, para que la representación resultante sea lineal e incoherente.

Nota 4.6 (Aprendizaje contrastivo). Cabe señalar que el *aprendizaje contrastivo* convencional [**hadsell2006dimensionality**; **Van__den__Oord2018-qf**; **he2019momentum**] pretende alcanzar objetivos similares en cuanto a la representación de características, tal y como establecen las dos primeras propiedades. En el caso de datos que pertenecen a k clases diferentes, un par elegido al azar $(\boldsymbol{x}_i, \boldsymbol{x}_j)$ tiene una alta probabilidad de pertenecer a clases diferentes si k es grande.[26] Por lo tanto, es deseable que la representación $\boldsymbol{z}_i = f(\boldsymbol{x}_i, \boldsymbol{\theta})$ de una muestra $\boldsymbol{x}_i$ sea altamente incoherente con respecto a las

[26] Por ejemplo, cuando $k \geq 100$, un par aleatorio tiene un 99% de probabilidad de pertenecer a clases diferentes.

$\boldsymbol{z}_j$ de otras muestras $\boldsymbol{x}_j$, mientras que coherente con la característica de su versión transformada $\tau(\boldsymbol{x}_i)$, denotada como $\boldsymbol{z}(\tau(\boldsymbol{x}_i))$ para τ en cierto conjunto dado de aumentos $\mathcal{T}$. Por lo tanto, se propuso de manera heurística que, para fomentar el poder discriminativo de la representación aprendida, se podría intentar minimizar la denominada *pérdida contrastiva*:

$$\min_{\boldsymbol{\theta}} -\log \frac{\exp(\langle \boldsymbol{z}_i, \boldsymbol{z}(\tau(\boldsymbol{x}_i))\rangle)}{\sum_{j \neq i} \exp(\langle \boldsymbol{z}_i, \boldsymbol{z}_j \rangle)}, \tag{4.2.8}$$

que es pequeña siempre que el producto interno $\langle \boldsymbol{z}_i, \boldsymbol{z}(\tau(\boldsymbol{x}_i))\rangle$ sea grande y $\langle \boldsymbol{z}_i, \boldsymbol{z}_j \rangle$ sea pequeño para $i \neq j$. Esta pérdida contrastiva y muchas de sus variantes se han utilizado ampliamente en métodos populares de aprendizaje de características (autosupervisado), como las características de tipo DINO [**caron2021unsupervisedlearningvisualfeatures**; **DINOv2**] para datos visuales. Como veremos, basándonos en el marco de codificación con pérdidas estudiado en la sección anterior, podremos llegar a una función objetivo fundamentada que promueva ambas propiedades y, al mismo tiempo, simplifique significativamente prácticas como el aprendizaje de las características de tipo DINO (tal y como lo expondremos con gran detalle en el **??**).

La tercera propiedad también es importante porque queremos que las características aprendidas revelen todas las causas posibles por las que una clase es diferente de todas las demás. Por ejemplo, para distinguir una «manzana» de una «naranja», no solo nos fijamos en el color, sino también en la forma y en las hojas. Lo ideal sería que la dimensión de cada subespacio $\{\mathcal{S}_k\}$ fuera igual a la de la subvariedad correspondiente $\mathcal{M}_k$. Esta propiedad será importante si queremos que la función $f(\boldsymbol{x}, \theta)$ sea *invertible* para tareas como la generación de imágenes. Por ejemplo, si tomamos diferentes puntos de muestra del subespacio de características de «manzana», deberíamos poder decodificarlos, mediante una aplicación inversa $g(\boldsymbol{z}, \eta)$, para generar diversas imágenes de manzanas. Este par de aplicaciones mutuamente inversas f y g constituye un autocodificador que estudiaremos con gran detalle en el **??**. Obsérvese que las representaciones de características aprendidas a partir de la minimización de la entropía cruzada (4.2.2) normalmente no presentan esta propiedad, ya que adolecen del denominado fenómeno del *colapso neuronal* comentado anteriormente (véase la Nota 4.4).

En general, aunque las estructuras intrínsecas de cada clase o grupo puedan ser de baja dimensión, no son en absoluto simplemente lineales (o gaussianas) en su representación original $\boldsymbol{x}$, y es necesario convertirlas primero en lineales mediante alguna transformación no lineal.[27] Por lo tanto, en general, utilizamos la transformación no lineal $f(\boldsymbol{x}, \theta)$ para buscar una representación de los datos tal que los subespacios que representan todas las clases sean subespacios lineales máximamente incoherentes. Para ser más precisos, queremos aprender una función $\boldsymbol{z} = f(\boldsymbol{x}, \theta)$ que mapee cada una de las subvariedades $\mathcal{M}_k \subset \mathbb{R}^D$ (Figura 4.13 izquierda) a un subespacio *lineal* $\mathcal{S}_k \subset \mathbb{R}^d$ (Figura 4.13 derecha). En

[27] Analizaremos cómo se puede hacer esto de manera explícita en el **??**.

cierta medida, los subespacios múltiples resultantes $\{\mathcal{S}_k\}$ pueden considerarse como *componentes principales generalizadas* discriminantes [**GPCA**] o, si son ortogonales, *componentes independientes* [**hyvarinen2000independent**] de las características resultantes $\boldsymbol{z}$ para los datos originales $\boldsymbol{x}$. Como veremos en el siguiente Capítulo 5, las redes profundas desempeñan precisamente la función de modelar y llevar a cabo esta transformación no lineal de la distribución de los datos a representaciones discriminantes lineales.

4.2.3 El principio de reducción máxima de la tasa de codificación

Aunque las tres propiedades —*discriminación entre clases*, *compresibilidad intraclase* y *representación de máxima diversidad* — de las representaciones discriminativas lineales (LDR) son todas propiedades muy deseables de la representación aprendida $\boldsymbol{z}$, no son en absoluto fáciles de obtener: ¿Son estas propiedades compatibles entre sí, de modo que podamos esperar alcanzarlas todas a la vez? Si es así, ¿existe algún objetivo *sencillo pero basado en principios* que permita evaluar la calidad de las representaciones resultantes en función de todas estas propiedades? La clave para responder a estas preguntas es encontrar una «medida de compacidad» o «ganancia de información» fundamentada para la distribución de una variable aleatoria $\boldsymbol{z}$ o a partir de un número finito de muestras $\{\boldsymbol{z}_i\}_{i=1}^N$ de la misma. Dicha medida debería caracterizar de manera directa y precisa las propiedades geométricas o estadísticas intrínsecas de la distribución, en términos de su dimensión intrínseca o volumen. A diferencia de la entropía cruzada (4.2.2) o el cuello de botella de información (4.2.5), dicha medida no debe depender exclusivamente de las etiquetas de clase, de modo que pueda funcionar en entornos más generales, como los supervisados, autosupervisados, semisupervisados y no supervisados (como veremos en secciones posteriores).

Sin pérdida de generalidad, supongamos que la distribución $\mathcal{D}$ del vector aleatorio $\boldsymbol{x}$ se define sobre una mezcla de distribuciones, es decir, $\mathcal{D} = \cup_{k=1}^K \mathcal{D}_k$, donde cada $\mathcal{D}_k \subset \mathbb{R}^D$ tiene una dimensión intrínseca baja en el espacio ambiente de alta dimensión $\mathbb{R}^D$. Sea $\boldsymbol{X}_k \in \mathbb{R}^{D\times N_k}$ la matriz de datos cuyas columnas son muestras extraídas de la distribución $\mathcal{D}_k$, donde N_k indica el número de muestras para cada $k = 1, \ldots, K$. Entonces, utilizamos $\boldsymbol{X} = [\boldsymbol{X}_1, \ldots, \boldsymbol{X}_K] \in \mathbb{R}^{D\times N}$ para denotar todas las muestras, donde $N = \sum_{k=1}^K N_k$. Recordemos que también utilizamos $\boldsymbol{x}_i$ para denotar la i-ésima muestra de $\boldsymbol{X}$, es decir, $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$. Bajo una aplicación de codificación:

$$\boldsymbol{x} \xrightarrow{f(\boldsymbol{x})} \boldsymbol{z}, \tag{4.2.9}$$

las muestras de entrada se mapean a $\boldsymbol{z}_i = f(\boldsymbol{x}_i)$ para cada $i = 1, \ldots, N$. Abusando un poco de la notación, también escribimos $\boldsymbol{Z}_k = f(\boldsymbol{X}_k)$ y $\boldsymbol{Z} = f(\boldsymbol{X})$. Por lo tanto, tenemos $\boldsymbol{Z} = [\boldsymbol{Z}_1, \ldots, \boldsymbol{Z}_K]$ y $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots \boldsymbol{z}_N]$.

Por un lado, para que las características aprendidas sean discriminativas, es preferible que las características de las diferentes clases o clúster sean *lo*

más incoherentes posible entre sí. Por lo tanto, en conjunto deberían abarcar un espacio del mayor volumen (o dimensión) posible y la tasa de codificación de todo el conjunto $\boldsymbol{Z}$ debería ser lo más alta posible. Por otro lado, las características aprendidas de una misma clase o clúster deberían estar muy correlacionadas y ser coherentes. Por lo tanto, cada clase o grupo solo debería abarcar un espacio (o subespacio) de un volumen muy reducido y la tasa de codificación debe ser lo más baja posible. A continuación, explicaremos cómo medir la tasa de codificación de las características aprendidas.

La tasa de codificación de las características. Cabe destacar que una dificultad práctica a la hora de evaluar la tasa de codificación es que la distribución subyacente de las representaciones de las características $\boldsymbol{Z}$ suele ser desconocida. Para solucionar esto, podemos aproximar las características $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N]$ como muestras extraídas de una distribución gaussiana multivariante. Bajo esta hipótesis, tal y como se analiza en la Sección 4.1.3, la compacidad de las características $\boldsymbol{Z}$ *en su conjunto* puede medirse en términos de la longitud media de codificación por muestra, denominada *tasa de codificación*, sujeta a un nivel de precisión $\epsilon > 0$ (véase (4.1.30)) definida de la siguiente manera:

$$R_\epsilon(\boldsymbol{Z}) = \frac{1}{2}\operatorname{logdet}\left(\boldsymbol{I} + \frac{d}{N\epsilon^2}\boldsymbol{Z}\boldsymbol{Z}^\top\right). \tag{4.2.10}$$

Por otro lado, esperamos que una transformación no lineal $f(\boldsymbol{x})$ asigne a cada subvariedad específica de clase $\mathcal{M}_k \subset \mathbb{R}^D$ un subespacio lineal maximamente incoherente $\mathcal{S}_k \subset \mathbb{R}^D$, de manera que las características aprendidas $\boldsymbol{Z}$ se encuentren en una unión de subespacios de baja dimensión. Esta estructura permite una evaluación más precisa de la tasa de codificación mediante el análisis separado de cada subespacio. Recordemos que las columnas de $\boldsymbol{Z}_k$ representan las características de las muestras en $\boldsymbol{X}_k$ para cada $k = 1, \ldots, K$. La tasa de codificación de las características en $\boldsymbol{Z}_k$ se puede calcular de la siguiente manera:

$$R_\epsilon(\boldsymbol{Z}_k) = \frac{N_k}{2N}\log\det\left(\boldsymbol{I} + \frac{d}{N_k\epsilon^2}\boldsymbol{Z}_k\boldsymbol{Z}_k^\top\right) \tag{4.2.11}$$

Entonces, la suma de las tasas de codificación promedio de las características de cada clase es

$$R_\epsilon^c(\boldsymbol{Z}) \doteq \sum_{k=1}^{K} R_\epsilon(\boldsymbol{Z}_k), \tag{4.2.12}$$

Por lo tanto, una buena representación $\boldsymbol{Z}$ de $\boldsymbol{X}$ es aquella que logra una gran diferencia entre la tasa de codificación del conjunto y la de todas las clases:

$$\Delta R_\epsilon(\boldsymbol{Z}) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}). \tag{4.2.13}$$

Obsérvese que, tal y como hemos comentado anteriormente en este capítulo, esta diferencia puede interpretarse como la cantidad de «información ganada» al identificar los clústeres de baja dimensión correctos $\boldsymbol{Z}_k$ dentro del conjunto global $\boldsymbol{Z}$.

Si elegimos que nuestra función de mapeo de características $f(\cdot)$ sea una red neuronal profunda $f(\cdot, \theta)$ con parámetros de red θ, el proceso general de representación de características y la consiguiente reducción de la tasa se pueden ilustrar mediante el siguiente diagrama:

$$\boldsymbol{X} \xrightarrow{f(\boldsymbol{x},\theta)} \boldsymbol{Z} \xrightarrow{\epsilon} \Delta R_\epsilon(\boldsymbol{Z}). \tag{4.2.14}$$

Recordemos que ΔR_ϵ es *monótono* en la escala de las características $\boldsymbol{Z}$. Para garantizar una comparación justa entre las diferentes representaciones, es esencial *normalizar la escala* de las características aprendidas. Esto se puede lograr ya sea imponiendo que la norma de Frobenius de cada clase $\boldsymbol{Z}_k$ sea proporcional al número de características en $\boldsymbol{Z}_k \in \mathbb{R}^{d\times N_k}$, es decir, $\|\boldsymbol{Z}_k\|_F^2 = N_k$, o bien normalizando cada característica para que se encuentre en la esfera unitaria, es decir, $\boldsymbol{z}_i \in \mathbb{S}^{d-1}$, donde N_k denota el número de muestras en la clase k-ésima. Esta formulación ofrece una justificación natural de la necesidad de la «normalización por lotes» en la práctica del entrenamiento de redes neuronales profundas [**ioffe2015batch**].

Una vez que las representaciones son comparables, el objetivo pasa a ser el de aprender un conjunto de características $\boldsymbol{Z} = f(\boldsymbol{X}, \theta)$ tal que maximicen la reducción entre la tasa de codificación de todas las características y la de la suma de características con respecto a sus clases:

$$\begin{aligned} &\max_\theta \Delta R_\epsilon\big(\boldsymbol{Z}\big) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}), \\ &\text{s.t.} \quad \boldsymbol{Z} = f(\boldsymbol{X},\theta),\ \|\boldsymbol{Z}_k\|_F^2 = N_k,\ k = 1,\ldots,K. \end{aligned} \tag{4.2.15}$$

Denominamos esto el principio de la *reducción máxima de la tasa de codificación* (MCR^2), como una encarnación de la famosa cita de Aristóteles:

"*El todo es más que la suma de sus partes.*"

Para aprender la mejor representación, se requiere que *la totalidad sea superior en la máxima medida a la suma de sus partes.* En gran medida, la cantidad ΔR puede considerarse como una realización computable del término de información mutua $I(\boldsymbol{z}; \boldsymbol{y})$ introducido en el objetivo (4.2.7).

Ejemplo 4.11. Para aclarar mejor estas ideas, analicemos el ejemplo que se muestra en la Figura 4.15. Desde el punto de vista de la compresión, la representación de la derecha es *la más compacta* en el sentido de que la diferencia entre la tasa de codificación cuando todas las características se codifican como una sola gaussiana (azul) y la que se obtiene cuando las características se agrupan adecuadamente y se codifican como dos subespacios separados (verde) es máxima.[28] ■

Tengamos presente que el principio MCR^2 mencionado anteriormente está diseñado para problemas de aprendizaje supervisado, en los que se conocen las pertenencias a grupos (o etiquetas de clase). Existen diversas formas de

[28] Intuitivamente, el cociente entre el «volumen» de todo el espacio abarcado por todas las características y el que realmente ocupan las características es máximo.

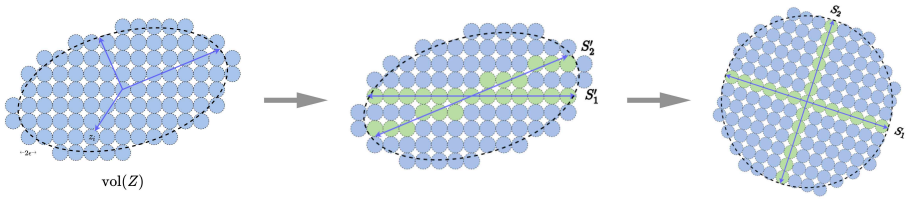


Figura 4.15: Tras identificar la distribución de los datos en baja dimensión, deseamos dar un paso más en la transformación de la distribución de los datos en una representación estructurada más informativa: R es el número de bolas de radio ϵ que cubren todo el espacio y R^c es la suma de los números correspondientes a todos los subespacios (las bolas verdes). ΔR es la diferencia entre ambas (el número de bolas azules).

aplicar este principio, o generalizaciones adecuadas, de manera que se eluda este requisito de las etiquetas de clase; analizaremos estas opciones más adelante en este capítulo y en el siguiente Capítulo 5. Para tener un primer acercamiento, extendemos el principio a problemas de aprendizaje no supervisado mediante la introducción de una matriz de pertenencia, que codifica la asignación (potencialmente suave) de cada punto de datos a grupos o clústeres latentes. Concretamente, sea $\boldsymbol{\Pi} = \{\boldsymbol{\Pi}_k\}_{k=1}^K \subset \mathbb{R}^{N\times N}$ un conjunto de matrices diagonales cuyos elementos diagonales codifican la pertenencia de las N muestras a K clases. Es decir, $\boldsymbol{\Pi}$ se encuentra en un simplex $\Omega \doteq \{\boldsymbol{\Pi} : \boldsymbol{\Pi}_k \geq \mathbf{0} : \sum_{k=1}^K \boldsymbol{\Pi}_k = \boldsymbol{I}_N\}$. Entonces, podemos definir la tasa de codificación promedio con respecto a la partición $\boldsymbol{\Pi}$ como

$$R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{\Pi}) \doteq \sum_{k=1}^K \frac{\operatorname{tr}(\boldsymbol{\Pi}_k)}{2N} \log\det\left(\boldsymbol{I} + \frac{d}{\operatorname{tr}(\boldsymbol{\Pi}_k)\epsilon^2}\boldsymbol{Z}\boldsymbol{\Pi}_k\boldsymbol{Z}^\top\right). \tag{4.2.16}$$

Dado $\boldsymbol{Z}$, $R_\epsilon^c(\boldsymbol{Z}|\boldsymbol{\Pi})$ es una función cóncava de $\boldsymbol{\Pi}$. En consecuencia, el principio MCR^2 para problemas de aprendizaje no supervisado se formula de la siguiente manera:

$$\begin{aligned} &\max_{\boldsymbol{\Pi},\theta} \Delta R_\epsilon\big(\boldsymbol{Z} \mid \boldsymbol{\Pi}\big) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{\Pi}) \\ &\text{s.t.} \quad \boldsymbol{Z} = f(\boldsymbol{X},\theta),\ \|\boldsymbol{Z}\boldsymbol{\Pi}_k\|_F^2 = N_k,\ k = 1,\ldots,K,\ \boldsymbol{\Pi} \in \Omega. \end{aligned} \tag{4.2.17}$$

En comparación con (4.2.15), esta formulación permite la optimización conjunta tanto de las pertenencias a grupos como de los parámetros de la red. En particular, cuando $\boldsymbol{\Pi}$ se establece como una matriz de pertenencia a grupos que asigna N puntos de datos a K grupos, el problema (4.2.17) puede recuperar el problema (4.2.15).

Nota 4.7 (Cálculo de las tasas de codificación). Es un resultado clásico que calcular el logaritmo del determinante de una matriz $n \times n$ requiere un tiempo de orden $\mathcal{O}(n^3)$. Por lo tanto, calcular las tasas de codificación ΔR_ϵ o R_ϵ

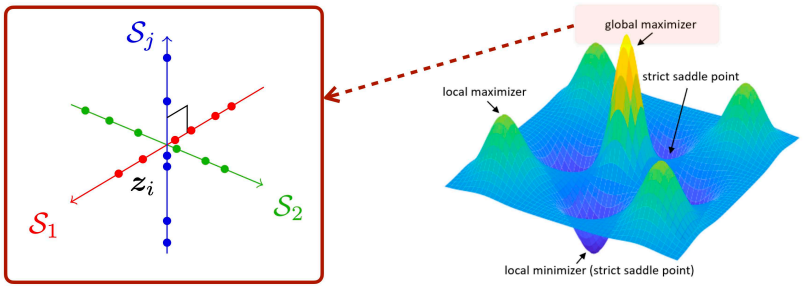


Figura 4.16: **Paisaje de optimización local:** Según el Teorema 4.2, el máximo global del objetivo de reducción de la tasa corresponde a una solución con subespacios mutuamente incoherentes.

(etc.) parece una tarea inmanejable a la escala del análisis de datos moderno. Hay varias formas de evadirlo, que utilizaremos sin mayor explicación en los experimentos que sirven de base al resto del libro.

- *Utilizar la estructura de la tasa de codificación.* Supongamos que $\boldsymbol{Z}$ es un elemento de $\mathbb{R}^{d\times n}$. Entonces se cumple que

$$\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top)=\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}^\top\boldsymbol{Z}) \tag{4.2.18}$$

  por lo que calcular el producto matricial y las tasas de codificación requiere un tiempo del orden de $\min\{d^2n+d^3, dn^2+n^3\}$.

- *Utilizar la simetría de la matriz de Gram.* Obsérvese que $\boldsymbol{Z}\boldsymbol{Z}^\top$ (y, de manera similar, $\boldsymbol{Z}^\top\boldsymbol{Z}$) es una matriz simétrica. Por lo tanto, resulta relativamente sencillo (aunque sigue siendo de tiempo cúbico) calcular sus valores propios $\lambda_i(\boldsymbol{Z}\boldsymbol{Z}^\top)$. Entonces, el logaritmo del determinante se puede expresar de la siguiente manera:

$$\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top)=\sum_{i=1}^{d}\log(1+\alpha\lambda_i(\boldsymbol{Z}\boldsymbol{Z}^\top)), \tag{4.2.19}$$

  (y análogamente para $\boldsymbol{Z}^\top\boldsymbol{Z}$). Sin embargo, este método se vuelve innecesario ante el siguiente, que aprovecha mejor la estructura de los datos de entrada y ofrece mejores resultados.

- *Utilizar la definitud positiva de los términos que intervienen en el determinante logarítmico.* Dado que $\boldsymbol{Z}\boldsymbol{Z}^\top$ (y, de manera análoga, $\boldsymbol{Z}^\top\boldsymbol{Z}$) es simétrico y semidefinido positivo, la cantidad $\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top$ es simétrica y definida positiva. Esto significa que podemos calcular la descomposición de Cholesky $\boldsymbol{L}\boldsymbol{L}^\top=\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top$, donde $\boldsymbol{L}\in\mathbb{R}^{d\times d}$ es una matriz triangular inferior con entradas diagonales positivas. Entonces

$$\operatorname{logdet}(\boldsymbol{I}+\alpha\boldsymbol{Z}\boldsymbol{Z}^\top)=2\sum_{i=1}^{d}\log(L_{ii}). \tag{4.2.20}$$

En la práctica, este método suele ser el más económico, especialmente cuando también se decide estratégicamente si calcular $\boldsymbol{Z}\boldsymbol{Z}^\top$ o $\boldsymbol{Z}^\top\boldsymbol{Z}$, aunque su complejidad temporal sigue siendo cúbica.

4.2.4 Propiedades de optimización en la reducción de la tasa de codificación

En este apartado, estudiamos las propiedades de optimización de la función MCR^2 mediante el análisis de sus soluciones óptimas y la estructura de su paisaje de optimización. Para sortear la dificultad técnica que plantean las redes neuronales, consideramos una versión simplificada del problema (4.2.15) de la siguiente manera:

$$\max_{\boldsymbol{Z}} \; R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}) \qquad \text{s.t.} \quad \|\boldsymbol{Z}_k\|_F^2 = N_k, \; k = 1, \ldots, K. \tag{4.2.21}$$

En teoría, el principio MCR^2 (4.2.21) se caracteriza por una gran generalizabilidad y puede aplicarse a representaciones $\boldsymbol{Z}$ de *cualquier* distribución, siempre y cuando las tasas R_ϵ y R_ϵ^c de dichas distribuciones puedan evaluarse con precisión y eficiencia. La representación óptima $\boldsymbol{Z}^*$ debería presentar algunas propiedades geométricas y estadísticas interesantes. Aquí revelamos algunas propiedades útiles de la representación óptima en el caso especial de los subespacios, que tienen muchas aplicaciones importantes en el aprendizaje automático. Cuando la representación deseada para $\boldsymbol{Z}$ consiste en múltiples subespacios, las tasas R_ϵ y R_ϵ^c en (4.2.21) vienen dadas por (4.2.10) y (4.2.12), respectivamente. En el máximo de reducción de tasa, MCR^2 alcanza sus representaciones óptimas, denotadas $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*, \ldots, \boldsymbol{Z}_K^*]$ con $\operatorname{rank}(\boldsymbol{Z}_k^*) \leq d_k$. Se puede demostrar que $\boldsymbol{Z}^*$ tiene las siguientes propiedades deseadas (véase [**yu2020learning**] para un enunciado formal y demostraciones detalladas).

Teorema 4.2 (**Caracterización de las soluciones globalmente óptimas**). *Supongamos que $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*, \ldots, \boldsymbol{Z}_K^*]$ es una solución óptima global del Problema (4.2.21). Se cumplen las siguientes afirmaciones:*

- Discriminación entre clases*: Siempre que el espacio envolvente sea suficientemente grande ($d \geq \sum_{k=1}^K d_k$), los subespacios son todos ortogonales entre sí,* es decir, *$(\boldsymbol{Z}_k^*)^\top \boldsymbol{Z}_l^* = \boldsymbol{0}$ para $k \neq l$.*
- Representación de máxima diversidad*: Siempre que la precisión de codificación sea suficientemente alta, es decir, $\epsilon^4 < c \cdot \min_k \left\{ \frac{N_k}{N} \frac{d^2}{d_k^2} \right\}$, donde $c > 0$ es una constante. Cada subespacio alcanza su dimensión máxima, es decir,* $\operatorname{rank}(\boldsymbol{Z}_k^*) = d_k$*. Además, los $d_k - 1$ valores singulares más grandes de $\boldsymbol{Z}_k^*$ son iguales.*

Este teorema indica que el principio MCR^2 favorece la incrustación de los datos en múltiples subespacios independientes (como se ilustra en Figura 4.16), con características distribuidas *isotrópicamente* en cada subespacio

(salvo, quizás, en una dimensión). Cabe destacar que este teorema también confirma que las características aprendidas por el principio MCR^2 presentan las propiedades discriminativas de baja dimensión deseadas que se analizaron en Sección 4.2.2. Además, de entre todas esas representaciones discriminativas, prefiere aquella de mayor dimensión en el espacio ambiente. Esto difiere sustancialmente del objetivo del cuello de botella de información (4.2.5).

MCR^2 regularizado. El teorema anterior caracteriza las propiedades de los óptimos globales de los objetivos de reducción de tasa. ¿Qué ocurre con otros óptimos, como los locales? Debido a las restricciones de la norma de Frobenius, resulta difícil analizar el problema (4.2.21) desde una perspectiva de la teoría de la optimización. Es por eso que consideramos la formulación lagrangiana de (4.2.21). Esto puede considerarse como una relajación estricta o incluso un problema equivalente al de (4.2.21), cuyas soluciones óptimas coinciden bajo ajustes específicos del parámetro de regularización; véase [**wang2024global**]. En concreto, la formulación que estudiamos, a la que en adelante nos referiremos como el *problema MCR^2 regularizado*, es la siguiente:

$$\max_{\boldsymbol{Z}} \; R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}) - \frac{\lambda}{2}\|\boldsymbol{Z}\|_F^2, \tag{4.2.22}$$

donde $\lambda > 0$ es el parámetro de regularización. Aunque el programa (4.2.22) es extremadamente no cóncavo e implica inversas de matrices en el cálculo de su gradiente, no obstante podemos caracterizar explícitamente sus óptimos locales y globales de la siguiente manera (véase [**wang2024global**] para una demostración detallada).

Teorema 4.3 (**Óptimos locales y globales**). *Sea N_k el número de muestras de entrenamiento en la k-ésima clase para cada $k \in \{1,\dots,K\}$, $N_{\max} \doteq \max\{N_1,\dots,N_K\}$, $\alpha = d/(N\epsilon^2)$, y $\alpha_k = d/(N_k\epsilon^2)$ con $k \in \{1,\dots,K\}$. Dada una precisión de codificación $\epsilon > 0$, si el parámetro de regularización satisface*

$$\lambda \in \left(0, \frac{d(\sqrt{N/N_{\max}} - 1)}{N(\sqrt{N/N_{\max}} + 1)\epsilon^2}\right], \tag{4.2.23}$$

en consecuencia se cumplen las siguientes afirmaciones:
*(i) (***Maximizadores locales***) $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*,\dots,\boldsymbol{Z}_K^*]$ es un maximizador local del problema (4.2.22) si y solo si el k-ésimo bloque admite la siguiente descomposición*

$$\boldsymbol{Z}_k^* = \left(\frac{\eta_k + \sqrt{\eta_k^2 - 4\lambda^2 N/N_k}}{2\lambda\alpha_k}\right)^{1/2} \boldsymbol{U}_k\boldsymbol{V}_k^\top, \tag{4.2.24}$$

donde (a) $r_k = \operatorname{rank}(\boldsymbol{Z}_k^*)$ satisface $r_k \in [0, \min\{N_k, d\})$ y $\sum_{k=1}^K r_k \le \min\{N, d\}$, (b) $\boldsymbol{U}_k \in \mathcal{O}^{d\times r_k}$ cumple que $\boldsymbol{U}_k^\top\boldsymbol{U}_l = \boldsymbol{0}$ para todo $k \neq l$, $\boldsymbol{V}_k \in \mathcal{O}^{N_k\times r_k}$, y (c) $\eta_k := (\alpha_k - \alpha) - \lambda(N/N_k + 1)$ para cada $k \in \{1,\dots,K\}$.

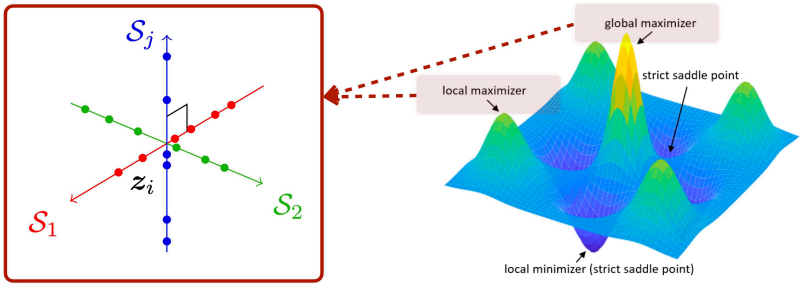


Figura 4.17: **Paisaje de optimización global:** Según [**sun2015nonconvex**; **lee2016gradient**], los Teoremas 4.3 y 4.4, tanto los máximos globales como los locales del objetivo de reducción de la tasa (regularizado) corresponden a una solución con subespacios mutuamente incoherentes. Todos los demás puntos críticos son puntos de silla estrictos.

*(ii) (***Maximizadores globales***)* $\boldsymbol{Z}^* = [\boldsymbol{Z}_1^*, \ldots, \boldsymbol{Z}_K^*]$ *es un maximizador global del problema* (4.2.22) *si y solo si (a) cumple todas las condiciones anteriores y* $\sum_{k=1}^K r_k = \min\{N, d\}$*, y (b) para todo* $k \neq l \in [K]$ *que satisfaga* $N_k < N_l$ *y* $r_l > 0$*, tenemos que* $r_k = \min\{N_k, d\}$*.*

Este teorema caracteriza explícitamente los óptimos locales y globales del problema (4.2.22). Intuitivamente, esto demuestra que las características representadas por cada maximizador local del problema (4.2.22) son de baja dimensión y discriminativas. Aunque hemos caracterizado las soluciones óptimas locales y globales en Teorema 4.3, sigue sin saberse si estas soluciones pueden calcularse de manera eficiente utilizando el algoritmo de descenso por gradiente (GD) para resolver el problema (4.2.22), ya que el GD puede quedarse atascado en otros puntos críticos, como un punto de silla. Afortunadamente, [**sun2015nonconvex**; **lee2016gradient**] demostraron que si una función es dos veces continuamente diferenciable y satisface la *propiedad de silla estricta*, es decir, cada punto crítico ya es, o un maximizador local o bien un punto de silla estricto[29], GD converge casi seguro a un maximizador local con inicialización aleatoria. Investigamos el paisaje de optimización global del problema (4.2.22) caracterizando todos sus puntos críticos de la siguiente manera (véase [**wang2024global**] para obtener demostraciones detalladas).

Teorema 4.4 (**Paisaje benigno de optimización global**)**.** *Dada una precisión de codificación* $\epsilon > 0$*, si el parámetro de regularización satisface* (4.2.23)*, se cumple que cualquier punto crítico* $\boldsymbol{Z}$ *del problema* (4.2.22) *es o bien un maximizador local o bien un punto de silla estricto.*

[29] Decimos que un punto crítico es un punto de silla estricto del problema (4.2.22) si tiene una dirección con curvatura estrictamente positiva [**sun2015nonconvex**]. Esto incluye tanto los puntos de silla clásicos con curvatura estrictamente positiva como los minimizadores locales estrictos.

En conjunto, los dos teoremas anteriores demuestran que las características aprendidas asociadas a cada maximizador local del objetivo de reducción de tasa —y no solo a los maximizadores globales— se estructuran como subespacios incoherentes de baja dimensión. Además, el objetivo de reducción de la tasa (regularizado) (4.2.15) presenta un paisaje muy benigno, con solo maximizadores locales y puntos de silla estrictos como puntos críticos, tal y como se ilustra en Figura 4.17. Según [**sun2015nonconvex**; **lee2016gradient**], los Teoremas 4.3 y 4.4 implican que las representaciones de baja dimensión y discriminativas (LDR) pueden encontrarse de manera eficiente aplicando el descenso por gradiente (estocástico) al objetivo de reducción de tasa (4.2.15) a partir de una inicialización aleatoria. Estos resultados también explican indirectamente por qué en la Sección 4.3.1, si la red elegida es lo suficientemente expresiva y está bien entrenada, la representación resultante suele ofrecer una representación lineal incoherente que probablemente corresponda a la solución óptima global. Se remite a los lectores interesados a [**wang2024global**] para consultar las demostraciones.

4.3 Aprendizaje de representaciones para datos de imagen

4.3.1 Aprendizaje supervisado de representaciones

A continuación, mostramos cómo el objetivo MCR^2 (4.2.15) ayuda a aprender mejores representaciones que la entropía cruzada (CE) (4.2.2) para la clasificación de imágenes. Aquí, para simplificar,[30] adoptamos una popular arquitectura de redes neuronales, la ResNet-18 [**he2016deep**], para modelar el mapeo de características $\boldsymbol{z} = f(\boldsymbol{x}, \theta)$. Optimizamos los parámetros de la red neuronal θ mediante el algoritmo de retropropagación (detallado en la **??**) para maximizar la reducción de la tasa de codificación. Se pueden encontrar más detalles sobre la implementación de este experimento en [**chan2021redunet**].

La Figura 4.18 ilustra cómo evolucionan las dos tasas y su diferencia (tanto para los datos de entrenamiento como para los de prueba) a lo largo de las épocas de entrenamiento: tras una fase inicial, R_ϵ aumenta gradualmente mientras que R_ϵ^c disminuye, lo que indica que las características $\boldsymbol{Z}$ se están expandiendo en su conjunto, mientras que cada clase $\boldsymbol{Z}_k$ se está comprimiendo.

En primer lugar, evaluamos el rendimiento con el conjunto de datos de clasificación de imágenes CIFAR-10 [**krizhevsky2009learning**] y los resultados se muestran en la tabla de la Figura 4.19. Como podemos ver, el objetivo MCR^2 alcanza un rendimiento comparable al de la pérdida de entropía cruzada (CE). Además, el objetivo MCR^2 es más robusto ante la corrupción en las etiquetas de clase, ya que aprende una representación conjunta (un subespacio) para cada clase, en lugar de intentar ajustarse a las etiquetas de las muestras individuales.

[30] Estudiaremos y justificaremos rigurosamente cómo diseñar o derivar una arquitectura profunda adecuada a partir del objetivo MCR^2 y para este en el Capítulo 5.

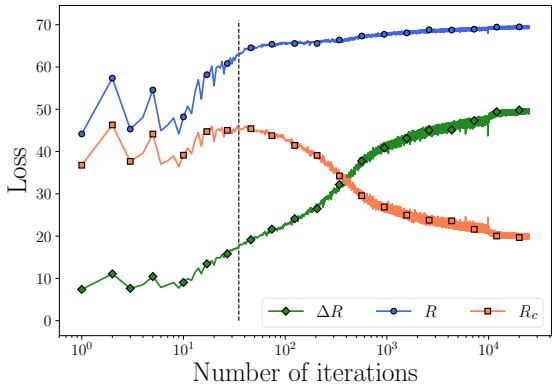


Figura 4.18: Evolución de R_ϵ, R_ϵ^c y ΔR_ϵ durante el proceso de entrenamiento.

	Ratio=0.0	Ratio=0.1	Ratio=0.2	Ratio=0.3	Ratio=0.4	Ratio=0.5
CE Training	0.939	0.909	0.861	0.791	0.724	0.603
MCR^2 Training	**0.940**	**0.911**	**0.897**	**0.881**	**0.866**	**0.843**

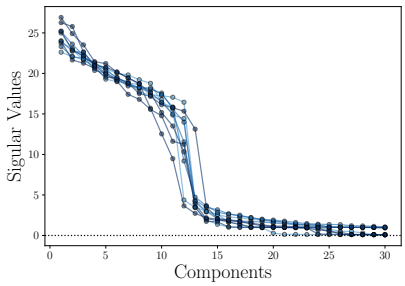
Figura 4.19: Comparación de la exactitud de clasificación entre el entrenamiento con la pérdida de entropía cruzada (CE) o el objetivo MCR^2, con diferentes proporciones de etiquetas de clase corrompidas aleatoriamente.

La Figura 4.20 muestra la distribución de los valores singulares de las características aprendidas $\boldsymbol{Z}_k$ por clase. A partir de la distribución de los valores singulares que se muestra en la Figura 4.20a, vemos que las características $\boldsymbol{z}$ aprendidas con MCR^2 para cada clase se distribuyen en un subespacio de dimensión alrededor de 10-12, mientras que las características de entropía cruzada (véase la Figura 4.20b) son de una dimensión mucho menor.[31]

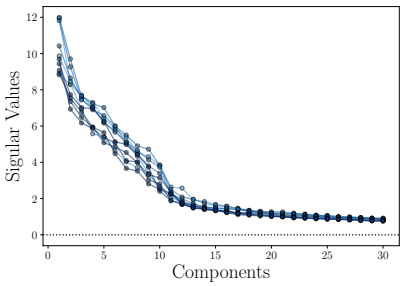
La Figura 4.21 muestra las similitudes coseno entre las características aprendidas, ordenadas por clase. También comparamos las similitudes de las características aprendidas utilizando la entropía cruzada (CE) (4.2.2) y el objetivo MCR^2 (4.2.15). En los gráficos se puede ver claramente que las características aprendidas por el objetivo MCR^2 para las diferentes clases son mucho más incoherentes (o independientes) que las características aprendidas utilizando la pérdida de entropía cruzada. Dentro de cada clase, las características también son menos coherentes y, por lo tanto, más diversas, tal y como indica la distribución de sus valores singulares que se muestra en la Figura 4.20a.

Visualizamos las imágenes que se proyectan a lo largo de diferentes vectores singulares en el subespacio de características de cada clase, a las que denominamos imágenes «principales». La Figura 4.22 muestra estas imágenes

[31] Si se continúa entrenando con más iteraciones, las características de CE podrían incluso colapsar en una sola dimensión, lo que se conoce como el fenómeno del colapso neuronal , ya comentado en la Nota 4.4.

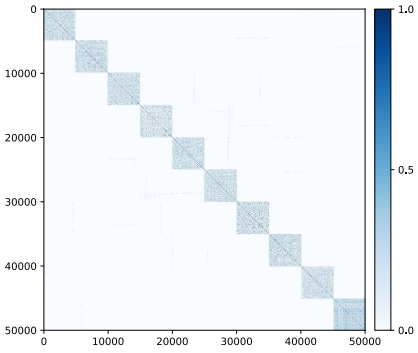


(a) Vectores de componentes principales (PCA) de las características de clases individuales, aprendidas por MCR^2.



(b) PCA de las características de las clases individuales aprendidas mediante la entropía cruzada.

Figura 4.20: Comparación de las componentes principales de las características aprendidas con MCR^2 frente a las obtenidas mediante entropía cruzada.



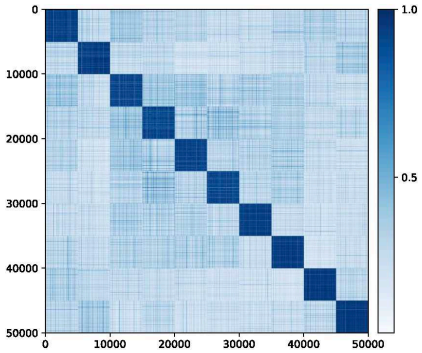


Figura 4.21: Similitud de coseno entre las características aprendidas usando el objetivo MCR^2 (**izquierda**) y la pérdida CE (**derecha**).

principales a lo largo de cada uno de los 10 vectores singulares principales. Como podemos ver, las imágenes correspondientes a cada vector singular son muy similares en cuanto a su aspecto y estructura, mientras que las imágenes correspondientes a diferentes vectores singulares son muy distintas entre sí.

A pesar de que el objetivo MCR^2 parece permitirnos aprender mejores representaciones, hay una aparente falta de justificación de las arquitecturas de red utilizadas en los experimentos anteriores. Aún no está claro por qué la red adoptada aquí (la ResNet-18) es adecuada para representar la función $f(\boldsymbol{x}, \theta)$ y mucho menos para interpretar los operadores de capa y los parámetros θ aprendidos en su interior. En el Capítulo 5, *mostraremos cómo derivar arquitecturas y componentes de redes en forma totalmente "transparente" o de "caja blanca" a partir del objetivo deseado (por ejemplo, la reducción de la tasa).*



(a) Bird (b) Ship

Figura 4.22: Las 10 mejores imágenes «principales» para las clases «Pájaro» y «Barco» en el conjunto de datos de imágenes CIFAR-10.

4.3.2 Aprendizaje no supervisado de representaciones

Como hemos visto en la sección anterior, la información sobre las clases facilita en cierta medida el aprendizaje de una representación estructurada de la distribución de datos (mixta). Sin embargo, a menudo no se dispone de dicha información, al menos no a gran escala.

A continuación, presentamos cómo extender el principio MCR^2 al entorno no supervisado. A diferencia del aprendizaje supervisado, en el que se conocen las etiquetas de clase, en el aprendizaje no supervisado se desconoce a qué grupo pertenecen las muestras de datos. En este caso, podemos aplicar cierta clase de aumentos o transformaciones, digamos τ con una distribución P_τ, a cada muestra. Por ejemplo, en el caso de las imágenes, los aumentos suelen incluir traslaciones, rotaciones o recortes de cada muestra de imagen. Podemos considerar que las muestras augmentadas de cada punto de datos pertenecen al mismo grupo o clúster.

Más concretamente, dado un conjunto de N muestras de datos $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N]$, generamos m vistas augmentadas $\{\tau_i\}_{i=1}^m$ para cada punto de datos $\boldsymbol{x}_k$, denotadas como $\tilde{\boldsymbol{X}}_k = [\tau_1(\boldsymbol{x}_k), \tau_2(\boldsymbol{x}_k), \ldots, \tau_m(\boldsymbol{x}_k)]$. Tratamos las muestras augmentadas $\tilde{\boldsymbol{X}}_i$ como la clase i-ésima con un total $K = N$ grupos/clústeres. Ahora podemos aplicar el principio MCR^2 tal como lo hicimos anteriormente en el caso del aprendizaje supervisado. Concretamente, aprendemos la función de mapeo de características $f(\boldsymbol{x}, \theta)$ maximizando la reducción de la tasa de codificación de la siguiente manera:

$$\begin{aligned} &\max_{\theta} \Delta R_\epsilon\big(\boldsymbol{Z}\big) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z}), \\ &\text{s.t.} \quad \boldsymbol{Z} = f(\tilde{\boldsymbol{X}}, \theta), \ \|\boldsymbol{Z}_k\|_F^2 = m, \ k = 1, \ldots, N. \end{aligned} \tag{4.3.1}$$

Aquí, $\tilde{\boldsymbol{X}} = [\tilde{\boldsymbol{X}}_1, \ldots, \tilde{\boldsymbol{X}}_N]$ y $\boldsymbol{Z}_k = f(\tilde{\boldsymbol{X}}_k, \theta)$ denotan las características de las muestras augmentadas de $\boldsymbol{x}_k$ para cada $k = 1, \ldots, N$.

Adoptemos de nuevo ResNet-18 como la función de mapeo de características $f(\boldsymbol{x}, \theta)$ y optimicemos los parámetros de la red θ para maximizar el objetivo no supervisado (4.3.1). Tras el entrenamiento con el conjunto de datos CIFAR-10, visualizamos la evolución de las dos tasas y su diferencia (tanto para los datos de entrenamiento como para los de prueba) a lo largo de las épocas de entrenamiento en la Figura 4.23a. En forma similar al caso del aprendizaje supervisado, tras una fase inicial, R_ϵ aumenta gradualmente mientras que R_ϵ^c disminuye, lo que indica que las características $\boldsymbol{Z}$ se están expandiendo en su conjunto, mientras que cada grupo $\boldsymbol{Z}_k$ se está comprimiendo. En comparación con el caso supervisado que se muestra en la Figura 4.18, podemos observar que, en el caso no supervisado, la tasa de codificación total R_ϵ aumenta rápidamente y el objetivo global MCR^2 se satura en un óptimo local. Para mitigar esta situación, podemos modificar la estrategia de optimización controlando la dinámica de R_ϵ y R_ϵ^c. En concreto, podemos reescalar las tasas sustituyendo la tasa de codificación $R_\epsilon(\boldsymbol{Z})$ en (4.3.1) por

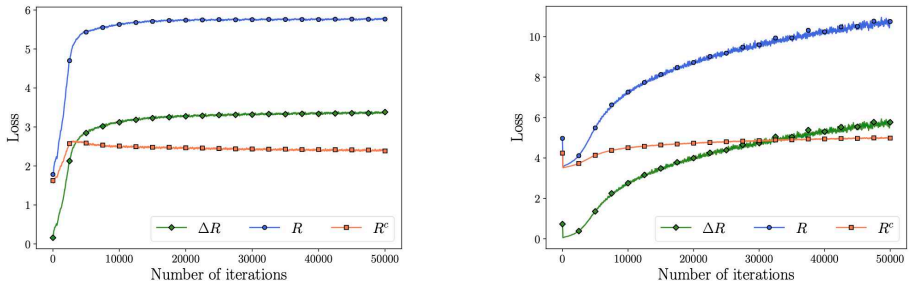
$$\tilde{R}_\epsilon(\boldsymbol{Z}) \doteq \frac{1}{2\gamma_1} \log\det\left(\boldsymbol{I} + \frac{\gamma_2 d}{Nm\epsilon^2}\boldsymbol{Z}\boldsymbol{Z}^\top\right).$$

Al establecer $\gamma_1 = \gamma_2 = N$, llegamos a un objetivo de aprendizaje modificado, denominado MCR^2-CTRL, cuya dinámica de entrenamiento se muestra en la Figura 4.23b. Como podemos ver, con este objetivo MCR^2 controlado, las características se comprimen primero y luego se expanden gradualmente, lo que da lugar a un valor más alto del objetivo global $\Delta R_\epsilon(\boldsymbol{Z})$.

Para evaluar la calidad de las representaciones aprendidas y verificar si presentan buenas estructuras de subespacios, adoptamos un algoritmo de agrupamiento de subespacios denominado EnSC [**you2016oracle**]. Como métricas de evaluación, tenemos en cuenta la información mutua normalizada (NMI), la precisión de agrupamiento (ACC) y el índice de Rand ajustado (ARI) [32]. A modo de comparación, analizamos métodos como JULE [**yang2016joint**], RTM [**nina2019decoder**], DEC [**xie2016unsupervised**], DAC [**chang2017deep**] y DCCM [**wu2019deep**]. Los resultados de la agrupación se resumen en la Figura 4.24. Como podemos ver, las características aprendidas al maximizar el objetivo MCR^2-CTRL alcanzan una mayor precisión de agrupación que las aprendidas por otros algoritmos.

Sin embargo, hay un inconveniente principal: el método de aprendizaje no supervisado basado en MCR^2 requiere una costosa operación log det de complejidad $O(d^3)$ *por muestra* al calcular R_ϵ^c. Esto se vuelve rápidamente prohibitivo en lo computacional a medida que intentamos escalar a conjuntos de datos más grandes. En lugar de minimizar la tasa de codificación exacta para cada grupo de muestras augmentadas $\tilde{\boldsymbol{X}}_i$, podemos aproximar el efecto de la compresión de características en cada grupo maximizando la *similitud coseno*

[32] Consulte el apéndice en [**yu2020learning**] para ver sus definiciones específicas.



(a) MCR^2 (b) MCR^2-ctrl

Figura 4.23: Evolución de las tasas de MCR^2 durante el proceso de entrenamiento para el aprendizaje no supervisado en CIFAR-10.

Table 2: Clustering results on CIFAR10, CIFAR100, and STL10 datasets.

Dataset	Metric	K-Means	JULE	RTM	DEC	DAC	DCCM	MCR^2-Ctrl
CIFAR10	NMI	0.087	0.192	0.197	0.257	0.395	0.496	**0.630**
	ACC	0.229	0.272	0.309	0.301	0.521	0.623	**0.684**
	ARI	0.049	0.138	0.115	0.161	0.305	0.408	**0.508**
CIFAR100	NMI	0.084	0.103	-	0.136	0.185	0.285	**0.387**
	ACC	0.130	0.137	-	0.185	0.237	0.327	**0.375**
	ARI	0.028	0.033	-	0.050	0.087	0.173	**0.178**
STL10	NMI	0.124	0.182	-	0.276	0.365	0.376	**0.446**
	ACC	0.192	0.182	-	0.359	0.470	0.482	**0.491**
	ARI	0.061	0.164	-	0.186	0.256	0.262	**0.290**

Figura 4.24: Comparación de la precisión de la agrupación entre el entrenamiento con el objetivo MCR^2-ctrl y otros algoritmos de aprendizaje no supervisado en los conjuntos de datos CIFAR-10, CIFAR-100 y STL-10.

entre las características de las muestras augmentadas:[33]

$$\begin{aligned} \max_{\theta} R_{\epsilon}(\boldsymbol{Z}) + \frac{\gamma}{N}\sum_{k=1}^{N}\sum_{i\neq j}\boldsymbol{z}_{k,j}^{\top}\boldsymbol{z}_{k,i}, \\ \text{s.t.} \quad \boldsymbol{z}_{i,j} = f(\tau_j(\boldsymbol{x}_i),\theta),\ \|\boldsymbol{z}_{i,j}\|_2^2 = 1, \end{aligned} \tag{4.3.2}$$

donde $\gamma > 0$ es un hiperparámetro que controla la intensidad del término de similitud coseno, ya que difiere de la tasa de codificación en magnitud. Este objetivo evita la costosa operación log det y es mucho más escalable. De hecho, este objetivo está estrechamente relacionado con métodos populares (sin supervisión) de *aprendizaje contrastivo*, tales como SimCLR [**chen2020simple**]

[33] Dejamos como ejercicio para los lectores que verifiquen bajo qué condiciones la minimización de la tasa de codificación por clase $R^c(\boldsymbol{Z})$ es (aproximadamente) equivalente a maximizar la similitud coseno entre las características $\boldsymbol{Z}$.

Método	Modelo	Épocas	20-NN	Sondeo Lineal
DINO	ViT-B	100	72.9	76.3
SimDINO	ViT-B	100	**74.9**	**77.3**
DINO	ViT-L	100	–	–
SimDINO	ViT-L	100	**75.6**	**77.4**

Tabla 4.1: **Rendimiento de la clasificación** en el conjunto de prueba de ImageNet-1k para DINO y SimDINO, mediante tanto la precisión del método de los k vecinos más cercanos ($k = 20$) como el sondeo lineal. SimDINO supera a DINO tanto en precisión como en estabilidad del entrenamiento. El entrenamiento de DINO en ViT-L es inestable y diverge rápidamente, mientras que SimDINO se entrena establemente.

y DINO [**caron2021emerging**], que han sido diseñados para aprender representaciones de alta calidad a partir de una gran cantidad de imágenes.

Como mostraremos más adelante en la **??** del **??**, este objetivo (4.3.2) puede implementarse de manera muy eficiente para aprender una representación altamente estructurada $\boldsymbol{Z}$ que es muy lineal y discriminativa. La representación de características aprendida ofrece un rendimiento de vanguardia en grandes conjuntos de datos del mundo real, comparable o incluso superior al de las características aprendidas por DINO, pero sin los considerables esfuerzos de ingeniería que requiere DINO. Por ello, denominamos a este objetivo *DINO simplificado* (SimDINO). A modo de avance, la Tabla 4.1 muestra que SimDINO supera a DINO en la clasificación de ImageNet-1k utilizando tanto la precisión de los k vecinos más cercanos como el sondeo lineal (es decir, el entrenamiento de un clasificador lineal sobre modelos preentrenados). Esto demuestra que el objetivo da lugar a representaciones muy estructuradas y linearizadas. También podemos utilizar estas representaciones para tareas posteriores, como la segmentación no supervisada[34], tal y como se ilustra en la Figura 4.25. En el **??** se presentarán más detalles sobre la implementación y los resultados experimentales de SimDINO.

4.4 Resumen y notas

Mensajes clave. En este capítulo, hemos estudiado los conceptos e ideas básicos subyacentes a la forma de aprender una distribución a partir de un número finito de muestras y obtener un esquema de codificación y decodificación *computable* para dicha distribución. En el caso de las distribuciones continuas y, posiblemente, de baja dimensión, hemos argumentado que dicho esquema es necesariamente *con pérdidas*; de ahí la importancia del concepto de tasa-distorsión. El ruido introducido por la distorsión, debido a la cuantización,

[34] En concreto, adoptamos MaskCut [**wang2023cut**], un enfoque no supervisado eficaz para extraer características de una red de visión con pesos congelados, a fin de realizar la detección de objetos y la segmentación de instancias.

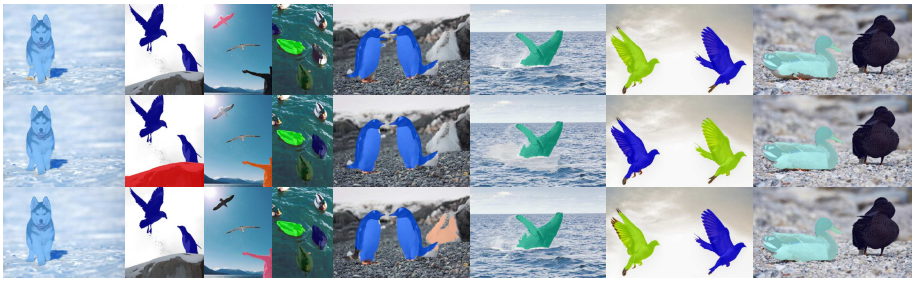


Figura 4.25: **Visualización de los resultados de la segmentación no supervisada** obtenidas con DINO ViT-B/16 (fila 1), SimDINO ViT-B/16 (fila 2) y SimDINO ViT-L/16 (fila 3).

desempeña un papel relacionado pero distinto del ruido que hemos estudiado en el capítulo anterior. Se podría considerar que el ruido considerado en el Capítulo 3 se encuentra «fuera» de la subvariedad del soporte de la distribución, mientras que el ruido analizado en este capítulo se encuentra «dentro» del soporte. De manera informal, podemos referirnos al proceso de difusión/eliminación de ruido fuera del soporte como «continuación» y al proceso de difusión/eliminación de ruido dentro del soporte como «percolación». Un análisis más unificado y una comprensión más profunda de su efecto combinado son objeto de futuros estudios.

En este capítulo, hemos introducido por primera vez el concepto de aprendizaje de una *representación* para una distribución. Hemos considerado el caso posiblemente más sencillo: el de la clasificación. Si se considera la etiqueta de clasificación $\boldsymbol{y}$ como un código fijo simple (con pérdidas enormes) para los datos $\boldsymbol{x}$, podríamos buscar una representación consistente[35] $\boldsymbol{z} = f(\boldsymbol{x})$ de los datos $\boldsymbol{x}$ de tal manera que tienda a maximizar la información mutua entre la representación y este código simple:

$$\max I(\boldsymbol{z}, \boldsymbol{y}) = H(\boldsymbol{z}) - H(\boldsymbol{z} \mid \boldsymbol{y}). \tag{4.4.1}$$

Además, exigimos que dicha representación $\boldsymbol{z}$ no solo sea consistente con $\boldsymbol{x}$, sino también mucho más *estructurada* que la distribución nativa de los datos de $\boldsymbol{x}$: las clases son linealmente discriminativas. Esto haría que la representación aprendida fuera mucho más eficiente en su uso que la distribución nativa de los datos para tareas posteriores como el reconocimiento, la segmentación y el completamiento. El objetivo de reducción máxima de la tasa de codificación introducido es una materialización natural y computable de estos requisitos. En el **??**, veremos cómo generalizar este objetivo a un contexto mucho más amplio, en el que $\boldsymbol{y}$ podría ser un código con pérdida más general (por ejemplo, un pie

[35] Es decir, podemos recuperar fielmente $\boldsymbol{x}$ a partir de $\boldsymbol{z}$, ya que aclararemos el concepto de representaciones «consistentes» más adelante en el **??**.

de imagen) o una variable aleatoria relacionada con $\boldsymbol{x}$.

4.5 Ejercicios y ampliaciones

Ejercicio 4.1. Demuestre las siguientes propiedades de la función $\log\det(\cdot)$.

1. Demuestre que

$$f(\boldsymbol{X}) = \log\det(\boldsymbol{X})$$

es una función cóncava. (**Pista:** La función $f(\boldsymbol{x})$ es cóncava si y solo si la función unidimensional $f(\boldsymbol{x} + t\boldsymbol{h})$ es cóncava para todos los valores de $\boldsymbol{x}$ y $\boldsymbol{h}$.)

2. Demuestre que

$$\log\det(\boldsymbol{I} + \boldsymbol{X}^\top\boldsymbol{X}) = \log\det(\boldsymbol{I} + \boldsymbol{X}\boldsymbol{X}^\top)$$

3. Sea $\boldsymbol{A} \in \mathbb{R}^{n\times n}$ una matriz definida positiva. Demuestre que:

$$\log\det(\boldsymbol{A}) = \sum_{i=1}^{n}\log(\lambda_i), \tag{4.5.1}$$

donde $\lambda_1, \lambda_2, \ldots, \lambda_n$ son los valores propios de $\boldsymbol{A}$.

4. Demuestre que el gradiente de la función $\log\det(\cdot)$ viene dado por

$$\nabla\log\det(\boldsymbol{A}) = \boldsymbol{A}^{-1},$$

para una matriz definida positiva $\boldsymbol{A}$.

Ejercicio 4.2. Sea $\boldsymbol{Z} = [\boldsymbol{Z}_1, \ldots, \boldsymbol{Z}_K] \in \mathbb{R}^{d\times N}$, donde cada $\boldsymbol{Z}_k \in \mathbb{R}^{d\times N_k}$ y $N = \sum_{k=1}^{K} N_k$.

1. Demuestre que

$$\log\det\left(\boldsymbol{I} + \alpha\boldsymbol{Z}\boldsymbol{Z}^\top\right) \leq \sum_{k=1}^{K}\log\det\left(\boldsymbol{I} + \alpha\boldsymbol{Z}_k\boldsymbol{Z}_k^\top\right),$$

donde $\alpha > 0$.

2. Derive la condición necesaria y suficiente bajo la cual se cumple la igualdad.

Ejercicio 4.3. Sean $\boldsymbol{z}_1$ y $\boldsymbol{z}_2$ dos vectores de longitud unitaria. Sea $\boldsymbol{Z} = [\boldsymbol{z}_1, \boldsymbol{z}_2]$. Demuestre que los tres objetivos siguientes son equivalentes:

- $\max \boldsymbol{z}_1^\top\boldsymbol{z}_2$.
- $\min \|\boldsymbol{z}_1 - \boldsymbol{z}_2\|_2^2$.

- $\max \log\det(\boldsymbol{I} + \boldsymbol{Z}\boldsymbol{Z}^\top)$.

Sin embargo, es posible que ya no sean necesariamente equivalentes cuando hay más de dos vectores en un espacio de alta dimensión. El $\log\det$ es más general, ya que mide el volumen total que abarcan todos los vectores en el espacio de alta dimensión.

Ejercicio 4.4. Derive la función de tasa-distorsión para el vector aleatorio gaussiano presentado en el Ejemplo 4.4.

Capítulo 5

Las representaciones profundas como optimización desenrollada

"*Lo que no puedo crear, no lo entiendo.*"
— Richard Feynman

En capítulos anteriores, hemos mostrado cómo identificar estructuras de baja dimensión en espacios de alta dimensión, *centrándonos principalmente en estructuras lineales*. Por ejemplo, introdujimos el análisis de componentes principales (PCA) para aprender el *denoiser* lineal $\hat{\boldsymbol{U}}^\top$ cuando los datos observados $\boldsymbol{x}$ siguen el modelo estadístico $\boldsymbol{x} = \boldsymbol{U}\boldsymbol{z} + \boldsymbol{\varepsilon}$. En este contexto, las representaciones aprendidas son los datos de entrada linealmente transformados $\hat{\boldsymbol{U}}^\top \boldsymbol{x}$. Bajo el supuesto de un modelo lineal, es posible aprender la estructura lineal de baja dimensión mediante algoritmos de optimización eficientes, con sólidas garantías teóricas. Más aún, el supuesto del modelo lineal abarca una amplia gama de aplicaciones y problemas, entre los que se incluyen el reconocimiento facial, la recuperación de imágenes por resonancia magnética y la recuperación de texturas estructurales [**Wright-Ma-2022**].

Por otro lado, el modelo lineal puede resultar limitado cuando se trata de aplicaciones del mundo real, especialmente cuando los datos de entrada $\boldsymbol{x}$ son complejos, como el habla y los lenguajes naturales, las imágenes y los videos, así como los movimientos robóticos. Las distribuciones de baja dimensión de dichos datos suelen ser *no lineales*. El tratamiento de la no linealidad cuenta con una larga trayectoria en diversas disciplinas, como la teoría de control, el procesamiento de señales y el reconocimiento de patrones. Se han realizado esfuerzos considerables para tratar de extender los métodos y soluciones de los modelos lineales con el fin de abordar la no linealidad, incluidos los primeros intentos de ampliar el PCA al PCA no lineal (como veremos con más detalle

en el **??**). En la mayoría de los casos, los métodos se diseñan partiendo de ciertas suposiciones sobre la distribución de los datos y se adaptan a problemas específicos.

Más recientemente, las redes neuronales profundas han logrado un éxito notable en una amplia gama de datos y aplicaciones. Una red neuronal

$$f(\cdot, \boldsymbol{\theta}) \colon \boldsymbol{x} \xrightarrow{f^{\text{pre}}} \boldsymbol{z}^1 \to \cdots \to \boldsymbol{z}^\ell \xrightarrow{f^\ell} \boldsymbol{z}^{\ell+1} \to \cdots \to \boldsymbol{z}^{L+1} = \boldsymbol{z}. \tag{5.0.1}$$

puede aprender características o representaciones efectivas para aplicaciones aguas abajo. Por ejemplo, una red neuronal profunda entrenada $f(\cdot, \boldsymbol{\theta})$ puede aplicarse para mapear imágenes a vectores de características, es decir, $\boldsymbol{z}_i = f(\boldsymbol{x}_i, \boldsymbol{\theta})$, mientras que se puede entrenar un clasificador lineal a partir de dichas representaciones $\{\boldsymbol{z}_i\}$. Un avance notable es AlexNet [**krizhevsky2012imagenet**], una red neuronal convolucional profunda entrenada con más de un millón de imágenes naturales, que superó (en tareas de predicción) a todos los enfoques anteriores basados en características definidas manualmente. Una de las diferencias clave entre AlexNet y los enfoques anteriores es que el primero *aprende los parámetros de la transformación no lineal a partir de enormes cantidades de datos* entrenando mediante retropropagación (BP) [**Back-Prop**], tal y como se detalla en la Sección **??** del **??**.

En la práctica habitual posterior, se modela el mapeo f con otras redes neuronales artificiales profundas diseñadas empíricamente y se aprenden los parámetros $\boldsymbol{\theta}$ a partir de una inicialización aleatoria mediante el algoritmo de retropropagación. Desde la aparición de AlexNet [**krizhevsky2012imagenet**], las arquitecturas de las redes profundas modernas siguen siendo revisadas y mejoradas en forma empírica. Arquitecturas de red como VGG [**simonyan2014very**], ResNet [**he2016deep**], DenseNet [**dense-net**], CNN, RNN o LSTM [**LSTM**], Transformer [**vaswani2017attention**] y mezclas de expertos (MoE) [**MoE**; **Fedus-2022**], etc., han seguido ampliando los límites del rendimiento. Como parte del esfuerzo por mejorar el rendimiento de las redes profundas, se ha examinado empíricamente casi todas las componentes de las redes y se han propuesto diversas revisiones y mejoras. No se limitan a funciones de activación no lineales [**maas2013rectifier**; **klambauer2017self**; **xu2015empirical**; **nwankpa2018activation**], conexiones de salto [**ronneberger2015u**; **he2016deep**], normalizaciones [**ioffe2015batch**; **ba2016layer**; **ulyanov2016instance**; **wu2018group**; **miyato2018spectral**], muestreo ascendente/descendente o agrupación [**scherer2010evaluation**], convoluciones [**lecun1998gradient**; **krizhevsky2012imagenet**], etc. Sin embargo, casi todas estas modificaciones se han desarrollado a lo largo de años de estudios empíricos de prueba y error o de ablación. Algunas prácticas recientes llegaron incluso al extremo de buscar estructuras de red y estrategias de entrenamiento efectivas mediante técnicas de búsqueda aleatoria a gran escala, como "Neural Architecture Search" (Búsqueda de Arquitectura Neuronal) [**NAS-1**; **Baker2017DesigningNN**], AutoML [**automl**] y "Learning to Learn"

(Aprender a Aprender)[**andrychowicz2016learning**].

A pesar de la amplia aplicación de las redes neuronales profundas, no está claro cuáles son los principios de diseño subyacentes de una red construida de este modo. En particular, no está claro qué función matemática desempeña cada capa de la red. En este capítulo, basándonos en los resultados de los capítulos anteriores, desarrollamos un marco teórico que proporcionará una interpretación matemática totalmente rigurosa del papel de una red profunda, incluyendo sus capas individuales y la red en su conjunto.

Para comprender las redes profundas y entender cómo deberían diseñarse mejor, debemos partir del objetivo del aprendizaje de representaciones. En los capítulos anteriores, hemos argumentado que el objetivo consiste en identificar la distribución de los datos, que es intrínsecamente de baja dimensión, y luego transformarla en una representación compacta y estructurada (por ejemplo, lineal por tramos). Como hemos visto en el capítulo anterior, el enfoque general para identificar una distribución de datos de baja dimensión consiste en un proceso de compresión que minimiza progresivamente la entropía o la tasa de codificación de la distribución. Sin embargo, hasta ahora hemos estado utilizando redes profundas diseñadas en forma empírica para modelar o aproximar las operaciones destinadas a optimizar estos objetivos, como la función de *score* para la eliminación de ruido (en la Sección 1.3.1) o la transformación que maximiza la reducción de la tasa (en la Sección 4.2.4).

Como hemos argumentado en el capítulo anterior, en la Sección 4.2 en particular, se puede evaluar la calidad de la representación resultante mediante la información que la representación refinada por sobre una representación «perezosa» que modela todos los datos como una gran distribución gaussiana.[1] En particular, si *utilizamos una mezcla de gaussianas (subespacios)*[2] *como distribuciones prototípicas para aproximar la distribución no lineal que nos interesa*, entonces podemos medir de manera eficiente la tasa de codificación de dicha representación utilizando la (suma de las) funciones de tasa-distorsión de las gaussianas asociadas. Entonces, la cantidad de *información ganada* o la entropía (relativa) reducida con dicho modelado puede medirse mediante la diferencia entre la tasa de codificación de la representación «perezosa» simplificada y la de la representación más refinada. Por lo tanto, el objetivo del aprendizaje de representaciones es maximizar esta ganancia de información, también conocida como el objetivo de reducción de tasa.

Como veremos en este capítulo, una vez que el objetivo del aprendizaje de representaciones queda claro, la función de una red neuronal profunda consiste precisamente en ayudar a optimizar ese objetivo de modo iterativo. Cada capa de una red neuronal profunda puede derivarse en forma natural como un paso de optimización iterativo destinado a maximizar de manera incremental la ganancia de información, incluyendo las populares arquitecturas de ResNet, CNN y Transformadores, así como otras variantes más avanzadas. En particular,

[1] que hemos visto en el capítulo anterior como una opción concreta de interpretación del conjunto de datos muestreado.

[2] que hemos estudiado a fondo en el capítulo anterior

este capítulo tiene como objetivo responder a las siguientes preguntas sobre las redes profundas:

- Sección 5.1 — dada una medida de calidad para una representación aprendida, ¿cómo construir el mapeo no lineal de los datos a la representación óptima mediante la optimización desenrollada del objetivo?
- Sección 5.2 — ¿cómo podría el mencionado enfoque de «desenrollado» ofrecer una interpretación fundamentada de las populares arquitecturas Transformers? Si es así, ¿cuáles son los mecanismos de optimización y los objetivos asociados?
- **??** — ¿cómo nos ayudaría este marco a diseñar arquitecturas profundas más eficientes o más parsimoniosas?

5.1 Redes profundas transparentes *white-box* mediante optimización desenrollada

Ahora bien, si estamos de acuerdo en que maximizar la reducción de tasa o la ganancia de información conduce a la representación deseada, tal como se analiza en la Sección 4.2, la pregunta que queda por resolver es cómo construir y aprender un mapeo (no lineal) desde los datos $\boldsymbol{X}$ hacia la representación óptima $\boldsymbol{Z}^*$. Esto implica diseñar una arquitectura de red y un algoritmo de aprendizaje que puedan captar de manera eficaz las estructuras subyacentes de los datos y materializar fielmente la representación óptima.

5.1.1 Redes profundas a partir del descenso por gradiente desenrollado

En el capítulo anterior, presentamos el objetivo de reducción de tasa (4.2.15) como un objetivo fundamentado para aprender representaciones discriminativas lineales de los datos. Sin embargo, no hemos especificado la arquitectura de la función de mapeo de características $\boldsymbol{z} = f(\boldsymbol{x}, \boldsymbol{\theta})$ para extraer dichas representaciones de los datos de entrada $\boldsymbol{x}$. Una opción sencilla es utilizar una red profunda convencional, como ResNet, para implementar $f(\boldsymbol{x}, \boldsymbol{\theta})$. Como hemos visto en el Sección 4.3.1, esta opción suele dar lugar a un rendimiento razonable en la práctica. No obstante, siguen existiendo varios problemas sin resolver al adoptar una red profunda arbitraria. Aunque la representación de características aprendida es ahora más interpretable, la red en sí misma sigue *sin* serlo. No está claro por qué cualquier red de «black-box» elegida es capaz siquiera de optimizar el objetivo MCR^2 deseado. Los buenos resultados empíricos (por ejemplo, con una ResNet) no justifican necesariamente la elección concreta de arquitecturas y operadores de la red: ¿Por qué es necesario un modelo de gran profundidad en capas? ¿Qué es lo que las capas adicionales pretenden mejorar o simplificar? ¿Qué anchura y profundidad son adecuadas? ¿O existe alguna justificación rigurosa para las convoluciones (en su popular

forma multicanal) y los operadores no lineales (por ejemplo, ReLU o softmax) que se utilizan?

En este capítulo, demostramos que el uso del ascenso por gradiente para maximizar la reducción de tasa $\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{\Pi})$, tal como se define en (4.2.15), conduce de manera natural a una red profunda transparente tipo *white-box* que realiza el mapeo deseado. Todas las capas de la red, los operadores lineales y no lineales, así como los parámetros se *construyen explícitamente siguiendo un enfoque de propagación puramente hacia adelante*. Además, estas arquitecturas de red se asemejan a las redes profundas existentes diseñadas empíricamente, lo que proporciona justificaciones fundamentadas para su diseño.

Ascenso por gradiente para la reducción de la tasa de codificación. En el capítulo anterior se vio que, para buscar una representación discriminativa lineal (LDR), matemáticamente estamos buscando, en esencia, una función continua $f(\cdot) : \boldsymbol{x} \mapsto \boldsymbol{z}$ que mapee los datos $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N] \in \mathbb{R}^{D\times N}$ (o características iniciales extraídas de los datos[3]) a una representación óptima $\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N] \in \mathbb{R}^{d\times N}$ que maximice el siguiente objetivo de reducción de la tasa de codificación (véase también (4.2.17)):

$$\Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{\Pi}) \doteq \underbrace{\frac{1}{2}\log\det\Big(\boldsymbol{I} + \alpha \boldsymbol{Z}\boldsymbol{Z}^\top\Big)}_{R_\epsilon(\boldsymbol{Z})} - \underbrace{\sum_{k=1}^{K}\frac{\gamma_k}{2}\log\det\Big(\boldsymbol{I} + \alpha_k \boldsymbol{Z}\boldsymbol{\Pi}_k\boldsymbol{Z}^\top\Big)}_{R_\epsilon^c(\boldsymbol{Z}\mid\boldsymbol{\Pi})}, \tag{5.1.1}$$

donde $\epsilon > 0$ es un error de cuantización prescrito y, para simplificar, denotamos[4]

$$\alpha \doteq \frac{d}{N\epsilon^2}, \quad \alpha_k \doteq \frac{d}{\operatorname{tr}(\boldsymbol{\Pi}_k)\epsilon^2}, \quad \gamma_k \doteq \frac{\operatorname{tr}(\boldsymbol{\Pi}_k)}{N}, \quad \text{for } k = 1, \ldots, K. \tag{5.1.2}$$

La pregunta se reduce, en realidad, a si existe una forma *constructiva* de encontrar una aplicación continua de este tipo $f(\cdot, \boldsymbol{\theta})$ de $\boldsymbol{x}$ a $\boldsymbol{z}$. Para ello, consideremos la maximización incremental del objetivo ΔR_ϵ como una función de $\boldsymbol{Z} \subseteq \mathbb{S}^{d-1}$. Aunque puede haber muchos esquemas de optimización entre los que elegir, por simplicidad consideraremos primero el esquema de «ascenso por gradiente proyectado» (PGA), posiblemente el más sencillo:[5]

$$\boldsymbol{Z}^{\ell+1} \propto \boldsymbol{Z}^\ell + \eta \cdot \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) \quad \text{s.t.} \quad \boldsymbol{Z}^{\ell+1} \subseteq \mathbb{S}^{d-1}, \quad \ell = 1, 2, \ldots, \tag{5.1.3}$$

para un tamaño de paso $\eta > 0$ y el iterado inicial dado por $\boldsymbol{Z}^0 = \boldsymbol{X}$.[6] Este esquema puede interpretarse como la forma en que se deben ajustar

[3] Como veremos, la necesidad de dicha extracción de características se abordará en la siguiente sección.

[4] Obsérvese que utilizamos una notación ligeramente simplificada en comparación con el Capítulo 3.

[5] Obsérvese que utilizamos el subíndice j en $\boldsymbol{Z}_j$ para indicar las características de la j-ésima clase y el superíndice ℓ en $\boldsymbol{Z}^\ell$ para indicar todas las características en la iteración o capa ℓ-ésima.

[6] Una vez más, para simplificar, aquí suponemos en primer lugar que las características

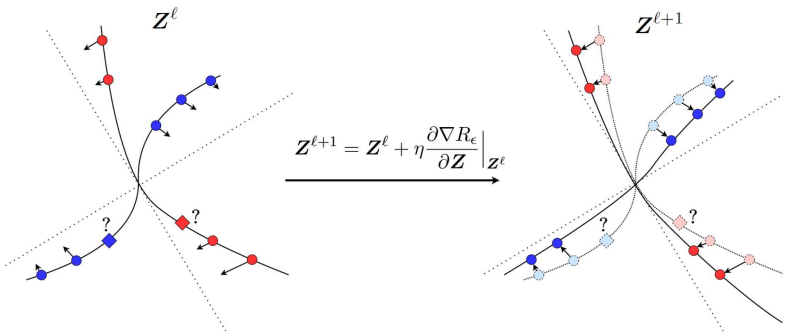


Figura 5.1: Deformación incremental mediante flujo de gradiente para, por un lado, aplanar los datos de cada clase en un subespacio y, por otro, separar las diferentes clases.

gradualmente las ubicaciones de las características actuales $\boldsymbol{Z}^\ell$, inicializadas a partir de los datos de entrada $\boldsymbol{X}$, para que el conjunto resultante $\boldsymbol{Z}^{\ell+1}$ mejore la reducción de la tasa ΔR_ϵ, tal como se ilustra en la Figura 5.1.

Un cálculo sencillo muestra que el gradiente $\partial \Delta R_\epsilon / \partial \boldsymbol{Z}$ implica evaluar las siguientes derivadas de los dos términos de ΔR_ϵ:

$$\frac{1}{2}\frac{\partial \log\det(\boldsymbol{I}+\alpha \boldsymbol{Z}\boldsymbol{Z}^\top)}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) = \underbrace{\alpha(\boldsymbol{I}+\alpha \boldsymbol{Z}^\ell(\boldsymbol{Z}^\ell)^\top)^{-1}}_{\boldsymbol{E}^\ell \in \mathbb{R}^{d\times d}} \boldsymbol{Z}^\ell, \tag{5.1.4}$$

$$\frac{1}{2}\frac{\partial \left(\gamma_k \log\det(\boldsymbol{I}+\alpha_k \boldsymbol{Z}\boldsymbol{\Pi}_k\boldsymbol{Z}^\top)\right)}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) = \gamma_k \underbrace{\alpha_k(\boldsymbol{I}+\alpha_k \boldsymbol{Z}^\ell\boldsymbol{\Pi}_k(\boldsymbol{Z}^\ell)^\top)^{-1}}_{\boldsymbol{C}_k^\ell \in \mathbb{R}^{d\times d}} \boldsymbol{Z}^\ell\boldsymbol{\Pi}_k. \tag{5.1.5}$$

Nótese que, en lo anterior, la matriz $\boldsymbol{E}^\ell$ solo depende de $\boldsymbol{Z}^\ell$ y tiene como objetivo *expandir* todas las características para aumentar la tasa de codificación global; la matriz $\boldsymbol{C}_k^\ell$, a su vez, depende de las características de la clase k y tiene como objetivo *comprimirlas* para reducir la tasa de codificación de cada clase. Entonces, el gradiente completo $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) \in \mathbb{R}^{d\times N}$ tiene la forma:

$$\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell) = \underbrace{\boldsymbol{E}^\ell}_{\text{Expansion}} \boldsymbol{Z}^\ell - \sum_{k=1}^{K} \gamma_k \underbrace{\boldsymbol{C}_k^\ell}_{\text{Compression}} \boldsymbol{Z}^\ell\boldsymbol{\Pi}_k. \tag{5.1.6}$$

Nota 5.1 (Interpretación de $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_j^\ell$ como operadores lineales). Para cualquier $\boldsymbol{z}^\ell \in \mathbb{R}^d$,

$$\boldsymbol{E}^\ell \boldsymbol{z}^\ell = \alpha(\boldsymbol{z}^\ell - \boldsymbol{Z}^\ell \boldsymbol{q}_\star^\ell), \quad \text{donde} \quad \boldsymbol{q}_\star^\ell \doteq \arg\min_{\boldsymbol{q}^\ell} \left\{\alpha\|\boldsymbol{z}^\ell - \boldsymbol{Z}^\ell\boldsymbol{q}^\ell\|_2^2 + \|\boldsymbol{q}^\ell\|_2^2\right\}. \tag{5.1.7}$$

iniciales $\boldsymbol{Z}^1$ son los propios datos. Obsérvese que aquí ℓ denota el número de iteraciones. Por lo tanto, los datos y las características tienen la misma dimensión d. Sin embargo, esto no tiene por qué ser así. Como veremos en la siguiente sección, las características iniciales pueden ser algunas características (elevadas) de los datos para empezar y, en principio, podrían tener una dimensión diferente (mucho mayor). Todas las iterados posteriores tienen la misma dimensión.

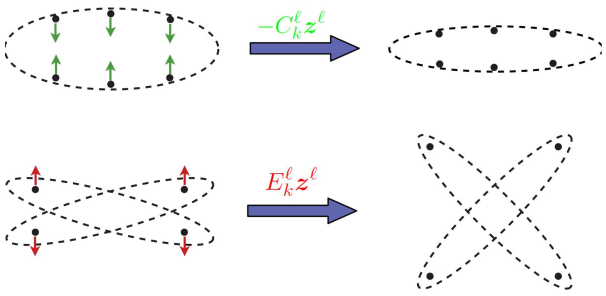


Figura 5.2: Interpretación de $\boldsymbol{C}_k^\ell$ y $\boldsymbol{E}^\ell$: $\boldsymbol{C}_k^\ell$ comprime cada clase contrayendo las características en un subespacio de baja dimensión; $\boldsymbol{E}^\ell$ expande todas las características contrastando y repeliendo características entre diferentes clases.

Obsérvese que $\boldsymbol{q}_\star^\ell$ es exactamente la solución de la regresión por crestas para todos los puntos de datos $\boldsymbol{Z}^\ell$ en cuestión. Por lo tanto, $\boldsymbol{E}^\ell$ (y de manera similar para $\boldsymbol{C}_k^\ell$) es aproximadamente (es decir, cuando N es lo suficientemente grande) la proyección sobre el complemento ortogonal del subespacio generado por las columnas de $\boldsymbol{Z}^\ell$. Otra forma de interpretar la matriz $\boldsymbol{E}^\ell$ es mediante la descomposición en valores propios de la matriz de covarianza $\boldsymbol{Z}^\ell(\boldsymbol{Z}^\ell)^\top$. Suponiendo que $\boldsymbol{Z}^\ell(\boldsymbol{Z}^\ell)^\top \doteq \boldsymbol{U}^\ell\boldsymbol{\Lambda}^\ell(\boldsymbol{U}^\ell)^\top$, donde $\boldsymbol{\Lambda}^\ell \doteq \operatorname{diag}\left(\lambda_1^\ell, \ldots, \lambda_d^\ell\right)$, tenemos

$$\boldsymbol{E}^\ell = \alpha \boldsymbol{U}^\ell \operatorname{diag}\left(\frac{1}{1+\alpha\lambda_1^\ell}, \ldots, \frac{1}{1+\alpha\lambda_d^\ell}\right)\left(\boldsymbol{U}^\ell\right)^\top. \tag{5.1.8}$$

Por lo tanto, la matriz $\boldsymbol{E}^\ell$ actúa sobre un vector $\boldsymbol{z}^\ell$ estirándolo de tal manera que las direcciones de gran varianza se reducen, mientras que se mantienen las direcciones de varianza cercana a cero. Estas son precisamente las direcciones (5.1.4) en las que desplazamos las características para que el volumen total se expanda y la tasa de codificación aumente, de ahí el signo positivo. Por el contrario, las direcciones asociadas a (5.1.5) son «residuos» de las características de cada clase que se desvían del subespacio al que se supone que pertenecen. Estas son exactamente las direcciones en las que las características deben comprimirse de nuevo hacia su respectivo subespacio, de ahí el signo negativo (véase la Figura 5.2).

En esencia, las operaciones lineales $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ en el método de ascenso por gradiente para la reducción de tasa se determinan mediante los datos de entrenamiento que realizan «autoregresiones». Los recientes avances en la comprensión de la regresión de cresta en una configuración sobreparametrizada [**yang2020rethinking**; **Wu2020OnTO**] indican que el uso de datos aparentemente muestreados en forma redundante (de cada subespacio) como regresores no conduce a sobreajuste.

Incremento del mapa de características guiado por gradientes. Nótese que en lo anterior, el ascenso por gradiente considera todas las características

$\boldsymbol{Z}^\ell = [\boldsymbol{z}_1^\ell, \ldots, \boldsymbol{z}_N^\ell]$ como variables libres. El incremento $\boldsymbol{Z}^{\ell+1} - \boldsymbol{Z}^\ell = \eta \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell)$ aún no proporciona una transformación en todo el dominio de características $\boldsymbol{z}^\ell \in \mathbb{R}^d$. Según la ecuación (5.1.6), el gradiente no puede evaluarse en un punto cuya pertenencia se desconoce, tal como se ilustra en la Figura 5.1. Por lo tanto, para hallar la $f(\boldsymbol{x}, \boldsymbol{\theta})$ óptima en forma explícita, podemos considerar la construcción de una pequeña transformación de incremento $g(\cdot, \theta^\ell)$ sobre la característica de la ℓ-ésima capa $\boldsymbol{z}^\ell$ para emular el esquema de gradiente (proyectado) anterior:

$$\boldsymbol{z}^{\ell+1} \;\propto\; \boldsymbol{z}^\ell + \eta \cdot g(\boldsymbol{z}^\ell, \theta^\ell) \quad \text{sujeto a} \quad \boldsymbol{z}^{\ell+1} \in \mathbb{S}^{d-1} \tag{5.1.9}$$

de tal manera que $\big[g(\boldsymbol{z}_1^\ell, \boldsymbol{\theta}^\ell), \ldots, g(\boldsymbol{z}_N^\ell, \boldsymbol{\theta}^\ell)\big] \approx \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell)$. Es decir, necesitamos aproximar el flujo del gradiente $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}$ que deforma localmente todas las características (de entrenamiento) $\{\boldsymbol{z}_i^\ell\}_{i=1}^N$ con un mapeo continuo $g(\boldsymbol{z}, \boldsymbol{\theta})$ definido en todo el espacio de características $\boldsymbol{z}^\ell \in \mathbb{R}^d$. Obsérvese que el incremento (5.1.9) puede interpretarse como una versión discretizada de una ecuación diferencial continua:

$$\dot{\boldsymbol{z}} = g(\boldsymbol{z}, \theta). \tag{5.1.10}$$

Por lo tanto, la red (profunda) así construida puede interpretarse como una determinada ecuación diferencial neuronal ordinaria [**chen2018neural**]. Sin embargo, a diferencia de la ODE neuronal, donde el flujo g se elige como estructura genérica, aquí nuestro $g(\boldsymbol{z}, \boldsymbol{\theta})$ tiene el propósito de emular el flujo de gradiente de la reducción de tasa en el conjunto de características (como se muestra en la Figura 5.1):

$$\dot{\boldsymbol{Z}} = \frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}},$$

y su estructura se deriva y se determina íntegramente a partir de este objetivo, sin ningún otro supuesto previo ni heurística.

Al examinar la estructura del gradiente (5.1.6), se sugiere que una opción natural para la transformación del incremento $g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell)$ es de la forma:

$$g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \;\doteq\; \boldsymbol{E}^\ell \boldsymbol{z}^\ell - \sum_{k=1}^{K} \gamma_k \pi_k(\boldsymbol{z}^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell \in \mathbb{R}^d, \tag{5.1.11}$$

donde $\pi_k(\boldsymbol{z}^\ell) \in [0, 1]$ indica la probabilidad de que $\boldsymbol{z}^\ell$ pertenezca a la clase k-ésima. Los parámetros del mapa de incremento $\boldsymbol{\theta}^\ell$ dependen: En primer lugar, de un conjunto de mapas lineales representados por $\boldsymbol{E}^\ell$ y $\{\boldsymbol{C}_k^\ell\}_{k=1}^K$ que dependen únicamente de las estadísticas de las características del entrenamiento $\boldsymbol{Z}^\ell$. En segundo lugar, de la pertenencia $\{\pi_k(\boldsymbol{z}^\ell)\}_{k=1}^K$ de cualquier característica $\boldsymbol{z}^\ell$. Nótese que, en las muestras de entrenamiento $\boldsymbol{Z}^\ell$, para las que se conocen las pertenencias $\boldsymbol{\Pi}_k$, la función $g(\boldsymbol{z}^\ell, \boldsymbol{\theta})$ así definida proporciona exactamente los valores del gradiente $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}}(\boldsymbol{Z}^\ell)$.

Dado que solo disponemos de la función de pertenencia para los datos de entrenamiento, la función $g(\cdot)$ definida en (5.1.11) solo puede evaluarse sobre los

datos de entrenamiento. Para extrapolar $g(\cdot)$ a todo el espacio de características, necesitamos estimar $\pi_k(\boldsymbol{z}^\ell)$ en su segundo término. En el aprendizaje profundo convencional, este mapa suele modelarse como una red profunda y aprenderse a partir de los datos de entrenamiento, por ejemplo, mediante *retropropagación*. Sin embargo, nuestro objetivo aquí no es (todavía) aprender un clasificador preciso $\pi_k(\boldsymbol{z}^\ell)$. En cambio, solo necesitamos una estimación lo suficientemente buena de la información de clase para que $g(\cdot)$ aproxime bien el gradiente $\frac{\partial \Delta R_\epsilon}{\partial \boldsymbol{Z}^\ell}$.

A partir de la interpretación geométrica de las aplicaciones lineales $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ dada por la Nota 5.1, el término $\boldsymbol{p}_k^\ell \doteq \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell$ puede considerarse como (aproximadamente) la proyección de $\boldsymbol{z}^\ell$ sobre el complemento ortogonal de cada clase j. Por lo tanto, $\|\boldsymbol{p}_j^\ell\|_2$ es pequeño si $\boldsymbol{z}^\ell$ está en la clase j y grande en caso contrario. Esto nos motiva a estimar su pertenencia basándonos en la siguiente función softmax:

$$\widehat{\boldsymbol{\pi}}(\boldsymbol{z}^\ell) \doteq \operatorname{softmax}\left(-\lambda \begin{bmatrix} \|\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell\|_2 \\ \vdots \\ \|\boldsymbol{C}_K^\ell \boldsymbol{z}^\ell\|_2 \end{bmatrix}\right) \tag{5.1.12}$$

$$= \frac{1}{\sum_{k=1}^K \exp(-\lambda \|\boldsymbol{C}_k^\ell \boldsymbol{z}^\ell\|_2)} \begin{bmatrix} \exp(-\lambda \|\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell\|_2) \\ \vdots \\ \exp(-\lambda \|\boldsymbol{C}_K^\ell \boldsymbol{z}^\ell\|_2) \end{bmatrix} \in [0,1]^K. \tag{5.1.13}$$

Por lo tanto, el segundo término de (5.1.11) puede aproximarse mediante esta pertenencia estimada:

$$\sum_{k=1}^K \gamma_k \pi_k(\boldsymbol{z}^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell \;\approx\; \sum_{k=1}^K \gamma_k \widehat{\pi}_k(\boldsymbol{z}^\ell) \boldsymbol{C}_k^\ell \boldsymbol{z}^\ell \;\doteq\; \boldsymbol{\sigma}\Big([\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell, \ldots, \boldsymbol{C}_K^\ell \boldsymbol{z}^\ell]\Big), \tag{5.1.14}$$

que se denota como un operador no lineal $\boldsymbol{\sigma}(\cdot)$ sobre las salidas de la característica $\boldsymbol{z}^\ell$ a través de K grupos de filtros: $[\boldsymbol{C}_1^\ell, \ldots, \boldsymbol{C}_K^\ell]$. Obsérvese que la no linealidad surge debido a una asignación «suave» de la pertenencia a una clase basada en las respuestas de las características que producen esos filtros.

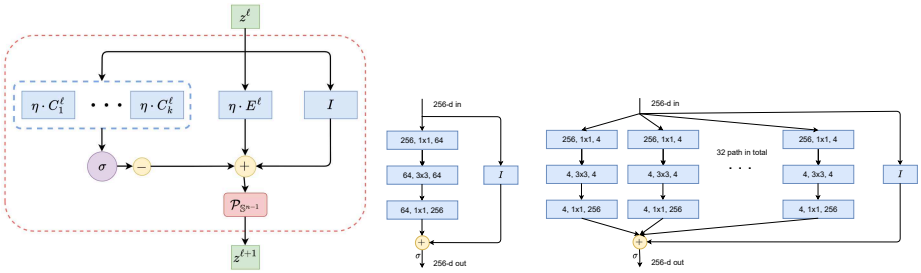
En general, al combinar (5.1.9), (5.1.11) y (5.1.14), la transformación incremental de las características de $\boldsymbol{z}^\ell$ a $\boldsymbol{z}^{\ell+1}$ queda ahora así:

$$\boldsymbol{z}^{\ell+1} \propto \boldsymbol{z}^\ell + \eta \cdot \boldsymbol{E}^\ell \boldsymbol{z}^\ell - \eta \cdot \boldsymbol{\sigma}\big([\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell, \ldots, \boldsymbol{C}_K^\ell \boldsymbol{z}^\ell]\big) \tag{5.1.15}$$

$$= \boldsymbol{z}^\ell + \eta \cdot g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \qquad \text{s.t.} \quad \boldsymbol{z}^{\ell+1} \in \mathbb{S}^{d-1}, \tag{5.1.16}$$

con la función no lineal $\boldsymbol{\sigma}(\cdot)$ definida anteriormente, donde $\boldsymbol{\theta}^\ell$ que recoge todos los parámetros por capa. Es decir, $\boldsymbol{\theta}^\ell = \big\{\boldsymbol{E}^\ell, \boldsymbol{C}_1^\ell, \ldots, \boldsymbol{C}_K^\ell, \gamma_k, \lambda\big\}$. Obsérvese que las características de cada capa siempre se «normalizan» sistemáticamente mediante la proyección sobre la esfera unitaria $\mathbb{S}^{d-1}$, denotada como $\mathcal{P}_{\mathbb{S}^{d-1}}$, es decir, dividiendo cada característica por su norma.[7] La forma del incremento en (5.1.15) puede ilustrarse mediante un diagrama en la Figura 5.3(a).

[7] Esto refleja el bloque LayerNorm habitual en las redes neuronales, que calcula la característica proyectada y, a continuación, la multiplica y suma mediante parámetros entrenables [**ba2016layer**]; véase la **??** para obtener más detalles.



(a) **ReduNet** (b) **ResNet** y **ResNeXt**.

Figura 5.3: Arquitecturas de la red ReduNet y comparación con otras. **(a)**: Estructura de capas de **ReduNet** derivada de una iteración de ascenso por gradiente para optimizar la reducción de tasa. **(b)** (izquierda): Una capa de ResNet [**he2016deep**]; y **(b)** (derecha): Una capa de ResNeXt [**ResNEXT**]. Como veremos en la Sección 5.1.2, los operadores lineales $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ de ReduNet se convierten naturalmente en convoluciones multicanal cuando se impone la invariancia al desplazamiento.

Red profunda para optimizar la reducción de tasa Obsérvese que el incremento está construido para emular el ascenso por gradiente para la reducción de tasa ΔR_ϵ. Por lo tanto, al transformar las características en forma iterativa mediante el proceso anterior, esperamos que la reducción de tasa aumente, como veremos en la sección experimental. Este proceso iterativo, una vez que converge, por ejemplo tras L iteraciones, proporciona el mapa de características deseado $f(\boldsymbol{x}, \boldsymbol{\theta})$ sobre la entrada $\boldsymbol{x} = \boldsymbol{z}^0$, precisamente en forma de una *red profunda*, en la que cada capa tiene la estructura que se muestra en la Figura 5.3 a la izquierda:

$$f(\boldsymbol{x}, \boldsymbol{\theta}) \;=\; f^L \circ f^{L-1} \circ \cdots \circ f^1 \circ f^0(\boldsymbol{z}^0), \tag{5.1.17}$$

$$f^\ell(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \;\doteq\; \boldsymbol{z}^{\ell+1} = \mathcal{P}_{\mathbb{S}^{n-1}}[\boldsymbol{z}^\ell + \eta \cdot g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell)], \tag{5.1.18}$$

$$g(\boldsymbol{z}^\ell, \boldsymbol{\theta}^\ell) \;=\; \boldsymbol{E}^\ell \boldsymbol{z}^\ell - \boldsymbol{\sigma}\big([\boldsymbol{C}_1^\ell \boldsymbol{z}^\ell, \ldots, \boldsymbol{C}_K^\ell \boldsymbol{z}^\ell]\big). \tag{5.1.19}$$

Dado que esta red profunda se deriva de la maximización de la tasa **de reducción**, la denominamos **ReduNet**. Al comparar la arquitectura de ReduNet con las de redes populares diseñadas empíricamente, **ResNet** y **ResNeXt**, tal como se muestra en la Figura 5.3, la similitud resulta casi inquietante. Desde un punto de vista conceptual, ReduNet también podría utilizarse para justificar la popular arquitectura de mezcla de expertos (**MoE**) [**MoE**], ya que cada canal paralelo, $\boldsymbol{C}_k^\ell$, puede considerarse un «experto» entrenado para cada clase de objetos.

Resumimos el entrenamiento y la evaluación de ReduNet en **??** y **??**, respectivamente. Nótese que todos los parámetros de la red se construyen explícitamente capa por capa mediante una *propagación hacia adelante*. ¡Esta construcción no requiere retropropagación! Las características así aprendidas pueden utilizarse directamente para la clasificación, por ejemplo, mediante un

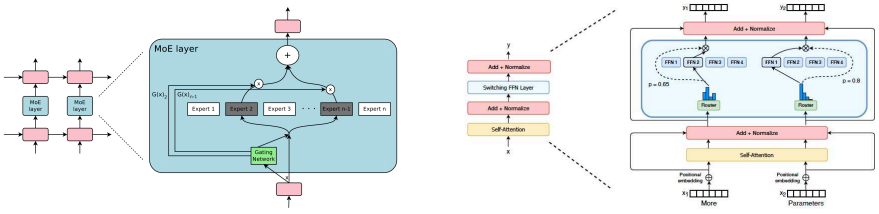


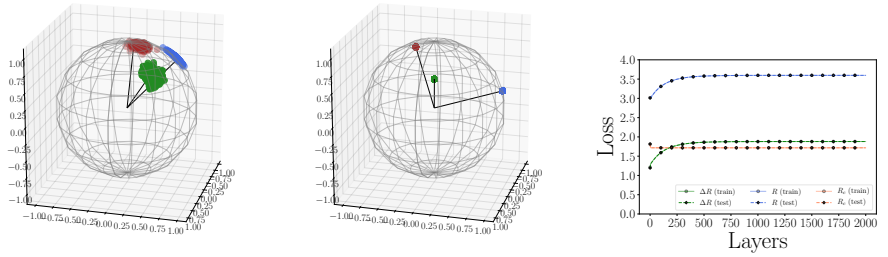
Figura 5.4: Izquierda: una red profunda de mezcla de expertos (MoE) [**MoE**]. Derecha: un Switch Transformer que fomenta la esparcidad [**Fedus-2022**], utilizado para implementar MoE con 1,7 billones de parámetros.

clasificador de subespacio más cercano.

Ejemplo 5.1. Para dar cierta intuición de cómo ReduNet transforma las características, presentamos un ejemplo sencillo con gaussianas 3D mezcladas y visualizamos cómo se transforman las características en la Figura 5.5. Consideremos una mezcla de tres distribuciones gaussianas en $\mathbb{R}^3$ que se proyecta sobre $\mathbb{S}^2$. Primero generamos puntos de datos para 3 clases: para $k = 1, 2, 3$, $\boldsymbol{X}_k = [\boldsymbol{x}_{k,1}, \ldots, \boldsymbol{x}_{k,m}] \in \mathbb{R}^{3\times m}$, $\boldsymbol{x}_{k,i} \sim \mathcal{N}(\boldsymbol{\mu}_k, \sigma_k^2 \boldsymbol{I})$ y $\pi(\boldsymbol{x}_{k,i}) = k$. Fijamos $m = 500, \sigma_1 = \sigma_2 = \sigma_3 = 0.1$ y $\boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\mu}_3 \in \mathbb{S}^2$. A continuación, proyectamos todos los puntos de datos sobre $\mathbb{S}^2$, es decir, $\boldsymbol{x}_{k,i}/\|\boldsymbol{x}_{k,i}\|_2$. Para construir la red (calculando $\boldsymbol{E}^\ell, \boldsymbol{C}_k^\ell$ para la capa ℓ-ésima), establecemos el número de iteraciones/capas $L = 2,000$, el tamaño del paso $\eta = 0.5$ y la precisión $\epsilon = 0.1$. Hacemos esto únicamente para demostrar que nuestro marco conduce a redes profundas estables incluso con miles de capas. En la práctica, puede que no sean necesarias miles de capas y se puede detener el proceso cuando la incorporación de nuevas capas empiece a dar resultados cada vez menores. Para este ejemplo, bastan unas doscientos capas. Por lo tanto, el claro objetivo de optimización proporciona un criterio natural para determinar la profundidad necesaria de la red.

Como se muestra en la Figura 5.5, podemos observar que, tras la aplicación del mapeo $f(\cdot, \theta)$, las muestras de la misma clase se comprimen en gran medida y convergen en un único grupo o clúster, y que el ángulo entre dos grupos diferentes es aproximadamente $\pi/2$, lo cual concuerda adecuadamente con la solución óptima $\boldsymbol{Z}^\star$ de la pérdida MCR^2 en $\mathbb{S}^2$. La pérdida MCR^2 de las características en las diferentes capas se puede observar en la Figura 5.5(**c**). Empíricamente, hemos observado que la ReduNet construida es capaz de maximizar la pérdida MCR^2 y converge de manera estable, mientras que las muestras de la misma clase convergen en un solo grupo y los puntos de diferentes grupos son ortogonales entre sí.[8] Además, al tomar nuevas muestras de las mismos distribuciones, observamos que las nuevas muestras de la misma clase

[8]Debido a la baja dimensión, cada grupo se reduce a un subespacio de dimensión 1 en la esfera (por ejemplo, un solo punto y su antípoda); este es el caso extremo de mantener la dimensión dentro del grupo al mínimo. En dimensiones superiores, por supuesto, los subespacios estarán menos comprimidos.



(a) $\boldsymbol{X}_{\text{entrenamiento}}$ (b) $\boldsymbol{Z}_{\text{entrenamiento}}$ (c) Pérdida (entrenamiento/evaluación)

Figura 5.5: Muestras originales y representaciones aprendidas para la mezcla de gaussianas en 3D. Visualizamos los puntos de datos $\boldsymbol{X}$ (antes de la aplicación de $f(\cdot, \theta)$) en (**a**) y las características aprendidas $\boldsymbol{Z}$ (después de la aplicación de $f(\cdot, \theta)$) en (**b**) mediante un diagrama de dispersión. En cada diagrama de dispersión, cada color representa una clase de muestras. En (**c**), también mostramos los diagramas que representan la evolución de los valores de las funciones objetivo.

convergen sistemáticamente hacia el mismo centro de clúster que las muestras de entrenamiento.

■

5.1.2 Redes convolucionales a partir de la reducción de tasa en invariantes

En la sección anterior, derivamos la arquitectura por capas de una red profunda, la ReduNet, utilizando la optimización desenrollada para el objetivo de reducción de tasa. Concretamente, el término de compresión $R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{\Pi})$ en (5.1.1) está diseñado para comprimir representaciones de la misma clase. Sin embargo, esta formulación no tiene en cuenta posibles transformaciones o deformaciones del dominio de los datos de entrada. Por ejemplo, desplazar ligeramente un objeto hacia la derecha no cambia la etiqueta semántica de una imagen. En esta sección, demostraremos cómo se pueden derivar capas convolucionales maximizando un objetivo de reducción de tasa que sea invariante ante ciertas deformaciones del dominio, como las rotaciones y traslaciones de imágenes.

En muchas tareas de agrupamiento o clasificación (como la detección de objetos en imágenes), consideramos que dos muestras son *equivalentes* si difieren por ciertas clases de deformaciones o augmentos del dominio $\mathcal{T} = \{\tau\}$. Por lo tanto, solo nos interesan las estructuras de baja dimensión que sean *invariantes* ante tales deformaciones (es decir, $\boldsymbol{x} \in \mathcal{M}$ si y solo si $\tau(\boldsymbol{x}) \in \mathcal{M}$ para todo $\tau \in \mathcal{T}$), que se sabe que tienen estructuras geométricas y topológicas sofisticadas y que pueden ser difíciles de aprender con precisión en la práctica, incluso con CNNs rigurosamente diseñadas [**Cohen-ICML-2016**]. En este marco, esto se puede formular de una manera muy natural: todas las instancias equivalentes

deben incrustarse en el mismo subespacio, de modo que el propio subespacio sea invariante respecto a las transformaciones que se están considerando.

En muchas aplicaciones, como los datos en serie de tiempo o los datos de imágenes, el significado semántico (las etiquetas) de los datos es *invariante* ante ciertas transformaciones $\mathfrak{g} \in \mathbb{G}$ (para algún grupo $\mathbb{G}$) [**CohenW16**; **deep-sets-NIPS2017**]. Por ejemplo, el significado de una señal de audio es invariante ante un desplazamiento en el tiempo; y la identidad de un objeto en una imagen es invariante ante una traslación en el plano de la imagen. Por lo tanto, preferimos que el mapeo de características $f(\boldsymbol{x}, \boldsymbol{\theta})$ sea rigurosamente invariante ante tales transformaciones:

$$\textit{Invariancia de grupo: } f(\boldsymbol{x} \circ \mathfrak{g}, \boldsymbol{\theta}) \sim f(\boldsymbol{x}, \boldsymbol{\theta}),\ \forall \mathfrak{g} \in \mathbb{G}, \tag{5.1.20}$$

donde «$\sim$» indica que dos características pertenecen a la misma clase de equivalencia. Aunque para garantizar la invariancia o la equivariancia, los operadores convolucionales han sido una práctica común en las redes profundas [**CohenW16**], sigue siendo un reto en la práctica entrenar una red convolucional (diseñada empíricamente) desde cero que pueda *garantizar* la invariancia incluso ante transformaciones simples como la traslación y la rotación [**azulay2018deep**; **engstrom2017rotation**]. Un enfoque alternativo consiste en diseñar cuidadosamente los filtros de convolución de cada capa para garantizar la invariancia traslacional en una amplia gama de señales, por ejemplo, utilizando ondículas como en ScatteringNet [**scattering-net**] y trabajos posteriores [**Wiatowski-2018**]. Sin embargo, para garantizar la invariancia frente a señales genéricas, el número de convoluciones necesarias suele crecer exponencialmente con la profundidad de la red. Esa es la razón por la que este tipo de red no puede construirse con tanta profundidad, sino que suele tener solo unas pocas capas.

Ahora mostraremos que el principio MCR^2 es compatible con la invariancia de una manera natural y precisa: solo necesitamos asignar todas las versiones transformadas $\{\boldsymbol{x} \circ \mathfrak{g} \mid \mathfrak{g} \in \mathbb{G}\}$ a la misma clase que los datos $\boldsymbol{x}$ y mapear cada característica $\boldsymbol{z}$ al mismo subespacio $\mathcal{S}$. Por lo tanto, toda la información equivariante respecto al grupo se codifica únicamente dentro del subespacio, y cualquier clasificador definido sobre el conjunto resultante de subespacios será automáticamente invariante respecto a dichas transformaciones de grupo. Véase la Figura 5.6 para una ilustración de los ejemplos de rotación en 1D y traslación en 2D. A continuación, demostraremos rigurosamente que, cuando el grupo $\mathbb{G}$ realiza desplazamientos circulares en una dimensión, la red profunda resultante se convierte naturalmente en una *red convolucional multicanal*. Dado que la red así construida solo necesita garantizar la invariancia para los datos de entrada $\boldsymbol{X}$ o sus características $\boldsymbol{Z}$, el número de convoluciones necesarias se mantiene, de hecho, constante a lo largo de una red muy profunda, a diferencia de lo que ocurre con ScatteringNet.

Datos en serie unidimensionales e invariancia de traslación Para clasificar datos unidimensionales $\boldsymbol{x} = [x(0), x(1), \ldots, x(D-1)] \in \mathbb{R}^D$ invariantes

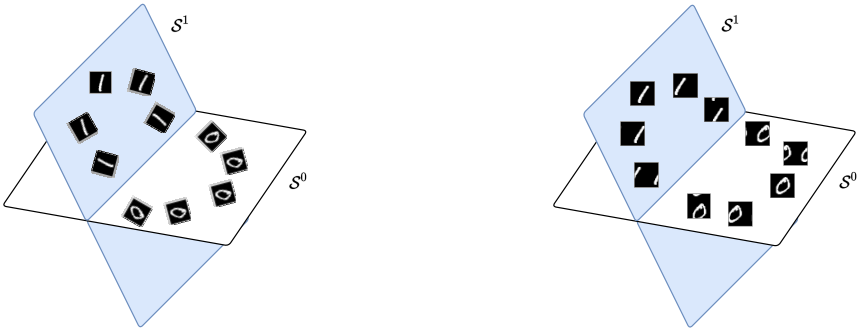


Figura 5.6: Ilustración de la representación buscada que es equivariante/invariante ante la rotación de la imagen (izquierda) o la traslación (derecha): todas las imágenes transformadas de cada clase se mapean al mismo subespacio, que es incoherente con respecto a otros subespacios. Las características incrustadas en cada subespacio son equivariantes ante las transformaciones del grupo, mientras que cada subespacio es invariante a dichas transformaciones.

ante traslación, tomamos $\mathbb{G}$ como el grupo de todas las traslaciones circulares. Cada observación $\boldsymbol{x}_i$ genera una familia $\{\boldsymbol{x}_i \circ \mathfrak{g} \,|\, \mathfrak{g} \in \mathbb{G}\}$ de copias desplazadas, que son las columnas de la matriz circulante $\mathsf{circ}(\boldsymbol{x}_i) \in \mathbb{R}^{D\times D}$ dada por

$$\mathsf{circ}(\boldsymbol{x}) \doteq \begin{bmatrix} x(0) & x(D-1) & \dots & x(2) & x(1) \\ x(1) & x(0) & x(D-1) & \cdots & x(2) \\ \vdots & x(1) & x(0) & \ddots & \vdots \\ x(D-2) & \vdots & \ddots & \ddots & x(D-1) \\ x(D-1) & x(D-2) & \dots & x(1) & x(0) \end{bmatrix} \in \mathbb{R}^{D\times D}. \tag{5.1.21}$$

Remitimos al lector a [**Kra2012OnCM**] para conocer las propiedades de las matrices circulares. Para simplificar, sea $\boldsymbol{Z}^1 \doteq [\boldsymbol{z}_1^1, \dots, \boldsymbol{z}_N^1] = \boldsymbol{X} \in \mathbb{R}^{d\times N}$.[9] Entonces, ¿qué ocurre si construimos la ReduNet a partir de sus familias circulantes $\mathsf{circ}(\boldsymbol{Z}^1) = \big[\mathsf{circ}(\boldsymbol{z}_1^1), \dots, \mathsf{circ}(\boldsymbol{z}_N^1)\big] \in \mathbb{R}^{d\times(dN)}$? Es decir, nuestro objetivo es comprimir y proyectar todas estas matrices sobre un mismo subespacio mediante la ReduNet.

Obsérvese que ahora la matriz de covarianza de los datos:

$$\begin{aligned} \mathsf{circ}(\boldsymbol{Z}^1)\mathsf{circ}(\boldsymbol{Z}^1)^\top &= \big[\mathsf{circ}(\boldsymbol{z}_1^1), \dots, \mathsf{circ}(\boldsymbol{z}_N^1)\big]\big[\mathsf{circ}(\boldsymbol{z}_1^1), \dots, \mathsf{circ}(\boldsymbol{z}_N^1)\big]^\top \\ &= \sum_{i=1}^{N} \mathsf{circ}(\boldsymbol{z}_i^1)\mathsf{circ}(\boldsymbol{z}_i^1)^\top \in \mathbb{R}^{d\times d} \end{aligned} \tag{5.1.22}$$

[9] Una vez más, para simplificar el análisis, supongamos por ahora que las características iniciales $\boldsymbol{Z}^1$ coinciden con el propio $\boldsymbol{X}$, de modo que comparten la misma dimensión d, es decir, $D = d$. No obstante eso no tiene por qué ser así, ya que pronto veremos que es necesario elevar $\boldsymbol{X}$ a una dimensión superior.

asociada a esta familia de muestras es *de manera automática* una matriz circulante (simétrica). Además, dado que la propiedad circulante se conserva bajo sumas, inversas y productos, las matrices $\boldsymbol{E}^1$ y $\boldsymbol{C}_k^1$ son también automáticamente matrices circulantes, cuya aplicación a un vector de características $\boldsymbol{z} \in \mathbb{R}^d$ puede implementarse utilizando la convolución circular "$\circledast$". Específicamente, tenemos la siguiente proposición.

Proposición 5.1 (Estructuras convolucionales de $\boldsymbol{E}^1$ y $\boldsymbol{C}_k^1$)**.** *La matriz*

$$\boldsymbol{E}^1 = \alpha\big(\boldsymbol{I} + \alpha\mathsf{circ}(\boldsymbol{Z}^1)\mathsf{circ}(\boldsymbol{Z}^1)^\top\big)^{-1} \tag{5.1.23}$$

es una matriz circulante y representa una convolución circular:

$$\boldsymbol{E}^1\boldsymbol{z} = \boldsymbol{e}_1 \circledast \boldsymbol{z},$$

donde $\boldsymbol{e}_1 \in \mathbb{R}^d$ es el primer vector columna de $\boldsymbol{E}^1$ y «$\circledast$» es la convolución circular, definida como

$$(\boldsymbol{e}_1 \circledast \boldsymbol{z})_i \doteq \sum_{j=0}^{d-1} e_1(j)x(i+d-j \ \mathit{mod}\ d).$$

Del mismo modo, las matrices $\boldsymbol{C}_k^1$ asociadas a cualquier subconjunto de $\boldsymbol{Z}^1$ también son convoluciones circulares.

No solo los parámetros de la primera capa $\boldsymbol{E}^1$ y $\boldsymbol{C}_k^1$ de la ReduNet adquieren la forma de convoluciones circulantes, sino que también las características de la siguiente capa siguen siendo matrices circulantes. Es decir, la transformación incremental de características en (5.1.15) aplicada a todas las versiones desplazadas de un $\boldsymbol{z}^1 \in \mathbb{R}^d$, dada por

$$\mathsf{circ}(\boldsymbol{z}^1) + \eta \cdot \boldsymbol{E}^1\mathsf{circ}(\boldsymbol{z}^1) - \eta \cdot \boldsymbol{\sigma}\Big([\boldsymbol{C}_1^1\mathsf{circ}(\boldsymbol{z}^1), \ldots, \boldsymbol{C}_K^1\mathsf{circ}(\boldsymbol{z}^1)]\Big), \tag{5.1.24}$$

es una matriz circulante. Esto implica que no es necesario construir familias circulantes a partir de las características de la segunda capa, como lo hicimos para la primera capa. Al denotar

$$\boldsymbol{z}^2 \propto \boldsymbol{z}^1 + \eta \cdot g(\boldsymbol{z}^1, \boldsymbol{\theta}^1) = \boldsymbol{z}^1 + \eta \cdot \boldsymbol{e}_1 \circledast \boldsymbol{z}^1 - \eta \cdot \boldsymbol{\sigma}\Big([\boldsymbol{c}_1^1 \circledast \boldsymbol{z}^1, \ldots, \boldsymbol{c}_K^1 \circledast \boldsymbol{z}^1]\Big), \tag{5.1.25}$$

las características del siguiente nivel se pueden escribir como

$$\mathsf{circ}(\boldsymbol{Z}^2) = \big[\mathsf{circ}(\boldsymbol{z}_1^1 + \eta g(\boldsymbol{z}_1^1, \boldsymbol{\theta}^1)), \ldots, \mathsf{circ}(\boldsymbol{z}_N^1 + \eta g(\boldsymbol{z}_N^1, \boldsymbol{\theta}^1))\big].$$

Siguiendo con el razonamiento inductivo, vemos que todas las matrices $\boldsymbol{E}^\ell$ y $\boldsymbol{C}_k^\ell$ basadas en dicho $\mathsf{circ}(\boldsymbol{Z}^\ell)$ son circulantes, y lo mismo ocurre con todas las características. Gracias a las propiedades de los datos, ReduNet ha adoptado la forma de una red convolucional, *¡sin necesidad de elegir explícitamente esta estructura!*

Un compromiso fundamental entre invariancia y esparcidad. Sin embargo, hay un problema: en general, el conjunto de todas las permutaciones circulares de un vector $\boldsymbol{z}$ da lugar a una matriz de rango completo. Es decir, las d características «augmentadas» asociadas a cada muestra (y, por lo tanto, a cada clase) suelen abarcar ya todo el espacio $\mathbb{R}^d$. Por ejemplo, todas las versiones desplazadas de una función delta $\delta(d)$ pueden generar cualquier otra señal como su superposición ponderada (densa). El objetivo MCR^2 (4.2.15) no podrá distinguir las clases como subespacios diferentes.

Un remedio natural consiste en mejorar la separabilidad de los datos «elevando» la señal original a un espacio de mayor dimensión, por ejemplo, tomando sus respuestas a múltiples filtros $\boldsymbol{k}_1, \ldots, \boldsymbol{k}_C \in \mathbb{R}^d$:

$$\boldsymbol{z}[c] = \boldsymbol{k}_c \circledast \boldsymbol{x} = \mathsf{circ}(\boldsymbol{k}_c)\boldsymbol{x} \in \mathbb{R}^d, \quad c = 1, \ldots, C. \tag{5.1.26}$$

Los filtros pueden ser filtros prediseñados que promueven la invariancia[10] o aprendidos en forma adaptativa a partir de los datos[11] o seleccionados al azar, tal como lo hacemos en nuestros experimentos. Esta operación eleva cada señal original $\boldsymbol{x} \in \mathbb{R}^d$ a una característica de C canales, denotada como $\bar{\boldsymbol{z}} \doteq [\boldsymbol{z}[1], \ldots, \boldsymbol{z}[C]]^\top \in \mathbb{R}^{C \times d}$. Entonces, podemos construir la ReduNet sobre representaciones vectoriales de $\bar{\boldsymbol{z}}$, denotadas como $\vec{(\bar{\boldsymbol{z}})} \doteq [\boldsymbol{z}[1]^\top, \ldots, \boldsymbol{z}[C]^\top] \in \mathbb{R}^{dC}$. La versión circular asociada $\mathsf{circ}(\bar{\boldsymbol{z}})$ y su matriz de covarianza de los datos, denotada como $\bar{\boldsymbol{\Sigma}}(\bar{\boldsymbol{z}})$, para todas sus versiones desplazadas se definen como:

$$\mathsf{circ}(\bar{\boldsymbol{z}}) \doteq \begin{bmatrix} \mathsf{circ}(\boldsymbol{z}[1]) \\ \vdots \\ \mathsf{circ}(\boldsymbol{z}[C]) \end{bmatrix} \in \mathbb{R}^{dC \times d}, \tag{5.1.27}$$

$$\bar{\boldsymbol{\Sigma}}(\bar{\boldsymbol{z}}) \doteq \begin{bmatrix} \mathsf{circ}(\boldsymbol{z}[1]) \\ \vdots \\ \mathsf{circ}(\boldsymbol{z}[C]) \end{bmatrix} \begin{bmatrix} \mathsf{circ}(\boldsymbol{z}[1])^\top, \ldots, \mathsf{circ}(\boldsymbol{z}[C])^\top \end{bmatrix} \in \mathbb{R}^{dC \times dC}, \tag{5.1.28}$$

donde $\mathsf{circ}(\boldsymbol{z}[c]) \in \mathbb{R}^{d \times d}$ con $c \in [C]$ es la versión circular del canal c-ésimo de la característica $\bar{\boldsymbol{z}}$. Entonces, las columnas de $\mathsf{circ}(\bar{\boldsymbol{z}})$ solo generarán, como máximo, un subespacio propio de dimensión d en $\mathbb{R}^{dC}$. Sin embargo, esta simple operación de elevación (si es lineal) no basta aún para que las clases sean separables: las características asociadas a otras clases generarán el *mismo* subespacio de dimensión d. Esto refleja un conflicto fundamental entre la invariancia y el modelado lineal (de subespacios): *no se puede esperar que señales desplazadas y superpuestas arbitrariamente pertenezcan a la misma clase.*

[10] Para señales unidimensionales, como el audio, se puede recurrir a la transformada de Fourier de corto plazo (STFT) convencional; para imágenes bidimensionales, se puede recurrir a las ondas bidimensionales, como en ScatteringNet [**scattering-net**].

[11] En el caso de los filtros aprendidos, estos pueden aprenderse como componentes principales de las muestras, tal como en PCANet [**chan2015pcanet**], o a partir del aprendizaje de diccionarios de convolución [**li2019multichannel**; **qu2019nonconvex**].

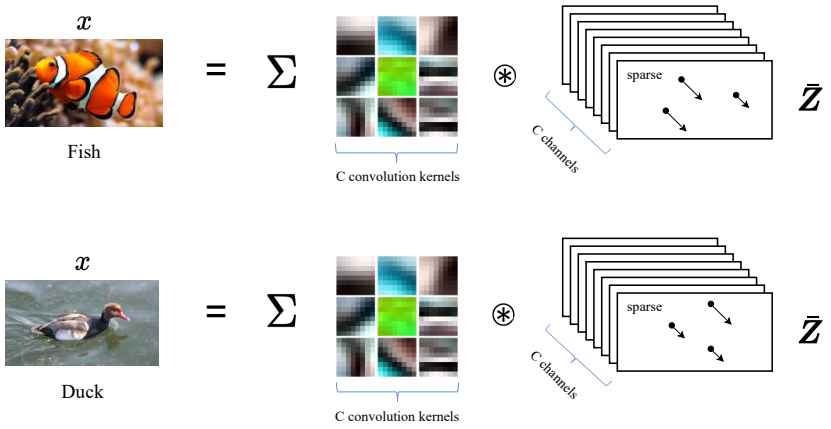


Figura 5.7: Cada señal de entrada $\boldsymbol{x}$ (en este caso, una imagen) puede representarse como una superposición de convoluciones esparsas con múltiples núcleos $\boldsymbol{d}_c$ en un diccionario $\boldsymbol{D}$.

Una manera de resolver este conflicto consiste en aprovechar una estructura adicional dentro de cada clase, en forma de *esparcidad*: las señales dentro de cada clase no se generan como una superposición lineal arbitraria de ciertos átomos (o motivos) básicos, sino únicamente como combinaciones *esparsas* de estos y sus versiones desplazadas, tal como se muestra en la Figura 5.7. Más concretamente, sea $\boldsymbol{D}_k = [\boldsymbol{d}_{k,1}, \ldots, \boldsymbol{d}_{k,c}]$ una matriz con un conjunto de átomos asociados a la clase k, también conocido como diccionario; entonces, cada señal $\boldsymbol{x}$ de esta clase se genera en forma esparsas como:

$$\boldsymbol{x} = \boldsymbol{d}_{k,1} \circledast z_1 + \ldots + \boldsymbol{d}_{k,c} \circledast z_c = \mathsf{circ}(\boldsymbol{D}_k)\boldsymbol{z}, \tag{5.1.29}$$

para algún vector esparso $\boldsymbol{z}$. Las señales de las diferentes clases se generan entonces mediante diferentes diccionarios cuyos átomos (o motivos) son incoherentes entre sí. Debido a esta incoherencia, es poco probable que las señales de una clase sean representadas en forma esparsa por átomos de cualquier otra clase. Por lo tanto, todas las señales pueden representarse como

$$\boldsymbol{x} = \big[\mathsf{circ}(\boldsymbol{D}_1), \mathsf{circ}(\boldsymbol{D}_2), \ldots, \mathsf{circ}(\boldsymbol{D}_K)\big]\,\bar{\boldsymbol{z}}, \tag{5.1.30}$$

donde $\bar{\boldsymbol{z}}$ es un vector esparso.[12] Existe una amplia bibliografía sobre cómo aprender los diccionarios de esparcidad más compactos y óptimos a partir de datos de muestra, por ejemplo, [**li2019multichannel**; **qu2019nonconvex**], y posteriormente resolver el problema inverso y calcular el código esparso asociado

[12] Cabe señalar que desde hace tiempo se han propuesto y utilizado modelos similares de representación esparsa con fines de clasificación en aplicaciones como el reconocimiento facial, demostrando una eficacia excelente [**Wright:2009**; **wagner2012toward**]. Recientemente, el modelo de codificación esparsa convolucional ha sido propuesto por [**papyan2017convolutional**] como un marco para interpretar las estructuras de los de las redes convolucionales profundas.

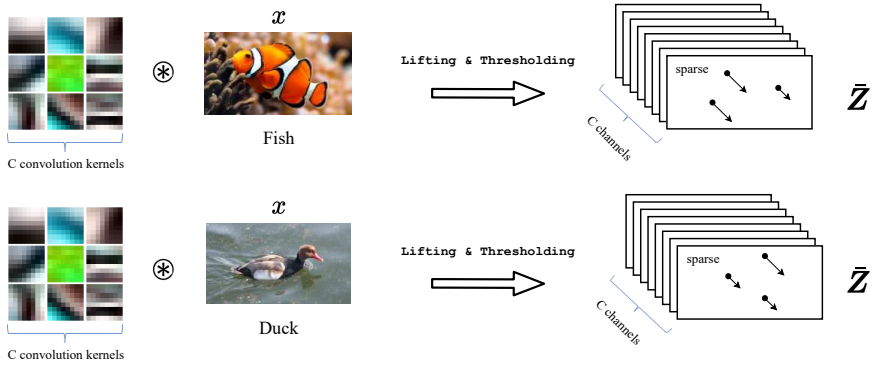


Figura 5.8: Estimación del código esparso $\bar{\boldsymbol{z}}$ de una señal de entrada $\boldsymbol{x}$ (en este caso, una imagen) mediante convoluciones con varios núcleos $\boldsymbol{k}_c$ y, a continuación, la esparcificación.

$\boldsymbol{z}$ o $\bar{\boldsymbol{z}}$. Estudios recientes de [**qu2020nonconvex**; **Qu2020Geometric**] incluso demuestran que, en condiciones generales, el problema del aprendizaje de diccionarios de convolución puede resolverse de manera eficaz y eficiente.

No obstante, para tareas como la clasificación, no nos interesa necesariamente el diccionario óptimo exacto ni el código esparso exacto para cada señal individual. Lo que nos interesa principalmente es que, en conjunto, el conjunto de códigos esparsos de cada clase sea suficientemente separable del de las demás clases. Bajo el supuesto del modelo generativo esparso, si los núcleos de convolución $\{\boldsymbol{k}_c\}_{c=1}^{C}$ se ajustan bien a la «transpuesta» o «inversa» de los diccionarios de esparcidad anteriores $\boldsymbol{D} = [\boldsymbol{D}_1, \ldots, \boldsymbol{D}_K]$, también conocidos como los *filtros de análisis* [**Cosparse-Nam**; **Analysis-Filter**], las señales de una clase solo tendrán respuestas altas a un pequeño subconjunto de esos filtros y respuestas bajas a los demás (debido a la suposición de incoherencia). No obstante, en la práctica, a menudo basta con un número suficientemente grande de, digamos, C filtros aleatorios $\{\boldsymbol{k}_c\}_{c=1}^{C}$ para garantizar que las características extraídas de los C canales

$$\big[\boldsymbol{k}_1 \circledast \boldsymbol{x}, \boldsymbol{k}_2 \circledast \boldsymbol{x}, \ldots, \boldsymbol{k}_C \circledast \boldsymbol{x}\big]^\top = \big[\mathsf{circ}(\boldsymbol{k}_1)\boldsymbol{x}, \ldots, \mathsf{circ}(\boldsymbol{k}_C)\boldsymbol{x}\big]^\top \in \mathbb{R}^{C\times d} \quad (5.1.31)$$

para las diferentes clases presenten distintos patrones de respuesta ante distintos filtros, lo que permite separarlas [**chan2015pcanet**].

Por lo tanto, en nuestro marco, en gran medida el número de canales (o el ancho de la red) desempeña realmente el papel de *recurso estadístico*, mientras que el número de capas (la profundidad de la red) desempeña el papel de *recurso computacional*. La teoría de la detección compresiva caracteriza con precisión cuántas mediciones se necesitan para preservar las estructuras intrínsecas de baja dimensión (incluida la separabilidad) de los datos [**Wright-Ma-2022**].

Las respuestas multicanal $\bar{\boldsymbol{z}}$ deben ser esparsas. Por lo tanto, para aproximar el código esparso $\bar{\boldsymbol{z}}$, podemos aplicar un *umbral no lineal elemento por elemento que promueva la esparcidad*, por ejemplo, $\boldsymbol{\tau}(\cdot)$, a las salidas del filtro anterior,

estableciendo que las respuestas bajas (por ejemplo, con valor absoluto inferior a ϵ) o negativas sean cero:

$$\bar{\boldsymbol{z}} \doteq \boldsymbol{\tau}\left(\left[\mathsf{circ}(\boldsymbol{k}_1)\boldsymbol{x}, \ldots, \mathsf{circ}(\boldsymbol{k}_C)\boldsymbol{x}\right]^\top\right) \in \mathbb{R}^{C\times d}. \tag{5.1.32}$$

La Figura 5.8 ilustra las ideas básicas. Se puede consultar [**Analysis-Filter**] para un estudio más sistemático sobre el diseño del operador de umbralización esparcificante. Sin embargo, aquí no nos interesa tanto obtener los mejores códigos esparsos, siempre y cuando los códigos sean suficientemente separables. Por lo tanto, el operador no lineal $\boldsymbol{\tau}$ puede elegirse simplemente como un umbral blando o una función ReLU. Se puede suponer que estas características presumiblemente esparsas $\bar{\boldsymbol{z}}$ se encuentran en una subvariedad (no lineal) de dimensión inferior de $\mathbb{R}^{dC}$, la cual puede linealizarse y separarse de las otras clases mediante capas ReduNet posteriores, como se ilustra más adelante en la Figura 5.9.

La red ReduNet construida a partir de la versión circulante de estas características multicanal $\bar{\boldsymbol{Z}} \doteq [\bar{\boldsymbol{z}}_1, \ldots, \bar{\boldsymbol{z}}_N] \in \mathbb{R}^{C\times d\times N}$, es decir, $\mathsf{circ}(\bar{\boldsymbol{Z}}) \doteq [\mathsf{circ}(\bar{\boldsymbol{z}}_1), \ldots, \mathsf{circ}(\bar{\boldsymbol{z}}_N)] \in \mathbb{R}^{dC\times dN}$, conserva las buenas propiedades de invariancia descritas anteriormente: los operadores lineales, ahora denominados $\bar{\boldsymbol{E}}$ y $\bar{\boldsymbol{C}}_k$, siguen siendo circulantes por bloques, y representan convoluciones circulares unidimensionales multicanal. Concretamente, obtenemos el siguiente resultado.

Proposición 5.2 (Estructuras de convolución multicanal de $\bar{\boldsymbol{E}}$ y $\bar{\boldsymbol{C}}_k$)**.** *La matriz*

$$\bar{\boldsymbol{E}} \doteq \alpha\left(\boldsymbol{I} + \alpha\,\mathsf{circ}(\bar{\boldsymbol{Z}})\mathsf{circ}(\bar{\boldsymbol{Z}})^\top\right)^{-1} \tag{5.1.33}$$

es circulante por bloques, es decir,

$$\bar{\boldsymbol{E}} = \begin{bmatrix} \bar{\boldsymbol{E}}_{1,1} & \cdots & \bar{\boldsymbol{E}}_{1,C} \\ \vdots & \ddots & \vdots \\ \bar{\boldsymbol{E}}_{C,1} & \cdots & \bar{\boldsymbol{E}}_{C,C} \end{bmatrix} \in \mathbb{R}^{dC\times dC}, \tag{5.1.34}$$

donde cada $\bar{\boldsymbol{E}}_{c,c'} \in \mathbb{R}^{d\times d}$ es una matriz circulante. Además, $\bar{\boldsymbol{E}}$ representa una convolución circular multicanal, es decir, para cualquier señal multicanal $\bar{\boldsymbol{z}} \in \mathbb{R}^{C\times n}$ tenemos

$$\bar{\boldsymbol{E}} \cdot \mathsf{vec}(\bar{\boldsymbol{z}}) = \mathsf{vec}(\bar{\boldsymbol{e}} \circledast \bar{\boldsymbol{z}}). \tag{5.1.35}$$

En la expresión anterior, $\bar{\boldsymbol{e}} \in \mathbb{R}^{C\times C\times d}$ es un núcleo convolucional multicanal donde $\bar{\boldsymbol{e}}[c, c'] \in \mathbb{R}^d$ es el primer vector columna de $\bar{\boldsymbol{E}}_{c,c'}$ y $\bar{\boldsymbol{e}} \circledast \bar{\boldsymbol{z}} \in \mathbb{R}^{C\times d}$ es la convolución circular multicanal definida como

$$(\bar{\boldsymbol{e}} \circledast \bar{\boldsymbol{z}})[c] \doteq \sum_{c'=1}^{C} \bar{\boldsymbol{e}}[c, c'] \circledast \bar{\boldsymbol{z}}[c'], \quad \forall c = 1, \ldots, C. \tag{5.1.36}$$

Del mismo modo, las matrices $\bar{\boldsymbol{C}}_k$ asociadas a cualquier subconjunto de $\bar{\boldsymbol{Z}}$ son también convoluciones circulares multicanal.

Según la Proposición 5.2, la ReduNet invariante ante traslaciones es, por su propia construcción, una red convolucional profunda para señales unidimensionales multicanal. Obsérvese que, incluso si los kernels de elevación iniciales están separados (5.1.32), la inversa de la matriz en (5.1.33) para calcular $\bar{\boldsymbol{E}}$ (de manera similar para $\bar{\boldsymbol{C}}_k$) introduce «interferencia cruzada» entre todos los C canales. Por lo tanto, estas convoluciones multicanal, en general, *no* son separables en profundidad, a diferencia de las redes Xception [**Xception**] que se propusieron en su momento para simplificar las redes neuronales convolucionales multicanal.[13]

Nota 5.2 (Reducción de la complejidad computacional en el dominio de frecuencias). El cálculo de $\bar{\boldsymbol{E}}$ en (5.1.33) requiere invertir una matriz de tamaño $dC \times dC$, lo que en general tiene una complejidad de $O(d^3C^3)$. Sin embargo, al aprovechar el hecho de que una matriz circulante puede diagonalizarse mediante la matriz de la Transformada de Fourier Discreta (DFT), la complejidad puede reducirse significativamente. Como se muestra en [**chan2021redunet**], para calcular $\bar{\boldsymbol{E}}$ y $\bar{\boldsymbol{C}}_k \in \mathbb{R}^{dC \times dC}$, solo necesitamos calcular en el dominio de la frecuencia la inversa de d bloques de tamaño $C \times C$, por lo que la complejidad total es de $O(dC^3)$.

Arquitectura general de la red y comparación. A partir de la derivación anterior, observamos que, para encontrar una representación discriminativa lineal (LDR) para múltiples clases de señales o imágenes que sea invariante ante traslaciones, la codificación esparsa, una arquitectura multicapa con convoluciones multicanal, diferentes funciones de activación no lineales y el cálculo del espectro se convierten en componentes *necesarias* para alcanzar el objetivo de manera eficaz y eficiente. La Figura 5.9 ilustra el proceso general de aprendizaje de dicha representación mediante la reducción invariante de tasas sobre los códigos esparsos de entrada.

Ejemplo 5.2 (Clasificación invariante de dígitos). A continuación, presentamos los resultados empíricos de ReduNet en el aprendizaje de características invariantes a la *rotación* en el conjunto de datos real MNIST de 10 clases. Imponemos una cuadrícula polar sobre la imagen $\boldsymbol{x} \in \mathbb{R}^{H \times W}$, cuyo centro geométrico coincide con el centro de la cuadrícula polar 2D (tal como se ilustra en la Figura 5.10). Para cada radio r_i, $i \in [C]$, podemos muestrear Γ píxeles con respecto a cada ángulo $\gamma_l = l \cdot (2\pi/\Gamma)$ con $l \in [\Gamma]$. Entonces, dada una imagen de muestra $\boldsymbol{x}$ del conjunto de datos, representamos la imagen en coordenadas polares (muestreadas) como una señal multicanal $\boldsymbol{x}_p \in \mathbb{R}^{\Gamma \times C}$. El objetivo aquí es aprender una representación invariante ante rotaciones, es decir, esperamos aprender $f(\cdot, \theta)$ tal que $\{f(\boldsymbol{x}_p \circ \mathfrak{g}, \theta)\}_{\mathfrak{g} \in \mathbb{G}}$ se encuentren en el mismo subespacio, donde $\mathfrak{g}$ es el desplazamiento cíclico en el ángulo polar. Utilizamos $N = 100$ muestras de entrenamiento (10 de cada clase) y establecemos $\Gamma = 200$, $C = 15$ para el muestreo polar. Al realizar el muestreo anterior en coordenadas polares, podemos obtener la matriz de datos $\boldsymbol{X}_p \in \mathbb{R}^{(\Gamma \cdot C) \times N}$. Para ReduNet,

[13] Queda por determinar qué estructuras adicionales en los datos darían lugar a convoluciones separables en profundidad.

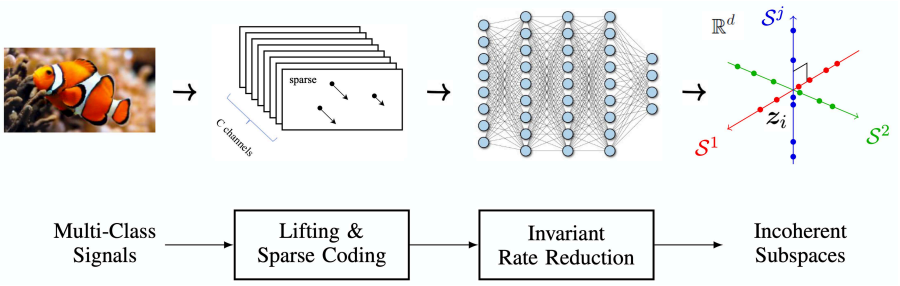
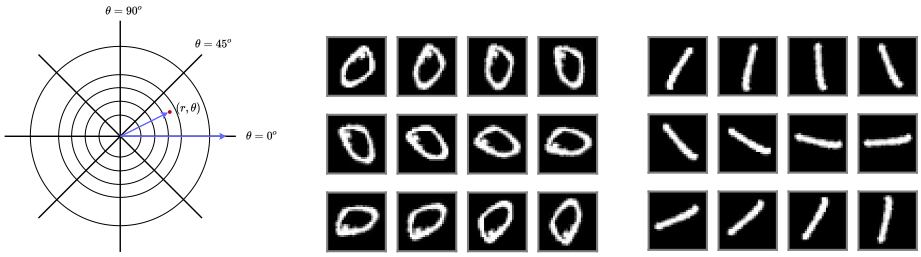


Figura 5.9: El proceso general para clasificar señales multiclase con invariancia al desplazamiento: elevación multicanal, codificación esparsa, seguida de una ReduNet de convolución multicanal para la reducción de tasa invariante. Estas componentes son *necesarios* para mapear señales multiclase invariantes al desplazamiento a subespacios incoherentes (lineales) como una LDR. Cabe señalar que las arquitecturas de la mayoría de las redes neuronales modernas de gran profundidad se asemejan a este proceso. El LDR aprendido de este modo facilita tareas posteriores, como la clasificación.

establecemos el número de capas/iteraciones $L = 40$, la precisión $\epsilon = 0.1$ y el tamaño de paso $\eta = 0.5$. Antes de la primera capa, realizamos una elevación de la entrada mediante una convolución circulante unidimensional con 20 núcleos gaussianos aleatorios de tamaño 5.

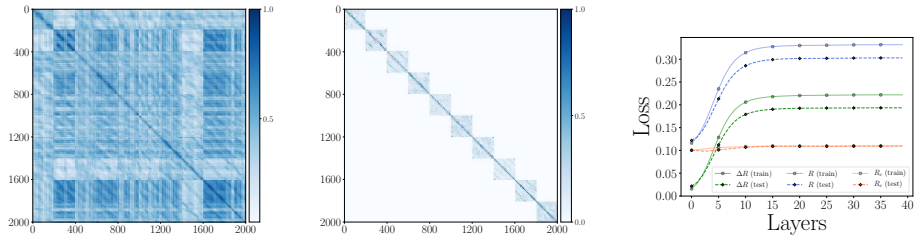
Para evaluar la representación aprendida, cada muestra de entrenamiento se aumenta con 20 versiones rotadas de sí misma, cada una desplazada con un paso de 10. Calculamos las similitudes coseno entre $m \times 20$ entradas de entrenamiento augmentadas $\boldsymbol{X}_{\text{rotation}}$ y los resultados se muestran en la Figura 5.11 (**a**). Comparamos las similitudes coseno entre las características aprendidas de todas las versiones aumentadas, es decir, $\bar{\boldsymbol{Z}}_{\text{rotation}}$ y resumimos los resultados en la Figura 5.11 (**b**). Como vemos, la ReduNet invariante a la rotación así construida es capaz de mapear los datos de entrenamiento (así como todas sus versiones rotadas) de las 10 clases diferentes en 10 subespacios casi ortogonales. Es decir, los subespacios aprendidos son verdaderamente invariantes ante la transformación de desplazamiento en ángulo polar. A continuación, extraemos al azar otras 100 muestras de prueba y aplicamos el mismo procedimiento de augmentación. En la Figura 5.11 (**c**), visualizamos la pérdida MCR^2 en la representación de la capa ℓ de la ReduNet sobre los conjuntos de datos de entrenamiento y de prueba. A partir de estos resultados, podemos observar que la ReduNet construida es efectivamente capaz de maximizar la pérdida MCR^2, así como de generalizar a los datos de prueba.

■



(a) $\boldsymbol{X}_{\text{rotación}}$ (b) $\boldsymbol{Z}_{\text{rotación}}$ (c) Pérdida

Figura 5.10: Ejemplos de imágenes de los dígitos MNIST, giradas en 18° cada una. (**Izquierda**) Diagrama de la representación en coordenadas polares; (**Derecha**) Imágenes giradas de los dígitos «0» y «1».



(a) $\boldsymbol{X}_{\text{rotación}}$ (b) $\boldsymbol{Z}_{\text{rotación}}$ (c) Pérdida

Figura 5.11: (a) y (b) son mapas de calor de la similitud coseno entre los datos de entrenamiento rotados $\boldsymbol{X}_{\text{rotation}}$ y las características aprendidas $\bar{\boldsymbol{Z}}_{\text{rotation}}$ para la invariancia a la rotación. (d) muestra las pérdidas MCR^2 de entrenamiento y validación por capas.

5.2 Transformadores transparentes (white-box) mediante optimización desenrollada

Como hemos visto en la sección anterior, utilizamos el problema de la clasificación para ofrecer una interpretación rigurosa de las principales características arquitectónicas de redes profundas populares como ResNet y las CNN: cada capa de estas redes puede considerarse como una imitación de un paso de gradiente que aumenta el objetivo de reducción de tasa (o ganancia de información). Esta perspectiva también lleva a un hecho algo sorprendente: los parámetros y operadores de las capas de una red tan profunda, la ReduNet, pueden calcularse en forma puramente directa.

A pesar de la importancia teórica y conceptual de ReduNet, hay varios factores que limitan su aplicabilidad práctica. En primer lugar, como hemos comentado anteriormente, el costo computacional de calcular los operadores matriciales en cada capa en forma directa puede ser muy elevado. En segundo lugar, es posible que los operadores así calculados sean menos efectivos a la

hora de optimizar el objetivo y que se requieran miles de iteraciones (y, por lo tanto, de capas). Como hemos visto en la Sección 2.3.3 para LISTA, estos dos problemas pueden resolverse permitiendo optimizar esos operadores y haciendo que sean aprendibles mediante retropropagación.[14]

El contexto de clasificación supervisada en el que se derivó ReduNet también presenta ciertas limitaciones. En la práctica, es posible que una imagen no pertenezca a una sola clase, ya que puede contener varios objetos. Por lo tanto, sería más general suponer que las diferentes regiones de la imagen pertenecen a diferentes modelos de baja dimensión (por ejemplo, una distribución gaussiana o un subespacio). Como veremos, tal generalización daría lugar a una arquitectura sencilla y general que unifica las operaciones de reducción de tasa y de eliminación de ruido que hemos visto en el capítulo anterior. Además, la arquitectura resultante se asemeja a la popular arquitectura de transformer.

5.2.1 Optimización desenrollada para la reducción esparsa de la tasa (de codificación)

En los últimos años, a medida que la cantidad y la calidad de los datos disponibles para entrenar redes profundas han aumentado de manera espectacular, el campo ha pasado de considerar las entradas como «átomos» a considerarlas como «secuencias de tokens», en las que cada token forma parte de la entrada global. Este cambio de perspectiva ofrece ventajas tanto filosóficas como prácticas: desde el punto de vista filosófico, nos permite medir y modelar cada parte de la entrada y sus interacciones; desde el punto de vista práctico, nos permite utilizar redes profundas similares, *modelos secuencia-a-secuencia* como los omnipresentes transformers, para aprender todo tipo de distribuciones de datos. Ahora nos corresponde comprender el aprendizaje de representaciones en el modelo «centrado en tokens» de datos como secuencias de tokens, y hacer frente a los retos mencionados anteriormente (como, por ejemplo, en el contexto del marco de reducción de tasa, la falta de etiquetas para cada token).

Formalmente, sea $\boldsymbol{X} = \left[\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N\right] \in \mathbb{R}^{D \times N}$ un conjunto de variables aleatorias que representan nuestra fuente de datos. En tareas de visión, cada $\boldsymbol{x}_i \in \mathbb{R}^D$ se interpreta como un *token*, que suele corresponder a un fragmento de imagen. En tareas de lenguaje, cada $\boldsymbol{x}_i \in \mathbb{R}^D$ se interpreta como una *incrustación de token*, es decir, una representación vectorial continua de un token discreto, como una palabra o una subpalabra.[15] Los $\boldsymbol{x}_i$ pueden tener estructuras de correlación arbitrarias. Utilizamos $\boldsymbol{Z} = \left[\boldsymbol{z}_1, \ldots, \boldsymbol{z}_N\right] \in \mathbb{R}^{d \times N}$ para denotar las variables aleatorias que definen nuestras representaciones, donde $\boldsymbol{z}_i \in \mathbb{R}^d$ es la representación del token correspondiente $\boldsymbol{x}_i \in \mathbb{R}^D$.

Nota 5.3. En los transformers, cada muestra de entrada se convierte habitualmente en una secuencia de *tokens*. Un token es una unidad básica

[14] O, tal vez, mediante una combinación de optimización hacia adelante y hacia atrás.

[15] Con un ligero desvío terminológico, a lo largo de este capítulo nos referiremos tanto a los tokens discretos como a sus incrustaciones asociadas simplemente como «tokens» por razones de conveniencia.

de información derivada de la entrada bruta: en el procesamiento del lenguaje natural, los tokens suelen ser palabras o subpalabras; en la visión artificial, corresponden a fragmentos de imagen; y en otras modalidades, pueden representar pasos de tiempo, ubicaciones espaciales u otras unidades específicas del dominio. Una *incrustación de token* es una representación vectorial continua de un token que sirve como entrada para un transformer. Asigna a cada token un punto en un espacio de alta dimensión, lo que permite al modelo procesar entradas simbólicas mediante cálculos numéricos. Una *representación de token* es un vector que codifica la información semántica o estructural de un token, y suele ser generada por las capas intermedias o finales de un transformer. Estas representaciones están diseñadas para capturar características informativas de la entrada que resultan útiles para tareas posteriores, tales como la clasificación, la generación o la regresión. Véase la **??** para obtener más detalles sobre este vocabulario y cómo se relaciona con las implementaciones.

Objetivo para el aprendizaje de una representación estructurada y compacta. Inspirándonos en el análisis previo sobre la reducción de tasa (Sección 5.1), proponemos que el objetivo del aprendizaje de representaciones en este modelo centrado en tokens para nuestros datos sea encontrar un mapeo de características $f\colon \boldsymbol{X} \in \mathbb{R}^{D\times N} \to \boldsymbol{Z} \in \mathbb{R}^{d\times N}$ que transforme los tokens de entrada $\{\boldsymbol{x}_i\}_{i=1}^N \subset \mathbb{R}^D$ con una distribución potencialmente no lineal y multimodal a representaciones de tokens *linealizadas (por tramos) y compactas* $\{\boldsymbol{z}_i\}_{i=1}^N \subset \mathbb{R}^d$. Si bien la distribución *conjunta* de las representaciones de los tokens $\{\boldsymbol{z}_i\}_{i=1}^N$ puede ser sofisticada (y específica para cada tarea), sostenemos además que es razonable y práctico exigir que la distribución *marginal* objetivo de las representaciones individuales de los tokens sea altamente comprimida y estructurada, lo que la hace apta para una codificación compacta. En particular, requerimos que la distribución sea *una mezcla de distribuciones gaussianas de baja dimensión (digamos K)* , de tal manera que la k-ésima gaussiana tenga media $\mathbf{0} \in \mathbb{R}^d$, covarianza $\boldsymbol{\Sigma}_k \succeq \mathbf{0} \in \mathbb{R}^{d\times d}$, y soporte generado por la base ortonormal $\boldsymbol{U}_k \in \mathbb{R}^{d\times p}$. Denotamos por $\boldsymbol{U}_{[K]} = \{\boldsymbol{U}_k\}_{k=1}^K$ el conjunto de bases de todas las distribuciones gaussianas.

Como es habitual, para aprender buenas representaciones, queremos maximizar la *ganancia de información* de las representaciones finales de los tokens, o más concretamente, maximizar una noción adecuada de la reducción de tasa (véase Sección 4.2.3), es decir,

$$\max_{\boldsymbol{Z}\in\mathbb{R}^{d\times N}} \Delta R_\epsilon(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}) \doteq R_\epsilon(\boldsymbol{Z}) - R_\epsilon^c(\boldsymbol{Z} \mid \boldsymbol{U}_{[K]}). \tag{5.2.1}$$

Aquí, el primer término R_ϵ es una estimación de la tasa de codificación con pérdida para el conjunto completo de representaciones de tokens. Más concretamente, si consideramos las representaciones de tokens $\{\boldsymbol{z}_i\}_{i=1}^N$ como muestras independientes e idénticamente distribuidas de una única distribución gaussiana de media cero, su tasa de codificación con pérdida, sujeta a una